



# DATA STRUCTURE



RAHUL SHRIDHAR BANSOD

# Data Structures Tutorial

11001  
10001 11100110  
0010 110001 11000110  
00101001 01011010 1100  
010101 1100000100 100  
00011111 10 1001110  
00101 11010 10  
10010 101  
00100  
01001  
00110  
0000110

## Data Structure

Data Structures (DS) tutorial provides basic and advanced concepts of Data Structure. Our Data Structure tutorial is designed for beginners and professionals.

Data Structure is a way to store and organize data so that it can be used efficiently.

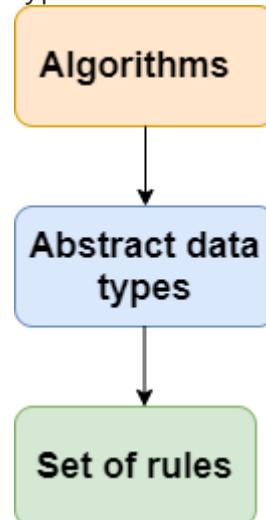
Our Data Structure tutorial includes all topics of Data Structure such as Array, Pointer, Structure, Linked List, Stack, Queue, Graph, Searching, Sorting, Programs, etc.

### What is Data Structure?

The data structure name indicates itself that organizing the data in memory. There are many ways of organizing the data in the memory as we have already seen one of the data structures, i.e., array in C language. Array is a collection of memory elements in which data is stored sequentially, i.e., one after another. In other words, we can say that array stores the elements in a continuous manner. This organization of

data is done with the help of an array of data structures. There are also other ways to organize the data in memory. Let's see the different types of data structures.

The data structure is not any programming language like C, C++, java, etc. It is a set of algorithms that we can use in any programming language to structure the data in the memory. To structure the data in memory, 'n' number of algorithms were proposed, and all these algorithms are known as Abstract data types. These abstract data types are the set of rules.



### Types of Data Structures

There are two types of data structures:

Primitive data structure

Non-primitive data structure

#### Primitive Data structure

The primitive data structures are primitive data types. The int, char, float, double, and pointer are the primitive data structures that can hold a single value.

#### Non-Primitive Data structure

The non-primitive data structure is divided into two types:

Linear data structure

Non-linear data structure

### **Linear Data Structure**

The arrangement of data in a sequential manner is known as a linear data structure. The data structures used for this purpose are Arrays, Linked list, Stacks, and Queues. In these data structures, one element is connected to only one another element in a linear form.

**When one element is connected to the 'n' number of elements known as a non-linear data structure. The best example is trees and graphs. In this case, the elements are arranged in a random manner.**

We will discuss the above data structures in brief in the coming topics. Now, we will see the common operations that we can perform on these data structures.

**Data structures can also be classified as:**

**Static data structure:** It is a type of data structure where the size is allocated at the compile time. Therefore, the maximum size is fixed.

**Dynamic data structure:** It is a type of data structure where the size is allocated at the run time. Therefore, the maximum size is flexible.

### **Major Operations**

The major or the common operations that can be performed on the data structures are:

**Searching:** We can search for any element in a data structure.

**Sorting:** We can sort the elements of a data structure either in an ascending or descending order.

**Insertion:** We can also insert the new element in a data structure.

**Updation:** We can also update the element, i.e., we can replace the element with another element.

**Deletion:** We can also perform the delete operation to remove the element from the data structure.

### **Which Data Structure?**

A data structure is a way of organizing the data so that it can be used efficiently. Here, we have used the word efficiently, which in terms of both the space and time. For example, a stack is an ADT (Abstract data type) which uses either arrays or linked list data structure for the implementation. Therefore, we conclude that we require some data structure to implement a particular ADT. An ADT tells **what** is to be done and data structure tells **how** it is to be done. In other words, we can say that ADT gives us the blueprint while data structure provides the implementation part. Now the question arises: how can one get to know which data structure to be used for a particular ADT?.

As the different data structures can be implemented in a particular ADT, but the different implementations are compared for time and space. For example, the Stack ADT can be implemented by both Arrays and linked list. Suppose the array is providing time efficiency while the linked list is providing space efficiency, so the one which is the best suited for

the current user's requirements will be selected.

## Advantages of Data structures

**The following are the advantages of a data structure:**

**Efficiency:** If the choice of a data structure for implementing a particular ADT is proper, it makes the program very efficient in terms of time and space.

**Reusability:** The data structure provides reusability means that multiple client programs can use the data structure.

**Abstraction:** The data structure specified by an ADT also provides the level of abstraction. The client cannot see the internal working of the data structure, so it does not have to worry about the implementation part. The client can only see the interface.

---

## Data Structures Index

---

### DS Basics

[DS Tutorial](#)

[DS Introduction](#)

[DS Algorithm](#)

[Ds Asymptotic Analysis](#)

[DS Pointer](#)

[DS Structure](#)

### DS Array

[Array](#)

[2D Array](#)

### DS Linked List

[Linked List](#)

[Insertion at beginning](#)

[Insertion at end](#)

[Insertion after specified node](#)

[Deletion at beginning](#)

[Deletion at end](#)

[Deletion after specified node](#)

[Traversing](#)

[Searching](#)

[Doubly Linked List](#)

[Insertion at beginning](#)

[Insertion at end](#)

[Insertion after specified node](#)

[Deletion at beginning](#)

[Deletion at end](#)

[Deletion of node having given data](#)

[Traversing](#)

[Searching](#)

[Circular Linked List](#)

[Insertion at beginning](#)

[Insertion at end](#)

[Deletion at beginning](#)

[Deletion at the end](#)

[Traversing](#)

[Searching](#)

[Circular Doubly List](#)

[Insertion at beginning](#)

[Insertion at end](#)

[Deletion at beginning](#)

[Deletion at the end](#)

### DS Stack

[DS Stack](#)

[Array Implementation](#)

[Linked List Implementation](#)

### DS Queue

[DS Queue](#)

[Array Implementation](#)

[Linked List Implementation](#)

[Circular Queue](#)

### DS Tree

[Tree](#)

[Binary Tree](#)

[Pre-order Traversal](#)

[In-order Traversal](#)

[Post-order Traversal](#)

[Binary Search Tree](#)

[Searching in BST](#)



[Insertion in BST](#)

[Deletion in BST](#)

[AVL Tree](#)

[Insertion in AVL Tree](#)

[LL Rotation](#)

[LR Rotation](#)

[RL Rotation](#)

[RR Rotation](#)

[Deletion in AVL Tree](#)

[B Tree](#)

[B+ Tree](#)

[Red Black Tree](#)

### **DS Graph**

[DS Graph](#)

[Graph Implementation](#)

[BFS Algorithm](#)

[DFS Algorithm](#)

[Spanning Tree](#)

[Prim's Algorithm](#)

[Kruskal's Algorithm](#)

### **DS Searching**

[Linear Search](#)

[Binary Search](#)

### **DS Sorting**

[Bubble Sort](#)

[Bucket Sort](#)

[Comb Sort](#)

[Counting Sort](#)

[Heap Sort](#)

[Insertion Sort](#)

[Merge Sort](#)

[Quick Sort](#)

[Radix Sort](#)

[Selection Sort](#)

[Shell Sort](#)

[Bitonic Sort](#)

[Cocktail Sort](#)

[Cycle Sort](#)

[Tim Sort](#)

### **Interview Questions**

### DS Interview Questions

#### **Singly Linked List Programs**

[Program to create and display a singly linked list](#)

[Program to create a singly linked list of n nodes and count the number of nodes](#)

[Program to create a singly linked list of n nodes and display it in reverse order](#)

[Program to delete a new node from the beginning of the singly linked list](#)

[Program to delete a new node from the middle of the singly linked list](#)

[Program to delete a node from the end of the singly linked list](#)

[Program to determine whether a singly linked list is the palindrome](#)

[Program to find the maximum and minimum value node from a singly linked list](#)

[Program to insert a new node at the middle of the singly linked list](#)

[Program to insert a new node at the beginning of the singly linked list](#)

[Program to insert a new node at the end of the singly linked list](#)

[Program to remove duplicate elements from a singly linked list](#)

[Program to search an element in a singly linked list](#)

[Program to sort the elements of the singly linked list](#)

[Program to swap nodes in a singly linked list without swapping data](#)

[Program to swap the last element of the singly linked list from the first one](#)

#### **Doubly Linked List Programs**

[Program to Convert a Given Binary Tree to Doubly Linked List](#)

[Program to Create a Doubly Linked List From a Ternary Tree](#)

[Program to Create a Doubly Linked List of N Nodes and Count the Number of Nodes](#)

[Program to Create a Doubly Linked List of N Nodes and Display it in Reverse Order](#)

[Program to Create and Display a Doubly Linked List](#)

[Program to Delete a New Node From the Beginning of the Doubly Linked List](#)

[Program to Delete a New Node From the End of the Doubly Linked List](#)

[Program to Delete a New Node From the Middle of the Doubly Linked List](#)

[Program to Find the Maximum and Minimum Value Node From a Doubly Linked List](#)

[Program to Insert a New Node at the Beginning of the Doubly Linked List](#)

[Program to Insert a New Node at the End of Doubly Linked List](#)

[Program to Insert a New Node at the Middle of Doubly Linked List](#)

[Program to Remove Duplicate Elements From a Doubly Linked List](#)

[Program to Rotate Doubly Linked List by N Nodes](#)

[Program to Search an Element in a Doubly Linked List](#)

[Program to Sort the Elements of the Doubly Linked List](#)

### **Circular Linked List Programs**

[Program to Create a Circular Linked List of N Nodes and Count the Number of Nodes](#)

[Program to Create a Circular Linked List of N Nodes and Display it in Reverse Order](#)

[Program to Create and Display a Circular Linked List](#)

[Program to Delete a New Node From the Beginning of the Circular Linked List](#)

[Program to Delete a New Node From the End of the Circular Linked List](#)

[Program to Delete a New Node From the Middle of the Circular Linked List](#)

[Program to Find the Maximum and Minimum Value Node From a Circular Linked List](#)

[Program to Insert a New Node at the Beginning of the Circular Linked List](#)

[Program to Insert a New Node at the End of the Circular Linked List](#)

[Program to Insert a New Node at the Middle of the Circular Linked List](#)

[Program to Remove Duplicate Elements From a Circular Linked List](#)

[Program to Search an Element in a Circular Linked List](#)

[Program to Sort the Elements of the Circular Linked List](#)

### **Tree Programs**

[Program to Calculate the Difference Between the Sum of the Odd Level and Even Level Nodes of a Binary Tree](#)

[Program to Construct a Binary Search Tree and Perform Deletion and Inorder Traversal](#)

[Program to Convert Binary Tree to Binary Search Tree](#)

[Program to Determine Whether all Leaves are at Same Level](#)

[Program to Determine Whether two Trees are Identical](#)

[Program to Find Maximum Width of a Binary Tree](#)

[Program to Find the Largest Element in a Binary Tree](#)

[Program to Find the Maximum Depth or Height of a Tree](#)

[Program to Find the Nodes Which are at the Maximum Distance in a Binary Tree](#)  
[Program to Find the Smallest Element in a Binary Tree](#)  
[Program to Find the Sum of all the Nodes of a Binary Tree](#)  
[Program to Find the Total Number of Possible Binary Search Trees with N Keys](#)  
[Program to Implement Binary Tree using the Linked List](#)  
[Program to Search a Node in a Binary Tree](#)

## Prerequisite

Before learning Data Structure, you must have the basic knowledge of C.

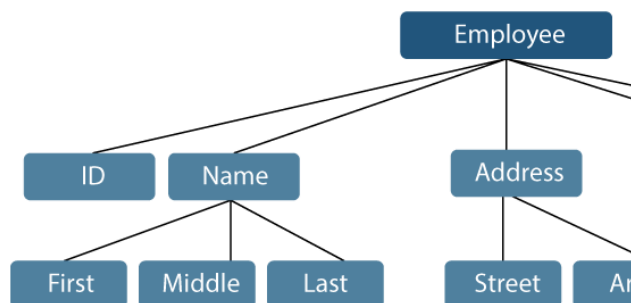
## Audience

Our Data Structure tutorial is designed to help beginners and professionals.

## Problem

We assure that you will not find any problem in this Data Structure tutorial. But if there is any mistake, please post it in the contact form.

# An Introduction to Data Structures



**Figure 1:** Representation of Data Items

Since the invention of computers, people have been using the term "**Data**" to refer to Computer Information, either transmitted or stored. However, there is data that exists in order types as well. Data can be numbers or texts written on a piece of paper, in the form of bits and bytes stored inside the memory of electronic devices, or facts stored within a person's mind. As the world started modernizing, this data became a significant aspect of everyone's day-to-day life, and various implementations allowed them to store it differently.

**Data** is a collection of facts and figures or a set of values or values of a specific format that refers to a single set of item values. The data items are then classified into sub-items, which is the group of items that are not known as the simple primary form of the item.

Let us consider an example where an employee name can be broken down into three sub-items: First, Middle, and Last. However, an ID assigned to an employee will generally be considered a single item.

In the example mentioned above, the items such as ID, Age, Gender, First, Middle, Last, Street, Locality, etc., are elementary data items. In contrast, the Name and the Address are group data items.

## What is Data Structure?

**Data Structure** is a branch of Computer Science. The study of data structure allows us to understand the organization of data and the management of the data flow in order to increase the efficiency of any process or program. Data Structure is a particular way of storing and organizing data in the memory of the computer so that these data can easily be retrieved and efficiently utilized in the future when required. The data can be managed in various ways, like the logical or mathematical model for a specific organization of data is known as a data structure.

The scope of a particular data model depends on two factors:

First, it must be loaded enough into the structure to reflect the definite correlation of the data with a real-world object.

Second, the formation should be so straightforward that one can adapt to process the data efficiently whenever necessary.

Some examples of Data Structures are Arrays, Linked Lists, Stack, Queue, Trees, etc. Data Structures are widely used in almost every aspect of Computer Science, i.e., Compiler Design, Operating Systems, Graphics, Artificial Intelligence, and many more.

Data Structures are the main part of many Computer Science Algorithms as they allow the programmers to manage the data in an effective way. It plays a crucial role in improving the performance of a program or software, as the main objective of the software is to store and retrieve the user's data as fast as possible.

## Basic Terminologies related to Data Structures

Data Structures are the building blocks of any software or program. Selecting the suitable data structure for a program is an extremely challenging task for a programmer. The following are some fundamental terminologies used whenever the data structures are involved:

**Data:** We can define data as an elementary value or a collection of values. For example, the Employee's name and ID are the data related to the Employee.

**Data Items:** A Single unit of value is known as Data Item.

**Group Items:** Data Items that have subordinate data items are known as Group Items. For example, an employee's name can have a first, middle, and last name.

**Elementary Items:** Data Items that are unable to divide into sub-items are known as Elementary Items. For example, the ID of an Employee.

**Entity and Attribute:** A class of certain objects is represented by an Entity. It consists of different Attributes. Each Attribute symbolizes the specific property of that Entity. For example,

| Attributes | ID   | Name           | Gender | Job Title          |
|------------|------|----------------|--------|--------------------|
| Values     | 1234 | Stacey M. Hill | Female | Software Developer |

Entities with similar attributes form an **Entity Set**. Each attribute of an entity set has a range of values, the set of all possible values that could be assigned to the specific attribute.

The term "information" is sometimes utilized for data with given attributes of meaningful or processed data.

**Field:** A single elementary unit of information symbolizing the Attribute of an Entity is known as Field.

**Record:** A collection of different data items are known as a Record. For example, if we talk about the employee entity, then its name, id, address, and job title can be grouped to form the record for the employee.

**File:** A collection of different Records of one entity type is known as a File. For example, if there are 100 employees, there will be 25 records in the related file containing data about each employee.

## Understanding the Need for Data Structures

As applications are becoming more complex and the amount of data is increasing every day, which may lead to problems with data searching, processing speed, multiple requests handling, and many more. Data Structures support different methods to organize, manage, and store data efficiently. With the help of Data Structures, we can easily traverse the data items. Data Structures provide Efficiency, Reusability, and Abstraction.

### Why should we learn Data Structures?

Data Structures and Algorithms are two of the key aspects of Computer Science.

Data Structures allow us to organize and store data, whereas Algorithms allow us to process that data meaningfully.

Learning Data Structures and Algorithms will help us become better Programmers.

We will be able to write code that is more effective and reliable.

We will also be able to solve problems more quickly and efficiently.

## Understanding the Objectives of Data Structures

Data Structures satisfy two complementary objectives:

**Correctness:** Data Structures are designed to operate correctly for all kinds of inputs based on the domain of interest. In other words, correctness forms the primary objective of Data Structure, which always depends upon the problems that the Data Structure is meant to solve.

**Efficiency:** Data Structures also require to be efficient. It should process the data quickly without utilizing many computer resources like memory space. In a real-time

state, the efficiency of a data structure is a key factor in determining the success and failure of the process.

## Understanding some Key Features of Data Structures

Some of the Significant Features of Data Structures are:

**Robustness:** Generally, all computer programmers aim to produce software that yields correct output for every possible input, along with efficient execution on all hardware platforms. This type of robust software must manage both valid and invalid inputs.

**Adaptability:** Building software applications like Web Browsers, Word Processors, and Internet Search Engine include huge software systems that require correct and efficient working or execution for many years. Moreover, software evolves due to emerging technologies or ever-changing market conditions.

**Reusability:** The features like Reusability and Adaptability go hand in hand. It is known that the programmer needs many resources to build any software, making it a costly enterprise. However, if the software is developed in a reusable and adaptable way, then it can be applied in most future applications. Thus, by executing quality data structures, it is possible to build reusable software, which appears to be cost-effective and timesaving.

## Classification of Data Structures

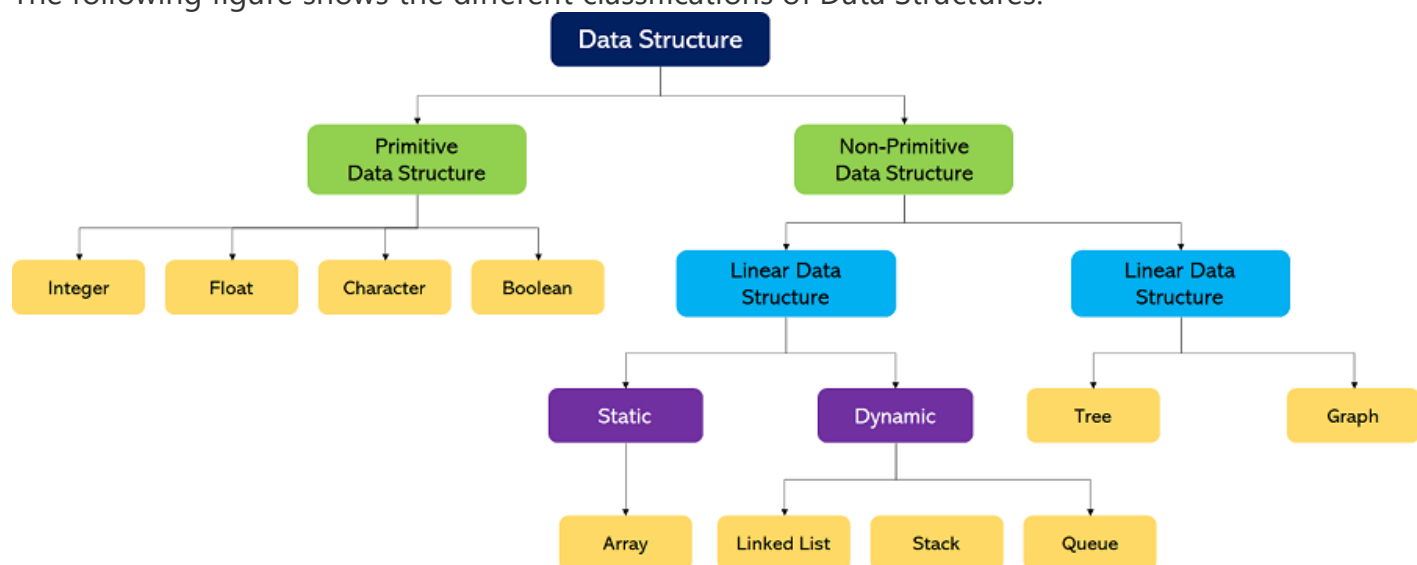
A Data Structure delivers a structured set of variables related to each other in various ways. It forms the basis of a programming tool that signifies the relationship between the data elements and allows programmers to process the data efficiently.

We can classify Data Structures into two categories:

Primitive Data Structure

Non-Primitive Data Structure

The following figure shows the different classifications of Data Structures.



**Figure 2:** Classifications of Data Structures

## Primitive Data Structures

**Primitive Data Structures** are the data structures consisting of the numbers and the characters that come **in-built** into programs.

These data structures can be manipulated or operated directly by machine-level instructions.

Basic data types like **Integer**, **Float**, **Character**, and **Boolean** come under the Primitive Data Structures.

These data types are also called **Simple data types**, as they contain characters that can't be divided further

## Non-Primitive Data Structures

**Non-Primitive Data Structures** are those data structures derived from Primitive Data Structures.

These data structures can't be manipulated or operated directly by machine-level instructions.

The focus of these data structures is on forming a set of data elements that is either **homogeneous** (same data type) or **heterogeneous** (different data types).

Based on the structure and arrangement of data, we can divide these data structures into two sub-categories -

Linear Data Structures

Non-Linear Data Structures

### Linear Data Structures

A data structure that preserves a linear connection among its data elements is known as a Linear Data Structure. The arrangement of the data is done linearly, where each element consists of the successors and predecessors except the first and the last data element. However, it is not necessarily true in the case of memory, as the arrangement may not be sequential.

Based on memory allocation, the Linear Data Structures are further classified into two types:

**Static Data Structures:** The data structures having a fixed size are known as Static Data Structures. The memory for these data structures is allocated at the compiler time, and their size cannot be changed by the user after being compiled; however, the data stored in them can be altered.

The **Array** is the best example of the Static Data Structure as they have a fixed size, and its data can be modified later.

**Dynamic Data Structures:** The data structures having a dynamic size are known as Dynamic Data Structures. The memory of these data structures is allocated at the run time, and their size varies during the run time of the code. Moreover, the user can change the size as well as the data elements stored in these data structures at the run time of the code.

**Linked Lists**, **Stacks**, and **Queues** are common examples of dynamic data structures



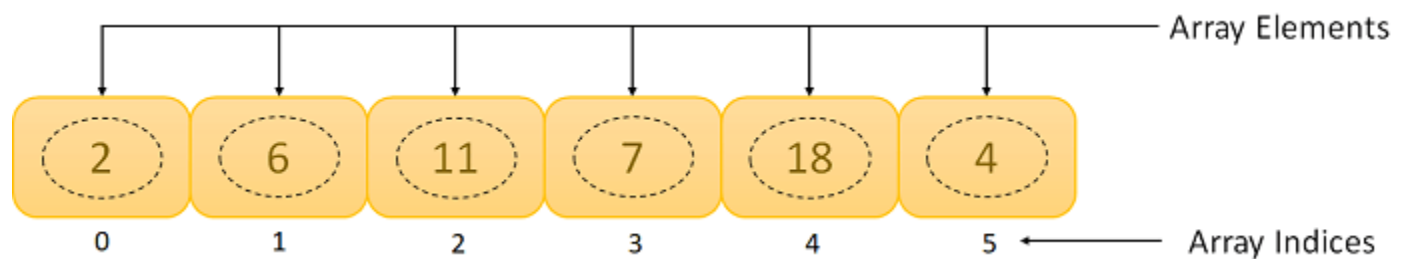
## Types of Linear Data Structures

The following is the list of Linear Data Structures that we generally use:

### 1. Arrays

An **Array** is a data structure used to collect multiple data elements of the same data type into one variable. Instead of storing multiple values of the same data types in separate variable names, we could store all of them together into one variable. This statement doesn't imply that we will have to unite all the values of the same data type in any program into one array of that data type. But there will often be times when some specific variables of the same data types are all related to one another in a way appropriate for an array.

An Array is a list of elements where each element has a unique place in the list. The data elements of the array share the same variable name; however, each carries a different index number called a subscript. We can access any data element from the list with the help of its location in the list. Thus, the key feature of the arrays to understand is that the data is stored in contiguous memory locations, making it possible for the users to traverse through the data elements of the array using their respective indexes.



**Figure 3.** An Array

**Arrays can be classified into different types:**

**One-Dimensional Array:** An Array with only one row of data elements is known as a One-Dimensional Array. It is stored in ascending storage location.

**Two-Dimensional Array:** An Array consisting of multiple rows and columns of data elements is called a Two-Dimensional Array. It is also known as a Matrix.

**Multidimensional Array:** We can define Multidimensional Array as an Array of Arrays. Multidimensional Arrays are not bounded to two indices or two dimensions as they can include as many indices as per the need.

**Some Applications of Array:**

We can store a list of data elements belonging to the same data type.

Array acts as an auxiliary storage for other data structures.

The array also helps store data elements of a binary tree of the fixed count.

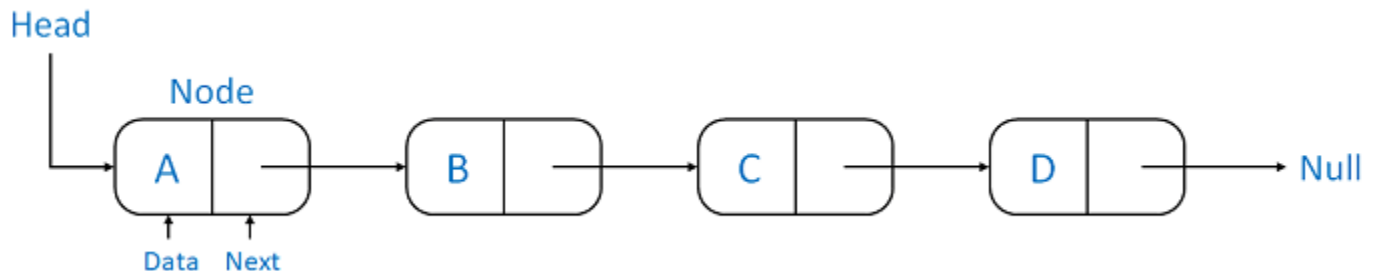
Array also acts as a storage of matrices.

### 2. Linked Lists

A **Linked List** is another example of a linear data structure used to store a collection of data elements dynamically. Data elements in this data structure are represented by the



Nodes, connected using links or pointers. Each node contains two fields, the information field consists of the actual data, and the pointer field consists of the address of the subsequent nodes in the list. The pointer of the last node of the linked list consists of a null pointer, as it points to nothing. Unlike the Arrays, the user can dynamically adjust the size of a Linked List as per the requirements.



**Figure 4.** A Linked List

**Linked Lists can be classified into different types:**

**Singly Linked List:** A Singly Linked List is the most common type of Linked List. Each node has data and a pointer field containing an address to the next node.

**Doubly Linked List:** A Doubly Linked List consists of an information field and two pointer fields. The information field contains the data. The first pointer field contains an address of the previous node, whereas another pointer field contains a reference to the next node. Thus, we can go in both directions (backward as well as forward).

**Circular Linked List:** The Circular Linked List is similar to the Singly Linked List. The only key difference is that the last node contains the address of the first node, forming a circular loop in the Circular Linked List.

**Some Applications of Linked Lists:**

The Linked Lists help us implement stacks, queues, binary trees, and graphs of predefined size.

We can also implement Operating System's function for dynamic memory management.

Linked Lists also allow polynomial implementation for mathematical operations.

We can use Circular Linked List to implement Operating Systems or application functions that Round Robin execution of tasks.

Circular Linked List is also helpful in a Slide Show where a user requires to go back to the first slide after the last slide is presented.

Doubly Linked List is utilized to implement forward and backward buttons in a browser to move forward and backward in the opened pages of a website.

### 3. Stacks

A **Stack** is a Linear Data Structure that follows the **LIFO** (Last In, First Out) principle that allows operations like insertion and deletion from one end of the Stack, i.e., Top. Stacks can be implemented with the help of contiguous memory, an Array, and non-contiguous memory, a Linked List. Real-life examples of Stacks are piles of books, a deck of cards, piles of money, and many more.



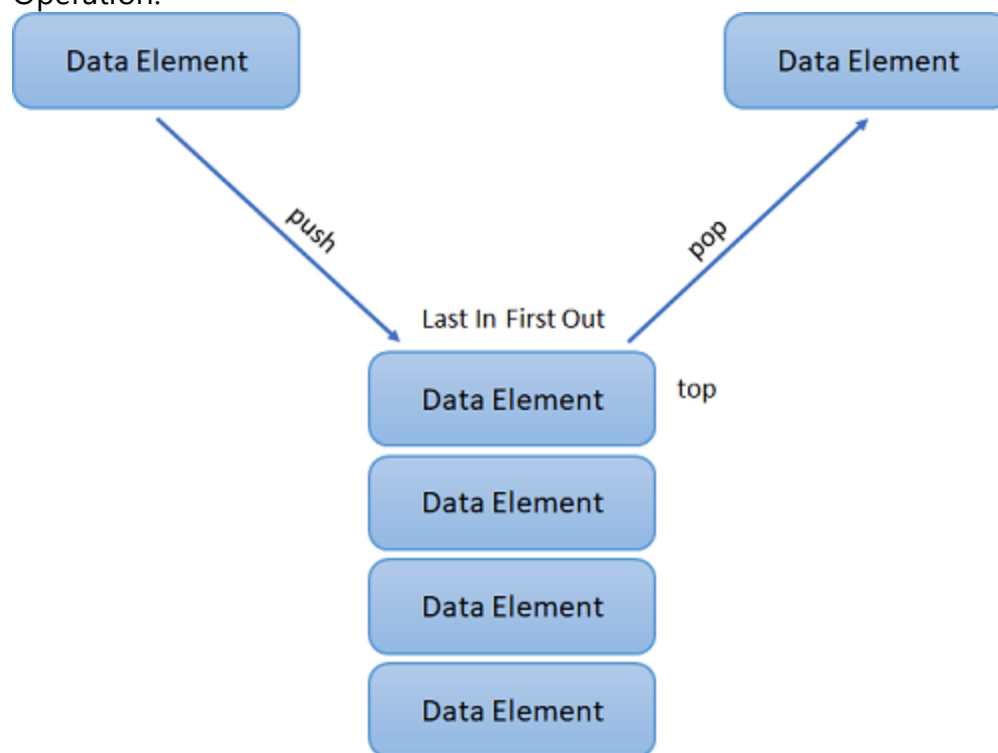
**Figure 5.** A Real-life Example of Stack

The above figure represents the real-life example of a Stack where the operations are performed from one end only, like the insertion and removal of new books from the top of the Stack. It implies that the insertion and deletion in the Stack can be done only from the top of the Stack. We can access only the Stack's tops at any given time.

**The primary operations in the Stack are as follows:**

**Push:** Operation to insert a new element in the Stack is termed as Push Operation.

**Pop:** Operation to remove or delete elements from the Stack is termed as Pop Operation.



**Figure 6.** A Stack

**Some Applications of Stacks:**

The Stack is used as a Temporary Storage Structure for recursive operations.

Stack is also utilized as Auxiliary Storage Structure for function calls, nested operations, and deferred/postponed functions.

We can manage function calls using Stacks.

Stacks are also utilized to evaluate the arithmetic expressions in different programming languages.

Stacks are also helpful in converting infix expressions to postfix expressions.

Stacks allow us to check the expression's syntax in the programming environment.

We can match parenthesis using Stacks.

Stacks can be used to reverse a String.

Stacks are helpful in solving problems based on backtracking.

We can use Stacks in depth-first search in graph and tree traversal.

Stacks are also used in Operating System functions.

Stacks are also used in UNDO and REDO functions in an edit.

#### 4. Queues

A **Queue** is a linear data structure similar to a Stack with some limitations on the insertion and deletion of the elements. The insertion of an element in a Queue is done at one end, and the removal is done at another or opposite end. Thus, we can conclude that the Queue data structure follows FIFO (First In, First Out) principle to manipulate the data elements. Implementation of Queues can be done using Arrays, Linked Lists, or Stacks. Some real-life examples of Queues are a line at the ticket counter, an escalator, a car wash, and many more.



**Figure 7.** A Real-life Example of Queue

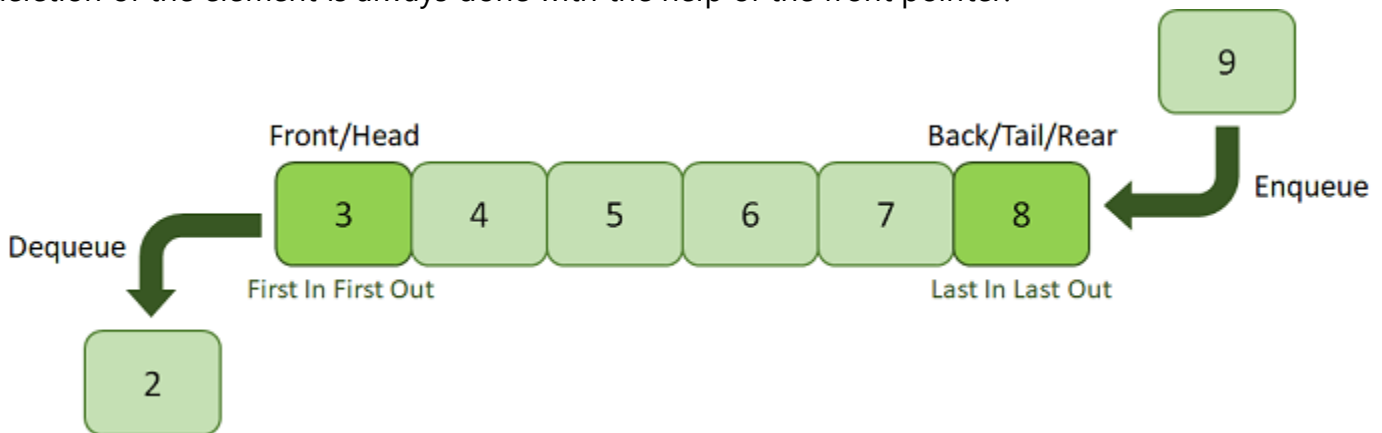
The above image is a real-life illustration of a movie ticket counter that can help us understand the Queue where the customer who comes first is always served first. The customer arriving last will undoubtedly be served last. Both ends of the Queue are open and can execute different operations. Another example is a food court line where the

customer is inserted from the rear end while the customer is removed at the front end after providing the service they asked for.

The following are the primary operations of the Queue:

**Enqueue:** The insertion or Addition of some data elements to the Queue is called Enqueue. The element insertion is always done with the help of the rear pointer.

**Dequeue:** Deleting or removing data elements from the Queue is termed Dequeue. The deletion of the element is always done with the help of the front pointer.



**Figure 8.** A Queue

### Some Applications of Queues:

Queues are generally used in the breadth search operation in Graphs.

Queues are also used in Job Scheduler Operations of Operating Systems, like a keyboard buffer queue to store the keys pressed by users and a print buffer queue to store the documents printed by the printer.

Queues are responsible for CPU scheduling, Job scheduling, and Disk Scheduling.

Priority Queues are utilized in file-downloading operations in a browser.

Queues are also used to transfer data between peripheral devices and the CPU.

Queues are also responsible for handling interrupts generated by the User Applications for the CPU.

## Non-Linear Data Structures

Non-Linear Data Structures are data structures where the data elements are not arranged in sequential order. Here, the insertion and removal of data are not feasible in a linear manner. There exists a hierarchical relationship between the individual data items.

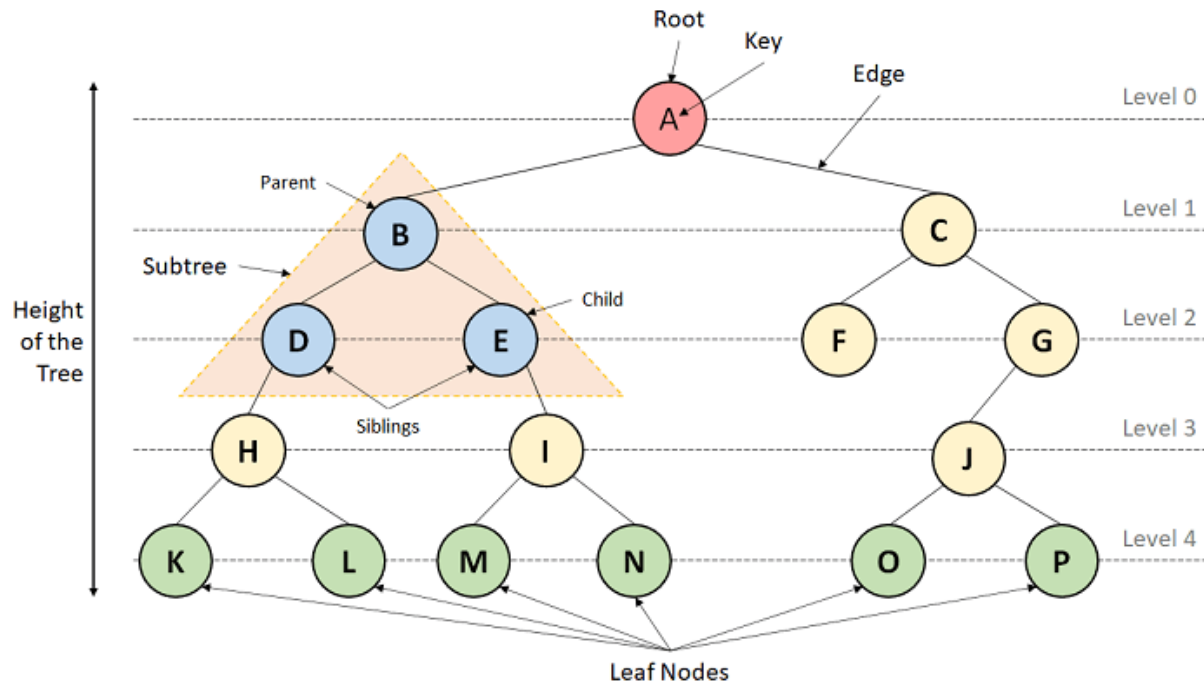
## Types of Non-Linear Data Structures

The following is the list of Non-Linear Data Structures that we generally use:

### 1. Trees

A Tree is a Non-Linear Data Structure and a hierarchy containing a collection of nodes such that each node of the tree stores a value and a list of references to other nodes (the "children").

The Tree data structure is a specialized method to arrange and collect data in the computer to be utilized more effectively. It contains a central node, structural nodes, and sub-nodes connected via edges. We can also say that the tree data structure consists of roots, branches, and leaves connected.



**Figure 9.** A Tree

**Trees can be classified into different types:**

**Binary Tree:** A Tree data structure where each parent node can have at most two children is termed a Binary Tree.

**Binary Search Tree:** A Binary Search Tree is a Tree data structure where we can easily maintain a sorted list of numbers.

**AVL Tree:** An AVL Tree is a self-balancing Binary Search Tree where each node maintains extra information known as a Balance Factor whose value is either -1, 0, or +1.

**B-Tree:** A B-Tree is a special type of self-balancing Binary Search Tree where each node consists of multiple keys and can have more than two children.

**Some Applications of Trees:**

Trees implement hierarchical structures in computer systems like directories and file systems.

Trees are also used to implement the navigation structure of a website.

We can generate code like Huffman's code using Trees.

Trees are also helpful in decision-making in Gaming applications.

Trees are responsible for implementing priority queues for priority-based OS scheduling functions.

Trees are also responsible for parsing expressions and statements in the compilers of different programming languages.

We can use Trees to store data keys for indexing for Database Management System (DBMS).

Spanning Trees allows us to route decisions in Computer and Communications Networks.

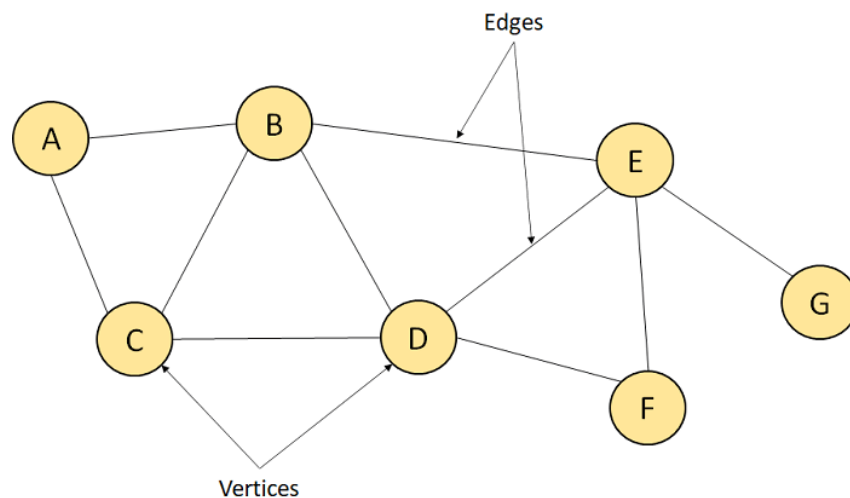
Trees are also used in the path-finding algorithm implemented in Artificial Intelligence (AI), Robotics, and Video Games Applications.

## 2. Graphs

A Graph is another example of a Non-Linear Data Structure comprising a finite number of nodes or vertices and the edges connecting them. The Graphs are utilized to address problems of the real world in which it denotes the problem area as a network such as social networks, circuit networks, and telephone networks. For instance, the nodes or vertices of a Graph can represent a single user in a telephone network, while the edges represent the link between them via telephone.

The Graph data structure,  $G$  is considered a mathematical structure comprised of a set of vertices,  $V$  and a set of edges,  $E$  as shown below:

$$G = (V, E)$$



**Figure 10.** A Graph

The above figure represents a Graph having seven vertices A, B, C, D, E, F, G, and ten edges [A, B], [A, C], [B, C], [B, D], [B, E], [C, D], [D, E], [D, F], [E, F], and [E, G].

**Depending upon the position of the vertices and edges, the Graphs can be classified into different types:**

**Null Graph:** A Graph with an empty set of edges is termed a Null Graph.

**Trivial Graph:** A Graph having only one vertex is termed a Trivial Graph.

**Simple Graph:** A Graph with neither self-loops nor multiple edges is known as a Simple Graph.

**Multi Graph:** A Graph is said to be Multi if it consists of multiple edges but no self-loops.

**Pseudo Graph:** A Graph with self-loops and multiple edges is termed a Pseudo Graph.

**Non-Directed Graph:** A Graph consisting of non-directed edges is known as a Non-Directed Graph.

**Directed Graph:** A Graph consisting of the directed edges between the vertices is known as a Directed Graph.

**Connected Graph:** A Graph with at least a single path between every pair of vertices is termed a Connected Graph.

**Disconnected Graph:** A Graph where there does not exist any path between at least one pair of vertices is termed a Disconnected Graph.

**Regular Graph:** A Graph where all vertices have the same degree is termed a Regular Graph.

**Complete Graph:** A Graph in which all vertices have an edge between every pair of vertices is known as a Complete Graph.

**Cycle Graph:** A Graph is said to be a Cycle if it has at least three vertices and edges that form a cycle.

**Cyclic Graph:** A Graph is said to be Cyclic if and only if at least one cycle exists.

**Acyclic Graph:** A Graph having zero cycles is termed an Acyclic Graph.

**Finite Graph:** A Graph with a finite number of vertices and edges is known as a Finite Graph.

**Infinite Graph:** A Graph with an infinite number of vertices and edges is known as an Infinite Graph.

**Bipartite Graph:** A Graph where the vertices can be divided into independent sets A and B, and all the vertices of set A should only be connected to the vertices present in set B with some edges is termed a Bipartite Graph.

**Planar Graph:** A Graph is said to be a Planar if we can draw it in a single plane with two edges intersecting each other.

**Euler Graph:** A Graph is said to be Euler if and only if all the vertices are even degrees.

**Hamiltonian Graph:** A Connected Graph consisting of a Hamiltonian circuit is known as a Hamiltonian Graph.

### **Some Applications of Graphs:**

Graphs help us represent routes and networks in transportation, travel, and communication applications.

Graphs are used to display routes in GPS.

Graphs also help us represent the interconnections in social networks and other network-based applications.

Graphs are utilized in mapping applications.

Graphs are responsible for the representation of user preference in e-commerce applications.

Graphs are also used in Utility networks in order to identify the problems posed to local or municipal corporations.

Graphs also help to manage the utilization and availability of resources in an organization.

Graphs are also used to make document link maps of the websites in order to display the connectivity between the pages through hyperlinks.

Graphs are also used in robotic motions and neural networks.

## Basic Operations of Data Structures

In the following section, we will discuss the different types of operations that we can perform to manipulate data in every data structure:

**Traversal:** Traversing a data structure means accessing each data element exactly once so it can be administered. For example, traversing is required while printing the names of all the employees in a department.

**Search:** Search is another data structure operation which means to find the location of one or more data elements that meet certain constraints. Such a data element may or may not be present in the given set of data elements. For example, we can use the search operation to find the names of all the employees who have the experience of more than 5 years.

**Insertion:** Insertion means inserting or adding new data elements to the collection. For example, we can use the insertion operation to add the details of a new employee the company has recently hired.

**Deletion:** Deletion means to remove or delete a specific data element from the given list of data elements. For example, we can use the deleting operation to delete the name of an employee who has left the job.

**Sorting:** Sorting means to arrange the data elements in either Ascending or Descending order depending on the type of application. For example, we can use the sorting operation to arrange the names of employees in a department in alphabetical order or estimate the top three performers of the month by arranging the performance of the employees in descending order and extracting the details of the top three.

**Merge:** Merge means to combine data elements of two sorted lists in order to form a single list of sorted data elements.

**Create:** Create is an operation used to reserve memory for the data elements of the program. We can perform this operation using a declaration statement. The creation of data structure can take place either during the following:

Compile-time

Run-time

For example, the **malloc()** function is used in C Language to create data structure.

**Selection:** Selection means selecting a particular data from the available data. We can select any particular data by specifying conditions inside the loop.



**Update:** The Update operation allows us to update or modify the data in the data structure. We can also update any particular data by specifying some conditions inside the loop, like the Selection operation.

**Splitting:** The Splitting operation allows us to divide data into various subparts decreasing the overall process completion time.

## Understanding the Abstract Data Type

As per the **National Institute of Standards and Technology (NIST)**, a data structure is an arrangement of information, generally in the memory, for better algorithm efficiency. Data Structures include linked lists, stacks, queues, trees, and dictionaries. They could also be a theoretical entity, like the name and address of a person.

From the definition mentioned above, we can conclude that the operations in data structure include:

A high level of abstractions like addition or deletion of an item from a list.

Searching and sorting an item in a list.

Accessing the highest priority item in a list.

Whenever the data structure does such operations, it is known as an **Abstract Data Type (ADT)**.

We can define it as a set of data elements along with the operations on the data. The term "abstract" refers to the fact that the data and the fundamental operations defined on it are being studied independently of their implementation. It includes what we can do with the data, not how we can do it.

An ADI implementation contains a storage structure in order to store the data elements and algorithms for fundamental operation. All the data structures, like an array, linked list, queue, stack, etc., are examples of ADT.

## Understanding the Advantages of using ADTs

In the real world, programs evolve as a consequence of new constraints or requirements, so modifying a program generally requires a change in one or multiple data structures. For example, suppose we want to insert a new field into an employee's record to keep track of more details about each employee. In that case, we can improve the efficiency of the program by replacing an Array with a Linked structure. In such a situation, rewriting every procedure that utilizes the modified structure is unsuitable. Hence, a better alternative is to separate a data structure from its implementation information. This is the principle behind the usage of Abstract Data Types (ADT).

## Some Applications of Data Structures

The following are some applications of Data Structures:

Data Structures help in the organization of data in a computer's memory.

Data Structures also help in representing the information in databases.

Data Structures allows the implementation of algorithms to search through data (For example, search engine).

We can use the Data Structures to implement the algorithms to manipulate data (For example, word processors).

We can also implement the algorithms to analyse data using Data Structures (For example, data miners).

Data Structures support algorithms to generate the data (For example, a random number generator).

Data Structures also support algorithms to compress and decompress the data (For example, a zip utility).

We can also use Data Structures to implement algorithms to encrypt and decrypt the data (For example, a security system).

With the help of Data Structures, we can build software that can manage files and directories (For example, a file manager).

We can also develop software that can render graphics using Data Structures. (For example, a web browser or 3D rendering software).

Apart from those, as mentioned earlier, there are many other applications of Data Structures that can help us build any desired software.

## DS Algorithm

### What is an Algorithm?

An algorithm is a process or a set of rules required to perform calculations or some other problem-solving operations especially by a computer. The formal definition of an algorithm is that it contains the finite set of instructions which are being carried in a specific order to perform the specific task. It is not the complete program or code; it is just a solution (logic) of a problem, which can be represented either as an informal description using a Flowchart or Pseudocode.

### Characteristics of an Algorithm

**The following are the characteristics of an algorithm:**

**Input:** An algorithm has some input values. We can pass 0 or some input value to an algorithm.

**Output:** We will get 1 or more output at the end of an algorithm.

**Unambiguity:** An algorithm should be unambiguous which means that the instructions in an algorithm should be clear and simple.

**Finiteness:** An algorithm should have finiteness. Here, finiteness means that the algorithm should contain a limited number of instructions, i.e., the instructions should be countable.

**Effectiveness:** An algorithm should be effective as each instruction in an algorithm affects the overall process.

**Language independent:** An algorithm must be language-independent so that the instructions in an algorithm can be implemented in any of the languages with the same output.

## Dataflow of an Algorithm

**Problem:** A problem can be a real-world problem or any instance from the real-world problem for which we need to create a program or the set of instructions. The set of instructions is known as an algorithm.

**Algorithm:** An algorithm will be designed for a problem which is a step by step procedure.

**Input:** After designing an algorithm, the required and the desired inputs are provided to the algorithm.

**Processing unit:** The input will be given to the processing unit, and the processing unit will produce the desired output.

**Output:** The output is the outcome or the result of the program.

## Why do we need Algorithms?

**We need algorithms because of the following reasons:**

**Scalability:** It helps us to understand the scalability. When we have a big real-world problem, we need to scale it down into small-small steps to easily analyze the problem.

**Performance:** The real-world is not easily broken down into smaller steps. If the problem can be easily broken into smaller steps means that the problem is feasible.

Let's understand the algorithm through a real-world example. Suppose we want to make a lemon juice, so following are the steps required to make a lemon juice:

Step 1: First, we will cut the lemon into half.

Step 2: Squeeze the lemon as much you can and take out its juice in a container.

Step 3: Add two tablespoon sugar in it.

Step 4: Stir the container until the sugar gets dissolved.

Step 5: When sugar gets dissolved, add some water and ice in it.

Step 6: Store the juice in a fridge for 5 to minutes.

Step 7: Now, it's ready to drink.

The above real-world can be directly compared to the definition of the algorithm. We cannot perform the step 3 before the step 2, we need to follow the specific order to make lemon juice. An algorithm also says that each and every instruction should be followed in a specific order to perform a specific task.

Now we will look an example of an algorithm in programming.

We will write an algorithm to add two numbers entered by the user.

**The following are the steps required to add two numbers entered by the user:**

Step 1: Start

Step 2: Declare three variables a, b, and sum.

Step 3: Enter the values of a and b.

Step 4: Add the values of a and b and store the result in the sum variable, i.e.,  $sum = a + b$ .

Step 5: Print sum

Step 6: Stop

## Factors of an Algorithm

**The following are the factors that we need to consider for designing an algorithm:**

**Modularity:** If any problem is given and we can break that problem into small-small modules or small-small steps, which is a basic definition of an algorithm, it means that this feature has been perfectly designed for the algorithm.

**Correctness:** The correctness of an algorithm is defined as when the given inputs produce the desired output, which means that the algorithm has been designed algorithm. The analysis of an algorithm has been done correctly.

**Maintainability:** Here, maintainability means that the algorithm should be designed in a very simple structured way so that when we redefine the algorithm, no major change will be done in the algorithm.

**Functionality:** It considers various logical steps to solve the real-world problem.

**Robustness:** Robustness means that how an algorithm can clearly define our problem.

**User-friendly:** If the algorithm is not user-friendly, then the designer will not be able to explain it to the programmer.

**Simplicity:** If the algorithm is simple then it is easy to understand.

**Extensibility:** If any other algorithm designer or programmer wants to use your algorithm then it should be extensible.

## Importance of Algorithms

**Theoretical importance:** When any real-world problem is given to us and we break the problem into small-small modules. To break down the problem, we should know all the theoretical aspects.

**Practical importance:** As we know that theory cannot be completed without the practical implementation. So, the importance of algorithm can be considered as both theoretical and practical.

## Issues of Algorithms

**The following are the issues that come while designing an algorithm:**

**How to design algorithms:** As we know that an algorithm is a step-by-step procedure so we must follow some steps to design an algorithm.

**How to analyze algorithm efficiency**

## Approaches of Algorithm

**The following are the approaches used after considering both the theoretical and practical importance of designing an algorithm:**

**Brute force algorithm:** The general logic structure is applied to design an algorithm. It is also known as an exhaustive search algorithm that searches all the possibilities to provide the required solution. Such algorithms are of two types:

**Optimizing:** Finding all the solutions of a problem and then take out the best solution or if the value of the best solution is known then it will terminate if the best solution is known.

**Sacrificing:** As soon as the best solution is found, then it will stop.

**Divide and conquer:** It is a very implementation of an algorithm. It allows you to design an algorithm in a step-by-step variation. It breaks down the algorithm to solve the problem in different methods. It allows you to break down the problem into different methods, and valid output is produced for the valid input. This valid output is passed to some other function.

**Greedy algorithm:** It is an algorithm paradigm that makes an optimal choice on each iteration with the hope of getting the best solution. It is easy to implement and has a faster execution time. But, there are very rare cases in which it provides the optimal solution.

**Dynamic programming:** It makes the algorithm more efficient by storing the intermediate results. It follows five different steps to find the optimal solution for the problem:

It breaks down the problem into a subproblem to find the optimal solution.

After breaking down the problem, it finds the optimal solution out of these subproblems.

Stores the result of the subproblems is known as memorization.

Reuse the result so that it cannot be recomputed for the same subproblems.

Finally, it computes the result of the complex program.

**Branch and Bound Algorithm:** The branch and bound algorithm can be applied to only integer programming problems. This approach divides all the sets of feasible solutions into smaller subsets. These subsets are further evaluated to find the best solution.

**Randomized Algorithm:** As we have seen in a regular algorithm, we have predefined input and required output. Those algorithms that have some defined set of inputs and required output, and follow some described steps are known as deterministic algorithms. What happens that when the random variable is introduced in the randomized algorithm?. In a randomized algorithm, some random bits are introduced by the algorithm and added in the input to produce the output, which is random in nature. Randomized algorithms are simpler and efficient than the deterministic algorithm.

**Backtracking:** Backtracking is an algorithmic technique that solves the problem recursively and removes the solution if it does not satisfy the constraints of a problem. The major categories of algorithms are given below:

**Sort:** Algorithm developed for sorting the items in a certain order.

**Search:** Algorithm developed for searching the items inside a data structure.

**Delete:** Algorithm developed for deleting the existing element from the data structure.

**Insert:** Algorithm developed for inserting an item inside a data structure.

**Update:** Algorithm developed for updating the existing element inside a data structure.

## Algorithm Analysis

The algorithm can be analyzed in two levels, i.e., first is before creating the algorithm, and second is after creating the algorithm. The following are the two analysis of an algorithm:

**Priori Analysis:** Here, priori analysis is the theoretical analysis of an algorithm which is done before implementing the algorithm. Various factors can be considered before implementing the algorithm like processor speed, which has no effect on the implementation part.

**Posterior Analysis:** Here, posterior analysis is a practical analysis of an algorithm. The practical analysis is achieved by implementing the algorithm using any programming language. This analysis basically evaluate that how much running time and space taken by the algorithm.

## Algorithm Complexity

The performance of the algorithm can be measured in two factors:

**Time complexity:** The time complexity of an algorithm is the amount of time required to complete the execution. The time complexity of an algorithm is denoted by the big O notation. Here, big O notation is the asymptotic notation to represent the time complexity. The time complexity is mainly calculated by counting the number of steps to finish the execution. Let's understand the time complexity through an example.

```
sum=0;  
// Suppose we have to calculate the sum of n numbers.  
for i=1 to n  
sum=sum+i;  
// when the loop ends then sum holds the sum of the n numbers  
return sum;
```

In the above code, the time complexity of the loop statement will be atleast n, and if the value of n increases, then the time complexity also increases. While the complexity of the code, i.e., return sum will be constant as its value is not dependent on the value of n and will provide the result in one step only. We generally consider the worst-time complexity as it is the maximum time taken for any given input size.

**Space complexity:** An algorithm's space complexity is the amount of space required to solve a problem and produce an output. Similar to the time complexity, space complexity is also expressed in big O notation.

For an algorithm, the space is required for the following purposes:

- To store program instructions

- To store constant values

- To store variable values

- To track the function calls, jumping statements, etc.

**Auxiliary space:** The extra space required by the algorithm, excluding the input size, is known as an auxiliary space. The space complexity considers both the spaces, i.e., auxiliary space, and space used by the input.

So,

**Space complexity = Auxiliary space + Input size.**

## Types of Algorithms

**The following are the types of algorithm:**

**Search Algorithm**

**Sort Algorithm**

**Search Algorithm**

On each day, we search for something in our day to day life. Similarly, with the case of computer, huge data is stored in a computer that whenever the user asks for any data then the computer searches for that data in the memory and provides that data to the user. There are mainly two techniques available to search the data in an array:

**Linear search**

**Binary search**

**Linear Search**

Linear search is a very simple algorithm that starts searching for an element or a value from the beginning of an array until the required element is not found. It compares the element to be searched with all the elements in an array, if the match is found, then it returns the index of the element else it returns -1. This algorithm can be implemented on the unsorted list.

**Binary Search**

A Binary algorithm is the simplest algorithm that searches the element very quickly. It is used to search the element from the sorted list. The elements must be stored in sequential order or the sorted manner to implement the binary algorithm. Binary search cannot be implemented if the elements are stored in a random manner. It is used to find the middle element of the list.

## Sorting Algorithms

Sorting algorithms are used to rearrange the elements in an array or a given data structure either in an ascending or descending order. The comparison operator decides the new order of the elements.

## Why do we need a sorting algorithm?

An efficient sorting algorithm is required for optimizing the efficiency of other algorithms like binary search algorithm as a binary search algorithm requires an array to be sorted in a particular order, mainly in ascending order.

It produces information in a sorted order, which is a human-readable format.

Searching a particular element in a sorted list is faster than the unsorted list.

## Asymptotic Analysis



As we know that data structure is a way of organizing the data efficiently and that efficiency is measured either in terms of time or space. So, the ideal data structure is a structure that occupies the least possible time to perform all its operation and the memory space. Our focus would be on finding the time complexity rather than space complexity, and by finding the time complexity, we can decide which data structure is the best for an algorithm.

The main question arises in our mind that on what basis should we compare the time complexity of data structures?. The time complexity can be compared based on operations performed on them. Let's consider a simple example.

Suppose we have an array of 100 elements, and we want to insert a new element at the beginning of the array. This becomes a very tedious task as we first need to shift the elements towards the right, and we will add new element at the starting of the array.

Suppose we consider the linked list as a data structure to add the element at the beginning. The linked list contains two parts, i.e., data and address of the next node. We simply add the address of the first node in the new node, and head pointer will now point to the newly added node. Therefore, we conclude that adding the data at the beginning of the linked list is faster than the arrays. In this way, we can compare the data structures and select the best possible data structure for performing the operations.

## How to find the Time Complexity or running time for performing the operations?

The measuring of the actual running time is not practical at all. The running time to perform any operation depends on the size of the input. Let's understand this statement through a simple example.

Suppose we have an array of five elements, and we want to add a new element at the beginning of the array. To achieve this, we need to shift each element towards right, and suppose each element takes one unit of time. There are five elements, so five units of time would be taken. Suppose there are 1000 elements in an array, then it takes 1000 units of time to shift. It concludes that time complexity depends upon the input size. Therefore, if the input size is  $n$ , then  $f(n)$  is a function of  $n$  that denotes the time complexity.

## How to calculate $f(n)$ ?

Calculating the value of  $f(n)$  for smaller programs is easy but for bigger programs, it's not that easy. We can compare the data structures by comparing their  $f(n)$  values. We can compare the data structures by comparing their  $f(n)$  values. We will find the growth rate of  $f(n)$  because there might be a possibility that one data structure for a smaller input size is better than the other one but not for the larger sizes. Now, how to find  $f(n)$ . Let's look at a simple example.

$$f(n) = 5n^2 + 6n + 12$$

where  $n$  is the number of instructions executed, and it depends on the size of the input.



When  $n=1$

$$\% \text{ of running time due to } 5n^2 = \frac{5}{5+6+12} * 100 = 21.74\%$$

$$\% \text{ of running time due to } 6n = \frac{6}{5+6+12} * 100 = 26.09\%$$

$$\% \text{ of running time due to } 12 = \frac{12}{5+6+12} * 100 = 52.17\%$$

From the above calculation, it is observed that most of the time is taken by 12. But, we have to find the growth rate of  $f(n)$ , we cannot say that the maximum amount of time is taken by 12. Let's assume the different values of  $n$  to find the growth rate of  $f(n)$ .

| $n$  | $5n^2$ | $6n$   | 12      |
|------|--------|--------|---------|
| 1    | 21.74% | 26.09% | 52.17%  |
| 10   | 87.41% | 10.49% | 2.09%   |
| 100  | 98.79% | 1.19%  | 0.02%   |
| 1000 | 99.88% | 0.12%  | 0.0002% |

As we can observe in the above table that with the increase in the value of  $n$ , the running time of  $5n^2$  increases while the running time of  $6n$  and 12 also decreases. Therefore, it is observed that for larger values of  $n$ , the squared term consumes almost 99% of the time. As the  $n^2$  term is contributing most of the time, so we can eliminate the rest two terms.

**Therefore,**

$$f(n) = 5n^2$$

Here, we are getting the approximate time complexity whose result is very close to the actual result. And this approximate measure of time complexity is known as an Asymptotic complexity. Here, we are not calculating the exact running time, we are eliminating the unnecessary terms, and we are just considering the term which is taking most of the time.

In mathematical analysis, asymptotic analysis of algorithm is a method of defining the mathematical boundation of its run-time performance. Using the asymptotic analysis, we can easily conclude the average-case, best-case and worst-case scenario of an algorithm.

It is used to mathematically calculate the running time of any operation inside an algorithm.

**Example:** Running time of one operation is  $x(n)$  and for another operation, it is calculated as  $f(n^2)$ . It refers to running time will increase linearly with an increase in ' $n$ ' for the first operation, and running time will increase exponentially for the second

operation. Similarly, the running time of both operations will be the same if  $n$  is significantly small.

Usually, the time required by an algorithm comes under three types:

**Worst case:** It defines the input for which the algorithm takes a huge time.

**Average case:** It takes average time for the program execution.

**Best case:** It defines the input for which the algorithm takes the lowest time

## Asymptotic Notations

The commonly used asymptotic notations used for calculating the running time complexity of an algorithm is given below:

Big oh Notation ( $O$ )

Omega Notation ( $\Omega$ )

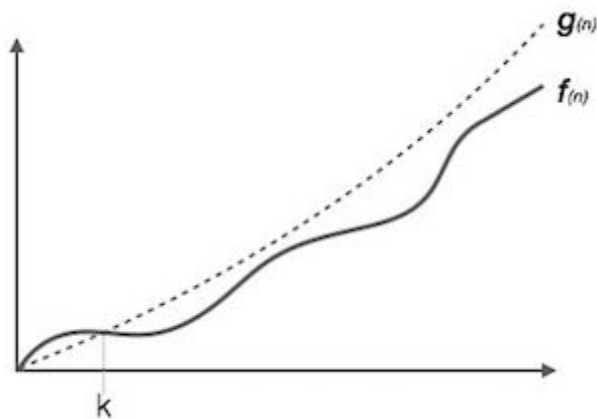
Theta Notation ( $\Theta$ )

### Big oh Notation ( $O$ )

Big  $O$  notation is an asymptotic notation that measures the performance of an algorithm by simply providing the order of growth of the function.

This notation provides an upper bound on a function which ensures that the function never grows faster than the upper bound. So, it gives the least upper bound on a function so that the function never grows faster than this upper bound.

It is the formal way to express the upper boundary of an algorithm running time. It measures the worst case of time complexity or the algorithm's longest amount of time to complete its operation. It is represented as shown below:



**For example:**

If  $f(n)$  and  $g(n)$  are the two functions defined for positive integers, then  $f(n) = O(g(n))$  as  $f(n)$  is big oh of  $g(n)$  or  $f(n)$  is on the order of  $g(n)$  if there exists constants  $c$  and  $n_0$  such that:

$$f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

This implies that  $f(n)$  does not grow faster than  $g(n)$ , or  $g(n)$  is an upper bound on the function  $f(n)$ . In this case, we are calculating the growth rate of the function which

eventually calculates the worst time complexity of a function, i.e., how worst an algorithm can perform.

### Let's understand through examples

Example 1:  $f(n)=2n+3$  ,  $g(n)=n$

Now, we have to find **Is  $f(n)=O(g(n))$ ?**

To check  $f(n)=O(g(n))$ , it must satisfy the given condition:

$$f(n) \leq c \cdot g(n)$$

First, we will replace  $f(n)$  by  $2n+3$  and  $g(n)$  by  $n$ .

$$2n+3 \leq c \cdot n$$

Let's assume  $c=5$ ,  $n=1$  then

$$2 \cdot 1 + 3 \leq 5 \cdot 1$$

$$5 \leq 5$$

For  $n=1$ , the above condition is true.

If  $n=2$

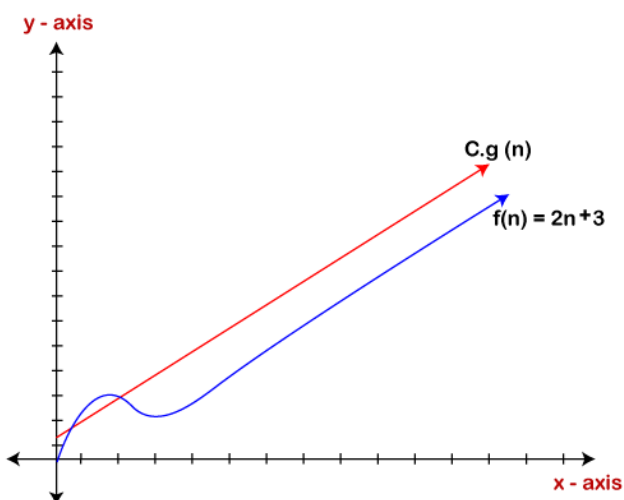
$$2 \cdot 2 + 3 \leq 5 \cdot 2$$

$$7 \leq 10$$

For  $n=2$ , the above condition is true.

We know that for any value of  $n$ , it will satisfy the above condition, i.e.,  $2n+3 \leq c \cdot n$ . If the value of  $c$  is equal to 5, then it will satisfy the condition  $2n+3 \leq c \cdot n$ . We can take any value of  $n$  starting from 1, it will always satisfy. Therefore, we can say that for some constants  $c$  and for some constants  $n_0$ , it will always satisfy  $2n+3 \leq c \cdot n$ . As it is satisfying the above condition, so  $f(n)$  is big oh of  $g(n)$  or we can say that  $f(n)$  grows linearly.

Therefore, it concludes that  $c \cdot g(n)$  is the upper bound of the  $f(n)$ . It can be represented graphically as:



The idea of using big o notation is to give an upper bound of a particular function, and eventually it leads to give a worst-time complexity. It provides an assurance that a particular function does not behave suddenly as a quadratic or a cubic fashion, it just behaves in a linear manner in a worst-case.

## Omega Notation ( $\Omega$ )

It basically describes the best-case scenario which is opposite to the big o notation. It is the formal way to represent the lower bound of an algorithm's running time. It measures the best amount of time an algorithm can possibly take to complete or the best-case time complexity.

It determines what is the fastest time that an algorithm can run.

If we required that an algorithm takes at least certain amount of time without using an upper bound, we use big-  $\Omega$  notation i.e. the Greek letter "omega". It is used to bound the growth of running time for large input size.

If  **$f(n)$**  and  **$g(n)$**  are the two functions defined for positive integers, then  **$f(n) = \Omega(g(n))$**  as  **$f(n)$  is Omega of  $g(n)$**  or  $f(n)$  is on the order of  $g(n)$  if there exists constants  $c$  and  $n_0$  such that:

**$f(n) \geq c \cdot g(n)$  for all  $n \geq n_0$  and  $c > 0$**

**Let's consider a simple example.**

If  $f(n) = 2n+3$ ,  $g(n) = n$ ,

Is  $f(n) = \Omega(g(n))$ ?

It must satisfy the condition:

**$f(n) \geq c \cdot g(n)$**

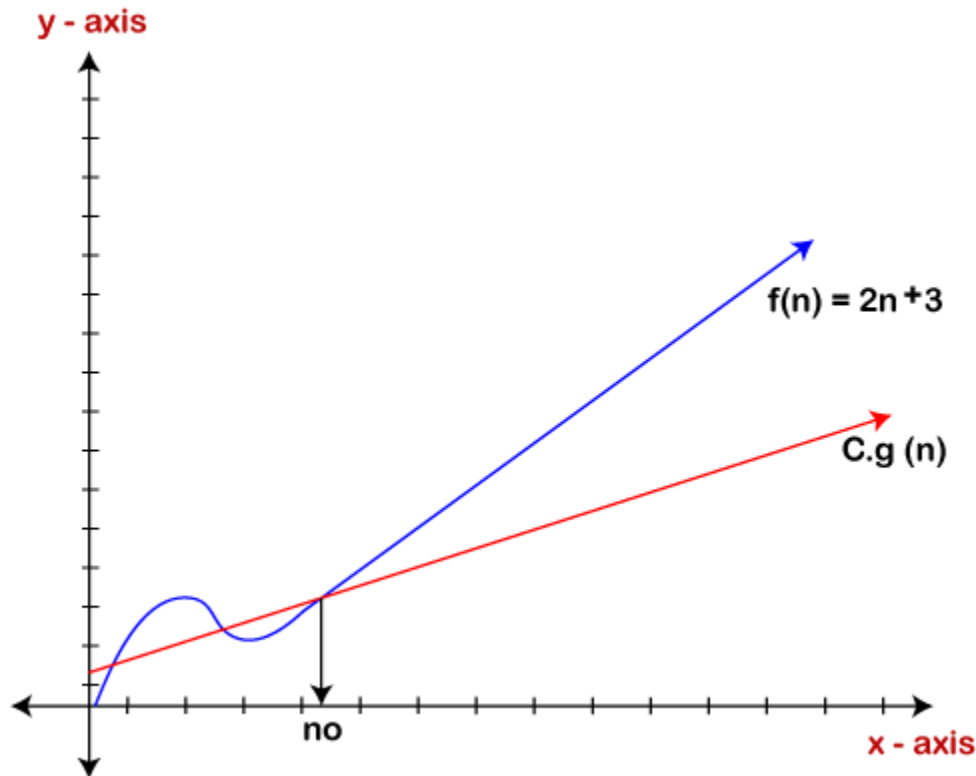
To check the above condition, we first replace  $f(n)$  by  $2n+3$  and  $g(n)$  by  $n$ .

**$2n+3 \geq c \cdot n$**

Suppose  $c=1$

**$2n+3 \geq n$**  (This equation will be true for any value of  $n$  starting from 1).

Therefore, it is proved that  $g(n)$  is big omega of  $2n+3$  function.



As we can see in the above figure that  $g(n)$  function is the lower bound of the  $f(n)$  function when the value of  $c$  is equal to 1. Therefore, this notation gives the fastest running time. But, we are not more interested in finding the fastest running time, we are interested in calculating the worst-case scenarios because we want to check our algorithm for larger input that what is the worst time that it will take so that we can take further decision in the further process.

### Theta Notation ( $\theta$ )

The theta notation mainly describes the average case scenarios.

It represents the realistic time complexity of an algorithm. Every time, an algorithm does not perform worst or best, in real-world problems, algorithms mainly fluctuate between the worst-case and best-case, and this gives us the average case of the algorithm.

Big theta is mainly used when the value of worst-case and the best-case is same.

It is the formal way to express both the upper bound and lower bound of an algorithm running time.

Let's understand the big theta notation mathematically:

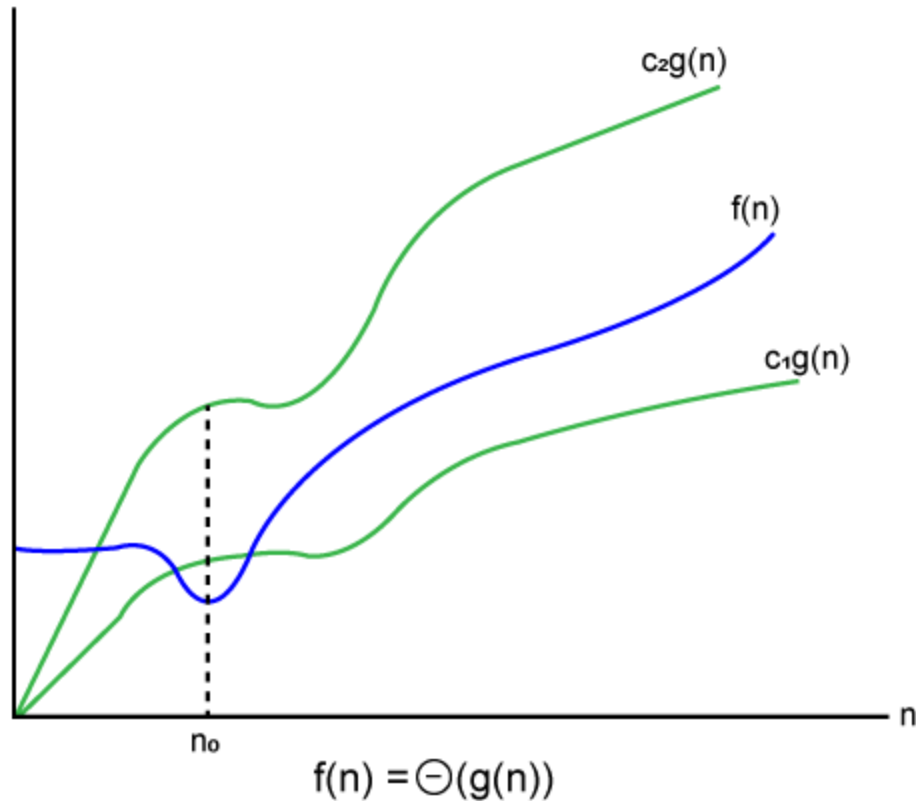
Let  $f(n)$  and  $g(n)$  be the functions of  $n$  where  $n$  is the steps required to execute the program then:

$$f(n) = \theta g(n)$$

The above condition is satisfied only if when

$$c1.g(n) < f(n) \leq c2.g(n)$$

where the function is bounded by two limits, i.e., upper and lower limit, and  $f(n)$  comes in between. The condition  $f(n) = \Theta(g(n))$  will be true if and only if  $c_1.g(n)$  is less than or equal to  $f(n)$  and  $c_2.g(n)$  is greater than or equal to  $f(n)$ . The graphical representation of theta notation is given below:



Let's consider the same example where

$$f(n) = 2n + 3$$

$$g(n) = n$$

As  $c_1.g(n)$  should be less than  $f(n)$  so  $c_1$  has to be 1 whereas  $c_2.g(n)$  should be greater than  $f(n)$  so  $c_2$  is equal to 5. The  $c_1.g(n)$  is the lower limit of the  $f(n)$  while  $c_2.g(n)$  is the upper limit of the  $f(n)$ .

$$c_1.g(n) \leq f(n) \leq c_2.g(n)$$

Replace  $g(n)$  by  $n$  and  $f(n)$  by  $2n + 3$

$$c_1.n \leq 2n + 3 \leq c_2.n$$

$$\text{if } c_1 = 1, c_2 = 2, n = 1$$

$$1 * 1 \leq 2 * 1 + 3 \leq 2 * 1$$

$$1 \leq 5 \leq 2 \text{ // for } n = 1, \text{ it satisfies the condition } c_1.g(n) \leq f(n) \leq c_2.g(n)$$

**If  $n = 2$**

$$1 * 2 \leq 2 * 2 + 3 \leq 2 * 2$$

$$2 \leq 7 \leq 4 \text{ // for } n = 2, \text{ it satisfies the condition } c_1.g(n) \leq f(n) \leq c_2.g(n)$$

Therefore, we can say that for any value of  $n$ , it satisfies the condition  $c_1.g(n) \leq f(n) \leq c_2.g(n)$ . Hence, it is proved that  $f(n)$  is big theta of  $g(n)$ . So, this is the average-case scenario which provides the realistic time complexity.

### Why we have three different asymptotic analysis?

As we know that big omega is for the best case, big oh is for the worst case while big theta is for the average case. Now, we will find out the average, worst and the best case of the linear search algorithm.

Suppose we have an array of  $n$  numbers, and we want to find the particular element in an array using the linear search. In the linear search, every element is compared with the searched element on each iteration. Suppose, if the match is found in a first iteration only, then the best case would be  $\Omega(1)$ , if the element matches with the last element, i.e.,  $n$ th element of the array then the worst case would be  $O(n)$ . The average case is the mid of the best and the worst-case, so it becomes  **$\theta(n/1)$ . The constant terms can be ignored in the time complexity so average case would be  $\theta(n)$ .**

So, three different analysis provide the proper bounding between the actual functions. Here, bounding means that we have upper as well as lower limit which assures that the algorithm will behave between these limits only, i.e., it will not go beyond these limits.

### Common Asymptotic Notations

|             |   |                    |
|-------------|---|--------------------|
| constant    | - | $\theta(1)$        |
| linear      | - | $\theta(n)$        |
| logarithmic | - | $\theta(\log n)$   |
| $n \log n$  | - | $\theta(n \log n)$ |
| exponential | - | $2^{\theta(n)}$    |
| cubic       | - | $\theta(n^3)$      |
| polynomial  | - | $n^{\theta(1)}$    |
| quadratic   | - | $\theta(n^2)$      |

## Pointer

Pointer is used to points the address of the value stored anywhere in the computer memory. To obtain the value stored at the location is known as dereferencing the pointer. Pointer improves the performance for repetitive process such as:

Traversing String

Lookup Tables

Control Tables

## Tree Structures

Pointer Improves Performance for



## Pointer Details

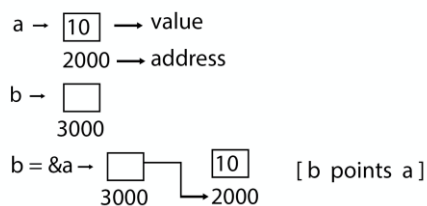
**Pointer arithmetic:** There are four arithmetic operators that can be used in pointers: ++, --, +, -

**Array of pointers:** You can define arrays to hold a number of pointers.

**Pointer to pointer:** C allows you to have pointer on a pointer and so on.

**Passing pointers to functions in C:** Passing an argument by reference or by address enable the passed argument to be changed in the calling function by the called function.

**Return pointer from functions in C:** C allows a function to return a pointer to the local variable, static variable and dynamically allocated memory as well.



## Program

### Pointer

```
#include <stdio.h>
```

```
int main( )
```

```
{
```

```
int a = 5;
```

```
int *b;
```

```
b = &a;
```

```
printf ("value of a = %d\n", a);
```

```
printf ("value of a = %d\n", *(&a));
```

```
printf ("value of a = %d\n", *b);
```

```
printf ("address of a = %u\n", &a);
```

```
printf ("address of a = %d\n", b);
```

```
printf ("address of b = %u\n", &b);
```



```
printf ("value of b = address of a = %u", b);  
return 0;  
}
```

## Output

```
value of a = 5  
value of a = 5  
address of a = 3010494292  
address of a = -1284473004  
address of b = 3010494296  
value of b = address of a = 3010494292
```

## Program

### Pointer to Pointer

```
#include <stdio.h>
```

```
int main( )  
{  
    int a = 5;  
    int *b;  
    int **c;  
    b = &a;  
    c = &b;  
    printf ("value of a = %d\n", a);  
    printf ("value of a = %d\n", *(&a));  
    printf ("value of a = %d\n", *b);  
    printf ("value of a = %d\n", **c);  
    printf ("value of b = address of a = %u\n", b);  
    printf ("value of c = address of b = %u\n", c);  
    printf ("address of a = %u\n", &a);  
    printf ("address of a = %u\n", b);  
    printf ("address of a = %u\n", *c);  
    printf ("address of b = %u\n", &b);  
    printf ("address of b = %u\n", c);  
    printf ("address of c = %u\n", &c);  
    return 0;  
}
```

### Pointer to Pointer

```
value of a = 5  
value of a = 5  
value of a = 5  
value of a = 5  
value of b = address of a = 2831685116  
value of c = address of b = 2831685120
```

address of a = 2831685116  
address of a = 2831685116  
address of a = 2831685116  
address of b = 2831685120  
address of b = 2831685120  
address of c = 2831685128

## Structure

A structure is a composite data type that defines a grouped list of variables that are to be placed under one name in a block of memory. It allows different variables to be accessed by using a single pointer to the structure.

### Syntax

```
struct structure_name
{
    data_type member1;
    data_type member2;
    .
    .
    data_type member;
};
```

### Advantages

It can hold variables of different data types.

We can create objects containing different types of attributes.

It allows us to re-use the data layout across programs.

It is used to implement other data structures like linked lists, stacks, queues, trees, graphs etc.

### Program

```
#include<stdio.h>
#include<conio.h>
void main( )
{
    struct employee
    {
        int id ;
        float salary ;
        int mobile ;
    };
    struct employee e1,e2,e3 ;
    clrscr();
    printf ("\nEnter ids, salary & mobile no. of 3 employee\n")
```

```

scanf ("%d %f %d", &e1.id, &e1.salary, &e1.mobile);
scanf ("%d%f %d", &e2.id, &e2.salary, &e2.mobile);
scanf ("%d %f %d", &e3.id, &e3.salary, &e3.mobile);
printf ("\n Entered Result ");
printf ("\n%d %f %d", e1.id, e1.salary, e1.mobile);
printf ("\n%d%f %d", e2.id, e2.salary, e2.mobile);
printf ("\n%d %f %d", e3.id, e3.salary, e3.mobile);
getch();
}

```

## Array in Data Structure

In this article, we will discuss the array in data structure. Arrays are defined as the collection of similar types of data items stored at contiguous memory locations. It is one of the simplest data structures where each data element can be randomly accessed by using its index number.

In C programming, they are the derived data types that can store the primitive type of data such as int, char, double, float, etc. For example, if we want to store the marks of a student in 6 subjects, then we don't need to define a different variable for the marks in different subjects. Instead, we can define an array that can store the marks in each subject at the contiguous memory locations.

### Properties of array

There are some of the properties of an array that are listed as follows -

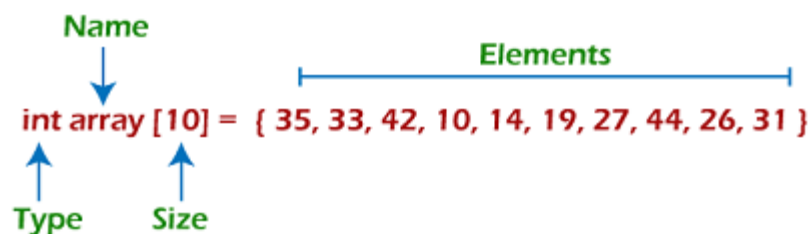
Each element in an array is of the same data type and carries the same size that is 4 bytes.

Elements in the array are stored at contiguous memory locations from which the first element is stored at the smallest memory location.

Elements of the array can be randomly accessed since we can calculate the address of each element of the array with the given base address and the size of the data element.

### Representation of an array

We can represent an array in various ways in different programming languages. As an illustration, let's see the declaration of array in C language -



As per the above illustration, there are some of the following important points -  
Index starts with 0.

The array's length is 10, which means we can store 10 elements.  
Each element in the array can be accessed via its index.

## Why are arrays required?

Arrays are useful because -

Sorting and searching a value in an array is easier.

Arrays are best to process multiple values quickly and easily.

**Arrays are good for storing multiple values in a single variable** - In computer programming, most cases require storing a large number of data of a similar type. To store such an amount of data, we need to define a large number of variables. It would be very difficult to remember the names of all the variables while writing the programs. Instead of naming all the variables with a different name, it is better to define an array and store all the elements into it.

## Memory allocation of an array

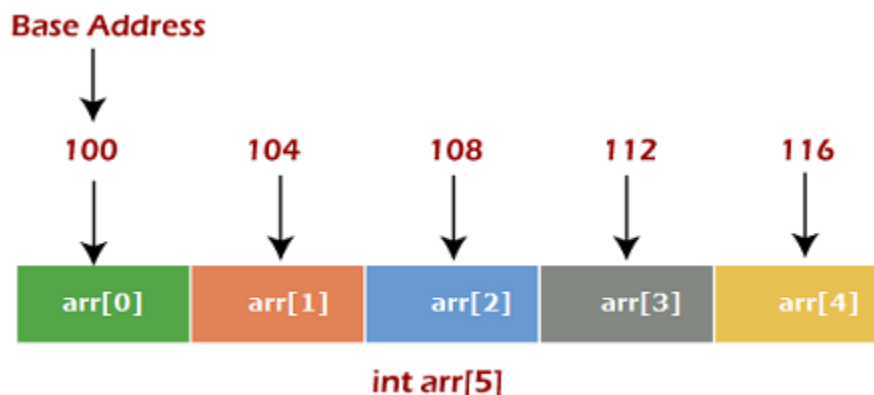
As stated above, all the data elements of an array are stored at contiguous locations in the main memory. The name of the array represents the base address or the address of the first element in the main memory. Each element of the array is represented by proper indexing.

We can define the indexing of an array in the below ways -

0 (zero-based indexing): The first element of the array will be `arr[0]`.

1 (one-based indexing): The first element of the array will be `arr[1]`.

n (n - based indexing): The first element of the array can reside at any random index number.



In the above image, we have shown the memory allocation of an array `arr` of size 5. The array follows a 0-based indexing approach. The base address of the array is 100 bytes. It is the address of `arr[0]`. Here, the size of the data type used is 4 bytes; therefore, each element will take 4 bytes in the memory.

## How to access an element from the array?

We required the information given below to access any random element from the array

-

Base Address of the array.

Size of an element in bytes.

Type of indexing, array follows.

The formula to calculate the address to access an array element -

Byte address of element  $A[i] = \text{base address} + \text{size} * (i - \text{first index})$

Here, size represents the memory taken by the primitive data types. As an instance, **int** takes 2 bytes, **float** takes 4 bytes of memory space in C programming.

We can understand it with the help of an example -

Suppose an array,  $A[-10 \dots +2]$  having Base address (BA) = 999 and size of an element = 2 bytes, find the location of  $A[-1]$ .

$L(A[-1]) = 999 + 2 \times [(-1) - (-10)]$

$= 999 + 18$

$= 1017$

## Basic operations

Now, let's discuss the basic operations supported in the array -

Traversal - This operation is used to print the elements of the array.

Insertion - It is used to add an element at a particular index.

Deletion - It is used to delete an element from a particular index.

Search - It is used to search an element using the given index or by the value.

Update - It updates an element at a particular index.

## Traversal operation

This operation is performed to traverse through the array elements. It prints all array elements one after another. We can understand it with the below program -

```
#include <stdio.h>
```

```
void main() {  
    int Arr[5] = {18, 30, 15, 70, 12};  
    int i;  
    printf("Elements of the array are:\n");  
    for(i = 0; i < 5; i++) {  
        printf("Arr[%d] = %d, ", i, Arr[i]);  
    }  
}
```

### Output

```
Elements of the array are:  
Arr[0] = 18, Arr[1] = 30, Arr[2] = 15, Arr[3] = 70, Arr[4] = 12,
```

## Insertion operation

This operation is performed to insert one or more elements into the array. As per the requirements, an element can be added at the beginning, end, or at any index of the array. Now, let's see the implementation of inserting an element into the array.

```
#include <stdio.h>
```

```

int main()
{
    int arr[20] = { 18, 30, 15, 70, 12 };
    int i, x, pos, n = 5;
    printf("Array elements before insertion\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");

    x = 50; // element to be inserted
    pos = 4;
    n++;

    for (i = n-1; i >= pos; i--)
        arr[i] = arr[i - 1];
    arr[pos - 1] = x;
    printf("Array elements after insertion\n");
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
    return 0;
}

```

### Output

```

Array elements before insertion
18 30 15 70 12
Array elements after insertion
18 30 15 50 70 12

```

## Deletion operation

As the name implies, this operation removes an element from the array and then reorganizes all of the array elements.

```
#include <stdio.h>
```

```

void main() {
    int arr[] = {18, 30, 15, 70, 12};
    int k = 30, n = 5;
    int i, j;

    printf("Given array elements are :\n");

    for(i = 0; i < n; i++) {

```

```

    printf("arr[%d] = %d, ", i, arr[i]);
}

j = k;

while(j < n) {
    arr[j-1] = arr[j];
    j = j + 1;
}

n = n -1;

printf("\nElements of array after deletion:\n");

for(i = 0; i<n; i++) {
    printf("arr[%d] = %d, ", i, arr[i]);
}
}

```

### Output

```

Given array elements are :
arr[0] = 18, arr[1] = 30, arr[2] = 15, arr[3] = 70, arr[4] = 12,
Elements of array after deletion:
arr[0] = 18, arr[1] = 30, arr[2] = 15, arr[3] = 70,

```

## Search operation

This operation is performed to search an element in the array based on the value or index.

```
#include <stdio.h>
```

```

void main() {
    int arr[5] = {18, 30, 15, 70, 12};
    int item = 70, i, j=0 ;

    printf("Given array elements are :\n");

    for(i = 0; i<5; i++) {
        printf("arr[%d] = %d, ", i, arr[i]);
    }
    printf("\nElement to be searched = %d", item);
    while(j < 5){
        if( arr[j] == item ) {

```

```

        break;
    }

    j = j + 1;
}

printf("\nElement %d is found at %d position", item, j+1);
}

```

### Output

```

Given array elements are :
arr[0] = 18, arr[1] = 30, arr[2] = 15, arr[3] = 70, arr[4] = 12,
Element to be searched = 70
Element 70 is found at 4 position

```

## Update operation

This operation is performed to update an existing array element located at the given index.

```
#include <stdio.h>
```

```

void main() {
    int arr[5] = {18, 30, 15, 70, 12};
    int item = 50, i, pos = 3;

    printf("Given array elements are :\n");

    for(i = 0; i<5; i++) {
        printf("arr[%d] = %d, ", i, arr[i]);
    }

    arr[pos-1] = item;
    printf("\nArray elements after updation :\n");

    for(i = 0; i<5; i++) {
        printf("arr[%d] = %d, ", i, arr[i]);
    }
}

```

### Output

```

Given array elements are :
arr[0] = 18, arr[1] = 30, arr[2] = 15, arr[3] = 70, arr[4] = 12,
Array elements after updation :
arr[0] = 18, arr[1] = 30, arr[2] = 50, arr[3] = 70, arr[4] = 12,

```

## Complexity of Array operations



Time and space complexity of various array operations are described in the following table.

### Time Complexity

| Operation | Average Case | Worst Case |
|-----------|--------------|------------|
| Access    | $O(1)$       | $O(1)$     |
| Search    | $O(n)$       | $O(n)$     |
| Insertion | $O(n)$       | $O(n)$     |
| Deletion  | $O(n)$       | $O(n)$     |

### Space Complexity

In array, space complexity for worst case is  $O(n)$ .

## Advantages of Array

Array provides the single name for the group of variables of the same type. Therefore, it is easy to remember the name of all the elements of an array.

Traversing an array is a very simple process; we just need to increment the base address of the array in order to visit each element one by one.

Any element in the array can be directly accessed by using the index.

## Disadvantages of Array

Array is homogenous. It means that the elements with similar data type can be stored in it.

In array, there is static memory allocation that is size of an array cannot be altered.

There will be wastage of memory if we store less number of elements than the declared size.

## Conclusion

In this article, we have discussed the special data structure, i.e., array, and the basic operations performed on it. Arrays provide a unique way to structure the stored data such that it can be easily accessed and can be queried to fetch the value using the index.

## 2D Array

2D array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns.

However, 2D arrays are created to implement a relational database look alike data structure. It provides ease of holding bulk of data at once which can be passed to any number of functions wherever required.

## How to declare 2D Array

The syntax of declaring two dimensional array is very much similar to that of a one dimensional array, given as follows.

```
int arr[max_rows][max_columns];
```

however, It produces the data structure which looks like following.

|     | 0         | 1         | 2         | .... | n-1         |
|-----|-----------|-----------|-----------|------|-------------|
| 0   | a[0][0]   | a[0][1]   | a[0][2]   | .... | a[0][n-1]   |
| 1   | a[1][0]   | a[1][1]   | a[1][2]   | .... | a[1][n-1]   |
| 2   | a[2][0]   | a[2][1]   | a[2][2]   | .... | a[2][n-1]   |
| 3   | a[3][0]   | a[3][1]   | a[3][2]   | .... | a[3][n-1]   |
| 4   | a[4][0]   | a[4][1]   | a[4][2]   | .... | a[4][n-1]   |
| .   | .         | .         | .         | .... | .           |
| .   | .         | .         | .         | .... | .           |
| n-1 | a[n-1][0] | a[n-1][1] | a[n-1][2] | .... | a[n-1][n-1] |

**a[n][n]**

Above image shows the two dimensional array, the elements are organized in the form of rows and columns. First element of the first row is represented by a[0][0] where the number shown in the first index is the number of that row while the number shown in the second index is the number of the column.

## How do we access data in a 2D array

Due to the fact that the elements of 2D arrays can be random accessed. Similar to one dimensional arrays, we can access the individual cells in a 2D array by using the indices of the cells. There are two indices attached to a particular cell, one is its row number while the other is its column number.

However, we can store the value stored in any particular cell of a 2D array to some variable x by using the following syntax.

```
int x = a[i][j];
```

where i and j is the row and column number of the cell respectively.

We can assign each cell of a 2D array to 0 by using the following code:

```
for ( int i=0; i<n; i++)
{
    for ( int j=0; j<n; j++)
    {
        a[i][j] = 0;
    }
}
```

```
}  
}
```

## Initializing 2D Arrays

We know that, when we declare and initialize one dimensional array in C programming simultaneously, we don't need to specify the size of the array. However this will not work with 2D arrays. We will have to define at least the second dimension of the array.

The syntax to declare and initialize the 2D array is given as follows.

```
int arr[2][2] = {0,1,2,3};
```

The number of elements that can be present in a 2D array will always be equal to **(number of rows \* number of columns)**.

**Example :** Storing User's data into a 2D array and printing it.

**C Example :**

```
#include <stdio.h>  
void main ()  
{  
    int arr[3][3],i,j;  
    for (i=0;i<3;i++)  
    {  
        for (j=0;j<3;j++)  
        {  
            printf("Enter a[%d][%d]: ",i,j);  
            scanf("%d",&arr[i][j]);  
        }  
    }  
    printf("\n printing the elements ....\n");  
    for(i=0;i<3;i++)  
    {  
        printf("\n");  
        for (j=0;j<3;j++)  
        {  
            printf("%d\t",arr[i][j]);  
        }  
    }  
}
```

## Java Example

```
import java.util.Scanner;  
publicclass TwoDArray {  
    publicstaticvoid main(String[] args) {  
        int[][] arr = newint[3][3];  
        Scanner sc = new Scanner(System.in);
```

```

for (inti =0;i<3;i++)
{
    for(intj=0;j<3;j++)
    {
        System.out.print("Enter Element");
        arr[i][j]=sc.nextInt();
        System.out.println();
    }
}
System.out.println("Printing Elements...");
for(inti=0;i<3;i++)
{
    System.out.println();
    for(intj=0;j<3;j++)
    {
        System.out.print(arr[i][j]+"\\t");
    }
}
}
}

```

## C# Example

using System;

```

public class Program
{
    public static void Main()
    {
        int[,] arr = new int[3,3];
        for (int i=0;i<3;i++)
        {
            for (int j=0;j<3;j++)
            {
                Console.WriteLine("Enter Element");
                arr[i,j]= Convert.ToInt32(Console.ReadLine());
            }
        }
        Console.WriteLine("Printing Elements...");
        for (int i=0;i<3;i++)
        {
            Console.WriteLine();

```

```

    for (int j=0;j<3;j++)
    {
        Console.Write(arr[i,j]+ " ");
    }
}
}

```

## Mapping 2D array to 1D array

When it comes to map a 2 dimensional array, most of us might think that why this mapping is required. However, 2 D arrays exists from the user point of view. 2D arrays are created to implement a relational database table lookalike data structure, in computer memory, the storage technique for 2D array is similar to that of an one dimensional array.

The size of a two dimensional array is equal to the multiplication of number of rows and the number of columns present in the array. We do need to map two dimensional array to the one dimensional array in order to store them in the memory.

A 3 X 3 two dimensional array is shown in the following image. However, this array needs to be mapped to a one dimensional array in order to store it into the memory.

|   | 0     | 1     | 2     |              |
|---|-------|-------|-------|--------------|
| 0 | (0,0) | (0,1) | (0,2) | Column Index |
| 1 | (1,0) | (1,1) | (1,2) |              |
| 2 | (2,0) | (2,1) | (2,2) |              |

Row Index

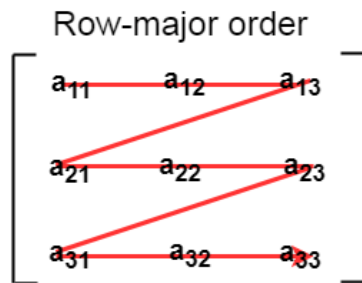
There are two main techniques of storing 2D array elements into memory

### 1. Row Major ordering

In row major ordering, all the rows of the 2D array are stored into the memory contiguously. Considering the array shown in the above image, its memory allocation according to row major order is shown as follows.

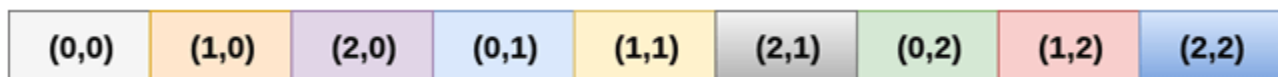
|       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| (0,0) | (0,1) | (0,2) | (1,0) | (1,1) | (1,2) | (2,0) | (2,1) | (2,2) |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|

first, the 1<sup>st</sup> row of the array is stored into the memory completely, then the 2<sup>nd</sup> row of the array is stored into the memory completely and so on till the last row.

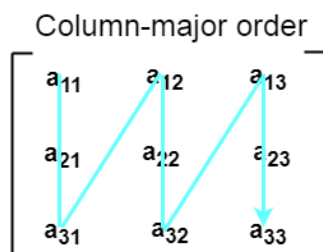


## 2. Column Major ordering

According to the column major ordering, all the columns of the 2D array are stored into the memory contiguously. The memory allocation of the array which is shown in the above image is given as follows.



first, the 1<sup>st</sup> column of the array is stored into the memory completely, then the 2<sup>nd</sup> row of the array is stored into the memory completely and so on till the last column of the array.



## Calculating the Address of the random element of a 2D array

Due to the fact that, there are two different techniques of storing the two dimensional array into the memory, there are two different formulas to calculate the address of a random element of the 2D array.

### By Row Major Order

If array is declared by  $a[m][n]$  where  $m$  is the number of rows while  $n$  is the number of columns, then address of an element  $a[i][j]$  of the array stored in row major order is calculated as,

$$\text{Address}(a[i][j]) = \text{B. A.} + (i * n + j) * \text{size}$$

where, B. A. is the base address or the address of the first element of the array  $a[0][0]$ .

**Example :**

a[10...30, 55...75], base address of the array (BA) = 0, size of an element = 4 bytes .  
Find the location of a[15][68].

$$\text{Address}(a[15][68]) = 0 + ((15 - 10) \times (68 - 55 + 1) + (68 - 55)) \times 4$$

$$\begin{aligned} &= (5 \times 14 + 13) \times 4 \\ &= 83 \times 4 \\ &= 332 \text{ answer} \end{aligned}$$

## By Column major order

If array is declared by a[m][n] where m is the number of rows while n is the number of columns, then address of an element a[i][j] of the array stored in row major order is calculated as,

$$\text{Address}(a[i][j]) = ((j \times m) + i) \times \text{Size} + \text{BA}$$

where BA is the base address of the array.

### Example:

A [-5 ... +20][20 ... 70], BA = 1020, Size of element = 8 bytes. Find the location of a[0][30].

$$\text{Address } [A[0][30]] = ((30-20) \times 24 + 5) \times 8 + 1020 = 245 \times 8 + 1020 = 2980 \text{ bytes}$$

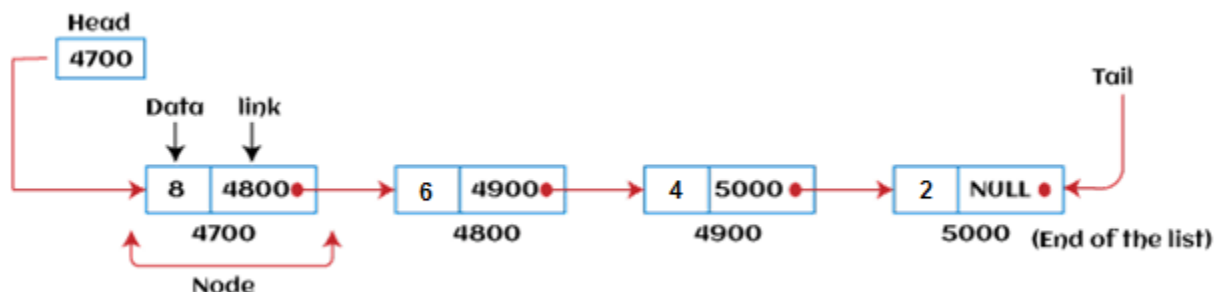
## Linked list

In this article, we will see the introduction of linked list.

Linked list is a linear data structure that includes a series of connected nodes. Linked list can be defined as the nodes that are randomly stored in the memory. A node in the linked list contains two parts, i.e., first is the data part and second is the address part. The last node of the list contains a pointer to the null. After array, linked list is the second most used data structure. In a linked list, every link contains a connection to another link.

## Representation of a Linked list

Linked list can be represented as the connection of nodes in which each node points to the next node of the list. The representation of the linked list is shown below -



Till now, we have been using array data structure to organize the group of elements that are to be stored individually in the memory. However, Array has several advantages and disadvantages that must be known to decide the data structure that will be used throughout the program. Now, the question arises why we should use linked list over array?

## Why use linked list over array?

Linked list is a data structure that overcomes the limitations of arrays. Let's first see some of the limitations of arrays -

The size of the array must be known in advance before using it in the program.

Increasing the size of the array is a time taking process. It is almost impossible to expand the size of the array at run time.

All the elements in the array need to be contiguously stored in the memory. Inserting an element in the array needs shifting of all its predecessors.

Linked list is useful because -

It allocates the memory dynamically. All the nodes of the linked list are non-contiguously stored in the memory and linked together with the help of pointers.

In linked list, size is no longer a problem since we do not need to define its size at the time of declaration. List grows as per the program's demand and limited to the available memory space.

## How to declare a linked list?

It is simple to declare an array, as it is of single type, while the declaration of linked list is a bit more typical than array. Linked list contains two parts, and both are of different types, i.e., one is the simple variable, while another is the pointer variable. We can declare the linked list by using the user-defined data type **structure**.

The declaration of linked list is given as follows -

```
struct node
{
    int data;
    struct node *next;
}
```

In the above declaration, we have defined a structure named as **node** that contains two variables, one is **data** that is of integer type, and another one is **next** that is a pointer which contains the address of next node.

Now, let's move towards the types of linked list.

## Types of Linked list

Linked list is classified into the following types -

**Singly-linked list** - Singly linked list can be defined as the collection of an ordered set of elements. A node in the singly linked list consists of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node, while the link part of the node stores the address of its immediate successor.

**Doubly linked list** - Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly-linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer), and pointer to the previous node (previous pointer).

**Circular singly linked list** - In a circular singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.

**Circular doubly linked list** - Circular doubly linked list is a more complex type of data structure in which a node contains pointers to its previous node as well as the next node. Circular doubly



linked list doesn't contain NULL in any of the nodes. The last node of the list contains the address of the first node of the list. The first node of the list also contains the address of the last node in its previous pointer.

Now, let's see the benefits and limitations of using the Linked list.

## Advantages of Linked list

The advantages of using the Linked list are given as follows -

**Dynamic data structure** - The size of the linked list may vary according to the requirements. Linked list does not have a fixed size.

**Insertion and deletion** - Unlike arrays, insertion, and deletion in linked list is easier. Array elements are stored in the consecutive location, whereas the elements in the linked list are stored at a random location. To insert or delete an element in an array, we have to shift the elements for creating the space. Whereas, in linked list, instead of shifting, we just have to update the address of the pointer of the node.

**Memory efficient** - The size of a linked list can grow or shrink according to the requirements, so memory consumption in linked list is efficient.

**Implementation** - We can implement both stacks and queues using linked list.

## Disadvantages of Linked list

The limitations of using the Linked list are given as follows -

**Memory usage** - In linked list, node occupies more memory than array. Each node of the linked list occupies two types of variables, i.e., one is a simple variable, and another one is the pointer variable.

**Traversal** - Traversal is not easy in the linked list. If we have to access an element in the linked list, we cannot access it randomly, while in case of array we can randomly access it by index. For example, if we want to access the 3rd node, then we need to traverse all the nodes before it. So, the time required to access a particular node is large.

**Reverse traversing** - Backtracking or reverse traversing is difficult in a linked list. In a doubly-linked list, it is easier but requires more memory to store the back pointer.

## Applications of Linked list

The applications of the Linked list are given as follows -

With the help of a linked list, the polynomials can be represented as well as we can perform the operations on the polynomial.

A linked list can be used to represent the sparse matrix.

The various operations like student's details, employee's details, or product details can be implemented using the linked list as the linked list uses the structure data type that can hold different data types.

Using linked list, we can implement stack, queue, tree, and other various data structures.

The graph is a collection of edges and vertices, and the graph can be represented as an adjacency matrix and adjacency list. If we want to represent the graph as an adjacency matrix, then it can be implemented as an array. If we want to represent the graph as an adjacency list, then it can be implemented as a linked list.

A linked list can be used to implement dynamic memory allocation. The dynamic memory allocation is the memory allocation done at the run-time.

## Operations performed on Linked list

The basic operations that are supported by a list are mentioned as follows -

**Insertion** - This operation is performed to add an element into the list.

**Deletion** - It is performed to delete an operation from the list.

**Display** - It is performed to display the elements of the list.

**Search** - It is performed to search an element from the list using the given key.

## Complexity of Linked list

Now, let's see the time and space complexity of the linked list for the operations search, insert, and delete.

### 1. Time Complexity

| Operations | Average case time complexity | Worst-case time complexity |
|------------|------------------------------|----------------------------|
| Insertion  | $O(1)$                       | $O(1)$                     |
| Deletion   | $O(1)$                       | $O(1)$                     |
| Search     | $O(n)$                       | $O(n)$                     |

Where 'n' is the number of nodes in the given tree.

### 2. Space Complexity

| Operations | Space complexity |
|------------|------------------|
| Insertion  | $O(n)$           |
| Deletion   | $O(n)$           |
| Search     | $O(n)$           |

The space complexity of linked list is  **$O(n)$** .

So, that's all about the introduction of linked list. Hope the article will be helpful and informative to you.

## Types of Linked List

Before knowing about the types of a linked list, we should know what is **linked list**. So, to know about the linked list, click on the link given below:

### Types of Linked list

**The following are the types of linked list:**

Singly Linked list

Doubly Linked list

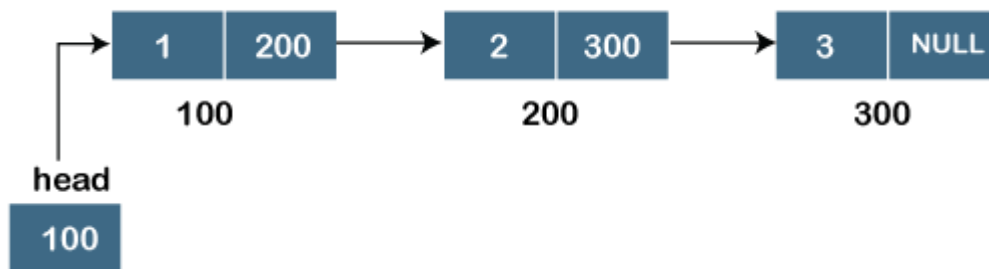
Circular Linked list

Doubly Circular Linked list

## Singly Linked list

It is the commonly used linked list in programs. If we are talking about the linked list, it means it is a singly linked list. The singly linked list is a data structure that contains two parts, i.e., one is the data part, and the other one is the address part, which contains the address of the next or the successor node. The address part in a node is also known as a **pointer**.

Suppose we have three nodes, and the addresses of these three nodes are 100, 200 and 300 respectively. The representation of three nodes as a linked list is shown in the below figure:



We can observe in the above figure that there are three different nodes having address 100, 200 and 300 respectively. The first node contains the address of the next node, i.e., 200, the second node contains the address of the last node, i.e., 300, and the third node contains the NULL value in its address part as it does not point to any node. The pointer that holds the address of the initial node is known as a **head pointer**.

The linked list, which is shown in the above diagram, is known as a singly linked list as it contains only a single link. In this list, only forward traversal is possible; we cannot traverse in the backward direction as it has only one link in the list.

### Representation of the node in a singly linked list

```
struct node
{
    int data;
    struct node *next;
}
```

In the above representation, we have defined a user-defined structure named a **node** containing two members, the first one is data of integer type, and the other one is the pointer (next) of the node type.

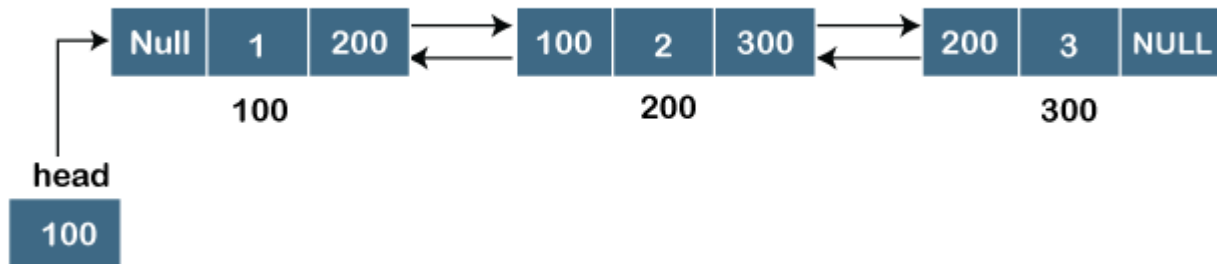
To know more about a singly linked list, click on the link given below:

<https://www.javatpoint.com/singly-linked-list>

## Doubly linked list

As the name suggests, the doubly linked list contains two pointers. We can define the doubly linked list as a linear data structure with three parts: the data part and the other two address part. In other words, a doubly linked list is a list that has three parts in a single node, includes one data part, a pointer to its previous node, and a pointer to the next node.

Suppose we have three nodes, and the address of these nodes are 100, 200 and 300, respectively. The representation of these nodes in a doubly-linked list is shown below:



As we can observe in the above figure, the node in a doubly-linked list has two address parts; one part stores the **address of the next** while the other part of the node stores the **previous node's address**. The initial node in the doubly linked list has the **NULL** value in the address part, which provides the address of the previous node.

### Representation of the node in a doubly linked list

struct node

```

{
    int data;
    struct node *next;
    struct node *prev;
}
  
```

In the above representation, we have defined a user-defined structure named **a node** with three members, one is **data** of integer type, and the other two are the pointers, i.e., **next and prev** of the node type. The **next pointer** variable holds the address of the next node, and the **prev pointer** holds the address of the previous node. The type of both the pointers, i.e., **next and prev** is **struct node** as both the pointers are storing the address of the node of the **struct node** type.

To know more about doubly linked list, click on the link given below:

<https://www.javatpoint.com/doubly-linked-list>

### Circular linked list

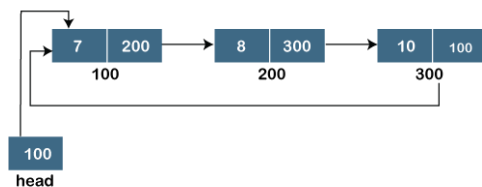
A circular linked list is a variation of a singly linked list. The only difference between the **singly linked list** and a **circular linked** list is that the last node does not point to any node in a singly linked list, so its link part contains a NULL value. On the other hand, the circular linked list is a list in which the last node connects to the first node, so the

link part of the last node holds the first node's address. The circular linked list has no starting and ending node. We can traverse in any direction, i.e., either backward or forward. The diagrammatic representation of the circular linked list is shown below:

struct node

```
{  
    int data;  
    struct node *next;  
}
```

A circular linked list is a sequence of elements in which each node has a link to the next node, and the last node is having a link to the first node. The representation of the circular linked list will be similar to the singly linked list, as shown below:

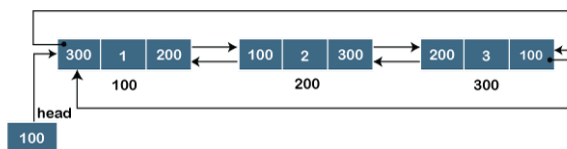


To know more about the circular linked list, click on the link given below:

<https://www.javatpoint.com/circular-singly-linked-list>

## Doubly Circular linked list

The doubly circular linked list has the features of both the **circular linked list** and **doubly linked list**.



The above figure shows the representation of the doubly circular linked list in which the last node is attached to the first node and thus creates a circle. It is a doubly linked list also because each node holds the address of the previous node also. The main difference between the doubly linked list and doubly circular linked list is that the doubly circular linked list does not contain the NULL value in the previous field of the node. As the doubly circular linked contains three parts, i.e., two address parts and one data part so its representation is similar to the doubly linked list.

struct node

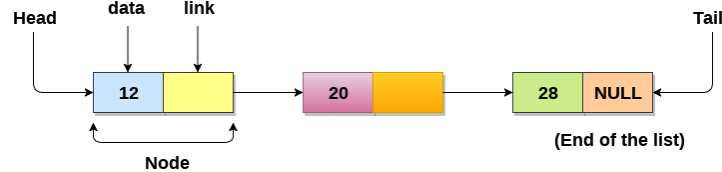
```
{  
    int data;  
    struct node *next;  
    struct node *prev;  
}
```

## Linked List

Linked List can be defined as collection of objects called **nodes** that are randomly stored in the memory.

A node contains two fields i.e. data stored at that particular address and the pointer which contains the address of the next node in the memory.

The last node of the list contains pointer to the null.



## Uses of Linked List

The list is not required to be contiguously present in the memory. The node can reside anywhere in the memory and linked together to make a list. This achieves optimized utilization of space.

list size is limited to the memory size and doesn't need to be declared in advance.

Empty node can not be present in the linked list.

We can store values of primitive types or objects in the singly linked list.

## Why use linked list over array?

Till now, we were using array data structure to organize the group of elements that are to be stored individually in the memory. However, Array has several advantages and disadvantages which must be known in order to decide the data structure which will be used throughout the program.

Array contains following limitations:

The size of array must be known in advance before using it in the program.

Increasing size of the array is a time taking process. It is almost impossible to expand the size of the array at run time.

All the elements in the array need to be contiguously stored in the memory. Inserting any element in the array needs shifting of all its predecessors.

Linked list is the data structure which can overcome all the limitations of an array. Using linked

| Data Structure     | Time Complexity |             |             |             |        |        |           |          | Space Compleity |
|--------------------|-----------------|-------------|-------------|-------------|--------|--------|-----------|----------|-----------------|
|                    | Average         |             |             |             | Worst  |        |           |          |                 |
|                    | Access          | Search      | Insertion   | Deletion    | Access | Search | Insertion | Deletion |                 |
| Singly Linked List | $\theta(n)$     | $\theta(n)$ | $\theta(1)$ | $\theta(1)$ | $O(n)$ | $O(n)$ | $O(1)$    | $O(1)$   | $O(n)$          |

list is useful because,

It allocates the memory dynamically. All the nodes of linked list are non-contiguously stored in the memory and linked together with the help of pointers.

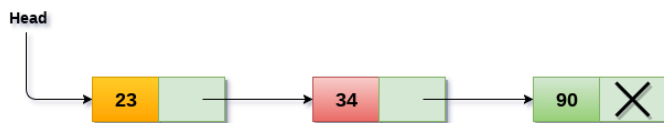
Sizing is no longer a problem since we do not need to define its size at the time of declaration. List grows as per the program's demand and limited to the available memory space.

## Singly linked list or One way chain

Singly linked list can be defined as the collection of ordered set of elements. The number of elements may vary according to need of the program. A node in the singly linked list consist of two parts: data part and link part. Data part of the node stores actual information that is to be represented by the node while the link part of the node stores the address of its immediate successor.

One way chain or singly linked list can be traversed only in one direction. In other words, we can say that each node contains only next pointer, therefore we can not traverse the list in the reverse direction.

Consider an example where the marks obtained by the student in three subjects are stored in a linked list as shown in the figure.



In the above figure, the arrow represents the links. The data part of every node contains the marks obtained by the student in the different subject. The last node in the list is identified by the null pointer which is present in the address part of the last node. We can have as many elements we require, in the data part of the list.

## Complexity

### Operations on Singly Linked List

There are various operations which can be performed on singly linked list. A list of all such operations is given below.

#### Node Creation

```
struct node
{
    int data;
    struct node *next;
};
struct node *head, *ptr;
ptr = (struct node *)malloc(sizeof(struct node *));
```

#### Insertion

The insertion into a singly linked list can be performed at different positions. Based on the position of the new node being inserted, the insertion is categorized into the following categories.

| SN | Operation | Description |
|----|-----------|-------------|
|----|-----------|-------------|

|   |                                                       |                                                                                                                                                                                                           |
|---|-------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <a href="#"><u>Insertion at beginning</u></a>         | It involves inserting any element at the front of the list. We just need to a few link adjustments to make the new node as the head of the list.                                                          |
| 2 | <a href="#"><u>Insertion at end of the list</u></a>   | It involves insertion at the last of the linked list. The new node can be inserted as the only node in the list or it can be inserted as the last one. Different logics are implemented in each scenario. |
| 3 | <a href="#"><u>Insertion after specified node</u></a> | It involves insertion after the specified node of the linked list. We need to skip the desired number of nodes in order to reach the node after which the new node will be inserted. .                    |

## Deletion and Traversing

The Deletion of a node from a singly linked list can be performed at different positions. Based on the position of the node being deleted, the operation is categorized into the following categories.

| SN | Operation                                              | Description                                                                                                                                                                                                        |
|----|--------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <a href="#"><u>Deletion at beginning</u></a>           | It involves deletion of a node from the beginning of the list. This is the simplest operation among all. It just need a few adjustments in the node pointers.                                                      |
| 2  | <a href="#"><u>Deletion at the end of the list</u></a> | It involves deleting the last node of the list. The list can either be empty or full. Different logic is implemented for the different scenarios.                                                                  |
| 3  | <a href="#"><u>Deletion after specified node</u></a>   | It involves deleting the node after the specified node in the list. we need to skip the desired number of nodes to reach the node after which the node will be deleted. This requires traversing through the list. |
| 4  | <a href="#"><u>Traversing</u></a>                      | In traversing, we simply visit each node of the list at least once in order to perform some specific operation on it, for example, printing data part of each node present in the list.                            |
| 5  | <a href="#"><u>Searching</u></a>                       | In searching, we match each element of the list with the given element. If the element is found on any of the location then location of that element is returned otherwise null is returned. .                     |

## Linked List in C: Menu Driven Program

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
```



```

    struct node *next;
};
struct node *head;

void begininsert ();
void lastinsert ();
void randominsert();
void begin_delete();
void last_delete();
void random_delete();
void display();
void search();
void main ()
{
    int choice =0;
    while(choice != 9)
    {
        printf("\n\n*****Main Menu*****\n");
        printf("\nChoose one option from the following list ...\n");
        printf("\n===== \n");
        printf("\n1.Insert in beginning\n2.Insert at last\n3.Insert at any random location\n4.Delete from Beginning\n5.Delete from last\n6.Delete node after specified location\n7.Search for an element\n8.Show\n9.Exit\n");
        printf("\nEnter your choice?\n");
        scanf("\n%d",&choice);
        switch(choice)
        {
            case 1:
                begininsert();
                break;
            case 2:
                lastinsert();
                break;
            case 3:
                randominsert();
                break;
            case 4:
                begin_delete();
                break;
            case 5:
                last_delete();
                break;
            case 6:

```

```

        random_delete();
        break;
    case 7:
        search();
        break;
    case 8:
        display();
        break;
    case 9:
        exit(0);
        break;
    default:
        printf("Please enter valid choice..");
    }
}
}
}
void beginsert()
{
    struct node *ptr;
    int item;
    ptr = (struct node *) malloc(sizeof(struct node *));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter value\n");
        scanf("%d",&item);
        ptr->data = item;
        ptr->next = head;
        head = ptr;
        printf("\nNode inserted");
    }
}

void lastinsert()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node*)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
}

```

```

}
else
{
    printf("\nEnter value?\n");
    scanf("%d",&item);
    ptr->data = item;
    if(head == NULL)
    {
        ptr -> next = NULL;
        head = ptr;
        printf("\nNode inserted");
    }
    else
    {
        temp = head;
        while (temp -> next != NULL)
        {
            temp = temp -> next;
        }
        temp->next = ptr;
        ptr->next = NULL;
        printf("\nNode inserted");
    }
}
}
}
void randominsert()
{
    int i,loc,item;
    struct node *ptr, *temp;
    ptr = (struct node *) malloc (sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter element value");
        scanf("%d",&item);
        ptr->data = item;
        printf("\nEnter the location after which you want to insert ");
        scanf("\n%d",&loc);
        temp=head;
        for(i=0;i<loc;i++)

```

```

    {
        temp = temp->next;
        if(temp == NULL)
        {
            printf("\ncan't insert\n");
            return;
        }

    }
    ptr ->next = temp ->next;
    temp ->next = ptr;
    printf("\nNode inserted");
}
}
void begin_delete()
{
    struct node *ptr;
    if(head == NULL)
    {
        printf("\nList is empty\n");
    }
    else
    {
        ptr = head;
        head = ptr->next;
        free(ptr);
        printf("\nNode deleted from the beginning ...\n");
    }
}
void last_delete()
{
    struct node *ptr,*ptr1;
    if(head == NULL)
    {
        printf("\nlist is empty");
    }
    else if(head -> next == NULL)
    {
        head = NULL;
        free(head);
        printf("\nOnly node of the list deleted ...\n");
    }
}
else

```

```

{
    ptr = head;
    while(ptr->next != NULL)
    {
        ptr1 = ptr;
        ptr = ptr ->next;
    }
    ptr1->next = NULL;
    free(ptr);
    printf("\nDeleted Node from the last ...\n");
}
}

void random_delete()
{
    struct node *ptr,*ptr1;
    int loc,i;
    printf("\n Enter the location of the node after which you want to perform deletion \n");
    scanf("%d",&loc);
    ptr=head;
    for(i=0;i<loc;i++)
    {
        ptr1 = ptr;
        ptr = ptr->next;

        if(ptr == NULL)
        {
            printf("\nCan't delete");
            return;
        }
    }
    ptr1 ->next = ptr ->next;
    free(ptr);
    printf("\nDeleted node %d ",loc+1);
}

void search()
{
    struct node *ptr;
    int item,i=0,flag;
    ptr = head;
    if(ptr == NULL)
    {
        printf("\nEmpty List\n");
    }
    else

```

```

{
    printf("\nEnter item which you want to search?\n");
    scanf("%d",&item);
    while (ptr!=NULL)
    {
        if(ptr->data == item)
        {
            printf("item found at location %d ",i+1);
            flag=0;
        }
        else
        {
            flag=1;
        }
        i++;
        ptr = ptr -> next;
    }
    if(flag==1)
    {
        printf("Item not found\n");
    }
}
}

```

```

void display()
{
    struct node *ptr;
    ptr = head;
    if(ptr == NULL)
    {
        printf("Nothing to print");
    }
    else
    {
        printf("\nprinting values . . . .\n");
        while (ptr!=NULL)
        {
            printf("\n%d",ptr->data);
            ptr = ptr -> next;
        }
    }
}
}

```

**Output:**

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

1

Enter value

1

Node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

2

Enter value?

2

Node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

3

Enter element value1

Enter the location after which you want to insert 1

Node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

8

printing values . . . .



1  
2  
1

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

2

Enter value?

123

Node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

1

Enter value  
1234

Node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

4

Node deleted from the begining ...

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

5

Deleted Node from the last ...

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

6

Enter the location of the node after which you want to perform deletion

1

Deleted node 2

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

8

printing values . . . . .

1

1

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

Enter your choice?

7

Enter item which you want to search?

1

item found at location 1

item found at location 2

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete node after specified location
- 7.Search for an element
- 8.Show
- 9.Exit

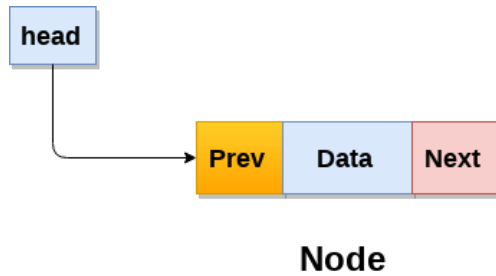
Enter your choice?

9

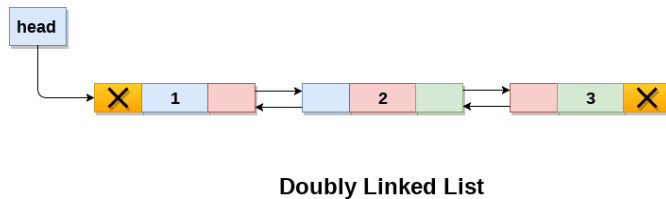
## Doubly linked list

Doubly linked list is a complex type of linked list in which a node contains a pointer to the previous as well as the next node in the sequence. Therefore, in a doubly linked list, a node consists of three parts: node data, pointer to the next node in sequence (next pointer) , pointer

to the previous node (previous pointer). A sample node in a doubly linked list is shown in the figure.



A doubly linked list containing three nodes having numbers from 1 to 3 in their data part, is shown in the following image.



In C, structure of a node in doubly linked list can be given as :

```
struct node
{
    struct node *prev;
    int data;
    struct node *next;
}
```

The **prev** part of the first node and the **next** part of the last node will always contain null indicating end in each direction.

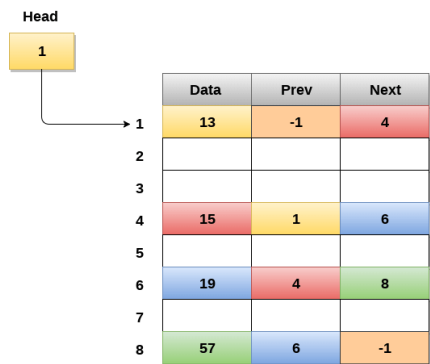
In a singly linked list, we could traverse only in one direction, because each node contains address of the next node and it doesn't have any record of its previous nodes. However, doubly linked list overcome this limitation of singly linked list. Due to the fact that, each node of the list contains the address of its previous node, we can find all the details about the previous node as well by using the previous address stored inside the previous part of each node.

## Memory Representation of a doubly linked list

Memory Representation of a doubly linked list is shown in the following image. Generally, doubly linked list consumes more space for every node and therefore, causes more expansive basic operations such as insertion and deletion. However, we can easily manipulate the elements of the list since the list maintains pointers in both the directions (forward and backward).

In the following image, the first element of the list that is i.e. 13 stored at address 1. The head pointer points to the starting address 1. Since this is the first element being added to the list therefore the **prev** of the list **contains** null. The next node of the list resides at address 4 therefore the first node contains 4 in its next pointer.

We can traverse the list in this way until we find any node containing null or -1 in its next part.



Memory Representation of a Doubly linked list

## Operations on doubly linked list

### Node Creation

```
struct node
{
    struct node *prev;
    int data;
    struct node *next;
};
struct node *head;
```

All the remaining operations regarding doubly linked list are described in the following table.

| SN | Operation                                              | Description                                                                       |
|----|--------------------------------------------------------|-----------------------------------------------------------------------------------|
| 1  | <a href="#">Insertion at beginning</a>                 | Adding the node into the linked list at beginning.                                |
| 2  | <a href="#">Insertion at end</a>                       | Adding the node into the linked list to the end.                                  |
| 3  | <a href="#">Insertion after specified node</a>         | Adding the node into the linked list after the specified node.                    |
| 4  | <a href="#">Deletion at beginning</a>                  | Removing the node from beginning of the list                                      |
| 5  | <a href="#">Deletion at the end</a>                    | Removing the node from end of the list.                                           |
| 6  | <a href="#">Deletion of the node having given data</a> | Removing the node which is present just after the node containing the given data. |

|   |                            |                                                                                                                                           |
|---|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 7 | <a href="#">Searching</a>  | Comparing each node data with the item to be searched and return the location of the item in the list if the item found else return null. |
| 8 | <a href="#">Traversing</a> | Visiting each node of the list at least once in order to perform some specific operation like searching, sorting, display, etc.           |

## Menu Driven Program in C to implement all the operations of doubly linked list

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    struct node *prev;
    struct node *next;
    int data;
};
struct node *head;
void insertion_beginning();
void insertion_last();
void insertion_specified();
void deletion_beginning();
void deletion_last();
void deletion_specified();
void display();
void search();
void main ()
{
    int choice =0;
    while(choice != 9)
    {
        printf("\n*****Main Menu*****\n");
        printf("\nChoose one option from the following list ...\n");
        printf("\n=====");
        printf("\n1.Insert in beginning\n2.Insert at last\n3.Insert at any random location\n4.Delete from Beginning\n5.Delete from last\n6.Delete the node after the given data\n7.Search\n8.Show\n9.Exit\n");
        printf("\nEnter your choice?\n");
        scanf("\n%d",&choice);
        switch(choice)
        {
            case 1:
                insertion_beginning();
```

```

        break;
        case 2:
            insertion_last();
        break;
        case 3:
            insertion_specified();
        break;
        case 4:
            deletion_beginning();
        break;
        case 5:
            deletion_last();
        break;
        case 6:
            deletion_specified();
        break;
        case 7:
            search();
        break;
        case 8:
            display();
        break;
        case 9:
            exit(0);
        break;
        default:
            printf("Please enter valid choice..");
    }
}
}
void insertion_beginning()
{
    struct node *ptr;
    int item;
    ptr = (struct node *)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter Item value");
        scanf("%d",&item);
    }
}

```



```

    if(head==NULL)
    {
        ptr->next = NULL;
        ptr->prev=NULL;
        ptr->data=item;
        head=ptr;
    }
    else
    {
        ptr->data=item;
        ptr->prev=NULL;
        ptr->next = head;
        head->prev=ptr;
        head=ptr;
    }
    printf("\nNode inserted\n");
}

}
void insertion_last()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node *) malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter value");
        scanf("%d",&item);
        ptr->data=item;
        if(head == NULL)
        {
            ptr->next = NULL;
            ptr->prev = NULL;
            head = ptr;
        }
        else
        {
            temp = head;
            while(temp->next!=NULL)
            {

```

```

        temp = temp->next;
    }
    temp->next = ptr;
    ptr->prev=temp;
    ptr->next = NULL;
}

}
printf("\nnode inserted\n");
}
void insertion_specified()
{
    struct node *ptr,*temp;
    int item,loc,i;
    ptr = (struct node *)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\n OVERFLOW");
    }
    else
    {
        temp=head;
        printf("Enter the location");
        scanf("%d",&loc);
        for(i=0;i<loc;i++)
        {
            temp = temp->next;
            if(temp == NULL)
            {
                printf("\n There are less than %d elements", loc);
                return;
            }
        }
        printf("Enter value");
        scanf("%d",&item);
        ptr->data = item;
        ptr->next = temp->next;
        ptr->prev = temp;
        temp->next = ptr;
        temp->next->prev=ptr;
        printf("\nnode inserted\n");
    }
}
void deletion_beginning()

```

```

{
    struct node *ptr;
    if(head == NULL)
    {
        printf("\n UNDERFLOW");
    }
    else if(head->next == NULL)
    {
        head = NULL;
        free(head);
        printf("\nnode deleted\n");
    }
    else
    {
        ptr = head;
        head = head -> next;
        head -> prev = NULL;
        free(ptr);
        printf("\nnode deleted\n");
    }
}

```

```

}
void deletion_last()
{
    struct node *ptr;
    if(head == NULL)
    {
        printf("\n UNDERFLOW");
    }
    else if(head->next == NULL)
    {
        head = NULL;
        free(head);
        printf("\nnode deleted\n");
    }
    else
    {
        ptr = head;
        if(ptr->next != NULL)
        {
            ptr = ptr -> next;
        }
        ptr -> prev -> next = NULL;
        free(ptr);
    }
}

```

```

        printf("\nnode deleted\n");
    }
}
void deletion_specified()
{
    struct node *ptr, *temp;
    int val;
    printf("\n Enter the data after which the node is to be deleted : ");
    scanf("%d", &val);
    ptr = head;
    while(ptr -> data != val)
        ptr = ptr -> next;
    if(ptr -> next == NULL)
    {
        printf("\nCan't delete\n");
    }
    else if(ptr -> next -> next == NULL)
    {
        ptr -> next = NULL;
    }
    else
    {
        temp = ptr -> next;
        ptr -> next = temp -> next;
        temp -> next -> prev = ptr;
        free(temp);
        printf("\nnode deleted\n");
    }
}
void display()
{
    struct node *ptr;
    printf("\n printing values...\n");
    ptr = head;
    while(ptr != NULL)
    {
        printf("%d\n", ptr->data);
        ptr = ptr->next;
    }
}
void search()
{
    struct node *ptr;
    int item, i=0, flag;

```

```

ptr = head;
if(ptr == NULL)
{
    printf("\nEmpty List\n");
}
else
{
    printf("\nEnter item which you want to search?\n");
    scanf("%d",&item);
    while (ptr!=NULL)
    {
        if(ptr->data == item)
        {
            printf("\nitem found at location %d ",i+1);
            flag=0;
            break;
        }
        else
        {
            flag=1;
        }
        i++;
        ptr = ptr -> next;
    }
    if(flag==1)
    {
        printf("\nItem not found\n");
    }
}
}

```

## Output

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search

8.Show  
9.Exit

Enter your choice?  
8

printing values...

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

1.Insert in begining  
2.Insert at last  
3.Insert at any random location  
4.Delete from Beginning  
5.Delete from last  
6.Delete the node after the given data  
7.Search  
8.Show  
9.Exit

Enter your choice?  
1

Enter Item value12

Node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

1.Insert in begining  
2.Insert at last  
3.Insert at any random location  
4.Delete from Beginning  
5.Delete from last  
6.Delete the node after the given data  
7.Search  
8.Show  
9.Exit

Enter your choice?

1

Enter Item value123

Node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

1

Enter Item value1234

Node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

8

printing values...

1234

123

12

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

2

Enter value89

node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

3

Enter the location1

Enter value12345



node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

8

printing values...

1234

123

12345

12

89

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

4

node deleted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

5

node deleted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

8

printing values...

123

12345

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

6

Enter the data after which the node is to be deleted : 123

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

8

printing values...

123

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining

- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

7

Enter item which you want to search?

123

item found at location 1

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location
- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

6

Enter the data after which the node is to be deleted : 123

Can't delete

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Insert at any random location

- 4.Delete from Beginning
- 5.Delete from last
- 6.Delete the node after the given data
- 7.Search
- 8.Show
- 9.Exit

Enter your choice?

9

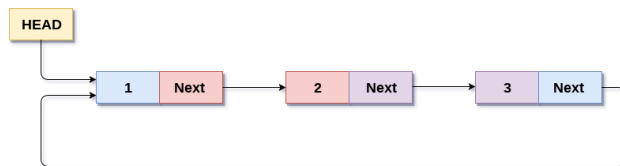
Exited..

## Circular Singly Linked List

In a circular Singly linked list, the last node of the list contains a pointer to the first node of the list. We can have circular singly linked list as well as circular doubly linked list.

We traverse a circular singly linked list until we reach the same node where we started. The circular singly linked list has no beginning and no ending. There is no null value present in the next part of any of the nodes.

The following image shows a circular singly linked list.



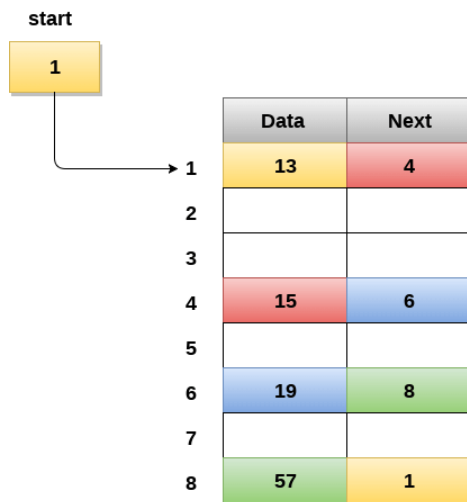
Circular Singly Linked List

Circular linked list are mostly used in task maintenance in operating systems. There are many examples where circular linked list are being used in computer science including browser surfing where a record of pages visited in the past by the user, is maintained in the form of circular linked lists and can be accessed again on clicking the previous button.

## Memory Representation of circular linked list:

In the following image, memory representation of a circular linked list containing marks of a student in 4 subjects. However, the image shows a glimpse of how the circular list is being stored in the memory. The start or head of the list is pointing to the element with the index 1 and containing 13 marks in the data part and 4 in the next part. Which means that it is linked with the node that is being stored at 4th index of the list.

However, due to the fact that we are considering circular linked list in the memory therefore the last node of the list contains the address of the first node of the list.



### Memory Representation of a circular linked list

We can also have more than one number of linked list in the memory with the different start pointers pointing to the different start nodes in the list. The last node is identified by its next part which contains the address of the start node of the list. We must be able to identify the last node of any linked list so that we can find out the number of iterations which need to be performed while traversing the list.

## Operations on Circular Singly linked list:

### Insertion

| SN | Operation                              | Description                                                      |
|----|----------------------------------------|------------------------------------------------------------------|
| 1  | <a href="#">Insertion at beginning</a> | Adding a node into circular singly linked list at the beginning. |
| 2  | <a href="#">Insertion at the end</a>   | Adding a node into circular singly linked list at the end.       |

### Deletion & Traversing

| SN | Operation                             | Description                                                                                                                                  |
|----|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <a href="#">Deletion at beginning</a> | Removing the node from circular singly linked list at the beginning.                                                                         |
| 2  | <a href="#">Deletion at the end</a>   | Removing the node from circular singly linked list at the end.                                                                               |
| 3  | <a href="#">Searching</a>             | Compare each element of the node with the given item and return the location at which the item is present in the list otherwise return null. |

## Menu-driven program in C implementing all operations on circular singly linked list

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
    struct node *next;
};
struct node *head;

void begininsert ();
void lastinsert ();
void randominsert();
void begin_delete();
void last_delete();
void random_delete();
void display();
void search();
void main ()
{
    int choice =0;
    while(choice != 7)
    {
        printf("\n*****Main Menu*****\n");
        printf("\nChoose one option from the following list ...\n");
        printf("\n===== \n");
        printf("\n1.Insert in begining\n2.Insert at last\n3.Delete from Beginning\n4.Delete from last\n5.Search for an element\n6.Show\n7.Exit\n");
        printf("\nEnter your choice?\n");
        scanf("\n%d",&choice);
        switch(choice)
        {
            case 1:
                begininsert();
                break;
            case 2:
                lastinsert();
                break;
```

```

        case 3:
            begin_delete();
            break;
        case 4:
            last_delete();
            break;
        case 5:
            search();
            break;
        case 6:
            display();
            break;
        case 7:
            exit(0);
            break;
        default:
            printf("Please enter valid choice..");
    }
}
}
void begininsert()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node *)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW");
    }
    else
    {
        printf("\nEnter the node data?");
        scanf("%d",&item);
        ptr->data = item;
        if(head == NULL)
        {
            head = ptr;
            ptr->next = head;
        }
        else
        {
            temp = head;
            while(temp->next != head)
                temp = temp->next;

```



```

        ptr->next = head;
        temp -> next = ptr;
        head = ptr;
    }
    printf("\nnode inserted\n");
}

}

void lastinsert()
{
    struct node *ptr,*temp;
    int item;
    ptr = (struct node *)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW\n");
    }
    else
    {
        printf("\nEnter Data?");
        scanf("%d",&item);
        ptr->data = item;
        if(head == NULL)
        {
            head = ptr;
            ptr -> next = head;
        }
        else
        {
            temp = head;
            while(temp -> next != head)
            {
                temp = temp -> next;
            }
            temp -> next = ptr;
            ptr -> next = head;
        }

        printf("\nnode inserted\n");
    }
}

}

void begin_delete()

```

```

{
    struct node *ptr;
    if(head == NULL)
    {
        printf("\nUNDERFLOW");
    }
    else if(head->next == head)
    {
        head = NULL;
        free(head);
        printf("\nnode deleted\n");
    }

    else
    {
        ptr = head;
        while(ptr -> next != head)
            ptr = ptr -> next;
        ptr->next = head->next;
        free(head);
        head = ptr->next;
        printf("\nnode deleted\n");
    }
}

void last_delete()
{
    struct node *ptr, *preptr;
    if(head==NULL)
    {
        printf("\nUNDERFLOW");
    }
    else if (head ->next == head)
    {
        head = NULL;
        free(head);
        printf("\nnode deleted\n");
    }
    else
    {
        ptr = head;
        while(ptr ->next != head)
        {
            preptr=ptr;

```

```

        ptr = ptr->next;
    }
    preptr->next = ptr -> next;
    free(ptr);
    printf("\nnode deleted\n");

}
}

```

```

void search()
{
    struct node *ptr;
    int item,i=0,flag=1;
    ptr = head;
    if(ptr == NULL)
    {
        printf("\nEmpty List\n");
    }
    else
    {
        printf("\nEnter item which you want to search?\n");
        scanf("%d",&item);
        if(head ->data == item)
        {
            printf("item found at location %d",i+1);
            flag=0;
        }
        else
        {
            while (ptr->next != head)
            {
                if(ptr->data == item)
                {
                    printf("item found at location %d ",i+1);
                    flag=0;
                    break;
                }
                else
                {
                    flag=1;
                }
                i++;
                ptr = ptr -> next;
            }

```

```

    }
    if(flag != 0)
    {
        printf("Item not found\n");
    }
}

}

void display()
{
    struct node *ptr;
    ptr=head;
    if(head == NULL)
    {
        printf("\nnothing to print");
    }
    else
    {
        printf("\n printing values ... \n");

        while(ptr -> next != head)
        {

            printf("%d\n", ptr -> data);
            ptr = ptr -> next;
        }
        printf("%d\n", ptr -> data);
    }

}

```

### Output:

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Delete from Beginning
- 4.Delete from last
- 5.Search for an element
- 6.Show
- 7.Exit

Enter your choice?

1

Enter the node data?10

node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Delete from Beginning
- 4.Delete from last
- 5.Search for an element
- 6.Show
- 7.Exit

Enter your choice?

2

Enter Data?20

node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Delete from Beginning
- 4.Delete from last
- 5.Search for an element
- 6.Show
- 7.Exit

Enter your choice?

2

Enter Data?30

node inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Delete from Beginning
- 4.Delete from last
- 5.Search for an element
- 6.Show
- 7.Exit

Enter your choice?

3

node deleted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Delete from Beginning
- 4.Delete from last
- 5.Search for an element
- 6.Show
- 7.Exit

Enter your choice?

4

node deleted

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last

- 3.Delete from Beginning
- 4.Delete from last
- 5.Search for an element
- 6.Show
- 7.Exit

Enter your choice?

5

Enter item which you want to search?

20

item found at location 1

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Delete from Beginning
- 4.Delete from last
- 5.Search for an element
- 6.Show
- 7.Exit

Enter your choice?

6

printing values ...

20

\*\*\*\*\*Main Menu\*\*\*\*\*

Choose one option from the following list ...

=====

- 1.Insert in begining
- 2.Insert at last
- 3.Delete from Beginning
- 4.Delete from last
- 5.Search for an element
- 6.Show
- 7.Exit

Enter your choice?

7

# Skip list in Data structure

## What is a skip list?

A skip list is a probabilistic data structure. The skip list is used to store a sorted list of elements or data with a linked list. It allows the process of the elements or data to view efficiently. In one single step, it skips several elements of the entire list, which is why it is known as a skip list.

The skip list is an extended version of the linked list. It allows the user to search, remove, and insert the element very quickly. It consists of a base list that includes a set of elements which maintains the link hierarchy of the subsequent elements.

## Skip list structure

It is built in two layers: The lowest layer and Top layer.

The lowest layer of the skip list is a common sorted linked list, and the top layers of the skip list are like an "express line" where the elements are skipped.

## Complexity table of the Skip list

| S. No | Complexity        | Average case | Worst case    |
|-------|-------------------|--------------|---------------|
| 1.    | Access complexity | $O(\log n)$  | $O(n)$        |
| 2.    | Search complexity | $O(\log n)$  | $O(n)$        |
| 3.    | Delete complexity | $O(\log n)$  | $O(n)$        |
| 4.    | Insert complexity | $O(\log n)$  | $O(n)$        |
| 5.    | Space complexity  | -            | $O(n \log n)$ |

## Working of the Skip list

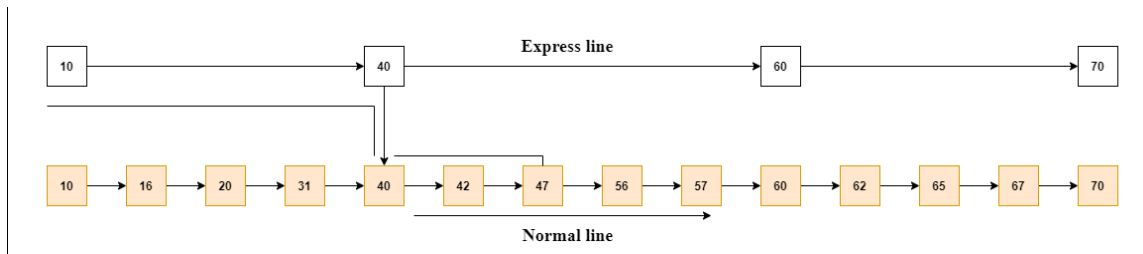
Let's take an example to understand the working of the skip list. In this example, we have 14 nodes, such that these nodes are divided into two layers, as shown in the diagram.

The lower layer is a common line that links all nodes, and the top layer is an express line that links only the main nodes, as you can see in the diagram.

Suppose you want to find 47 in this example. You will start the search from the first node of the express line and continue running on the express line until you find a node that is equal a 47 or more than 47.

You can see in the example that 47 does not exist in the express line, so you search for a node of less than 47, which is 40. Now, you go to the normal line with the help of 40, and search the 47, as shown in the diagram.





**Note:** Once you find a node like this on the "express line", you go from this node to a "normal lane" using a pointer, and when you search for the node in the normal line.

## Skip List Basic Operations

There are the following types of operations in the skip list.

**Insertion operation:** It is used to add a new node to a particular location in a specific situation.

**Deletion operation:** It is used to delete a node in a specific situation.

**Search Operation:** The search operation is used to search a particular node in a skip list.

### Algorithm of the insertion operation

Insertion (L, Key)

local update [0...Max\_Level + 1]

a = L → header

**for** i = L → level down to 0 **do**.

**while** a → forward[i] → key forward[i]

update[i] = a

a = a → forward[0]

lvl = random\_Level()

**if** lvl > L → level then

**for** i = L → level + 1 to lvl **do**

    update[i] = L → header

    L → level = lvl

a = makeNode(lvl, Key, value)

**for** i = 0 to level **do**

    a → forward[i] = update[i] → forward[i]

update[i] → forward[i] = a

### Algorithm of deletion operation

Deletion (L, Key)

local update [0... Max\_Level + 1]

a = L → header

**for** i = L → level down to 0 **do**.

**while** a → forward[i] → key forward[i]

    update[i] = a

a = a → forward[0]

**if** a → key = Key then

**for** i = 0 to L → level **do**

**if** update[i] → forward[i] ? a then **break**

```

    update[i] → forward[i] = a → forward[i]
    free(a)
    while L → level > 0 and L → header → forward[L → level] = NIL do
        L → level = L → level - 1

```

### Algorithm of searching operation

Searching (L, SKey)

a = L → header

loop invariant: a → key level down to 0 do.

while a → forward[i] → key forward[i]

a = a → forward[0]

if a → key = SKey then return a → value

else return failure

**Example 1:** Create a skip list, we want to insert these following keys in the empty skip list.

6 with level 1.

29 with level 1.

22 with level 4.

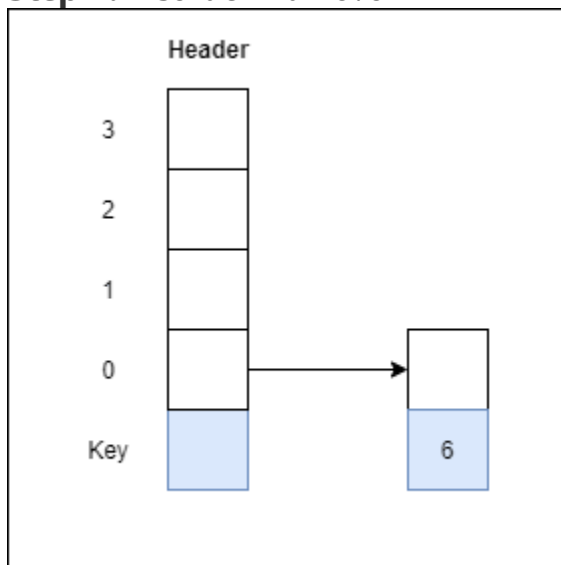
9 with level 3.

17 with level 1.

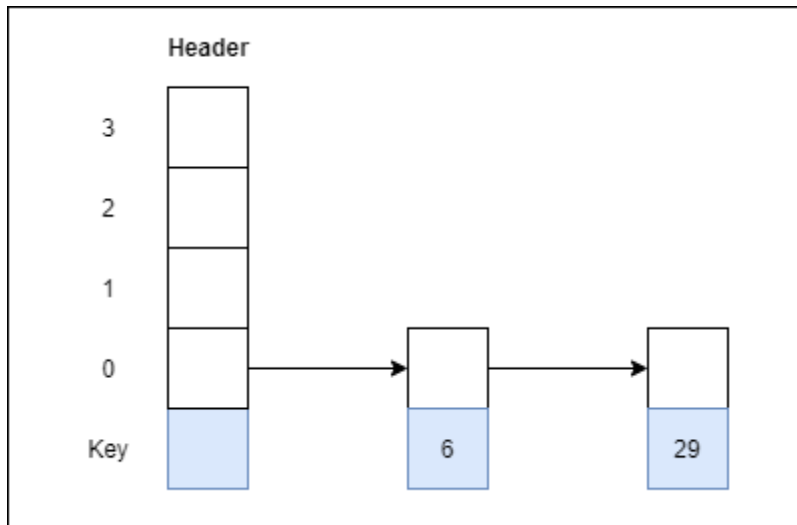
4 with level 2.

**Ans:**

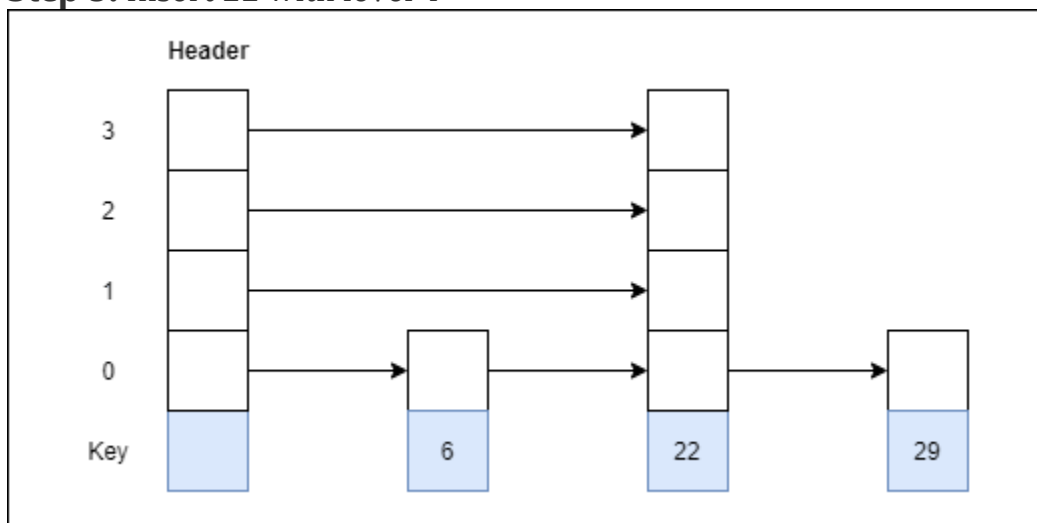
**Step 1:** Insert 6 with level 1



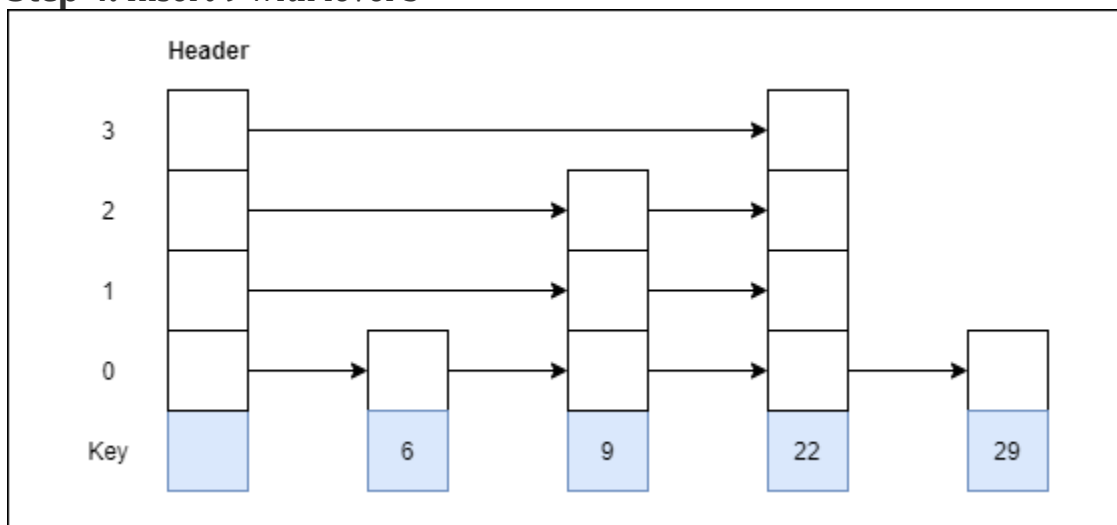
**Step 2:** Insert 29 with level 1



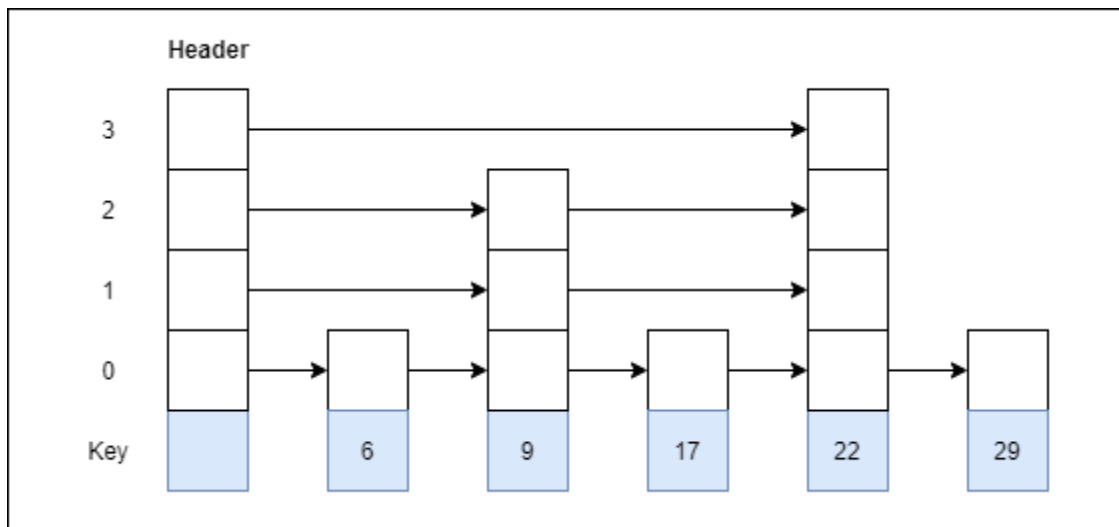
**Step 3:** Insert 22 with level 4



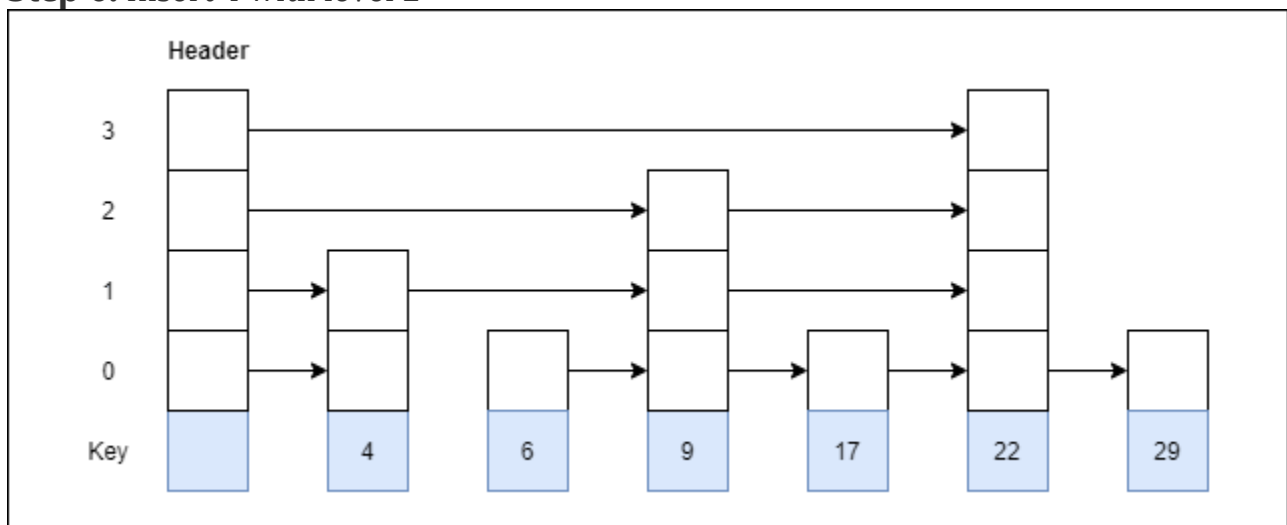
**Step 4:** Insert 9 with level 3



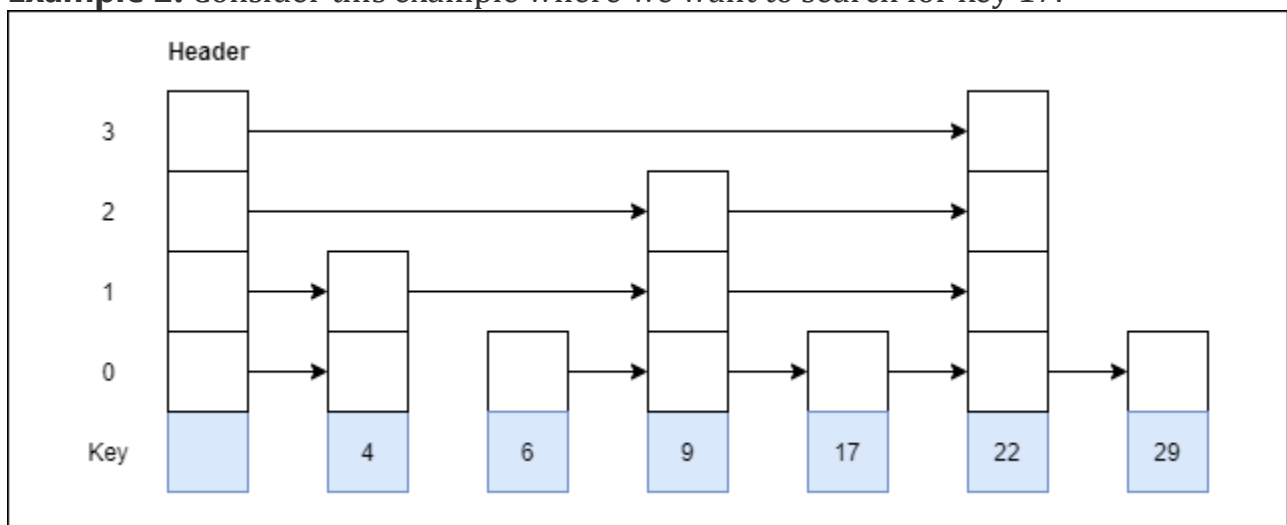
**Step 5:** Insert 17 with level 1



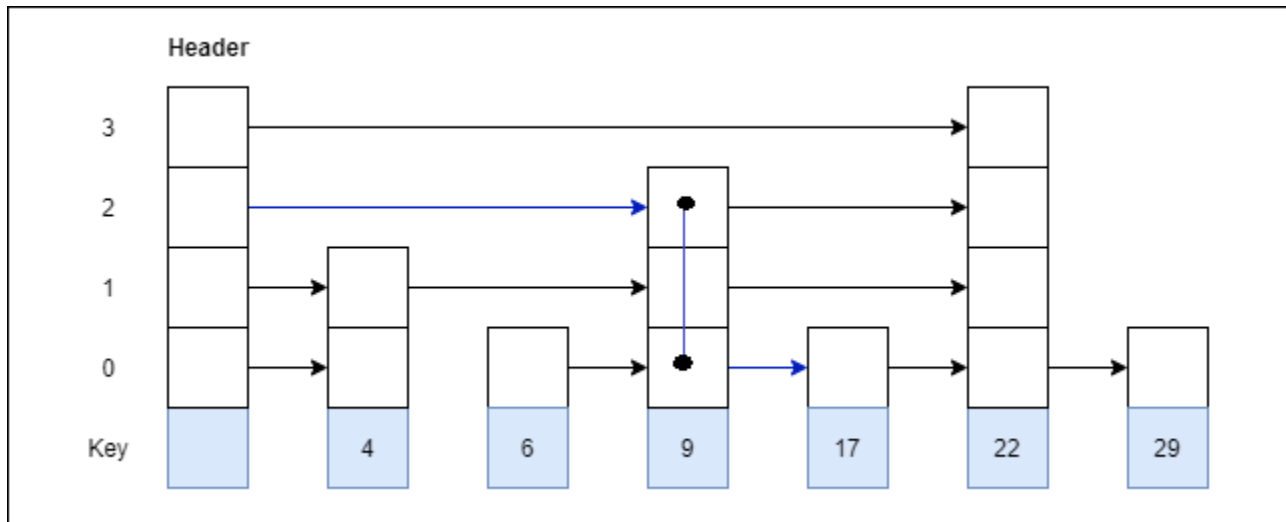
**Step 6:** Insert 4 with level 2



**Example 2:** Consider this example where we want to search for key 17.



**Ans:**



## Advantages of the Skip list

If you want to insert a new node in the skip list, then it will insert the node very fast because there are no rotations in the skip list.

The skip list is simple to implement as compared to the hash table and the binary search tree.

It is very simple to find a node in the list because it stores the nodes in sorted form.

The skip list algorithm can be modified very easily in a more specific structure, such as indexable skip lists, trees, or priority queues.

The skip list is a robust and reliable list.

## Disadvantages of the Skip list

It requires more memory than the balanced tree.

Reverse searching is not allowed.

The skip list searches the node much slower than the linked list.

## Applications of the Skip list

It is used in distributed applications, and it represents the pointers and system in the distributed applications.

It is used to implement a dynamic elastic concurrent queue with low lock contention.

It is also used with the QMap template class.

The indexing of the skip list is used in running median problems.

The skip list is used for the delta-encoding posting in the Lucene search.

## Skip list in Data structure

### What is a skip list?

A skip list is a probabilistic data structure. The skip list is used to store a sorted list of elements or data with a linked list. It allows the process of the elements or data to view efficiently. In one single step, it skips several elements of the entire list, which is why it is known as a skip list.

The skip list is an extended version of the linked list. It allows the user to search, remove, and insert the element very quickly. It consists of a base list that includes a set of elements which maintains the link hierarchy of the subsequent elements.

## Skip list structure

It is built in two layers: The lowest layer and Top layer.

The lowest layer of the skip list is a common sorted linked list, and the top layers of the skip list are like an "express line" where the elements are skipped.

## Complexity table of the Skip list

| S. No | Complexity        | Average case | Worst case    |
|-------|-------------------|--------------|---------------|
| 1.    | Access complexity | $O(\log n)$  | $O(n)$        |
| 2.    | Search complexity | $O(\log n)$  | $O(n)$        |
| 3.    | Delete complexity | $O(\log n)$  | $O(n)$        |
| 4.    | Insert complexity | $O(\log n)$  | $O(n)$        |
| 5.    | Space complexity  | -            | $O(n \log n)$ |

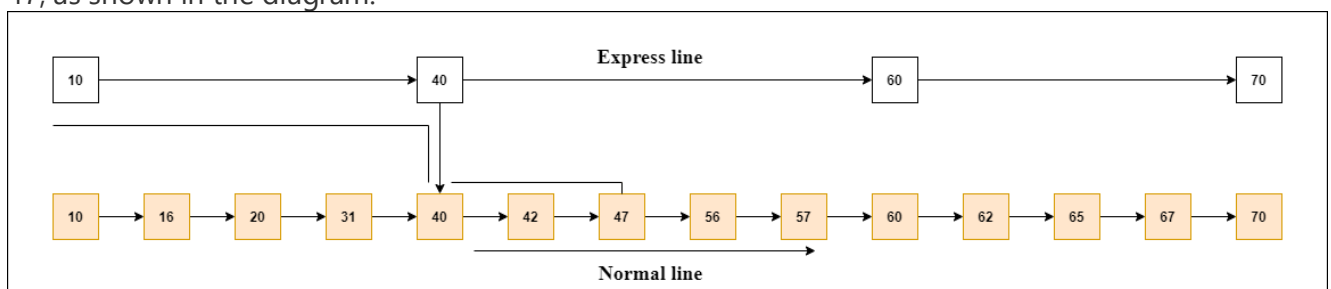
## Working of the Skip list

Let's take an example to understand the working of the skip list. In this example, we have 14 nodes, such that these nodes are divided into two layers, as shown in the diagram.

The lower layer is a common line that links all nodes, and the top layer is an express line that links only the main nodes, as you can see in the diagram.

Suppose you want to find 47 in this example. You will start the search from the first node of the express line and continue running on the express line until you find a node that is equal a 47 or more than 47.

You can see in the example that 47 does not exist in the express line, so you search for a node of less than 47, which is 40. Now, you go to the normal line with the help of 40, and search the 47, as shown in the diagram.



**Note:** Once you find a node like this on the "express line", you go from this node to a "normal lane" using a pointer, and when you search for the node in the normal line.

## Skip List Basic Operations

There are the following types of operations in the skip list.

**Insertion operation:** It is used to add a new node to a particular location in a specific situation.

**Deletion operation:** It is used to delete a node in a specific situation.

**Search Operation:** The search operation is used to search a particular node in a skip list.

### Algorithm of the insertion operation

Insertion (L, Key)

local update [0...Max\_Level + 1]

a = L → header

**for** i = L → level down to 0 **do**.

**while** a → forward[i] → key forward[i]

update[i] = a

a = a → forward[0]

lvl = random\_Level()

**if** lvl > L → level then

**for** i = L → level + 1 to lvl **do**

    update[i] = L → header

    L → level = lvl

a = makeNode(lvl, Key, value)

**for** i = 0 to level **do**

    a → forward[i] = update[i] → forward[i]

update[i] → forward[i] = a

### Algorithm of deletion operation

Deletion (L, Key)

local update [0... Max\_Level + 1]

a = L → header

**for** i = L → level down to 0 **do**.

**while** a → forward[i] → key forward[i]

    update[i] = a

a = a → forward[0]

**if** a → key = Key then

**for** i = 0 to L → level **do**

**if** update[i] → forward[i] ? a then **break**

        update[i] → forward[i] = a → forward[i]

    free(a)

**while** L → level > 0 and L → header → forward[L → level] = NIL **do**

        L → level = L → level - 1

### Algorithm of searching operation

Searching (L, SKey)

```

a = L → header
loop invariant: a → key level down to 0 do.
    while a → forward[i] → key forward[i]
    a = a → forward[0]
if a → key = SKey then return a → value
else return failure

```

**Example 1:** Create a skip list, we want to insert these following keys in the empty skip list.

6 with level 1.

29 with level 1.

22 with level 4.

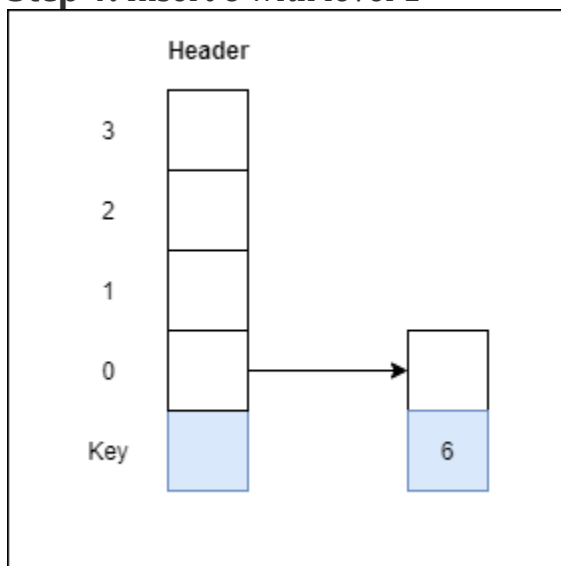
9 with level 3.

17 with level 1.

4 with level 2.

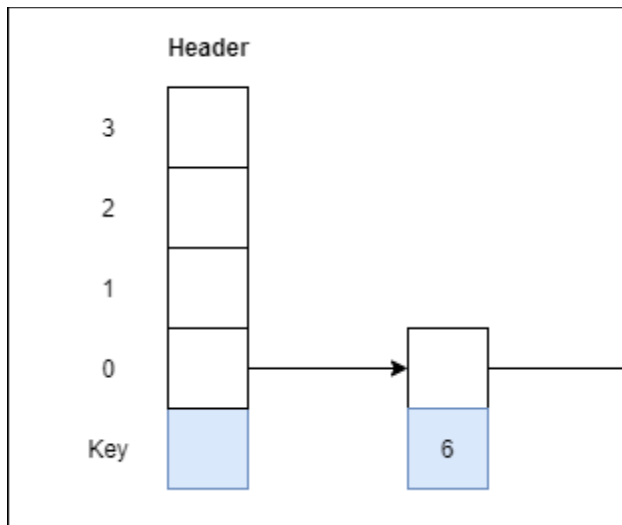
**Ans:**

**Step 1:** Insert 6 with level 1

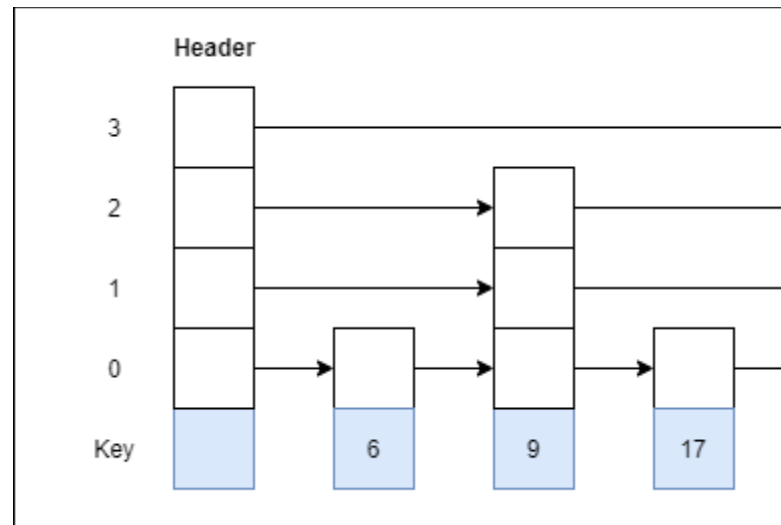


**Step 2:** Insert 29 with level 1

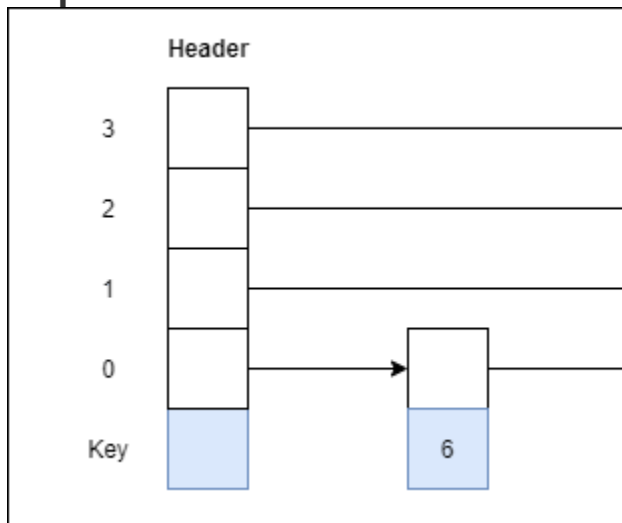




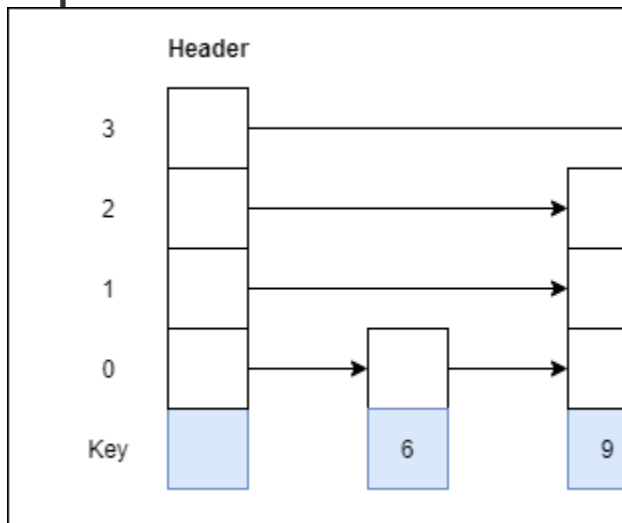
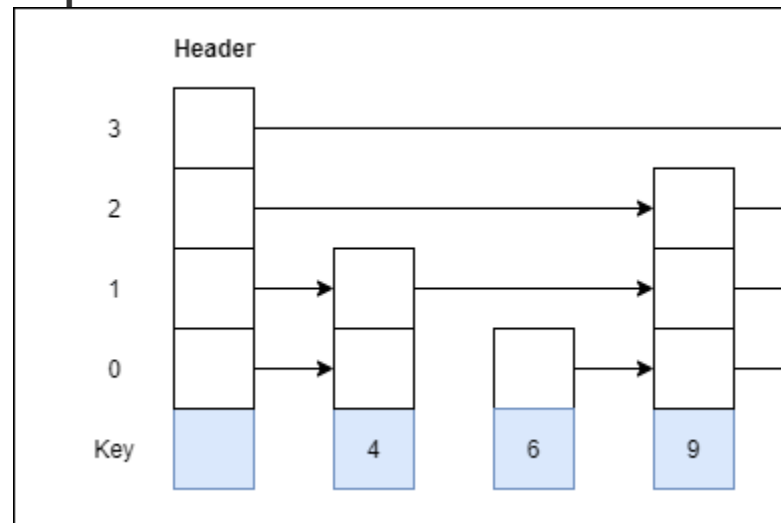
**Step 3:** Insert 22 with level 4



**Step 6:** Insert 4 with level 2

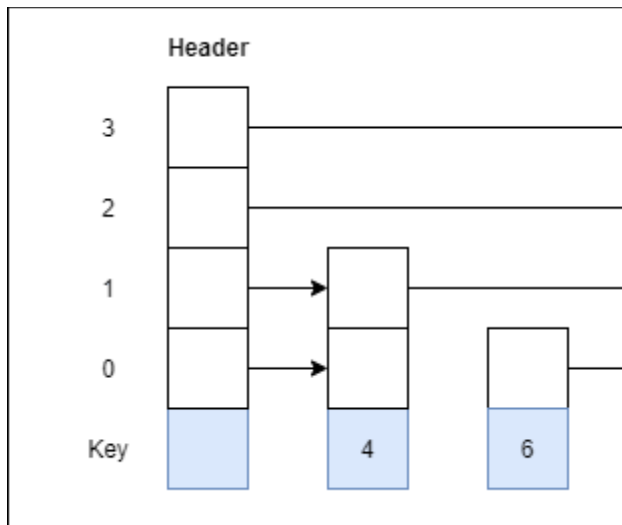


**Step 4:** Insert 9 with level 3

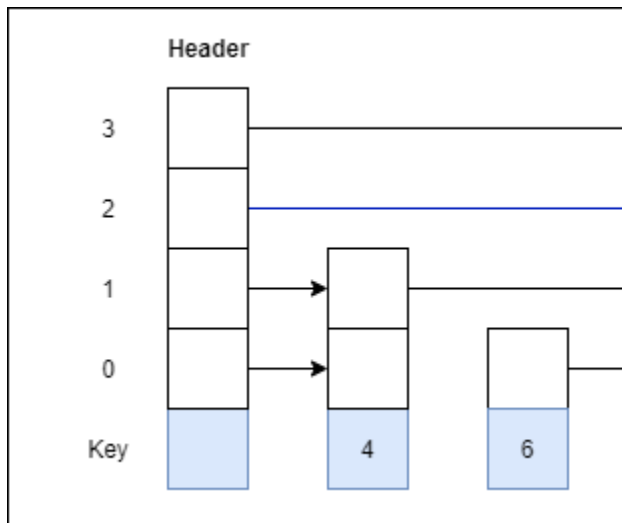


**Step 5:** Insert 17 with level 1

**Example 2:** Consider this example where we want to search for key 17.



**Ans:**



## Advantages of the Skip list

If you want to insert a new node in the skip list, then it will insert the node very fast because there are no rotations in the skip list.

The skip list is simple to implement as compared to the hash table and the binary search tree.

It is very simple to find a node in the list because it stores the nodes in sorted form.

The skip list algorithm can be modified very easily in a more specific structure, such as indexable skip lists, trees, or priority queues.

Increment the variable Top so that it can now refer to the next memory location.

The skip list is a robust and reliable list.

## Disadvantages of the Skip list

It requires more memory than the balanced tree.

Reverse searching is not allowed.

The skip list searches the node much slower than the linked list.

## Applications of the Skip list

It is used in distributed applications, and it represents the pointers and system in the distributed applications.

It is used to implement a dynamic elastic concurrent queue with low lock contention.

It is also used with the QMap template class.

The indexing of the skip list is used in running median problems.

The skip list is used for the delta-encoding posting in the Lucene search.

## Array implementation of Stack

In array implementation, the stack is formed by using the array. All the operations regarding the stack are performed using arrays. Let's see how each operation can be implemented on the stack using array data structure.

### Adding an element onto the stack (push operation)

Adding an element into the top of the stack is referred to as push operation. Push operation involves following two steps.

Add element at the position of incremented top. This is referred to as adding new element at the top of the stack.

Stack is overflown when we try to insert an element into a completely filled stack therefore, our main function must always avoid stack overflow condition.

**Algorithm:**

begin

**if** top = n then stack full

        top = top + 1

        stack (top) := item;

end

**Time Complexity :  $O(1)$**

## implementation of push algorithm in C language

**void** push (**int** val,**int** n) //n is size of the stack

```
{
    if (top == n )
        printf("\n Overflow");
    else
    {
        top = top +1;
        stack[top] = val;
    }
}
```

## Deletion of an element from a stack (Pop operation)

Deletion of an element from the top of the stack is called pop operation. The value of the variable top will be incremented by 1 whenever an item is deleted from the stack. The top most element of the stack is stored in an another variable and then the top is decremented by 1. the operation returns the deleted value that was stored in another variable as the result.

The underflow condition occurs when we try to delete an element from an already empty stack.

**Algorithm :**

begin

**if** top = 0 then stack empty;

        item := stack(top);

        top = top - 1;

end;

**Time Complexity :  $O(1)$**

## Implementation of POP algorithm using C language

**int** pop ()

```
{
    if(top == -1)
    {
        printf("Underflow");
        return 0;
    }
}
```

```

else
{
    return stack[top - - ];
}
}

```

## Visiting each element of the stack (Peek operation)

Peek operation involves returning the element which is present at the top of the stack without deleting it. Underflow condition can occur if we try to return the top element in an already empty stack.

### Algorithm :

PEEK (STACK, TOP)

Begin

```

if top = -1 then stack empty
item = stack[top]
return item

```

End

**Time complexity:  $O(n)$**

## Implementation of Peek algorithm in C language

```

int peek()
{
    if (top == -1)
    {
        printf("Underflow");
        return 0;
    }
    else
    {
        return stack [top];
    }
}

```

### C program

```

#include <stdio.h>
int stack[100],i,j,choice=0,n,top=-1;
void push();
void pop();
void show();
void main ()
{

    printf("Enter the number of elements in the stack ");
    scanf("%d",&n);
    printf("*****Stack operations using array*****");
}

```

```

printf("\n-----\n");
while(choice != 4)
{
    printf("Chose one from the below options...\n");
    printf("\n1.Push\n2.Pop\n3.Show\n4.Exit");
    printf("\n Enter your choice \n");
    scanf("%d",&choice);
    switch(choice)
    {
        case 1:
        {
            push();
            break;
        }
        case 2:
        {
            pop();
            break;
        }
        case 3:
        {
            show();
            break;
        }
        case 4:
        {
            printf("Exiting....");
            break;
        }
        default:
        {
            printf("Please Enter valid choice ");
        }
    };
}

```

```

void push ()
{
    int val;
    if (top == n )
        printf("\n Overflow");
    else
    {

```

```

        printf("Enter the value?");
        scanf("%d",&val);
        top = top + 1;
        stack[top] = val;
    }
}

```

```

void pop ()
{
    if(top == -1)
        printf("Underflow");
    else
        top = top - 1;
}

```

```

void show()
{
    for (i=top;i>=0;i--)
    {
        printf("%d\n",stack[i]);
    }
    if(top == -1)
    {
        printf("Stack is empty");
    }
}

```

### Java Program

```

import java.util.Scanner;
class Stack
{
    int top;
    int maxsize = 10;
    int[] arr = new int[maxsize];

```

```

    boolean isEmpty()
    {
        return (top < 0);
    }

```

```

    Stack()

```

```

    {
        top = -1;
    }

```

```

    boolean push (Scanner sc)
    {

```

```

    if(top == maxsize-1)
    {
        System.out.println("Overflow !!");
        return false;
    }
    else
    {
        System.out.println("Enter Value");
        int val = sc.nextInt();
        top++;
        arr[top]=val;
        System.out.println("Item pushed");
        return true;
    }
}
boolean pop ()
{
    if (top == -1)
    {
        System.out.println("Underflow !!");
        return false;
    }
    else
    {
        top --;
        System.out.println("Item popped");
        return true;
    }
}
void display ()
{
    System.out.println("Printing stack elements .....");
    for(int i = top; i>=0;i--)
    {
        System.out.println(arr[i]);
    }
}
}

public class Stack_Operations {
public static void main(String[] args) {
    int choice=0;
    Scanner sc = new Scanner(System.in);
    Stack s = new Stack();
    System.out.println("*****Stack operations using array*****\n");

```

```

System.out.println("\n-----\n");
while(choice != 4)
{
    System.out.println("\nChose one from the below options...\n");
    System.out.println("\n1.Push\n2.Pop\n3.Show\n4.Exit");
    System.out.println("\n Enter your choice \n");
    choice = sc.nextInt();
    switch(choice)
    {
        case 1:
        {
            s.push(sc);
            break;
        }
        case 2:
        {
            s.pop();
            break;
        }
        case 3:
        {
            s.display();
            break;
        }
        case 4:
        {
            System.out.println("Exiting....");
            System.exit(0);
            break;
        }
        default:
        {
            System.out.println("Please Enter valid choice ");
        }
    };
}
}
}

```

### C# Program

using System;

**public class** Stack

```

{
    int top;

```



```

int maxsize=10;
int[] arr = new int[10];
public static void Main()
{
    Stack st = new Stack();
    st.top=-1;
    int choice=0;
    Console.WriteLine("*****Stack operations using array*****");
    Console.WriteLine("\n-----\n");
    while(choice != 4)
    {
        Console.WriteLine("Chose one from the below options...\n");
        Console.WriteLine("\n1.Push\n2.Pop\n3.Show\n4.Exit");
        Console.WriteLine("\n Enter your choice \n");
        choice = Convert.ToInt32(Console.ReadLine());
        switch(choice)
        {
            case 1:
            {
                st.push();
                break;
            }
            case 2:
            {
                st.pop();
                break;
            }
            case 3:
            {
                st.show();
                break;
            }
            case 4:
            {
                Console.WriteLine("Exiting....");
                break;
            }
            default:
            {
                Console.WriteLine("Please Enter valid choice ");
                break;
            }
        }
    };
}

```

```
}
```

```
Boolean push ()
```

```
{
```

```
    int val;
```

```
    if(top == maxsize-1)
```

```
    {
```

```
        Console.WriteLine("\n Overflow");
```

```
        return false;
```

```
    }
```

```
    else
```

```
    {
```

```
        Console.WriteLine("Enter the value?");
```

```
        val = Convert.ToInt32(Console.ReadLine());
```

```
        top = top +1;
```

```
        arr[top] = val;
```

```
        Console.WriteLine("Item pushed");
```

```
        return true;
```

```
    }
```

```
}
```

```
Boolean pop ()
```

```
{
```

```
    if (top == -1)
```

```
    {
```

```
        Console.WriteLine("Underflow");
```

```
        return false;
```

```
    }
```

```
    else
```

```
    {
```

```
        top = top -1;
```

```
        Console.WriteLine("Item popped");
```

```
        return true;
```

```
    }
```

```
}
```

```
void show()
```

```
{
```

```
    for (int i=top;i>=0;i--)
```

```
    {
```

```
        Console.WriteLine(arr[i]);
```

```

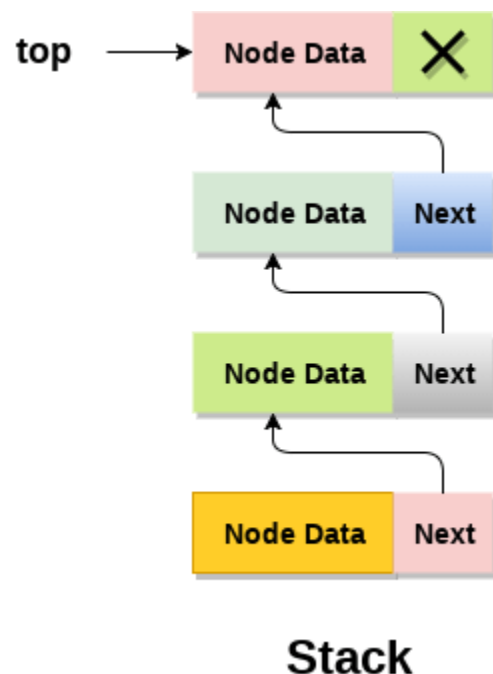
}
if(top == -1)
{
    Console.WriteLine("Stack is empty");
}
}
}

```

## Linked list implementation of stack

Instead of using array, we can also use linked list to implement stack. Linked list allocates the memory dynamically. However, time complexity in both the scenario is same for all the operations i.e. push, pop and peek.

In linked list implementation of stack, the nodes are maintained non-contiguously in the memory. Each node contains a pointer to its immediate successor node in the stack. Stack is said to be overflown if the space left in the memory heap is not enough to create a node.



The top most node in the stack always contains null in its address field. Lets discuss the way in which, each operation is performed in linked list implementation of stack.

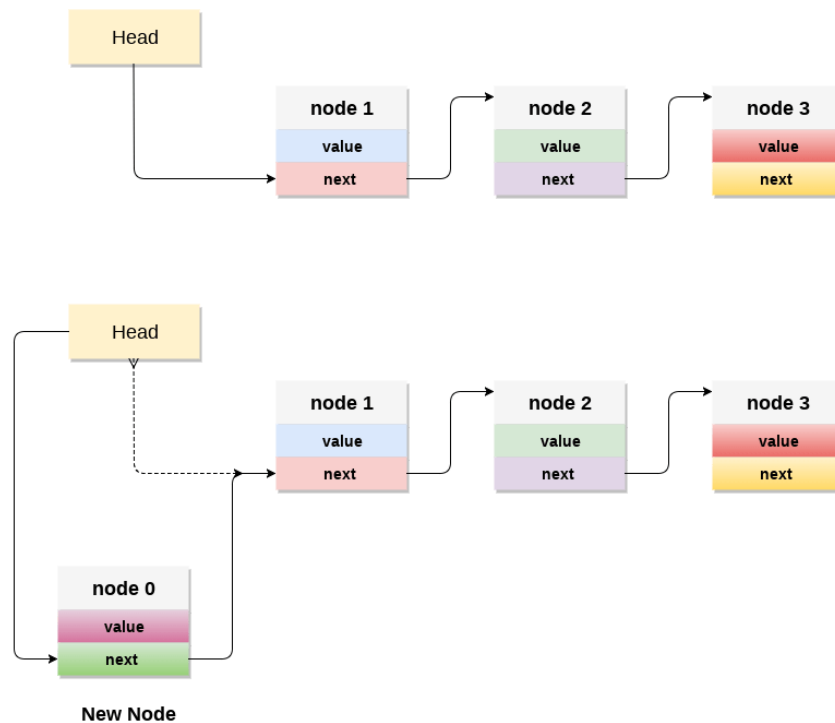
### Adding a node to the stack (Push operation)

Adding a node to the stack is referred to as **push** operation. Pushing an element to a stack in linked list implementation is different from that of an array implementation. In order to push an element onto the stack, the following steps are involved.

Create a node first and allocate memory to it.

If the list is empty then the item is to be pushed as the start node of the list. This includes assigning value to the data part of the node and assign null to the address part of the node. If there are some nodes in the list already, then we have to add the new element in the beginning of the list (to not violate the property of the stack). For this purpose, assign the address of the starting element to the address field of the new node and make the new node, the starting node of the list.

**Time Complexity :  $O(1)$**



## C implementation :

```
void push ()
{
    int val;
    struct node *ptr =(struct node*)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("not able to push the element");
    }
    else
    {
        printf("Enter the value");
        scanf("%d",&val);
        if(head==NULL)
        {
            ptr->val = val;
            ptr -> next = NULL;
```

```

        head=ptr;
    }
    else
    {
        ptr->val = val;
        ptr->next = head;
        head=ptr;
    }
    printf("Item pushed");
}
}

```

## Deleting a node from the stack (POP operation)

Deleting a node from the top of stack is referred to as **pop** operation. Deleting a node from the linked list implementation of stack is different from that in the array implementation. In order to pop an element from the stack, we need to follow the following steps :

**Check for the underflow condition:** The underflow condition occurs when we try to pop from an already empty stack. The stack will be empty if the head pointer of the list points to null.

**Adjust the head pointer accordingly:** In stack, the elements are popped only from one end, therefore, the value stored in the head pointer must be deleted and the node must be freed. The next node of the head node now becomes the head node.

**Time Complexity :  $O(1)$**

## C implementation

```

void pop()
{
    int item;
    struct node *ptr;
    if (head == NULL)
    {
        printf("Underflow");
    }
    else
    {
        item = head->val;
        ptr = head;
        head = head->next;
        free(ptr);
        printf("Item popped");
    }
}

```

## Display the nodes (Traversing)

Displaying all the nodes of a stack needs traversing all the nodes of the linked list organized in the form of stack. For this purpose, we need to follow the following steps.

Copy the head pointer into a temporary pointer.

Move the temporary pointer through all the nodes of the list and print the value field attached to every node.

**Time Complexity :  $O(n)$**

## C Implementation

**void** display()

```
{
    int i;
    struct node *ptr;
    ptr=head;
    if(ptr == NULL)
    {
        printf("Stack is empty\n");
    }
    else
    {
        printf("Printing Stack elements \n");
        while(ptr!=NULL)
        {
            printf("%d\n",ptr->val);
            ptr = ptr->next;
        }
    }
}
```

## Menu Driven program in C implementing all the stack operations using linked list :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void push();
```

```
void pop();
```

```
void display();
```

```
struct node
```

```
{
```

```
    int val;
```

```
    struct node *next;
```

```
};
```

```
struct node *head;
```

```
void main ()
```

```
{
```

```
    int choice=0;
```

```

printf("\n*****Stack operations using linked list*****\n");
printf("\n-----\n");
while(choice != 4)
{
    printf("\n\nChose one from the below options...\n");
    printf("\n1.Push\n2.Pop\n3.Show\n4.Exit");
    printf("\n Enter your choice \n");
    scanf("%d",&choice);
    switch(choice)
    {
        case 1:
        {
            push();
            break;
        }
        case 2:
        {
            pop();
            break;
        }
        case 3:
        {
            display();
            break;
        }
        case 4:
        {
            printf("Exiting....");
            break;
        }
        default:
        {
            printf("Please Enter valid choice ");
        }
    }
};
}
}
void push ()
{
    int val;
    struct node *ptr = (struct node*)malloc(sizeof(struct node));
    if(ptr == NULL)
    {
        printf("not able to push the element");
    }
}

```

```

    }
    else
    {
        printf("Enter the value");
        scanf("%d",&val);
        if(head==NULL)
        {
            ptr->val = val;
            ptr -> next = NULL;
            head=ptr;
        }
        else
        {
            ptr->val = val;
            ptr->next = head;
            head=ptr;
        }
        printf("Item pushed");
    }
}

```

```

void pop()
{
    int item;
    struct node *ptr;
    if (head == NULL)
    {
        printf("Underflow");
    }
    else
    {
        item = head->val;
        ptr = head;
        head = head->next;
        free(ptr);
        printf("Item popped");
    }
}

void display()
{
    int i;

```



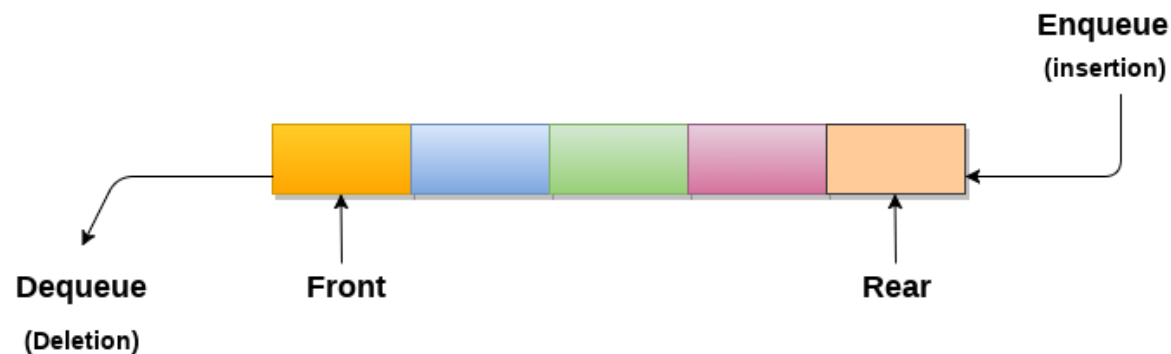
```

struct node *ptr;
ptr=head;
if(ptr == NULL)
{
    printf("Stack is empty\n");
}
else
{
    printf("Printing Stack elements \n");
    while(ptr!=NULL)
    {
        printf("%d\n",ptr->val);
        ptr = ptr->next;
    }
}
}

```

## Queue

1. A queue can be defined as an ordered list which enables insert operations to be performed at one end called **REAR** and delete operations to be performed at another end called **FRONT**.
2. Queue is referred to be as First In First Out list.
3. For example, people waiting in line for a rail ticket form a queue.



## Applications of Queue

Due to the fact that queue performs actions on first in first out basis which is quite fair for the ordering of actions. There are various applications of queues discussed as below.

Queues are widely used as waiting lists for a single shared resource like printer, disk, CPU.

Queues are used in asynchronous transfer of data (where data is not being transferred at the same rate between two processes) for eg. pipes, file IO, sockets.

Queues are used as buffers in most of the applications like MP3 media player, CD player, etc.

| Data Structure | Time Complexity |             |             |             |        |        |           |          | Space Complexity |
|----------------|-----------------|-------------|-------------|-------------|--------|--------|-----------|----------|------------------|
|                | Average         |             |             |             | Worst  |        |           |          |                  |
|                | Access          | Search      | Insertion   | Deletion    | Access | Search | Insertion | Deletion |                  |
| Queue          | $\theta(n)$     | $\theta(n)$ | $\theta(1)$ | $\theta(1)$ | $O(n)$ | $O(n)$ | $O(1)$    | $O(1)$   | $O(n)$           |

Queue are used to maintain the play list in media players in order to add and remove the songs from the play-list.

Queues are used in operating systems for handling interrupts.

## Complexity

## Types of Queue

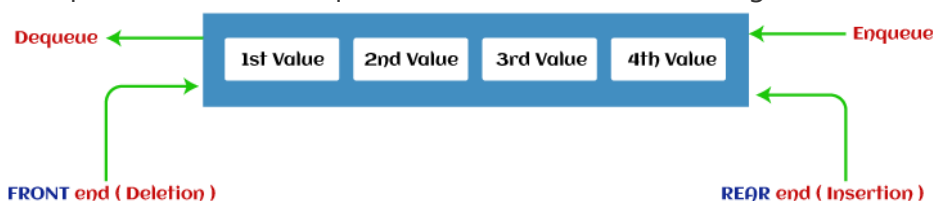
In this article, we will discuss the types of queue. But before moving towards the types, we should first discuss the brief introduction of the queue.

### What is a Queue?

Queue is the data structure that is similar to the queue in the real world. A queue is a data structure in which whatever comes first will go out first, and it follows the FIFO (First-In-First-Out) policy. Queue can also be defined as the list or collection in which the insertion is done from one end known as the **rear end** or the **tail** of the queue, whereas the deletion is done from another end known as the **front end** or the **head** of the queue.

The real-world example of a queue is the ticket queue outside a cinema hall, where the person who enters first in the queue gets the ticket first, and the last person enters in the queue gets the ticket at last. Similar approach is followed in the queue in data structure.

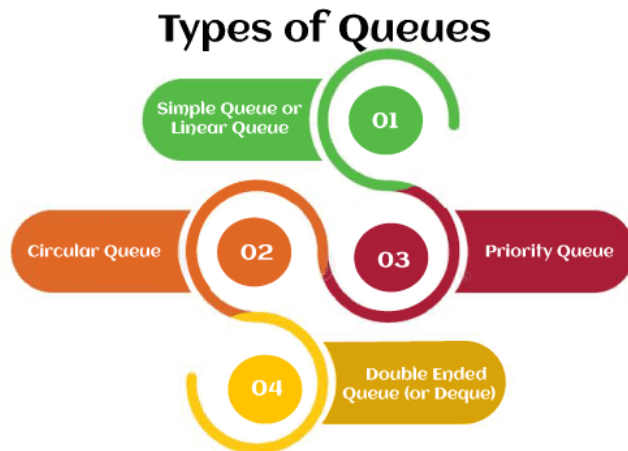
The representation of the queue is shown in the below image -



Now, let's move towards the types of queue.

## Types of Queue

There are four different types of queue that are listed as follows -



Simple Queue or Linear Queue

Circular Queue

Priority Queue

Double Ended Queue (or Deque)

Let's discuss each of the type of queue.

### Simple Queue or Linear Queue

In Linear Queue, an insertion takes place from one end while the deletion occurs from another end. The end at which the insertion takes place is known as the rear end, and the end at which the deletion takes place is known as front end. It strictly follows the FIFO rule.



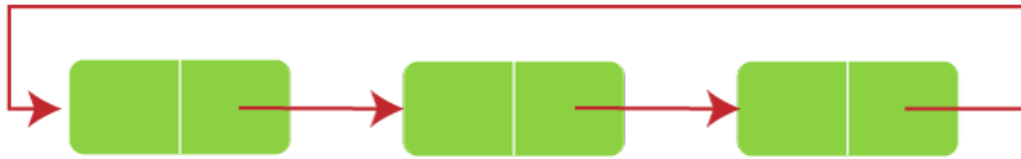
The major drawback of using a linear Queue is that insertion is done only from the rear end. If the first three elements are deleted from the Queue, we cannot insert more elements even though the space is available in a Linear Queue. In this case, the linear Queue shows the overflow condition as the rear is pointing to the last element of the Queue.

To know more about the queue in data structure, you can click the link

- <https://www.javatpoint.com/data-structure-queue>

### Circular Queue

In Circular Queue, all the nodes are represented as circular. It is similar to the linear Queue except that the last element of the queue is connected to the first element. It is also known as Ring Buffer, as all the ends are connected to another end. The representation of circular queue is shown in the below image -



**Circular Queue**

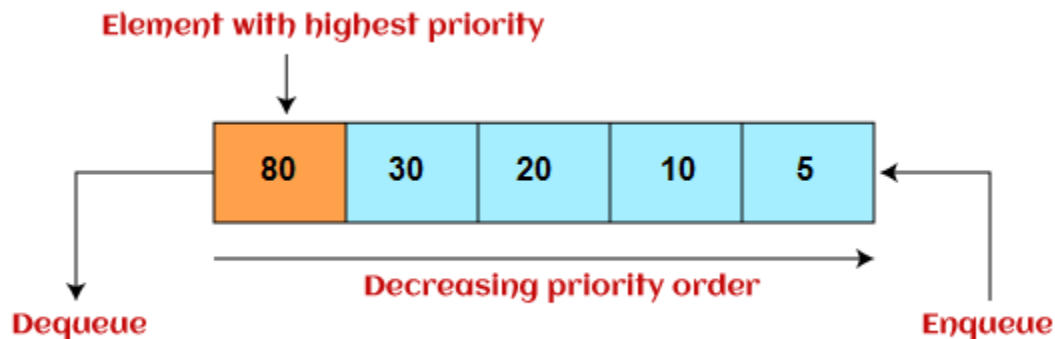
The drawback that occurs in a linear queue is overcome by using the circular queue. If the empty space is available in a circular queue, the new element can be added in an empty space by simply incrementing the value of rear. The main advantage of using the circular queue is better memory utilization.

To know more about the circular queue, you can click the link

- <https://www.javatpoint.com/circular-queue>

## Priority Queue

It is a special type of queue in which the elements are arranged based on the priority. It is a special type of queue data structure in which every element has a priority associated with it. Suppose some elements occur with the same priority, they will be arranged according to the FIFO principle. The representation of priority queue is shown in the below image -



Insertion in priority queue takes place based on the arrival, while deletion in the priority queue occurs based on the priority. Priority queue is mainly used to implement the CPU scheduling algorithms.

There are two types of priority queue that are discussed as follows -

**Ascending priority queue** - In ascending priority queue, elements can be inserted in arbitrary order, but only smallest can be deleted first. Suppose an array with elements 7, 5, and 3 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 3, 5, 7.

**Descending priority queue** - In descending priority queue, elements can be inserted in arbitrary order, but only the largest element can be deleted first. Suppose an array with elements 7, 3, and 5 in the same order, so, insertion can be done with the same sequence, but the order of deleting the elements is 7, 5, 3.

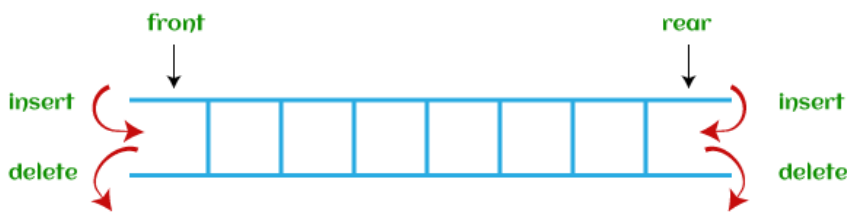
To learn more about the priority queue, you can click the link - <https://www.javatpoint.com/ds-priority-queue>

## Deque (or, Double Ended Queue)

In Deque or Double Ended Queue, insertion and deletion can be done from both ends of the queue either from the front or rear. It means that we can insert and delete elements from both front and rear ends of the queue. Deque can be used as a palindrome checker means that if we read the string from both ends, then the string would be the same.

Deque can be used both as stack and queue as it allows the insertion and deletion operations on both ends. Deque can be considered as stack because stack follows the LIFO (Last In First Out) principle in which insertion and deletion both can be performed only from one end. And in deque, it is possible to perform both insertion and deletion from one end, and Deque does not follow the FIFO principle.

The representation of the deque is shown in the below image -

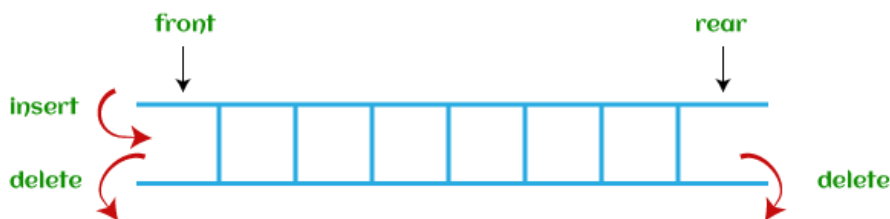


**double ended queue**

To know more about the deque, you can click the link - <https://www.javatpoint.com/ds-deque>

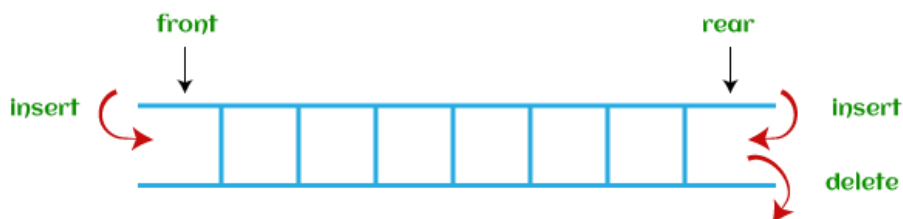
There are two types of deque that are discussed as follows -

**Input restricted deque** - As the name implies, in input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



**input restricted double ended queue**

**Output restricted deque** - As the name implies, in output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



**Output restricted double ended queue**

Now, let's see the operations performed on the queue.

## Operations performed on queue

The fundamental operations that can be performed on queue are listed as follows -

**Enqueue:** The Enqueue operation is used to insert the element at the rear end of the queue. It returns void.

**Dequeue:** It performs the deletion from the front-end of the queue. It also returns the element which has been removed from the front-end. It returns an integer value.

**Peek:** This is the third operation that returns the element, which is pointed by the front pointer in the queue but does not delete it.

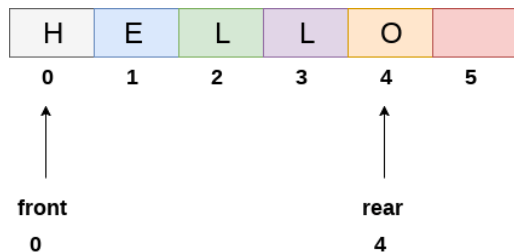
**Queue overflow (isfull):** It shows the overflow condition when the queue is completely full.

**Queue underflow (isempty):** It shows the underflow condition when the Queue is empty, i.e., no elements are in the Queue.

Now, let's see the ways to implement the queue.

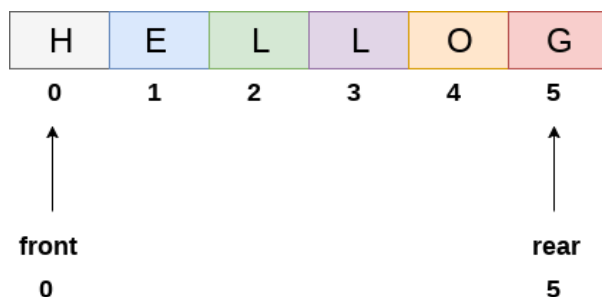
## Array representation of Queue

We can easily represent queue by using linear arrays. There are two variables i.e. front and rear, that are implemented in the case of every queue. Front and rear variables point to the position from where insertions and deletions are performed in a queue. Initially, the value of front and queue is -1 which represents an empty queue. Array representation of a queue containing 5 elements along with the respective values of front and rear, is shown in the following figure.



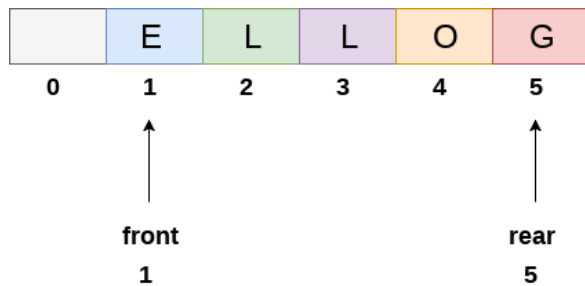
Queue

The above figure shows the queue of characters forming the English word "**HELLO**". Since, No deletion is performed in the queue till now, therefore the value of front remains -1. However, the value of rear increases by one every time an insertion is performed in the queue. After inserting an element into the queue shown in the above figure, the queue will look something like following. The value of rear will become 5 while the value of front remains same.



Queue after inserting an element

After deleting an element, the value of front will increase from -1 to 0. however, the queue will look something like following.



Queue after deleting an element

## Algorithm to insert any element in a queue

Check if the queue is already full by comparing rear to max - 1. if so, then return an overflow error.

If the item is to be inserted as the first element in the list, in that case set the value of front and rear to 0 and insert the element at the rear end.

Otherwise keep increasing the value of rear and insert each element one by one having rear as the index.

### Algorithm

**Step 1:** IF REAR = MAX - 1

Write OVERFLOW

Go to step

[END OF IF]

**Step 2:** IF FRONT = -1 and REAR = -1

SET FRONT = REAR = 0

ELSE

SET REAR = REAR + 1

[END OF IF]

**Step 3:** Set QUEUE[REAR] = NUM

**Step 4:** EXIT

### C Function

**void** insert (**int** queue[], **int** max, **int** front, **int** rear, **int** item)

```
{
    if (rear + 1 == max)
    {
        printf("overflow");
    }
    else
    {
        if(front == -1 && rear == -1)
        {
            front = 0;
            rear = 0;
        }
    }
}
```

```

    }
    else
    {
        rear = rear + 1;
    }
    queue[rear]=item;
}
}

```

## Algorithm to delete an element from the queue

If, the value of front is -1 or value of front is greater than rear , write an underflow message and exit.

Otherwise, keep increasing the value of front and return the item stored at the front end of the queue at each time.

### Algorithm

**Step 1:** IF FRONT = -1 or FRONT > REAR

Write UNDERFLOW

ELSE

SET VAL = QUEUE[FRONT]

SET FRONT = FRONT + 1

[END OF IF]

**Step 2:** EXIT

### C Function

**int** delete (**int** queue[], **int** max, **int** front, **int** rear)

```

{
    int y;
    if (front == -1 || front > rear)

    {
        printf("underflow");
    }
    else
    {
        y = queue[front];
        if(front == rear)
        {
            front = rear = -1;
        }
        else
            front = front + 1;

    }
    return y;
}
}

```



## Menu driven program to implement queue using array

```
#include<stdio.h>
#include<stdlib.h>
#define maxsize 5
void insert();
void delete();
void display();
int front = -1, rear = -1;
int queue[maxsize];
void main ()
{
    int choice;
    while(choice != 4)
    {
        printf("\n*****Main Menu*****\n");
        printf("\n=====
=====\\n");
        printf("\n1.insert an element\\n2.Delete an element\\n3.Display the queue\\n4.Exit\\n");
        printf("\nEnter your choice ?");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:
                insert();
                break;
            case 2:
                delete();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
                break;
            default:
                printf("\nEnter valid choice??\\n");
        }
    }
}

void insert()
{
    int item;
```

```

printf("\nEnter the element\n");
scanf("\n%d",&item);
if(rear == maxsize-1)
{
    printf("\nOVERFLOW\n");
    return;
}
if(front == -1 && rear == -1)
{
    front = 0;
    rear = 0;
}
else
{
    rear = rear+1;
}
queue[rear] = item;
printf("\nValue inserted ");

}
void delete()
{
    int item;
    if (front == -1 || front > rear)
    {
        printf("\nUNDERFLOW\n");
        return;
    }
    else
    {
        item = queue[front];
        if(front == rear)
        {
            front = -1;
            rear = -1 ;
        }
        else
        {
            front = front + 1;
        }
        printf("\nvalue deleted ");
    }
}

```

```

}

void display()
{
    int i;
    if(rear == -1)
    {
        printf("\nEmpty queue\n");
    }
    else
    { printf("\nprinting values ..... \n");
      for(i=front;i<=rear;i++)
      {
          printf("\n%d\n",queue[i]);
      }
    }
}

```

### Output:

```
*****Main Menu*****
```

```
=====
```

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?1

Enter the element  
123

Value inserted

```
*****Main Menu*****
```

```
=====
```

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?1

Enter the element  
90

Value inserted

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?2

value deleted

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?3

printing values .....

90

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

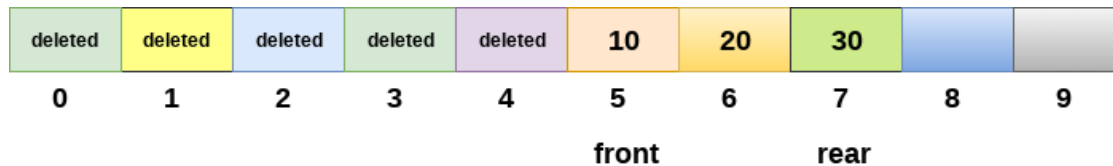
- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?4

## Drawback of array implementation

Although, the technique of creating a queue is easy, but there are some drawbacks of using this technique to implement a queue.

**Memory wastage :** The space of the array, which is used to store queue elements, can never be reused to store the elements of that queue because the elements can only be inserted at front end and the value of front might be so high so that, all the space before that, can never be filled.



## limitation of array representation of queue

The above figure shows how the memory space is wasted in the array representation of queue. In the above figure, a queue of size 10 having 3 elements, is shown. The value of the front variable is 5, therefore, we can not reinstate the values in the place of already deleted element before the position of front. That much space of the array is wasted and can not be used in the future (for this queue).

### Deciding the array size

One of the most common problem with array implementation is the size of the array which requires to be declared in advance. Due to the fact that, the queue can be extended at runtime depending upon the problem, the extension in the array size is a time taking process and almost impossible to be performed at runtime since a lot of reallocations take place. Due to this reason, we can declare the array large enough so that we can store queue elements as enough as possible but the main problem with this declaration is that, most of the array slots (nearly half) can never be reused. It will again lead to memory wastage.

## Linked List implementation of Queue

Due to the drawbacks discussed in the previous section of this tutorial, the array implementation can not be used for the large scale applications where the queues are implemented. One of the alternative of array implementation is linked list implementation of queue.

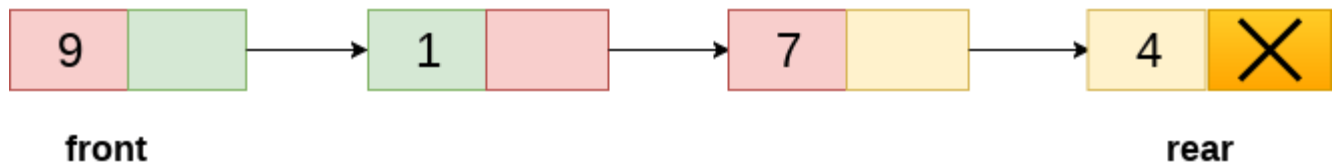
The storage requirement of linked representation of a queue with  $n$  elements is  $O(n)$  while the time requirement for operations is  $O(1)$ .

In a linked queue, each node of the queue consists of two parts i.e. data part and the link part. Each element of the queue points to its immediate next element in the memory.

In the linked queue, there are two pointers maintained in the memory i.e. front pointer and rear pointer. The front pointer contains the address of the starting element of the queue while the rear pointer contains the address of the last element of the queue.

Insertion and deletions are performed at rear and front end respectively. If front and rear both are NULL, it indicates that the queue is empty.

The linked representation of queue is shown in the following figure.



## Linked Queue

### Operation on Linked Queue

There are two basic operations which can be implemented on the linked queues. The operations are Insertion and Deletion.

#### Insert operation

The insert operation append the queue by adding an element to the end of the queue. The new element will be the last element of the queue.

Firstly, allocate the memory for the new node ptr by using the following statement.

```
Ptr = (struct node *) malloc (sizeof(struct node));
```

There can be the two scenario of inserting this new node ptr into the linked queue.

In the first scenario, we insert element into an empty queue. In this case, the condition **front = NULL** becomes true. Now, the new element will be added as the only element of the queue and the next pointer of front and rear pointer both, will point to NULL.

```
ptr -> data = item;
if(front == NULL)
{
    front = ptr;
    rear = ptr;
    front -> next = NULL;
    rear -> next = NULL;
}
```

In the second case, the queue contains more than one element. The condition front = NULL becomes false. In this scenario, we need to update the end pointer rear so that the next pointer of rear will point to the new node ptr. Since, this is a linked queue, hence we also need to make the rear pointer point to the newly added node **ptr**. We also need to make the next pointer of rear point to NULL.

```
rear -> next = ptr;
rear = ptr;
rear->next = NULL;
```

In this way, the element is inserted into the queue. The algorithm and the C implementation is given as follows.

### Algorithm

**Step 1:** Allocate the space for the new node PTR

**Step 2:** SET PTR -> DATA = VAL

**Step 3:** IF FRONT = NULL

SET FRONT = REAR = PTR

SET FRONT -> NEXT = REAR -> NEXT = NULL

ELSE

SET REAR -> NEXT = PTR

SET REAR = PTR

SET REAR -> NEXT = NULL

[END OF IF]

**Step 4:** END

## C Function

**void** insert(struct node \*ptr, **int** item; )

{

ptr = (struct node \*) malloc (sizeof(struct node));

**if**(ptr == NULL)

{

printf("\nOVERFLOW\n");

**return**;

}

**else**

{

ptr -> data = item;

**if**(front == NULL)

{

front = ptr;

rear = ptr;

front -> next = NULL;

rear -> next = NULL;

}

**else**

{

rear -> next = ptr;

rear = ptr;

rear->next = NULL;

}

}

}

## Deletion

Deletion operation removes the element that is first inserted among all the queue elements.

Firstly, we need to check either the list is empty or not. The condition front == NULL becomes true if the list is empty, in this case , we simply write underflow on the console and make exit.

Otherwise, we will delete the element that is pointed by the pointer front. For this purpose, copy the node pointed by the front pointer into the pointer ptr. Now, shift the front pointer, point to its next node and free the node pointed by the node ptr. This is done by using the following statements.

```
ptr = front;
    front = front -> next;
    free(ptr);
```

The algorithm and C function is given as follows.

## Algorithm

**Step 1:** IF FRONT = NULL

Write " Underflow "

Go to Step 5

[END OF IF]

**Step 2:** SET PTR = FRONT

**Step 3:** SET FRONT = FRONT -> NEXT

**Step 4:** FREE PTR

**Step 5:** END

## C Function

```
void delete (struct node *ptr)
{
    if(front == NULL)
    {
        printf("\nUNDERFLOW\n");
        return;
    }
    else
    {
        ptr = front;
        front = front -> next;
        free(ptr);
    }
}
```

## Menu-Driven Program implementing all the operations on Linked Queue

```
#include<stdio.h>
#include<stdlib.h>
struct node
{
    int data;
    struct node *next;
};
struct node *front;
```



```

struct node *rear;
void insert();
void delete();
void display();
void main ()
{
    int choice;
    while(choice != 4)
    {
        printf("\n*****Main Menu*****\n");
        printf("\n=====
=====\\n");
        printf("\n1.insert an element\n2.Delete an element\n3.Display the queue\n4.Exit\n");
        printf("\nEnter your choice ?");
        scanf("%d",& choice);
        switch(choice)
        {
            case 1:
                insert();
                break;
            case 2:
                delete();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
                break;
            default:
                printf("\nEnter valid choice??\\n");
        }
    }
}

void insert()
{
    struct node *ptr;
    int item;

    ptr = (struct node *) malloc (sizeof(struct node));
    if(ptr == NULL)
    {
        printf("\nOVERFLOW\\n");
        return;
    }
}

```

```

}
else
{
    printf("\nEnter value?\n");
    scanf("%d",&item);
    ptr -> data = item;
    if(front == NULL)
    {
        front = ptr;
        rear = ptr;
        front -> next = NULL;
        rear -> next = NULL;
    }
    else
    {
        rear -> next = ptr;
        rear = ptr;
        rear->next = NULL;
    }
}
}

void delete ()
{
    struct node *ptr;
    if(front == NULL)
    {
        printf("\nUNDERFLOW\n");
        return;
    }
    else
    {
        ptr = front;
        front = front -> next;
        free(ptr);
    }
}

void display()
{
    struct node *ptr;
    ptr = front;
    if(front == NULL)
    {
        printf("\nEmpty queue\n");
    }
}

```

```

else
{ printf("\nprinting values .....\\n");
  while(ptr != NULL)
  {
    printf("\\n%d\\n",ptr -> data);
    ptr = ptr -> next;
  }
}
}

```

### Output:

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?1

Enter value?

123

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?1

Enter value?

90

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?3

printing values .....

123

90

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?2

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?3

printing values .....

90

\*\*\*\*\*Main Menu\*\*\*\*\*

=====

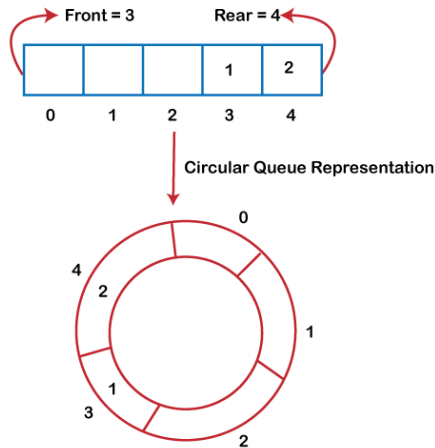
- 1.insert an element
- 2.Delete an element
- 3.Display the queue
- 4.Exit

Enter your choice ?4

## Circular Queue

Why was the concept of the circular queue introduced?

There was one limitation in the array implementation of [Queue](#). If the rear reaches to the end position of the Queue then there might be possibility that some vacant spaces are left in the beginning which cannot be utilized. So, to overcome such limitations, the concept of the circular queue was introduced.



As we can see in the above image, the rear is at the last position of the Queue and front is pointing somewhere rather than the 0<sup>th</sup> position. In the above array, there are only two elements and other three positions are empty. The rear is at the last position of the Queue; if we try to insert the element then it will show that there are no empty spaces in the Queue. There is one solution to avoid such wastage of memory space by shifting both the elements at the left and adjust the front and rear end accordingly. It is not a practically good approach because shifting all the elements will consume lots of time. The efficient approach to avoid the wastage of the memory is to use the circular queue data structure.

## What is a Circular Queue?

A circular queue is similar to a linear queue as it is also based on the FIFO (First In First Out) principle except that the last position is connected to the first position in a circular queue that forms a circle. It is also known as a **Ring Buffer**.

## Operations on Circular Queue

The following are the operations that can be performed on a circular queue:

**Front:** It is used to get the front element from the Queue.

**Rear:** It is used to get the rear element from the Queue.

**enQueue(value):** This function is used to insert the new value in the Queue. The new element is always inserted from the rear end.

**deQueue():** This function deletes an element from the Queue. The deletion in a Queue always takes place from the front end.

## Applications of Circular Queue

**The circular Queue can be used in the following scenarios:**

**Memory management:** The circular queue provides memory management. As we have already seen that in linear queue, the memory is not managed very efficiently. But in case of a circular queue, the memory is managed efficiently by placing the elements in a location which is unused.

**CPU Scheduling:** The operating system also uses the circular queue to insert the processes and then execute them.

**Traffic system:** In a computer-control traffic system, traffic light is one of the best examples of the circular queue. Each light of traffic light gets ON one by one after every jinterval of time. Like red light gets ON for one minute then yellow light for one minute and then green light. After green light, the red light gets ON.

## Enqueue operation

**The steps of enqueue operation are given below:**

First, we will check whether the Queue is full or not.

Initially the front and rear are set to -1. When we insert the first element in a Queue, front and rear both are set to 0.

When we insert a new element, the rear gets incremented, i.e.,  **$rear=rear+1$** .

## Scenarios for inserting an element

**There are two scenarios in which queue is not full:**

**If  $rear \neq max - 1$ ,** then rear will be incremented to  **$mod(maxsize)$**  and the new value will be inserted at the rear end of the queue.

**If  $front = 0$  and  $rear = max - 1$ ,** it means that queue is not full, then set the value of rear to 0 and insert the new element there.

**There are two cases in which the element cannot be inserted:**

When  **$front = 0$  &&  $rear = max-1$** , which means that front is at the first position of the Queue and rear is at the last position of the Queue.

$front == rear + 1$ ;

**Algorithm to insert an element in a circular queue**

**Step 1:** IF  $(REAR+1)\%MAX = FRONT$

Write " OVERFLOW "

Goto step 4

[End OF IF]

**Step 2:** IF  $FRONT = -1$  and  $REAR = -1$

SET  $FRONT = REAR = 0$

ELSE IF  $REAR = MAX - 1$  and  $FRONT \neq 0$

SET  $REAR = 0$

ELSE

SET  $REAR = (REAR + 1) \% MAX$

[END OF IF]

**Step 3:** SET  $QUEUE[REAR] = VAL$

**Step 4:** EXIT

## Dequeue Operation

The steps of dequeue operation are given below:

First, we check whether the Queue is empty or not. If the queue is empty, we cannot perform the dequeue operation.

When the element is deleted, the value of front gets decremented by 1.

If there is only one element left which is to be deleted, then the front and rear are reset to -1.

**Algorithm to delete an element from the circular queue**

**Step 1:** IF  $FRONT = -1$

Write " UNDERFLOW "

Goto Step 4

[END of IF]

**Step 2:** SET VAL = QUEUE[FRONT]

**Step 3:** IF FRONT = REAR

SET FRONT = REAR = -1

ELSE

IF FRONT = MAX -1

SET FRONT = 0

ELSE

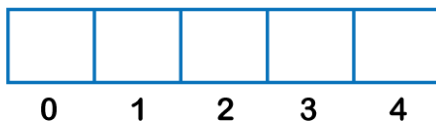
SET FRONT = FRONT + 1

[END of IF]

[END OF IF]

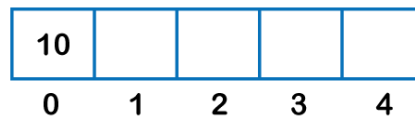
**Step 4:** EXIT

**Let's understand the enqueue and dequeue operation through the diagrammatic representation.**



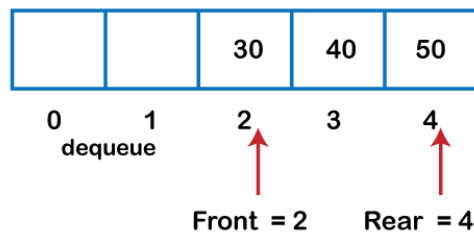
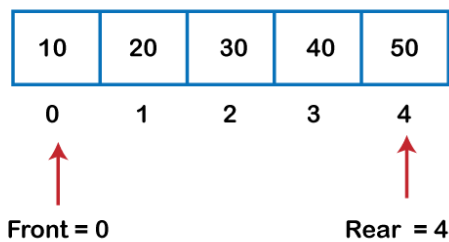
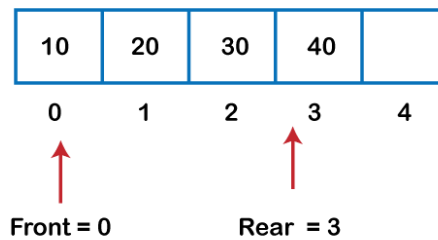
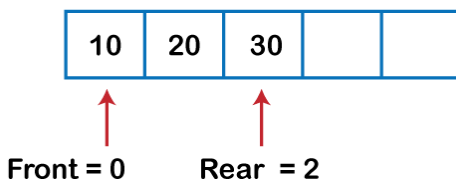
Front = -1

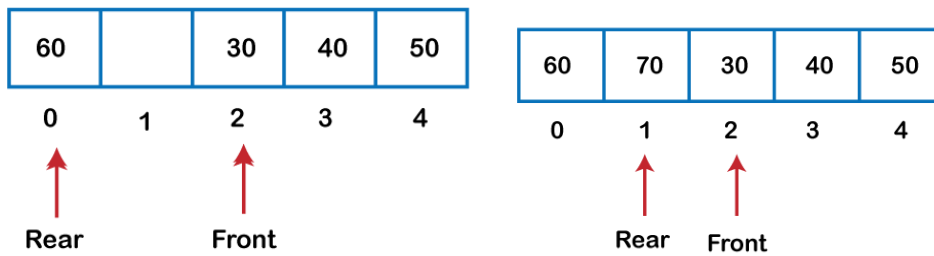
Rear = -1



Front = 0

Rear = 0





## Implementation of circular queue using Array

```
#include <stdio.h>
```

```
# define max 6
```

```
int queue[max]; // array declaration
```

```
int front=-1;
```

```
int rear=-1;
```

```
// function to insert an element in a circular queue
```

```
void enqueue(int element)
```

```
{
    if(front== -1 && rear== -1) // condition to check queue is empty
    {
        front=0;
        rear=0;
        queue[rear]=element;
    }
    else if((rear+1)%max==front) // condition to check queue is full
    {
        printf("Queue is overflow..");
    }
    else
    {
        rear=(rear+1)%max; // rear is incremented
        queue[rear]=element; // assigning a value to the queue at the rear position.
    }
}
```

```
// function to delete the element from the queue
```

```
int dequeue()
```

```
{
    if((front== -1) && (rear== -1)) // condition to check queue is empty
    {
        printf("\nQueue is underflow..");
    }
    else if(front==rear)
```



```

{
    printf("\nThe dequeued element is %d", queue[front]);
    front=-1;
    rear=-1;
}
else
{
    printf("\nThe dequeued element is %d", queue[front]);
    front=(front+1)%max;
}
}
// function to display the elements of a queue
void display()
{
    int i=front;
    if(front== -1 && rear== -1)
    {
        printf("\n Queue is empty..");
    }
    else
    {
        printf("\nElements in a Queue are :");
        while(i<=rear)
        {
            printf("%d,", queue[i]);
            i=(i+1)%max;
        }
    }
}
int main()
{
    int choice=1,x; // variables declaration

    while(choice<4 && choice!=0) // while loop
    {
        printf("\n Press 1: Insert an element");
        printf("\nPress 2: Delete an element");
        printf("\nPress 3: Display the element");
        printf("\nEnter your choice");
        scanf("%d", &choice);

        switch(choice)
        {

```

**case 1:**

```
printf("Enter the element which is to be inserted");
```

```
scanf("%d", &x);
```

```
enqueue(x);
```

```
break;
```

**case 2:**

```
dequeue();
```

```
break;
```

**case 3:**

```
display();
```

```
}}
```

```
return 0;
```

```
}
```

**Output:**

```
Press 1: Insert an element
Press 2: Delete an element
Press 3: Display the element
Enter your choice
1
Enter the element which is to be inserted
10

Press 1: Insert an element
Press 2: Delete an element
Press 3: Display the element
Enter your choice
1
Enter the element which is to be inserted
20

Press 1: Insert an element
Press 2: Delete an element
Press 3: Display the element
Enter your choice
1
Enter the element which is to be inserted
30

Press 1: Insert an element
Press 2: Delete an element
Press 3: Display the element
Enter your choice
3

Elements in a Queue are :10,20,30,
Press 1: Insert an element
Press 2: Delete an element
Press 3: Display the element
Enter your choice
2

The dequeued element is 10
```

## Implementation of circular queue using linked list

As we know that linked list is a linear data structure that stores two parts, i.e., data part and the address part where address part contains the address of the next node. Here, linked list is used to implement the circular queue; therefore, the linked list follows the properties of the Queue. When we are implementing the circular queue using linked list then both the **enqueue and dequeue** operations take  **$O(1)$**  time.

```
#include <stdio.h>
```

```
// Declaration of struct type node
```

```
struct node
```

```
{
```

```
    int data;
```

```

    struct node *next;
};
struct node *front=-1;
struct node *rear=-1;
// function to insert the element in the Queue
void enqueue(int x)
{
    struct node *newnode; // declaration of pointer of struct node type.
    newnode=(struct node *)malloc(sizeof(struct node)); // allocating the memory to the newnode
    newnode->data=x;
    newnode->next=0;
    if(rear==-1) // checking whether the Queue is empty or not.
    {
        front=rear=newnode;
        rear->next=front;
    }
    else
    {
        rear->next=newnode;
        rear=newnode;
        rear->next=front;
    }
}

```

```

// function to delete the element from the queue
void dequeue()
{
    struct node *temp; // declaration of pointer of node type
    temp=front;
    if((front==-1)&&(rear==-1)) // checking whether the queue is empty or not
    {
        printf("\nQueue is empty");
    }
    else if(front==rear) // checking whether the single element is left in the queue
    {
        front=rear=-1;
        free(temp);
    }
    else
    {
        front=front->next;
        rear->next=front;
        free(temp);
    }
}

```

```
    }  
}
```

```
// function to get the front of the queue
```

```
int peek()  
{  
    if((front== -1) &&(rear== -1))  
    {  
        printf("\nQueue is empty");  
    }  
    else  
    {  
        printf("\nThe front element is %d", front->data);  
    }  
}
```

```
// function to display all the elements of the queue
```

```
void display()  
{  
    struct node *temp;  
    temp=front;  
    printf("\n The elements in a Queue are : ");  
    if((front== -1) && (rear== -1))  
    {  
        printf("Queue is empty");  
    }  
  
    else  
    {  
        while(temp->next!=front)  
        {  
            printf("%d", temp->data);  
            temp=temp->next;  
        }  
        printf("%d", temp->data);  
    }  
}
```

```
void main()
```

```
{  
    enqueue(34);  
    enqueue(10);  
    enqueue(23);  
    display();  
}
```

```
    dequeue();  
    peek();  
}
```

**Output:**

```
The elements in a Queue are : 34,10,23  
The front element is 10  
  
...Program finished with exit code 24  
Press ENTER to exit console.
```

## Deque (or double-ended queue)

In this article, we will discuss the double-ended queue or deque. We should first see a brief description of the queue.

### What is a queue?

A queue is a data structure in which whatever comes first will go out first, and it follows the FIFO (First-In-First-Out) policy. Insertion in the queue is done from one end known as the **rear end** or the **tail**, whereas the deletion is done from another end known as the **front end** or the **head** of the queue.

The real-world example of a queue is the ticket queue outside a cinema hall, where the person who enters first in the queue gets the ticket first, and the person enters last in the queue gets the ticket at last.

### What is a Deque (or double-ended queue)

The deque stands for Double Ended Queue. Deque is a linear data structure where the insertion and deletion operations are performed from both ends. We can say that deque is a generalized version of the queue.

Though the insertion and deletion in a deque can be performed on both ends, it does not follow the FIFO rule. The representation of a deque is given as follows -



**Representation of deque**

### Types of deque

There are two types of deque -

Input restricted queue

Output restricted queue

#### **Input restricted Queue**

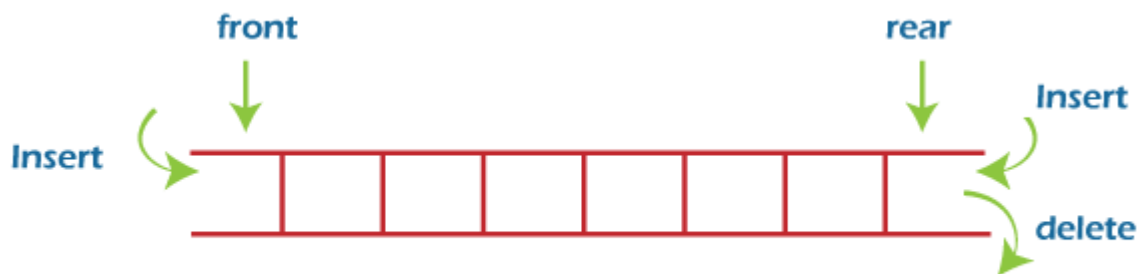
In input restricted queue, insertion operation can be performed at only one end, while deletion can be performed from both ends.



### input restricted double ended queue

#### Output restricted Queue

In output restricted queue, deletion operation can be performed at only one end, while insertion can be performed from both ends.



### Output restricted double ended queue

## Operations performed on deque

There are the following operations that can be applied on a deque -

Insertion at front

Insertion at rear

Deletion at front

Deletion at rear

We can also perform peek operations in the deque along with the operations listed above.

Through peek operation, we can get the deque's front and rear elements of the deque. So, in addition to the above operations, following operations are also supported in deque -

Get the front item from the deque

Get the rear item from the deque

Check whether the deque is full or not

Checks whether the deque is empty or not

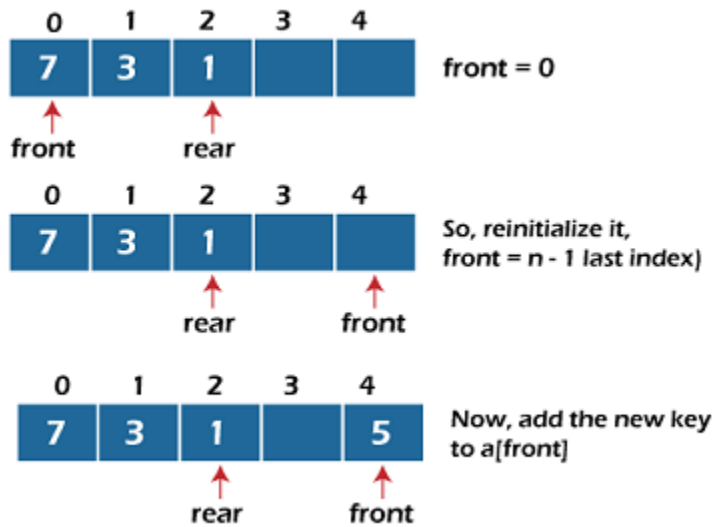
Now, let's understand the operation performed on deque using an example.

#### Insertion at the front end

In this operation, the element is inserted from the front end of the queue. Before implementing the operation, we first have to check whether the queue is full or not. If the queue is not full, then the element can be inserted from the front end by using the below conditions -

If the queue is empty, both rear and front are initialized with 0. Now, both will point to the first element.

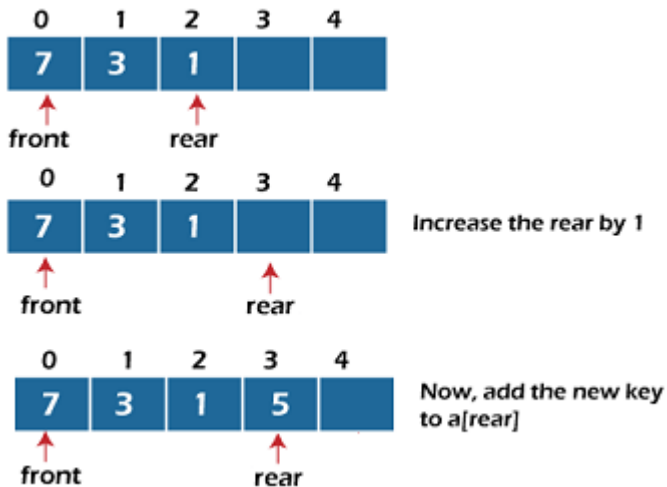
Otherwise, check the position of the front if the front is less than 1 ( $\text{front} < 1$ ), then reinitialize it by **front = n - 1**, i.e., the last index of the array.



### Insertion at the rear end

In this operation, the element is inserted from the rear end of the queue. Before implementing the operation, we first have to check again whether the queue is full or not. If the queue is not full, then the element can be inserted from the rear end by using the below conditions - If the queue is empty, both rear and front are initialized with 0. Now, both will point to the first element.

Otherwise, increment the rear by 1. If the rear is at last index (or size - 1), then instead of increasing it by 1, we have to make it equal to 0.



### Deletion at the front end

In this operation, the element is deleted from the front end of the queue. Before implementing the operation, we first have to check whether the queue is empty or not.

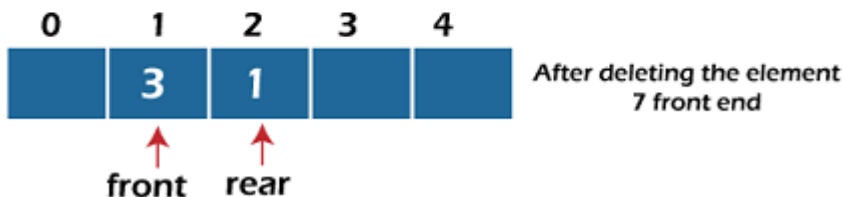
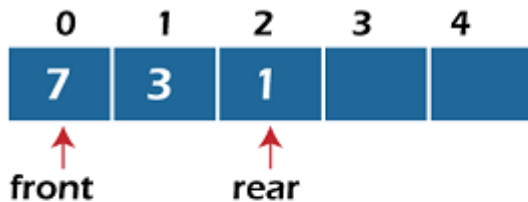


If the queue is empty, i.e.,  $\text{front} = -1$ , it is the underflow condition, and we cannot perform the deletion. If the queue is not full, then the element can be inserted from the front end by using the below conditions -

If the deque has only one element, set  $\text{rear} = -1$  and  $\text{front} = -1$ .

Else if  $\text{front}$  is at end (that means  $\text{front} = \text{size} - 1$ ), set  $\text{front} = 0$ .

Else increment the  $\text{front}$  by 1, (i.e.,  $\text{front} = \text{front} + 1$ ).



### Deletion at the rear end

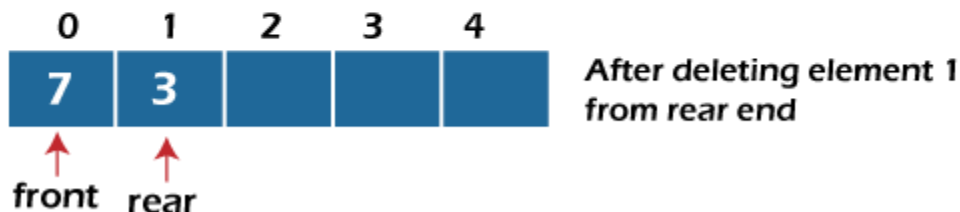
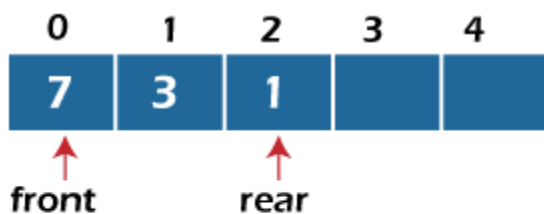
In this operation, the element is deleted from the rear end of the queue. Before implementing the operation, we first have to check whether the queue is empty or not.

If the queue is empty, i.e.,  $\text{front} = -1$ , it is the underflow condition, and we cannot perform the deletion.

If the deque has only one element, set  $\text{rear} = -1$  and  $\text{front} = -1$ .

If  $\text{rear} = 0$  ( $\text{rear}$  is at front), then set  $\text{rear} = n - 1$ .

Else, decrement the  $\text{rear}$  by 1 (or,  $\text{rear} = \text{rear} - 1$ ).



### Check empty

This operation is performed to check whether the deque is empty or not. If  $\text{front} = -1$ , it means that the deque is empty.

### Check full

This operation is performed to check whether the deque is full or not. If  $\text{front} = \text{rear} + 1$ , or  $\text{front} = 0$  and  $\text{rear} = n - 1$  it means that the deque is full.

The time complexity of all of the above operations of the deque is  $O(1)$ , i.e., constant.

## Applications of deque

Deque can be used as both stack and queue, as it supports both operations.

Deque can be used as a palindrome checker means that if we read the string from both ends, the string would be the same.

## Implementation of deque

Now, let's see the implementation of deque in C programming language.

```
#include <stdio.h>
```

```
#define size 5
```

```
int deque[size];
```

```
int f = -1, r = -1;
```

```
// insert_front function will insert the value from the front
```

```
void insert_front(int x)
```

```
{
    if((f==0 && r==size-1) || (f==r+1))
    {
        printf("Overflow");
    }
    else if((f==-1) && (r==-1))
    {
        f=r=0;
        deque[f]=x;
    }
    else if(f==0)
    {
        f=size-1;
        deque[f]=x;
    }
    else
    {
        f=f-1;
        deque[f]=x;
    }
}
```

```
// insert_rear function will insert the value from the rear
```

```
void insert_rear(int x)
```

```
{
    if((f==0 && r==size-1) || (f==r+1))
    {
```

```

        printf("Overflow");
    }
    else if((f == -1) && (r == -1))
    {
        r = 0;
        deque[r] = x;
    }
    else if(r == size - 1)
    {
        r = 0;
        deque[r] = x;
    }
    else
    {
        r++;
        deque[r] = x;
    }
}

```

// display function prints all the value of deque.

```

void display()
{
    int i = f;
    printf("\nElements in a deque are: ");

    while(i != r)
    {
        printf("%d ", deque[i]);
        i = (i + 1) % size;
    }
    printf("%d", deque[r]);
}

```

// getfront function retrieves the first value of the deque.

```

void getfront()
{
    if((f == -1) && (r == -1))
    {
        printf("Deque is empty");
    }
    else
    {
        printf("\nThe value of the element at front is: %d", deque[f]);
    }
}

```

```

    }

}

// getrear function retrieves the last value of the deque.
void getrear()
{
    if((f==-1) && (r==-1))
    {
        printf("Deque is empty");
    }
    else
    {
        printf("\nThe value of the element at rear is %d", deque[r]);
    }
}

// delete_front() function deletes the element from the front
void delete_front()
{
    if((f==-1) && (r==-1))
    {
        printf("Deque is empty");
    }
    else if(f==r)
    {
        printf("\nThe deleted element is %d", deque[f]);
        f=-1;
        r=-1;
    }
    else if(f==(size-1))
    {
        printf("\nThe deleted element is %d", deque[f]);
        f=0;
    }
    else
    {
        printf("\nThe deleted element is %d", deque[f]);
        f=f+1;
    }
}

```

// delete\_rear() function deletes the element from the rear

```
void delete_rear()
{
    if((f== -1) && (r== -1))
    {
        printf("Deque is empty");
    }
    else if(f==r)
    {
        printf("\nThe deleted element is %d", deque[r]);
        f=-1;
        r=-1;
    }
    else if(r==0)
    {
        printf("\nThe deleted element is %d", deque[r]);
        r=size-1;
    }
    else
    {
        printf("\nThe deleted element is %d", deque[r]);
        r=r-1;
    }
}
```

```
int main()
{
    insert_front(20);
    insert_front(10);
    insert_rear(30);
    insert_rear(50);
    insert_rear(80);
    display(); // Calling the display function to retrieve the values of deque
    getfront(); // Retrieve the value at front-end
    getrear(); // Retrieve the value at rear-end
    delete_front();
    delete_rear();
    display(); // calling display function to retrieve values after deletion
    return 0;
}
```

**Output:**

```
Elements in a deque are: 10 20 30 50 80
The value of the element at front is: 10
The value of the element at rear is 80
The deleted element is 10
The deleted element is 80
Elements in a deque are: 20 30 50
```

So, that's all about the article. Hope, the article will be helpful and informative to you.

## What is a priority queue?

A priority queue is an abstract data type that behaves similarly to the normal queue except that each element has some priority, i.e., the element with the highest priority would come first in a priority queue. The priority of the elements in a priority queue will determine the order in which elements are removed from the priority queue.

The priority queue supports only comparable elements, which means that the elements are either arranged in an ascending or descending order.

For example, suppose we have some values like 1, 3, 4, 8, 14, 22 inserted in a priority queue with an ordering imposed on the values is from least to the greatest. Therefore, the 1 number would be having the highest priority while 22 will be having the lowest priority.

### Characteristics of a Priority queue

A priority queue is an extension of a queue that contains the following characteristics:

Every element in a priority queue has some priority associated with it.

An element with the higher priority will be deleted before the deletion of the lesser priority.

If two elements in a priority queue have the same priority, they will be arranged using the FIFO principle.

#### Let's understand the priority queue through an example.

We have a priority queue that contains the following values:

**1, 3, 4, 8, 14, 22**

All the values are arranged in ascending order. Now, we will observe how the priority queue will look after performing the following operations:

**poll():** This function will remove the highest priority element from the priority queue. In the above priority queue, the '1' element has the highest priority, so it will be removed from the priority queue.

**add(2):** This function will insert '2' element in a priority queue. As 2 is the smallest element among all the numbers so it will obtain the highest priority.

**poll():** It will remove '2' element from the priority queue as it has the highest priority queue.

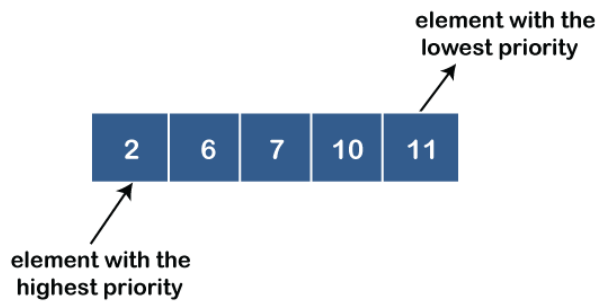
**add(5):** It will insert 5 element after 4 as 5 is larger than 4 and lesser than 8, so it will obtain the third highest priority in a priority queue.

### Types of Priority Queue

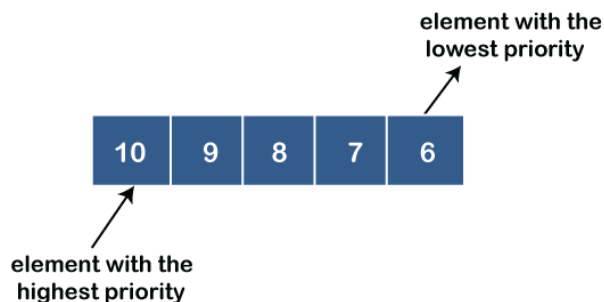
**There are two types of priority queue:**

**Ascending order priority queue:** In ascending order priority queue, a lower priority number is given as a higher priority in a priority. For example, we take the numbers from 1 to 5 arranged in an ascending order like 1,2,3,4,5; therefore, the smallest number, i.e., 1 is given as the highest

priority in a priority queue.



**Descending order priority queue:** In descending order priority queue, a higher priority number is given as a higher priority in a priority. For example, we take the numbers from 1 to 5 arranged in descending order like 5, 4, 3, 2, 1; therefore, the largest number, i.e., 5 is given as the highest priority in a priority queue.



## Representation of priority queue

Now, we will see how to represent the priority queue through a one-way list.

We will create the priority queue by using the list given below in which **INFO** list contains the data elements, **PNR** list contains the priority numbers of each data element available in the **INFO** list, and **LINK** basically contains the address of the next node.

|   | INFO | PNR | LINK |
|---|------|-----|------|
| 0 | 200  | 2   | 4    |
| 1 | 400  | 4   | 2    |
| 2 | 500  | 4   | 6    |
| 3 | 300  | 1   | 0    |
| 4 | 100  | 2   | 5    |
| 5 | 600  | 3   | 1    |
| 6 | 700  | 4   |      |

**Let's create the priority queue step by step.**

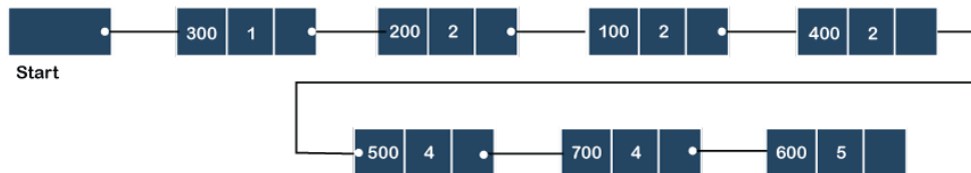
**In the case of priority queue, lower priority number is considered the higher priority, i.e., lower priority number = higher priority.**

**Step 1:** In the list, lower priority number is 1, whose data value is 333, so it will be inserted in the list as shown in the below diagram:

**Step 2:** After inserting 333, priority number 2 is having a higher priority, and data values associated with this priority are 222 and 111. So, this data will be inserted based on the FIFO principle; therefore 222 will be added first and then 111.

**Step 3:** After inserting the elements of priority 2, the next higher priority number is 4 and data elements associated with 4 priority numbers are 444, 555, 777. In this case, elements would be inserted based on the FIFO principle; therefore, 444 will be added first, then 555, and then 777.

**Step 4:** After inserting the elements of priority 4, the next higher priority number is 5, and the value associated with priority 5 is 666, so it will be inserted at the end of the queue.



## Implementation of Priority Queue

The priority queue can be implemented in four ways that include arrays, linked list, heap data structure and binary search tree. The heap data structure is the most efficient way of implementing the priority queue, so we will implement the priority queue using a heap data structure in this topic. Now, first we understand the reason why heap is the most efficient way among all the other data structures.

### Analysis of complexities using different implementations

| Implementation     | add         | Remove      | peek   |
|--------------------|-------------|-------------|--------|
| Linked list        | $O(1)$      | $O(n)$      | $O(n)$ |
| Binary heap        | $O(\log n)$ | $O(\log n)$ | $O(1)$ |
| Binary search tree | $O(\log n)$ | $O(\log n)$ | $O(1)$ |

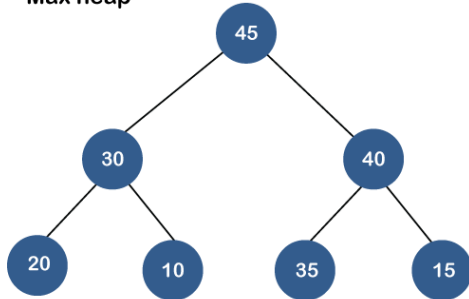
## What is Heap?

A heap is a tree-based data structure that forms a complete binary tree, and satisfies the heap property. If A is a parent node of B, then A is ordered with respect to the node B for all nodes A and B in a heap. It means that the value of the parent node could be more than or equal to the value of the child node, or the value of the parent node could be less than or equal to the value of the child node. Therefore, we can say that there are two types of heaps:



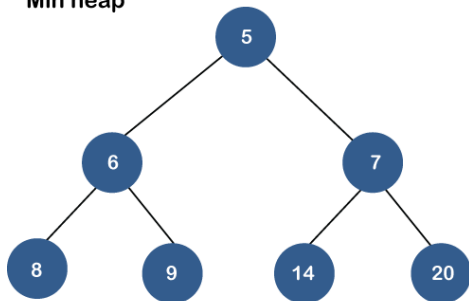
**Max heap:** The max heap is a heap in which the value of the parent node is greater than the value of the child nodes.

Max heap



**Min heap:** The min heap is a heap in which the value of the parent node is less than the value of the child nodes.

Min heap



Both the heaps are the binary heap, as each has exactly two child nodes.

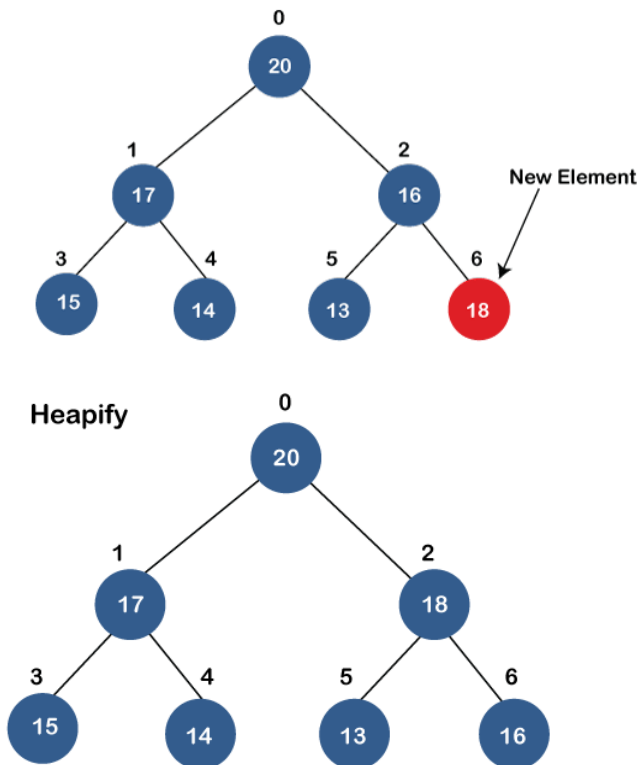
## Priority Queue Operations

The common operations that we can perform on a priority queue are insertion, deletion and peek. Let's see how we can maintain the heap data structure.

### Inserting the element in a priority queue (max heap)

If we insert an element in a priority queue, it will move to the empty slot by looking from top to bottom and left to right.

If the element is not in a correct place then it is compared with the parent node; if it is found out of order, elements are swapped. This process continues until the element is placed in a correct position.



### Removing the minimum element from the priority queue

As we know that in a max heap, the maximum element is the root node. When we remove the root node, it creates an empty slot. The last inserted element will be added in this empty slot. Then, this element is compared with the child nodes, i.e., left-child and right child, and swap with the smaller of the two. It keeps moving down the tree until the heap property is restored.

## Applications of Priority queue

**The following are the applications of the priority queue:**

It is used in the Dijkstra's shortest path algorithm.

It is used in prim's algorithm

It is used in data compression techniques like Huffman code.

It is used in heap sort.

It is also used in operating system like priority scheduling, load balancing and interrupt handling.

**Program to create the priority queue using the binary max heap.**

```
#include <stdio.h>
```

```
#include <stdio.h>
```

```
int heap[40];
```

```
int size=-1;
```

```
// retrieving the parent node of the child node
```

```
int parent(int i)
```

```
{
```

```

    return (i - 1) / 2;
}

// retrieving the left child of the parent node.
int left_child(int i)
{
    return i+1;
}

// retrieving the right child of the parent
int right_child(int i)
{
    return i+2;
}

// Returning the element having the highest priority
int get_Max()
{
    return heap[0];
}

//Returning the element having the minimum priority
int get_Min()
{
    return heap[size];
}

// function to move the node up the tree in order to restore the heap property.
void moveUp(int i)
{
    while (i > 0)
    {
        // swapping parent node with a child node
        if(heap[parent(i)] < heap[i]) {

            int temp;
            temp=heap[parent(i)];
            heap[parent(i)]=heap[i];
            heap[i]=temp;

        }

        // updating the value of i to i/2
        i=i/2;
    }
}

```

//function to move the node down the tree in order to restore the heap property.

```
void moveDown(int k)
{
    int index = k;

    // getting the location of the Left Child
    int left = left_child(k);

    if (left <= size && heap[left] > heap[index]) {
        index = left;
    }

    // getting the location of the Right Child
    int right = right_child(k);

    if (right <= size && heap[right] > heap[index]) {
        index = right;
    }

    // If k is not equal to index
    if (k != index) {
        int temp;
        temp=heap[index];
        heap[index]=heap[k];
        heap[k]=temp;
        moveDown(index);
    }
}
```

// Removing the element of maximum priority

```
void removeMax()
{
    int r= heap[0];
    heap[0]=heap[size];
    size=size-1;
    moveDown(0);
}
```

//inserting the element in a priority queue

```
void insert(int p)
{
    size = size + 1;
    heap[size] = p;
```

// move Up to maintain heap property

```

    moveUp(size);
}

//Removing the element from the priority queue at a given index i.
void delete(int i)
{
    heap[i] = heap[0] + 1;

    // move the node stored at ith location is shifted to the root node
    moveUp(i);

    // Removing the node having maximum priority
    removeMax();
}

int main()
{
    // Inserting the elements in a priority queue

    insert(20);
    insert(19);
    insert(21);
    insert(18);
    insert(12);
    insert(17);
    insert(15);
    insert(16);
    insert(14);
    int i=0;

    printf("Elements in a priority queue are : ");
    for(int i=0;i<=size;i++)
    {
        printf("%d ",heap[i]);
    }
    delete(2); // deleting the element whose index is 2.
    printf("\nElements in a priority queue after deleting the element are : ");
    for(int i=0;i<=size;i++)
    {
        printf("%d ",heap[i]);
    }
    int max=get_Max();
    printf("\nThe element which is having the highest priority is %d: ",max);
}

```

```

int min=get_Min();
printf("\nThe element which is having the minimum priority is : %d",min);
return 0;
}

```

**In the above program, we have created the following functions:**

**int parent(int i):** This function returns the index of the parent node of a child node, i.e., i.

**int left\_child(int i):** This function returns the index of the left child of a given index, i.e., i.

**int right\_child(int i):** This function returns the index of the right child of a given index, i.e., i.

**void moveUp(int i):** This function will keep moving the node up the tree until the heap property is restored.

**void moveDown(int i):** This function will keep moving the node down the tree until the heap property is restored.

**void removeMax():** This function removes the element which is having the highest priority.

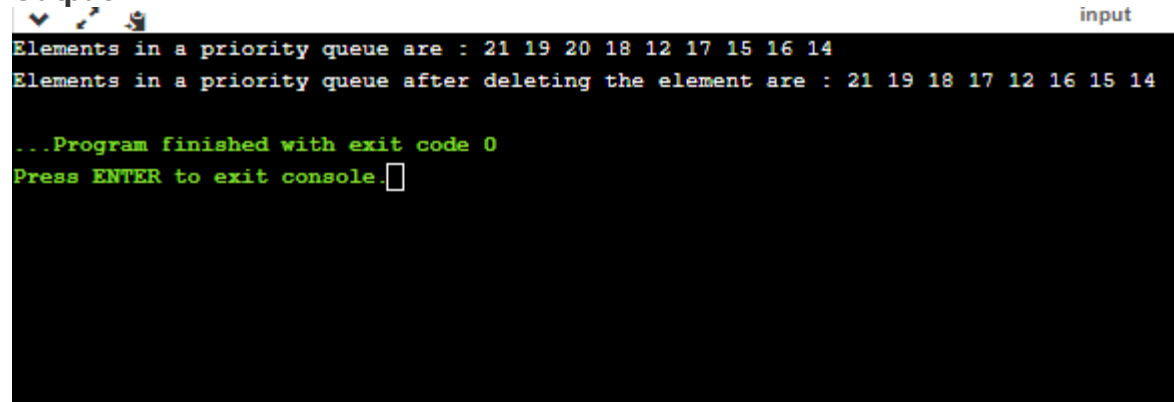
**void insert(int p):** It inserts the element in a priority queue which is passed as an argument in a function.

**void delete(int i):** It deletes the element from a priority queue at a given index.

**int get\_Max():** It returns the element which is having the highest priority, and we know that in max heap, the root node contains the element which has the largest value, and highest priority.

**int get\_Min():** It returns the element which is having the minimum priority, and we know that in max heap, the last node contains the element which has the smallest value, and lowest priority.

**Output**



```

input
Elements in a priority queue are : 21 19 20 18 12 17 15 16 14
Elements in a priority queue after deleting the element are : 21 19 18 17 12 16 15 14

...Program finished with exit code 0
Press ENTER to exit console

```

## Tree Data Structure

We read the linear data structures like an array, linked list, stack and queue in which all the elements are arranged in a sequential manner. The different data structures are used for different kinds of data.

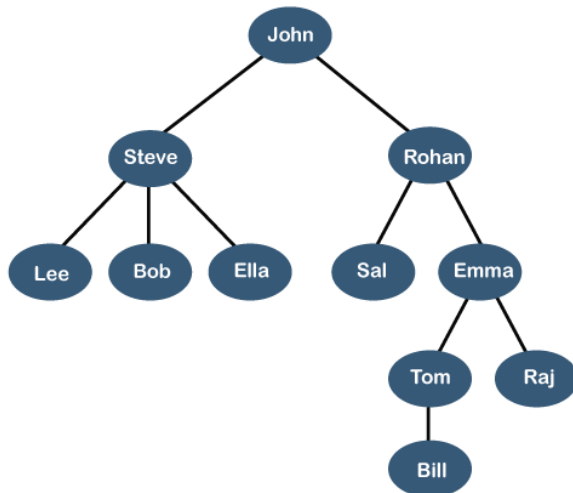
**Some factors are considered for choosing the data structure:**

**What type of data needs to be stored?:** It might be a possibility that a certain data structure can be the best fit for some kind of data.

**Cost of operations:** If we want to minimize the cost for the operations for the most frequently performed operations. For example, we have a simple list on which we have to perform the search operation; then, we can create an array in which elements are stored in sorted order to perform the **binary search**. The binary search works very fast for the simple list as it divides the search space into half.

**Memory usage:** Sometimes, we want a data structure that utilizes less memory.

**A tree** is also one of the data structures that represent hierarchical data. Suppose we want to show the employees and their positions in the hierarchical form then it can be represented as shown below:



The above tree shows the **organization hierarchy** of some company. In the above structure, **John** is the **CEO** of the company, and John has two direct reports named as **Steve** and **Rohan**. Steve has three direct reports named **Lee, Bob, Ella** where **Steve** is a manager. Bob has two direct reports named **Sal** and **Emma**. Emma has two direct reports named **Tom** and **Raj**. Tom has one direct report named **Bill**. This particular logical structure is known as a **Tree**. Its structure is similar to the real tree, so it is named a **Tree**. In this structure, the **root** is at the top, and its branches are moving in a downward direction. Therefore, we can say that the Tree data structure is an efficient way of storing the data in a hierarchical way.

**Let's understand some key points of the Tree data structure.**

A tree data structure is defined as a collection of objects or entities known as nodes that are linked together to represent or simulate hierarchy.

A tree data structure is a non-linear data structure because it does not store in a sequential manner. It is a hierarchical structure as elements in a Tree are arranged in multiple levels.

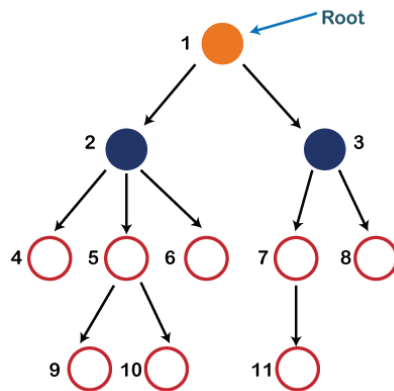
In the Tree data structure, the topmost node is known as a root node. Each node contains some data, and data can be of any type. In the above tree structure, the node contains the name of the employee, so the type of data would be a string.

Each node contains some data and the link or reference of other nodes that can be called children.

**Some basic terms used in Tree data structure.**

Let's consider the tree structure, which is shown below:

## Introduction to Trees



In the above structure, each node is labeled with some number. Each arrow shown in the above figure is known as a **link** between the two nodes.

**Root:** The root node is the topmost node in the tree hierarchy. In other words, the root node is the one that doesn't have any parent. In the above structure, node numbered 1 is **the root node of the tree**. If a node is directly linked to some other node, it would be called a parent-child relationship.

**Child node:** If the node is a descendant of any node, then the node is known as a child node.

**Parent:** If the node contains any sub-node, then that node is said to be the parent of that sub-node.

**Sibling:** The nodes that have the same parent are known as siblings.

**Leaf Node:-** The node of the tree, which doesn't have any child node, is called a leaf node. A leaf node is the bottom-most node of the tree. There can be any number of leaf nodes present in a general tree. Leaf nodes can also be called external nodes.

**Internal nodes:** A node has atleast one child node known as an **internal**

**Ancestor node:-** An ancestor of a node is any predecessor node on a path from the root to that node. The root node doesn't have any ancestors. In the tree shown in the above image, nodes 1, 2, and 5 are the ancestors of node 10.

**Descendant:** The immediate successor of the given node is known as a descendant of a node. In the above figure, 10 is the descendant of node 5.

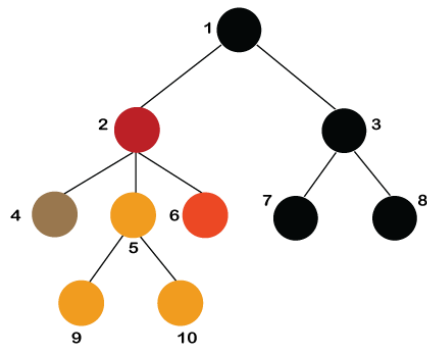
## Properties of Tree data structure

**Recursive data structure:** The tree is also known as a **recursive data structure**. A tree can be defined as recursively because the distinguished node in a tree data structure is known as a **root node**. The root node of the tree contains a link to all the roots of its subtrees. The left subtree is shown in the yellow color in the below figure, and the right subtree is shown in the red color.

The left subtree can be further split into subtrees shown in three different colors. Recursion means reducing something in a self-similar manner. So, this recursive property of the tree data



structure is implemented in various applications.



**Number of edges:** If there are  $n$  nodes, then there would be  $n-1$  edges. Each arrow in the structure represents the link or path. Each node, except the root node, will have at least one incoming link known as an edge. There would be one link for the parent-child relationship.

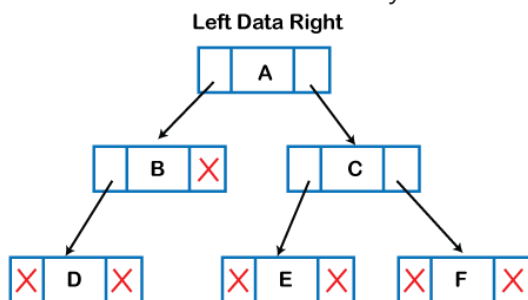
**Depth of node  $x$ :** The depth of node  $x$  can be defined as the length of the path from the root to the node  $x$ . One edge contributes one-unit length in the path. So, the depth of node  $x$  can also be defined as the number of edges between the root node and the node  $x$ . The root node has 0 depth.

**Height of node  $x$ :** The height of node  $x$  can be defined as the longest path from the node  $x$  to the leaf node.

Based on the properties of the Tree data structure, trees are classified into various categories.

## Implementation of Tree

The tree data structure can be created by creating the nodes dynamically with the help of the pointers. The tree in the memory can be represented as shown below:



The above figure shows the representation of the tree data structure in the memory. In the above structure, the node contains three fields. The second field stores the data; the first field stores the address of the left child, and the third field stores the address of the right child.

In programming, the structure of a node can be defined as:

```
struct node
{
    int data;
    struct node *left;
    struct node *right;
}
```

The above structure can only be defined for the binary trees because the binary tree can have utmost two children, and generic trees can have more than two children. The structure of the node for generic trees would be different as compared to the binary tree.

## Applications of trees

The following are the applications of trees:

**Storing naturally hierarchical data:** Trees are used to store the data in the hierarchical structure. For example, the file system. The file system stored on the disc drive, the file and folder are in the form of the naturally hierarchical data and stored in the form of trees.

**Organize data:** It is used to organize data for efficient insertion, deletion and searching. For example, a binary tree has a  $\log N$  time for searching an element.

**Trie:** It is a special kind of tree that is used to store the dictionary. It is a fast and efficient way for dynamic spell checking.

**Heap:** It is also a tree data structure implemented using arrays. It is used to implement priority queues.

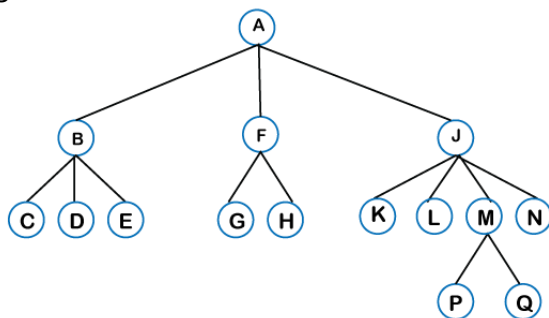
**B-Tree and B+Tree:** B-Tree and B+Tree are the tree data structures used to implement indexing in databases.

**Routing table:** The tree data structure is also used to store the data in routing tables in the routers.

## Types of Tree data structure

The following are the types of a tree data structure:

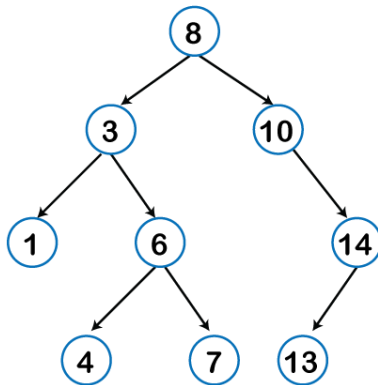
**General tree:** The general tree is one of the types of tree data structure. In the general tree, a node can have either 0 or maximum  $n$  number of nodes. There is no restriction imposed on the degree of the node (the number of nodes that a node can contain). The topmost node in a general tree is known as a root node. The children of the parent node are known as **subtrees**.



There can be  $n$  number of subtrees in a general tree. In the general tree, the subtrees are unordered as the nodes in the subtree cannot be ordered.

Every non-empty tree has a downward edge, and these edges are connected to the nodes known as **child nodes**. The root node is labeled with level 0. The nodes that have the same parent are known as **siblings**.

**Binary tree:** Here, binary name itself suggests two numbers, i.e., 0 and 1. In a binary tree, each node in a tree can have utmost two child nodes. Here, utmost means whether the node has 0 nodes, 1 node or 2 nodes.



**To know more about the binary tree, click on the link given below:**

<https://www.javatpoint.com/binary-tree>

**Binary Search tree:** Binary search tree is a non-linear data structure in which one node is connected to *n* number of nodes. It is a node-based data structure. A node can be represented in a binary search tree with three fields, i.e., data part, left-child, and right-child. A node can be connected to the utmost two child nodes in a binary search tree, so the node contains two pointers (left child and right child pointer).

Every node in the left subtree must contain a value less than the value of the root node, and the value of each node in the right subtree must be bigger than the value of the root node.

A node can be created with the help of a user-defined data type known as **struct**, as shown below:

```

struct node
{
    int data;
    struct node *left;
    struct node *right;
}
  
```

The above is the node structure with three fields: data field, the second field is the left pointer of the node type, and the third field is the right pointer of the node type.

**To know more about the binary search tree, click on the link given below:**

<https://www.javatpoint.com/binary-search-tree>

### **AVL tree**

It is one of the types of the binary tree, or we can say that it is a variant of the binary search tree. AVL tree satisfies the property of the **binary tree** as well as of the **binary search tree**. It is a self-balancing binary search tree that was invented by **Adelson Velsky Lindas**. Here, self-balancing means that balancing the heights of left subtree and right subtree. This balancing is measured in terms of the **balancing factor**.

We can consider a tree as an AVL tree if the tree obeys the binary search tree as well as a balancing factor. The balancing factor can be defined as the **difference between the height of the left subtree and the height of the right subtree**. The balancing factor's value must be either 0, -1, or 1; therefore, each node in the AVL tree should have the value of the balancing factor either as 0, -1, or 1.

**To know more about the AVL tree, click on the link given below:**

<https://www.javatpoint.com/avl-tree>

### **Red-Black Tree**

**The red-Black tree** is the binary search tree. The prerequisite of the Red-Black tree is that we should know about the binary search tree. In a binary search tree, the value of the left-subtree should be less than the value of that node, and the value of the right-subtree should be greater than the value of that node. As we know that the time complexity of binary search in the average case is  $\log_2 n$ , the best case is  $O(1)$ , and the worst case is  $O(n)$ .

When any operation is performed on the tree, we want our tree to be balanced so that all the operations like searching, insertion, deletion, etc., take less time, and all these operations will have the time complexity of  **$\log_2 n$** .

**The red-black tree** is a self-balancing binary search tree. AVL tree is also a height balancing binary search tree then **why do we require a Red-Black tree**. In the AVL tree, we do not know how many rotations would be required to balance the tree, but in the Red-black tree, a maximum of 2 rotations are required to balance the tree. It contains one extra bit that represents either the red or black color of a node to ensure the balancing of the tree.

### **Splay tree**

The splay tree data structure is also binary search tree in which recently accessed element is placed at the root position of tree by performing some rotation operations.

Here, **splaying** means the recently accessed node. It is a **self-balancing** binary search tree having no explicit balance condition like **AVL** tree.

It might be a possibility that height of the splay tree is not balanced, i.e., height of both left and right subtrees may differ, but the operations in splay tree takes order of  **$\log N$**  time where **n** is the number of nodes.

Splay tree is a balanced tree but it cannot be considered as a height balanced tree because after each operation, rotation is performed which leads to a balanced tree.

### **Treap**

Treap data structure came from the Tree and Heap data structure. So, it comprises the properties of both Tree and Heap data structures. In Binary search tree, each node on the left subtree must be equal or less than the value of the root node and each node on the right subtree must be equal or greater than the value of the root node. In heap data structure, both right and left subtrees contain larger keys than the root; therefore, we can say that the root node contains the lowest value.

In treap data structure, each node has both **key** and **priority** where key is derived from the Binary search tree and priority is derived from the heap data structure.

The **Treap** data structure follows two properties which are given below:

Right child of a node  $\geq$  current node and left child of a node  $\leq$  current node (binary tree)

Children of any subtree must be greater than the node (heap)

### **B-tree**

B-tree is a balanced **m-way** tree where **m** defines the order of the tree. Till now, we read that the node contains only one key but b-tree can have more than one key, and more than 2 children. It always maintains the sorted data. In binary tree, it is possible that leaf nodes can be at different levels, but in b-tree, all the leaf nodes must be at the same level.

**If order is m then node has the following properties:**

Each node in a b-tree can have maximum **m** children

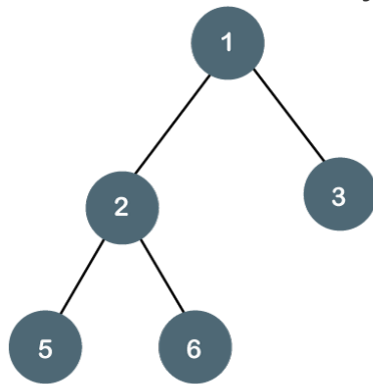
For minimum children, a leaf node has 0 children, root node has minimum 2 children and internal node has minimum ceiling of  $m/2$  children. For example, the value of  $m$  is 5 which means that a node can have 5 children and internal nodes can contain maximum 3 children. Each node has maximum  $(m-1)$  keys.

The root node must contain minimum 1 key and all other nodes must contain atleast **ceiling of  $m/2$  minus 1** keys.

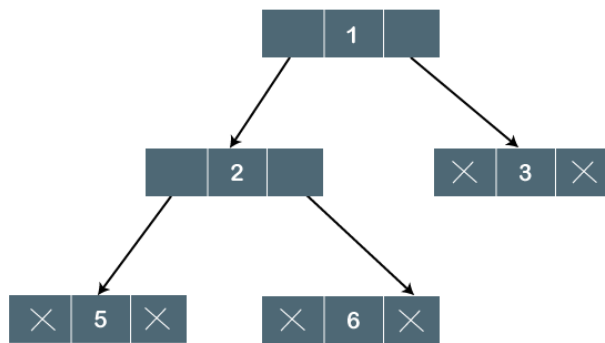
## Binary Tree

The Binary tree means that the node can have maximum two children. Here, binary name itself suggests that 'two'; therefore, each node can have either 0, 1 or 2 children.

**Let's understand the binary tree through an example.**



The above tree is a binary tree because each node contains the utmost two children. The logical representation of the above tree is given below:



In the above tree, node 1 contains two pointers, i.e., left and a right pointer pointing to the left and right node respectively. The node 2 contains both the nodes (left and right node); therefore, it has two pointers (left and right). The nodes 3, 5 and 6 are the leaf nodes, so all these nodes contain **NULL** pointer on both left and right parts.

## Properties of Binary Tree

At each level of  $i$ , the maximum number of nodes is  $2^i$ .

The height of the tree is defined as the longest path from the root node to the leaf node. The tree which is shown above has a height equal to 3. Therefore, the maximum number of nodes at height 3 is equal to  $(1+2+4+8) = 15$ . In general, the maximum number of nodes possible at height  $h$  is  $(2^0 + 2^1 + 2^2 + \dots + 2^h) = 2^{h+1} - 1$ .

The minimum number of nodes possible at height  $h$  is equal to  **$h+1$** .

If the number of nodes is minimum, then the height of the tree would be maximum. Conversely, if the number of nodes is maximum, then the height of the tree would be minimum.

If there are 'n' number of nodes in the binary tree.

**The minimum height can be computed as:**

As we know that,

$$n = 2^{h+1} - 1$$

$$n+1 = 2^{h+1}$$

Taking log on both the sides,

$$\log_2(n+1) = \log_2(2^{h+1})$$

$$\log_2(n+1) = h+1$$

$$h = \log_2(n+1) - 1$$

**The maximum height can be computed as:**

As we know that,

$$n = h+1$$

$$h = n-1$$

## Types of Binary Tree

There are four types of Binary tree:

**Full/ proper/ strict Binary tree**

**Complete Binary tree**

**Perfect Binary tree**

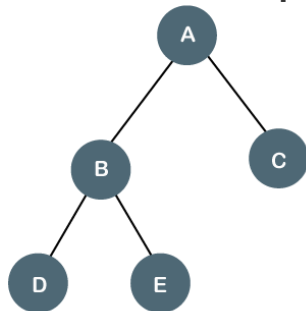
**Degenerate Binary tree**

**Balanced Binary tree**

### 1. Full/ proper/ strict Binary tree

The full binary tree is also known as a strict binary tree. The tree can only be considered as the full binary tree if each node must contain either 0 or 2 children. The full binary tree can also be defined as the tree in which each node must contain 2 children except the leaf nodes.

**Let's look at the simple example of the Full Binary tree.**



In the above tree, we can observe that each node is either containing zero or two children; therefore, it is a Full Binary tree.

### Properties of Full Binary Tree

The number of leaf nodes is equal to the number of internal nodes plus 1. In the above example, the number of internal nodes is 3; therefore, the number of leaf nodes is equal to 4.

The maximum number of nodes is the same as the number of nodes in the binary tree, i.e.,  $2^{h+1} - 1$ .

The minimum number of nodes in the full binary tree is  $2*h-1$ .

The minimum height of the full binary tree is  $\log_2(n+1) - 1$ .

The maximum height of the full binary tree can be computed as:

$$n = 2^h - 1$$

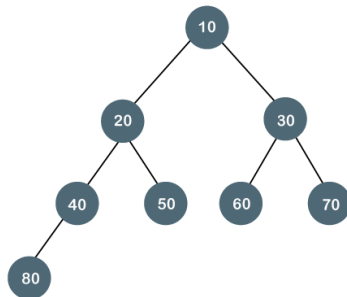
$$n+1 = 2^h$$

$$h = \log_2(n+1)$$

### Complete Binary Tree

The complete binary tree is a tree in which all the nodes are completely filled except the last level. In the last level, all the nodes must be as left as possible. In a complete binary tree, the nodes should be added from the left.

Let's create a complete binary tree.



The above tree is a complete binary tree because all the nodes are completely filled, and all the nodes in the last level are added at the left first.

### Properties of Complete Binary Tree

The maximum number of nodes in complete binary tree is  $2^{h+1} - 1$ .

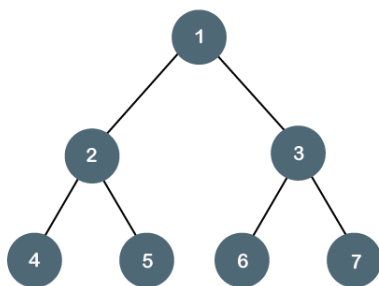
The minimum number of nodes in complete binary tree is  $2^h$ .

The minimum height of a complete binary tree is  $\log_2(n+1) - 1$ .

The maximum height of a complete binary tree is

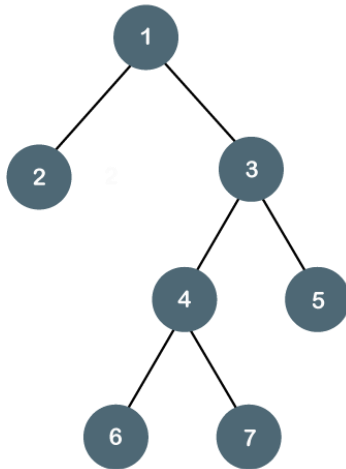
### Perfect Binary Tree

A tree is a perfect binary tree if all the internal nodes have 2 children, and all the leaf nodes are at the same level.



Let's look at a simple example of a perfect binary tree.

The below tree is not a perfect binary tree because all the leaf nodes are not at the same level.

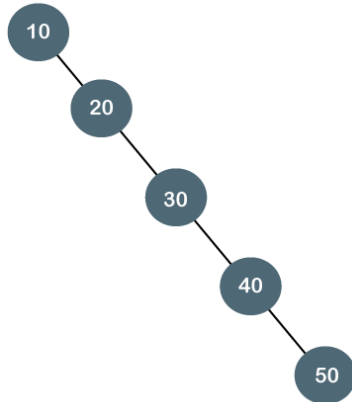


**Note:** All the perfect binary trees are the complete binary trees as well as the full binary tree, but vice versa is not true, i.e., all complete binary trees and full binary trees are the perfect binary trees.

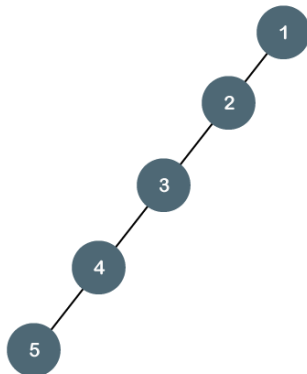
## Degenerate Binary Tree

The degenerate binary tree is a tree in which all the internal nodes have only one children.

**Let's understand the Degenerate binary tree through examples.**



The above tree is a degenerate binary tree because all the nodes have only one child. It is also known as a right-skewed tree as all the nodes have a right child only.



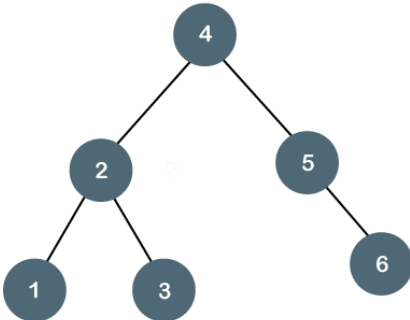
The above tree is also a degenerate binary tree because all the nodes have only one child. It is also known as a left-skewed tree as all the nodes have a left child only.



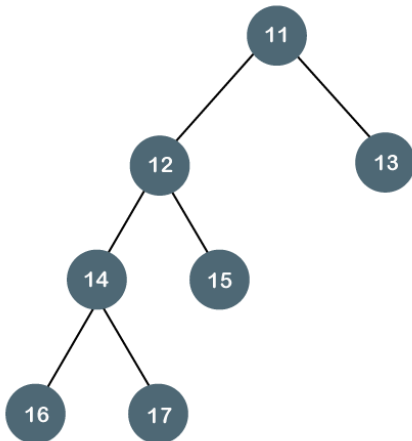
## Balanced Binary Tree

The balanced binary tree is a tree in which both the left and right trees differ by at most 1. For example, **AVL** and **Red-Black trees** are balanced binary tree.

**Let's understand the balanced binary tree through examples.**



The above tree is a balanced binary tree because the difference between the left subtree and right subtree is zero.



The above tree is not a balanced binary tree because the difference between the left subtree and the right subtree is greater than 1.

## Binary Tree Implementation

A Binary tree is implemented with the help of pointers. The first node in the tree is represented by the root pointer. Each node in the tree consists of three parts, i.e., data, left pointer and right pointer. To create a binary tree, we first need to create the node. We will create the node of user-defined as shown below:

```
struct node
{
    int data,
    struct node *left, *right;
}
```

In the above structure, **data** is the value, **left pointer** contains the address of the left node, and **right pointer** contains the address of the right node.

### Binary Tree program in C

```
#include<stdio.h>
struct node
```

```

{
    int data;
    struct node *left, *right;
}
void main()
{
    struct node *root;
    root = create();
}
struct node *create()
{
    struct node *temp;
    int data;
    temp = (struct node *)malloc(sizeof(struct node));
    printf("Press 0 to exit");
    printf("\nPress 1 for new node");
    printf("Enter your choice : ");
    scanf("%d", &choice);
    if(choice==0)
    {
        return 0;
    }
    else
    {
        printf("Enter the data:");
        scanf("%d", &data);
        temp->data = data;
        printf("Enter the left child of %d", data);
        temp->left = create();
        printf("Enter the right child of %d", data);
        temp->right = create();
        return temp;
    }
}

```

The above code is calling the create() function recursively and creating new node on each recursive call. When all the nodes are created, then it forms a binary tree structure. The process of visiting the nodes is known as tree traversal. There are three types traversals used to visit a node:

Inorder traversal

Preorder traversal

Postorder traversal

## Binary Search tree

In this article, we will discuss the Binary search tree. This article will be very helpful and informative to the students with technical background as it is an important topic of their course. Before moving directly to the binary search tree, let's first see a brief description of the tree.

## What is a tree?

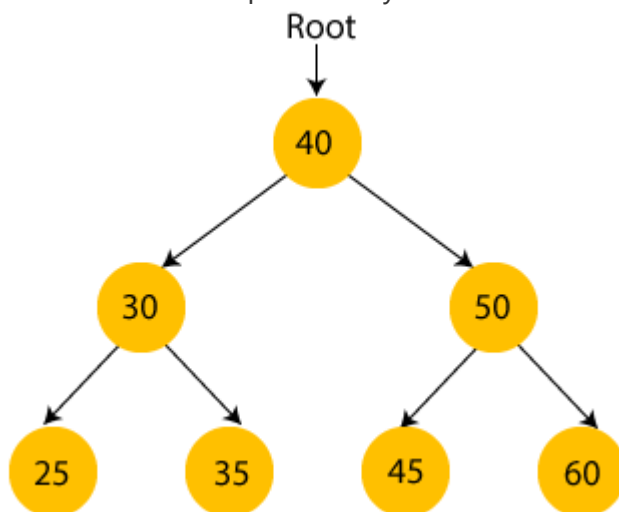
A tree is a kind of data structure that is used to represent the data in hierarchical form. It can be defined as a collection of objects or entities called as nodes that are linked together to simulate a hierarchy. Tree is a non-linear data structure as the data in a tree is not stored linearly or sequentially.

Now, let's start the topic, the Binary Search tree.

## What is a Binary Search tree?

A binary search tree follows some order to arrange the elements. In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.

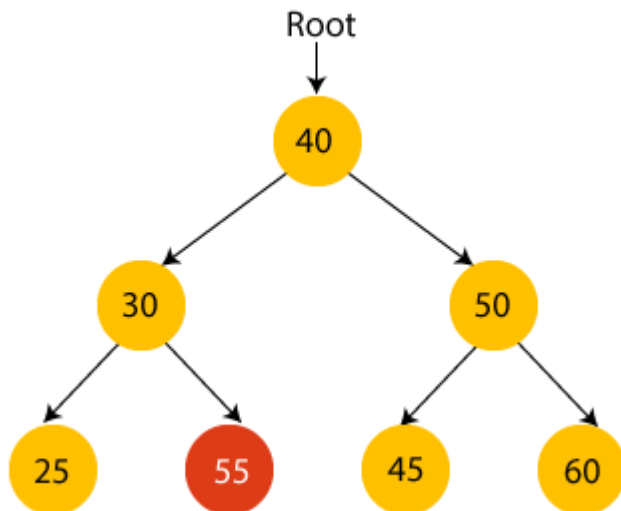
Let's understand the concept of Binary search tree with an example.



In the above figure, we can observe that the root node is 40, and all the nodes of the left subtree are smaller than the root node, and all the nodes of the right subtree are greater than the root node.

Similarly, we can see the left child of root node is greater than its left child and smaller than its right child. So, it also satisfies the property of binary search tree. Therefore, we can say that the tree in the above image is a binary search tree.

Suppose if we change the value of node 35 to 55 in the above tree, check whether the tree will be binary search tree or not.



In the above tree, the value of root node is 40, which is greater than its left child 30 but smaller than right child of 30, i.e., 55. So, the above tree does not satisfy the property of Binary search tree. Therefore, the above tree is not a binary search tree.

## Advantages of Binary search tree

Searching an element in the Binary search tree is easy as we always have a hint that which subtree has the desired element.

As compared to array and linked lists, insertion and deletion operations are faster in BST.

## Example of creating a binary search tree

Now, let's see the creation of binary search tree using an example.

Suppose the data elements are - **45, 15, 79, 90, 10, 55, 12, 20, 50**

First, we have to insert **45** into the tree as the root of the tree.

Then, read the next element; if it is smaller than the root node, insert it as the root of the left subtree, and move to the next element.

Otherwise, if the element is larger than the root node, then insert it as the root of the right subtree.

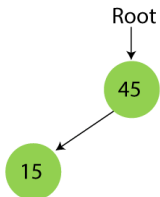
Now, let's see the process of creating the Binary search tree using the given data element. The process of creating the BST is shown below -

### Step 1 - Insert 45.



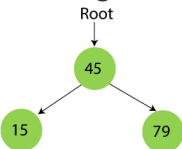
### Step 2 - Insert 15.

As 15 is smaller than 45, so insert it as the root node of the left subtree.



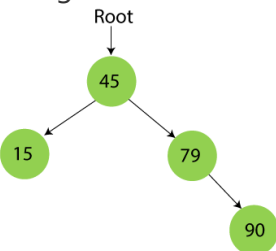
### Step 3 - Insert 79.

As 79 is greater than 45, so insert it as the root node of the right subtree.



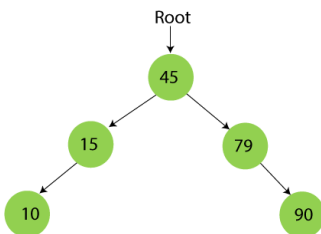
### Step 4 - Insert 90.

90 is greater than 45 and 79, so it will be inserted as the right subtree of 79.



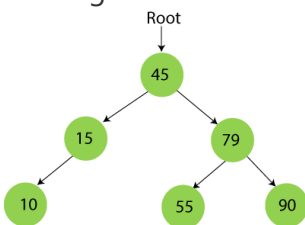
### Step 5 - Insert 10.

10 is smaller than 45 and 15, so it will be inserted as a left subtree of 15.



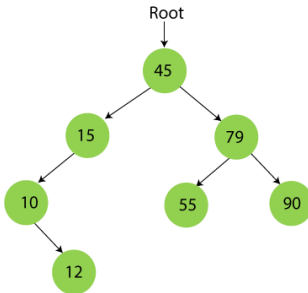
### Step 6 - Insert 55.

55 is larger than 45 and smaller than 79, so it will be inserted as the left subtree of 79.



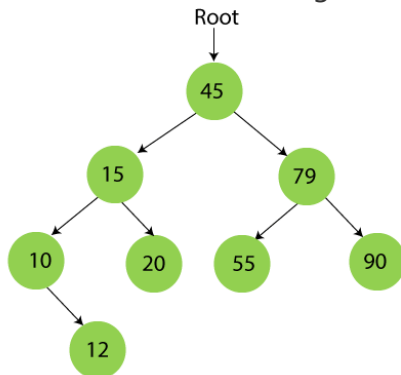
### Step 7 - Insert 12.

12 is smaller than 45 and 15 but greater than 10, so it will be inserted as the right subtree of 10.



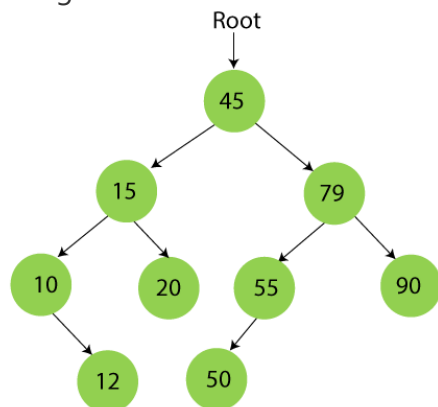
### Step 8 - Insert 20.

20 is smaller than 45 but greater than 15, so it will be inserted as the right subtree of 15.



### Step 9 - Insert 50.

50 is greater than 45 but smaller than 79 and 55. So, it will be inserted as a left subtree of 55.



Now, the creation of binary search tree is completed. After that, let's move towards the operations that can be performed on Binary search tree.

We can perform insert, delete and search operations on the binary search tree.

Let's understand how a search is performed on a binary search tree.

## Searching in Binary search tree

Searching means to find or locate a specific element or node in a data structure. In Binary search tree, searching a node is easy because elements in BST are stored in a specific order. The steps of searching a node in Binary Search tree are listed as follows -

First, compare the element to be searched with the root element of the tree.

If root is matched with the target element, then return the node's location.

If it is not matched, then check whether the item is less than the root element, if it is smaller than the root element, then move to the left subtree.

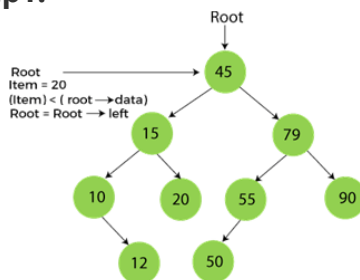
If it is larger than the root element, then move to the right subtree.

Repeat the above procedure recursively until the match is found.

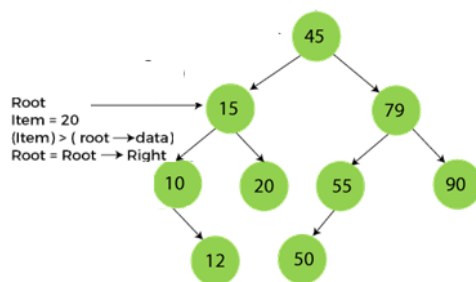
If the element is not found or not present in the tree, then return NULL.

Now, let's understand the searching in binary tree using an example. We are taking the binary search tree formed above. Suppose we have to find node 20 from the below tree.

**Step1:**



**Step2:**



**Step3:**



Now, let's see the algorithm to search an element in the Binary search tree.

## Algorithm to search an element in Binary search tree

Search (root, item)

Step 1 - if (item = root → data) or (root = NULL)

return root

else if (item < root → data)

return Search(root → left, item)

else

return Search(root → right, item)

END if

## Step 2 - END

Now let's understand how the deletion is performed on a binary search tree. We will also see an example to delete an element from the given tree.

### Deletion in Binary Search tree

In a binary search tree, we must delete a node from the tree by keeping in mind that the property of BST is not violated. To delete a node from BST, there are three possible situations occur -

The node to be deleted is the leaf node, or,

The node to be deleted has only one child, and,

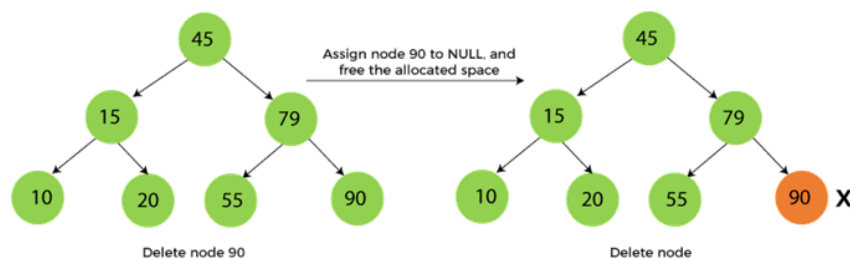
The node to be deleted has two children

We will understand the situations listed above in detail.

#### When the node to be deleted is the leaf node

It is the simplest case to delete a node in BST. Here, we have to replace the leaf node with NULL and simply free the allocated space.

We can see the process to delete a leaf node from BST in the below image. In below image, suppose we have to delete node 90, as the node to be deleted is a leaf node, so it will be replaced with NULL, and the allocated space will free.

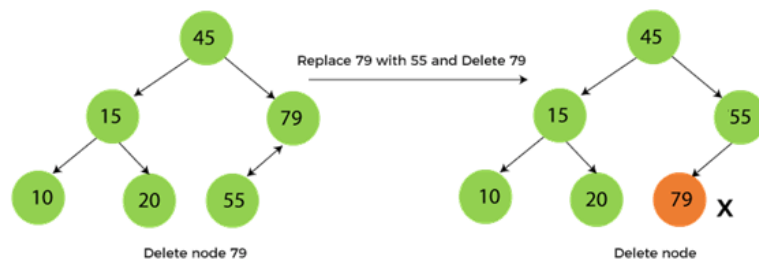


#### When the node to be deleted has only one child

In this case, we have to replace the target node with its child, and then delete the child node. It means that after replacing the target node with its child node, the child node will now contain the value to be deleted. So, we simply have to replace the child node with NULL and free up the allocated space.

We can see the process of deleting a node with one child from BST in the below image. In the below image, suppose we have to delete the node 79, as the node to be deleted has only one child, so it will be replaced with its child 55.

So, the replaced node 79 will now be a leaf node that can be easily deleted.



#### When the node to be deleted has two children

This case of deleting a node in BST is a bit complex among other two cases. In such a case, the steps to be followed are listed as follows -



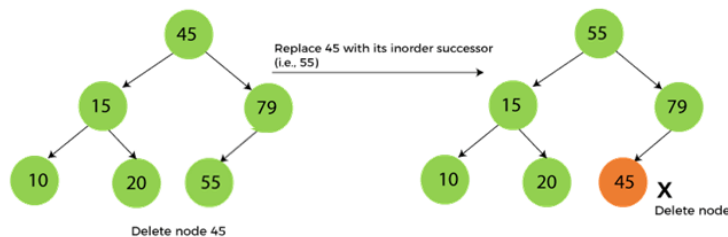
First, find the inorder successor of the node to be deleted.

After that, replace that node with the inorder successor until the target node is placed at the leaf of tree.

And at last, replace the node with NULL and free up the allocated space.

The inorder successor is required when the right child of the node is not empty. We can obtain the inorder successor by finding the minimum element in the right child of the node.

We can see the process of deleting a node with two children from BST in the below image. In the below image, suppose we have to delete node 45 that is the root node, as the node to be deleted has two children, so it will be replaced with its inorder successor. Now, node 45 will be at the leaf of the tree so that it can be deleted easily.

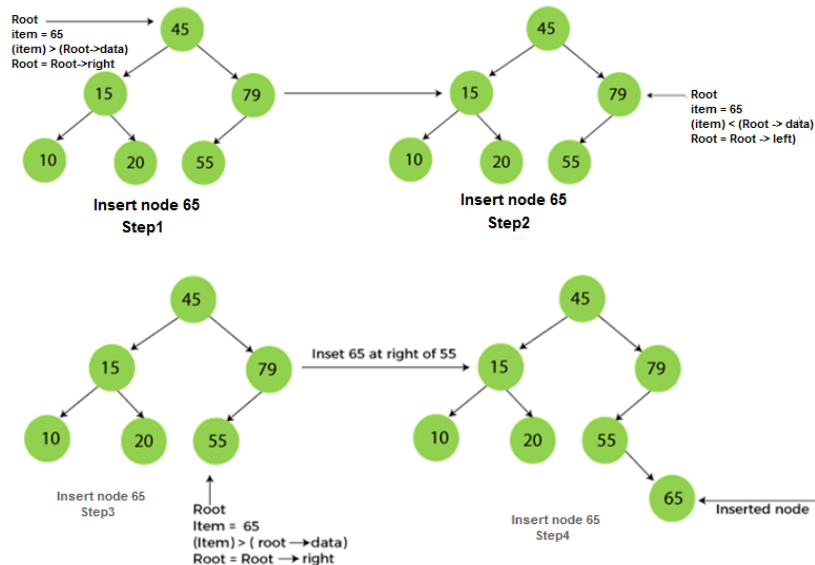


Now let's understand how insertion is performed on a binary search tree.

## Insertion in Binary Search tree

A new key in BST is always inserted at the leaf. To insert an element in BST, we have to start searching from the root node; if the node to be inserted is less than the root node, then search for an empty location in the left subtree. Else, search for the empty location in the right subtree and insert the data. Insert in BST is similar to searching, as we always have to maintain the rule that the left subtree is smaller than the root, and right subtree is larger than the root.

Now, let's see the process of inserting a node into BST using an example.



## The complexity of the Binary Search tree

Let's see the time and space complexity of the Binary search tree. We will see the time complexity for insertion, deletion, and searching operations in best case, average case, and worst case.

## 1. Time Complexity

| Operations | Best case time complexity | Average case time complexity | Worst case time complexity |
|------------|---------------------------|------------------------------|----------------------------|
| Insertion  | $O(\log n)$               | $O(\log n)$                  | $O(n)$                     |
| Deletion   | $O(\log n)$               | $O(\log n)$                  | $O(n)$                     |
| Search     | $O(\log n)$               | $O(\log n)$                  | $O(n)$                     |

Where 'n' is the number of nodes in the given tree.

## 2. Space Complexity

| Operations | Space complexity |
|------------|------------------|
| Insertion  | $O(n)$           |
| Deletion   | $O(n)$           |
| Search     | $O(n)$           |

The space complexity of all operations of Binary search tree is  $O(n)$ .

## Implementation of Binary search tree

Now, let's see the program to implement the operations of Binary Search tree.

**Program:** Write a program to perform operations of Binary Search tree in C++.

In this program, we will see the implementation of the operations of binary search tree. Here, we will see the creation, inorder traversal, insertion, and deletion operations of tree.

Here, we will see the inorder traversal of the tree to check whether the nodes of the tree are in their proper location or not. We know that the inorder traversal always gives us the data in ascending order. So, after performing the insertion and deletion operations, we perform the inorder traversal, and after traversing, if we get data in ascending order, then it is clear that the nodes are in their proper location.

```
#include <iostream>
```

```
using namespace std;
```

```
struct Node {
```

```
    int data;
```

```
    Node *left;
```

```
    Node *right;
```

```
};
```

```
Node* create(int item)
```

```
{
```

```

Node* node = new Node;
node->data = item;
node->left = node->right = NULL;
return node;
}
/*Inorder traversal of the tree formed*/
void inorder(Node *root)
{
    if (root == NULL)
        return;
    inorder(root->left); //traverse left subtree
    cout<< root->data << " "; //traverse root node
    inorder(root->right); //traverse right subtree
}
Node* findMinimum(Node* cur) /*To find the inorder successor*/
{
    while(cur->left != NULL) {
        cur = cur->left;
    }
    return cur;
}
Node* insertion(Node* root, int item) /*Insert a node*/
{
    if (root == NULL)
        return create(item); /*return new node if tree is empty*/
    if (item < root->data)
        root->left = insertion(root->left, item);
    else
        root->right = insertion(root->right, item);
    return root;
}
void search(Node* &cur, int item, Node* &parent)
{
    while (cur != NULL && cur->data != item)
    {
        parent = cur;
        if (item < cur->data)
            cur = cur->left;
        else
            cur = cur->right;
    }
}
void deletion(Node*& root, int item) /*function to delete a node*/
{

```

```

Node* parent = NULL;
Node* cur = root;
search(cur, item, parent); /*find the node to be deleted*/
if (cur == NULL)
    return;
if (cur->left == NULL && cur->right == NULL) /*When node has no children*/
{
    if (cur != root)
    {
        if (parent->left == cur)
            parent->left = NULL;
        else
            parent->right = NULL;
    }
    else
        root = NULL;
    free(cur);
}
else if (cur->left && cur->right)
{
    Node* succ = findMinimum(cur->right);
    int val = succ->data;
    deletion(root, succ->data);
    cur->data = val;
}
else
{
    Node* child = (cur->left)? cur->left: cur->right;
    if (cur != root)
    {
        if (cur == parent->left)
            parent->left = child;
        else
            parent->right = child;
    }
    else
        root = child;
    free(cur);
}
}
int main()
{
    Node* root = NULL;
    root = insertion(root, 45);
}

```

```

root = insertion(root, 30);
root = insertion(root, 50);
root = insertion(root, 25);
root = insertion(root, 35);
root = insertion(root, 45);
root = insertion(root, 60);
root = insertion(root, 4);
printf("The inorder traversal of the given binary tree is - \n");
inorder(root);
deletion(root, 25);
printf("\nAfter deleting node 25, the inorder traversal of the given binary tree is - \n");
inorder(root);
insertion(root, 2);
printf("\nAfter inserting node 2, the inorder traversal of the given binary tree is - \n");
inorder(root);
return 0;
}

```

### Output

After the execution of the above code, the output will be -

```

The inorder traversal of the given binary tree is -
4      25      30      35      45      45      50      60
After deleting node 25, the inorder traversal of the given binary tree is -
4      30      35      45      45      50      60
After inserting node 2, the inorder traversal of the given binary tree is -
2      4      30      35      45      45      50      60

```

So, that's all about the article. Hope the article will be helpful and informative to you.

## AVL Tree

AVL Tree is invented by GM Adelson - Velsky and EM Landis in 1962. The tree is named AVL in honour of its inventors.

AVL Tree can be defined as height balanced binary search tree in which each node is associated with a balance factor which is calculated by subtracting the height of its right sub-tree from that of its left sub-tree.

Tree is said to be balanced if balance factor of each node is in between -1 to 1, otherwise, the tree will be unbalanced and need to be balanced.

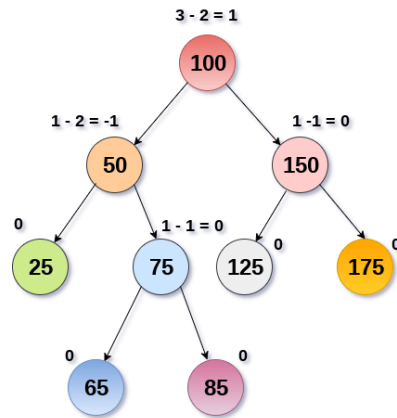
**Balance Factor (k) = height (left(k)) - height (right(k))**

If balance factor of any node is 1, it means that the left sub-tree is one level higher than the right sub-tree.

If balance factor of any node is 0, it means that the left sub-tree and right sub-tree contain equal height.

If balance factor of any node is -1, it means that the left sub-tree is one level lower than the right sub-tree.

An AVL tree is given in the following figure. We can see that, balance factor associated with each node is in between -1 and +1. therefore, it is an example of AVL tree.



AVL Tree

## Complexity

| Algorithm | Average case | Worst case  |
|-----------|--------------|-------------|
| Space     | $O(n)$       | $O(n)$      |
| Search    | $O(\log n)$  | $O(\log n)$ |
| Insert    | $O(\log n)$  | $O(\log n)$ |
| Delete    | $O(\log n)$  | $O(\log n)$ |

## Operations on AVL tree

Due to the fact that, AVL tree is also a binary search tree therefore, all the operations are performed in the same way as they are performed in a binary search tree. Searching and traversing do not lead to the violation in property of AVL tree. However, insertion and deletion are the operations which can violate this property and therefore, they need to be revisited.

| SN | Operation                 | Description                                                                                                                                                                                                                                          |
|----|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1  | <a href="#">Insertion</a> | Insertion in AVL tree is performed in the same way as it is performed in a binary search tree. However, it may lead to violation in the AVL tree property and therefore the tree may need balancing. The tree can be balanced by applying rotations. |
| 2  | <a href="#">Deletion</a>  | Deletion can also be performed in the same way as it is performed in a binary search tree. Deletion may also disturb the balance of the tree therefore, various types of rotations are used to rebalance the tree.                                   |

## Why AVL Tree?

AVL tree controls the height of the binary search tree by not letting it to be skewed. The time taken for all operations in a binary search tree of height  $h$  is  $O(h)$ . However, it can be extended to  $O(n)$  if the BST becomes skewed (i.e. worst case). By limiting this height to  $\log n$ , AVL tree imposes an upper bound on each operation to be  $O(\log n)$  where  $n$  is the number of nodes.

## AVL Rotations

We perform rotation in AVL tree only in case if Balance Factor is other than **-1, 0, and 1**. There are basically four types of rotations which are as follows:

L L rotation: Inserted node is in the left subtree of left subtree of A

R R rotation : Inserted node is in the right subtree of right subtree of A

L R rotation : Inserted node is in the right subtree of left subtree of A

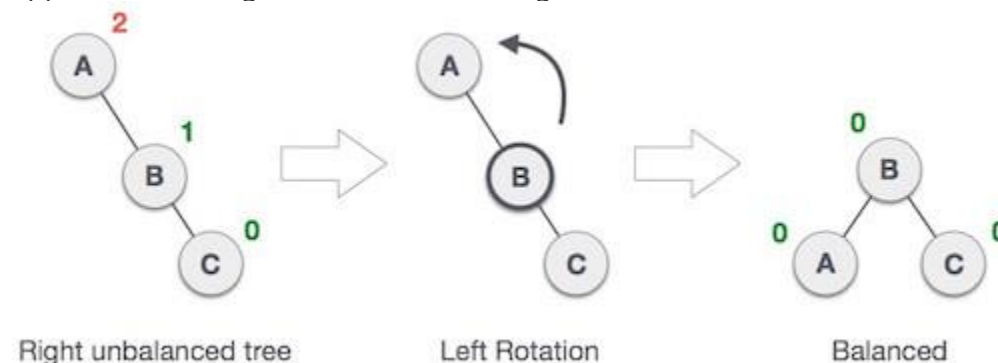
R L rotation : Inserted node is in the left subtree of right subtree of A

Where node A is the node whose balance Factor is other than -1, 0, 1.

The first two rotations LL and RR are single rotations and the next two rotations LR and RL are double rotations. For a tree to be unbalanced, minimum height must be at least 2, Let us understand each rotation

### 1. RR Rotation

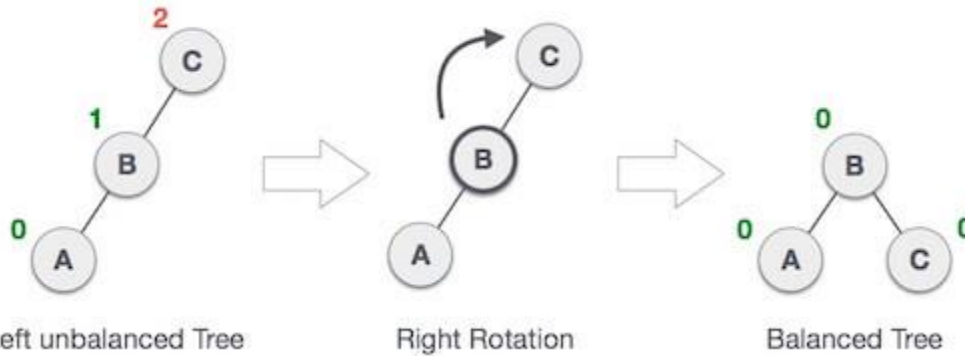
When BST becomes unbalanced, due to a node is inserted into the right subtree of the right subtree of A, then we perform RR rotation, [RR rotation](#) is an anticlockwise rotation, which is applied on the edge below a node having balance factor -2



In above example, node A has balance factor -2 because a node C is inserted in the right subtree of A right subtree. We perform the RR rotation on the edge below A.

### 2. LL Rotation

When BST becomes unbalanced, due to a node is inserted into the left subtree of the left subtree of C, then we perform LL rotation, [LL rotation](#) is clockwise rotation, which is applied on the edge below a node having balance factor 2.



In above example, node C has balance factor 2 because a node A is inserted in the left subtree of C left subtree. We perform the LL rotation on the edge below A.

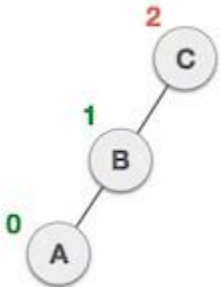
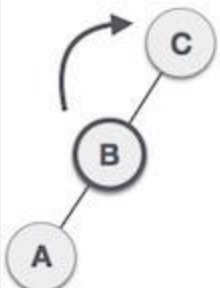
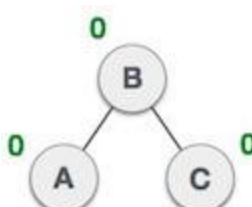
### 3. LR Rotation

Double rotations are bit tougher than single rotation which has already explained above. LR rotation = RR rotation + LL rotation, i.e., first RR rotation is performed on subtree and then LL rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

**Let us understand each and every step very clearly:**

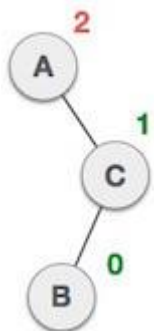
| State | Action                                                                                                                                                                                                                                              |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       | A node B has been inserted into the right subtree of A the left subtree of C, because of which C has become an unbalanced node having balance factor 2. This case is L R rotation where: Inserted node is in the right subtree of left subtree of C |
|       | As LR rotation = RR + LL rotation, hence RR (anticlockwise) on subtree rooted at A is performed first. By doing RR rotation, node <b>A</b> , has become the left subtree of <b>B</b> .                                                              |

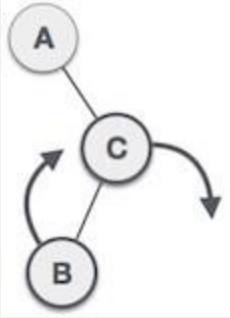
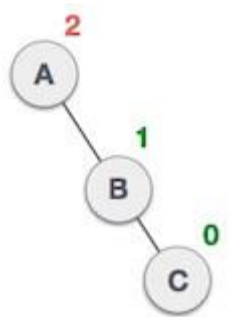
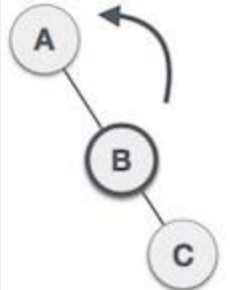
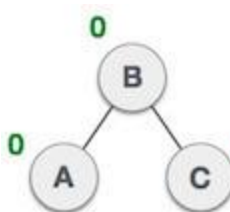


|                                                                                    |                                                                                                                                                            |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|   | <p>After performing RR rotation, node C is still unbalanced, i.e., having balance factor 2, as inserted node A is in the left of left of <b>C</b></p>      |
|   | <p>Now we perform LL clockwise rotation on full tree, i.e. on node C. node <b>C</b> has now become the right subtree of node B, A is left subtree of B</p> |
|  | <p>Balance factor of each node is now either -1, 0, or 1, i.e. BST is balanced now.</p>                                                                    |

## 4. RL Rotation

As already discussed, that double rotations are bit tougher than single rotation which has already explained above. RL rotation = LL rotation + RR rotation, i.e., first LL rotation is performed on subtree and then RR rotation is performed on full tree, by full tree we mean the first node from the path of inserted node whose balance factor is other than -1, 0, or 1.

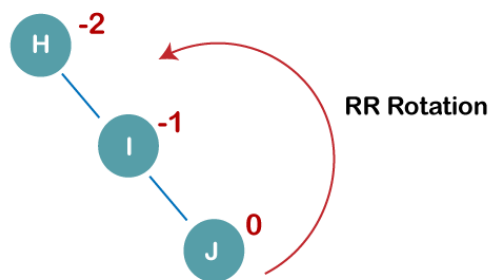
| State                                                                               | Action                                                                                                                                                                                                                                                                           |
|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p>A node <b>B</b> has been inserted into the left subtree of <b>C</b> the right subtree of <b>A</b>, because of which A has become an unbalanced node having balance factor - 2. This case is RL rotation where: Inserted node is in the left subtree of right subtree of A</p> |

|                                                                                     |                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p>As RL rotation = LL rotation + RR rotation, hence, LL (clockwise) on subtree rooted at <b>C</b> is performed first. By doing RR rotation, node <b>C</b> has become the right subtree of <b>B</b>.</p> |
|    | <p>After performing LL rotation, node <b>A</b> is still unbalanced, i.e. having balance factor -2, which is because of the right-subtree of the right-subtree node A.</p>                                |
|   | <p>Now we perform RR rotation (anticlockwise rotation) on full tree, i.e. on node A. node <b>C</b> has now become the right subtree of node B, and node A has become the left subtree of B.</p>          |
|  | <p>Balance factor of each node is now either -1, 0, or 1, i.e., BST is balanced now.</p>                                                                                                                 |

## Q: Construct an AVL tree having the following elements

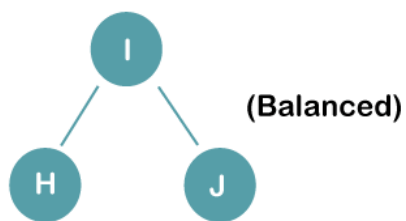
H, I, J, B, A, E, C, F, D, G, K, L

1. Insert H, I, J

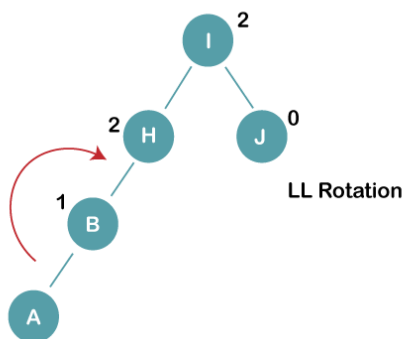


On inserting the above elements, especially in the case of H, the BST becomes unbalanced as the Balance Factor of H is -2. Since the BST is right-skewed, we will perform RR Rotation on node H.

**The resultant balance tree is:**

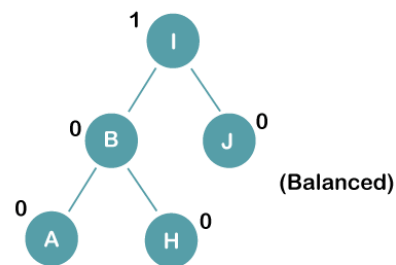


## 2. Insert B, A

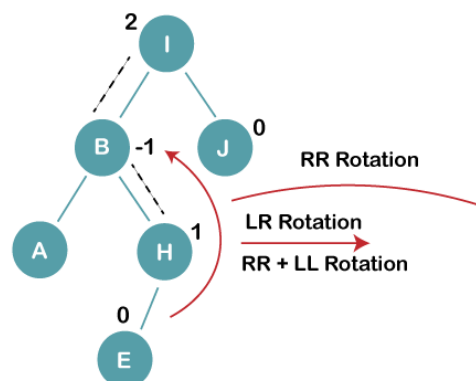


On inserting the above elements, especially in case of A, the BST becomes unbalanced as the Balance Factor of H and I is 2, we consider the first node from the last inserted node i.e. H. Since the BST from H is left-skewed, we will perform LL Rotation on node H.

**The resultant balance tree is:**



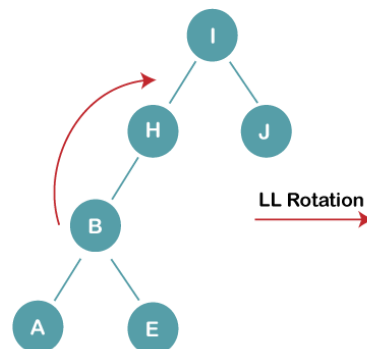
## 3. Insert E



On inserting E, BST becomes unbalanced as the Balance Factor of I is 2, since if we travel from E to I we find that it is inserted in the left subtree of right subtree of I, we will perform LR Rotation on node I. LR = RR + LL rotation

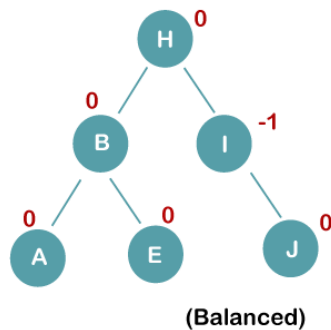
**3 a) We first perform RR rotation on node B**

**The resultant tree after RR rotation is:**

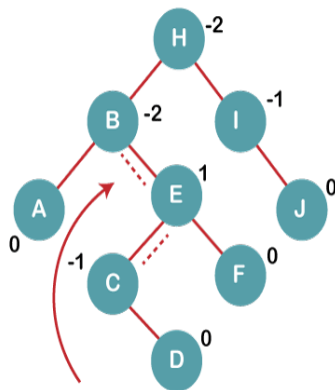


**3b) We first perform LL rotation on the node I**

The resultant balanced tree after LL rotation is:



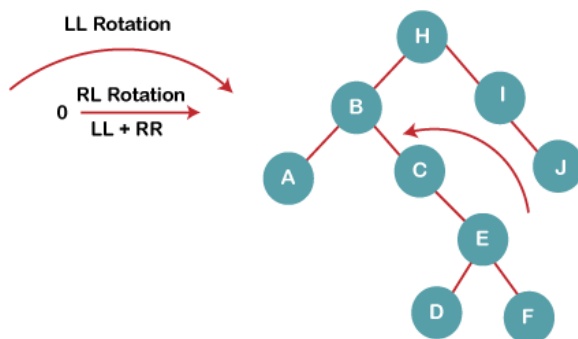
4. Insert C, F, D



On inserting C, F, D, BST becomes unbalanced as the Balance Factor of B and H is -2, since if we travel from D to B we find that it is inserted in the right subtree of left subtree of B, we will perform RL Rotation on node I. RL = LL + RR rotation.

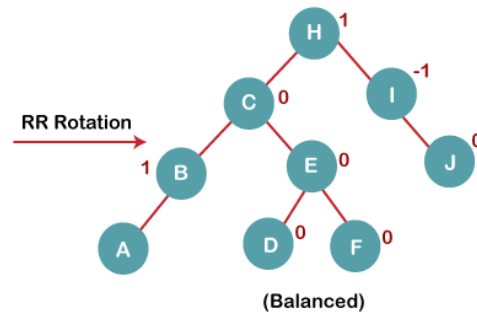
4a) We first perform LL rotation on node E

The resultant tree after LL rotation is:

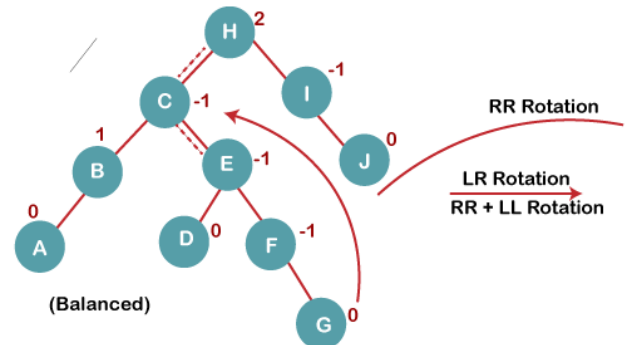


4b) We then perform RR rotation on node B

The resultant balanced tree after RR rotation is:



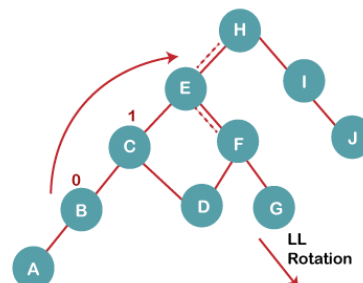
5. Insert G



On inserting G, BST become unbalanced as the Balance Factor of H is 2, since if we travel from G to H, we find that it is inserted in the left subtree of right subtree of H, we will perform LR Rotation on node I. LR = RR + LL rotation.

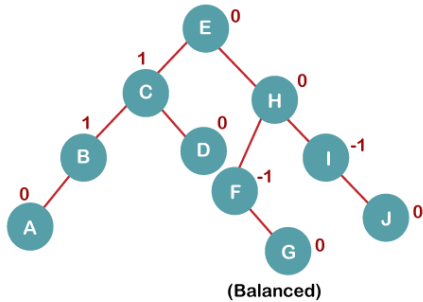
5 a) We first perform RR rotation on node C

The resultant tree after RR rotation is:

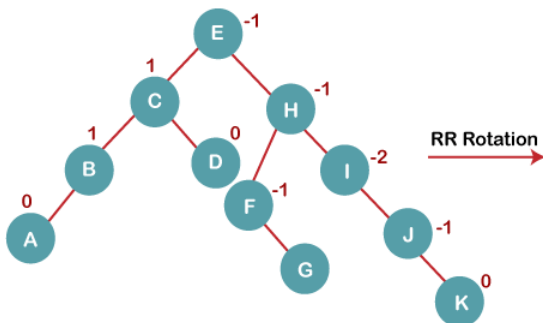


5 b) We then perform LL rotation on node H

The resultant balanced tree after LL rotation is:

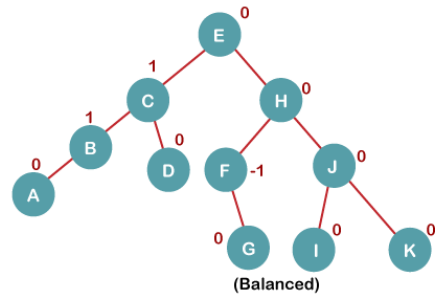


## 6. Insert K



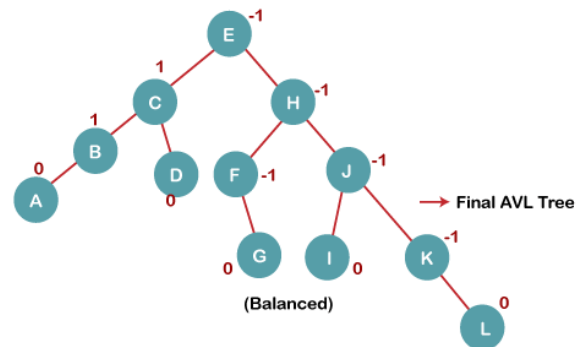
On inserting K, BST becomes unbalanced as the Balance Factor of I is -2. Since the BST is right-skewed from I to K, hence we will perform RR Rotation on the node I.

The resultant balanced tree after RR rotation is:



## 7. Insert L

On inserting the L tree is still balanced as the Balance Factor of each node is now either, -1, 0, +1. Hence the tree is a Balanced AVL tree



# B Tree

B Tree is a specialized m-way tree that can be widely used for disk access. A B-Tree of order m can have at most m-1 keys and m children. One of the main reason of using B tree is its capability to store large number of keys in a single node and large key values by keeping the height of the tree relatively small.

A B tree of order m contains all the properties of an M way tree. In addition, it contains the following properties.

Every node in a B-Tree contains at most m children.

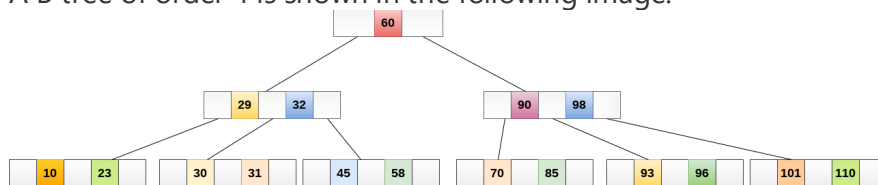
Every node in a B-Tree except the root node and the leaf node contain at least  $m/2$  children.

The root nodes must have at least 2 nodes.

All leaf nodes must be at the same level.

It is not necessary that, all the nodes contain the same number of children but, each node must have  $m/2$  number of nodes.

A B tree of order 4 is shown in the following image.



While performing some operations on B Tree, any property of B Tree may violate such as number of minimum children a node can have. To maintain the properties of B Tree, the tree may split or join.

## Operations

### Searching :

Searching in B Trees is similar to that in Binary search tree. For example, if we search for an item 49 in the following B Tree. The process will something like following :

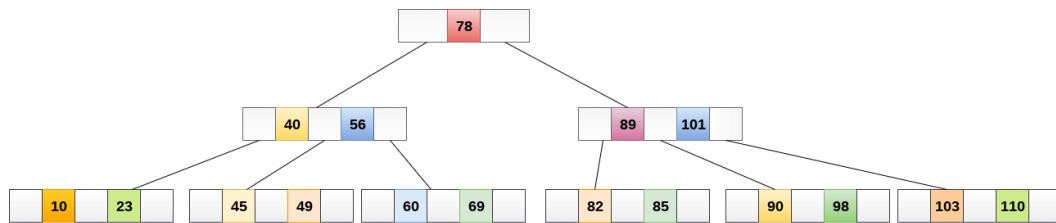
Compare item 49 with root node 78. since  $49 < 78$  hence, move to its left sub-tree.

Since,  $40 < 49 < 56$ , traverse right sub-tree of 40.

$49 > 45$ , move to right. Compare 49.

match found, return.

Searching in a B tree depends upon the height of the tree. The search algorithm takes  $O(\log n)$  time to search any element in a B tree.



## Inserting

Insertions are done at the leaf node level. The following algorithm needs to be followed in order to insert an item into B Tree.

Traverse the B Tree in order to find the appropriate leaf node at which the node can be inserted. If the leaf node contains less than  $m-1$  keys then insert the element in the increasing order.

Else, if the leaf node contains  $m-1$  keys, then follow the following steps.

Insert the new element in the increasing order of elements.

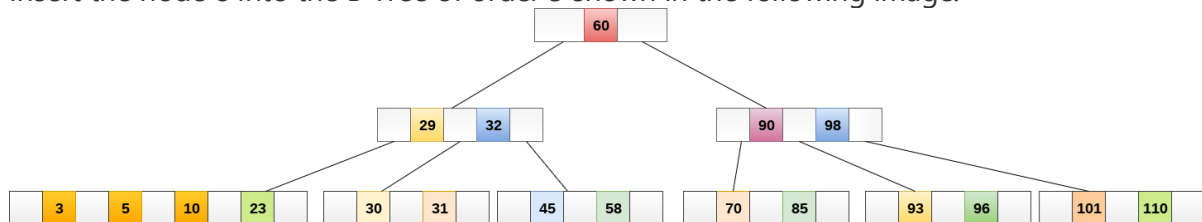
Split the node into the two nodes at the median.

Push the median element up to its parent node.

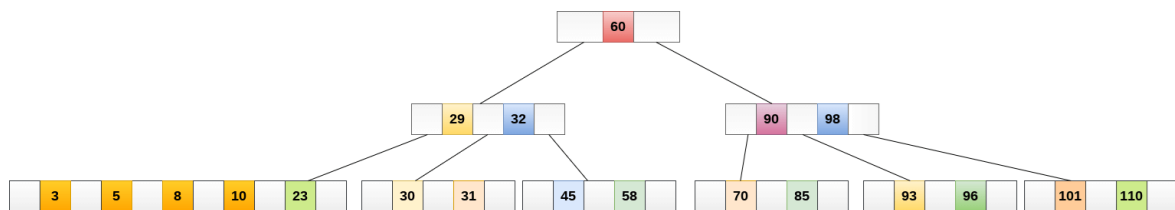
If the parent node also contains  $m-1$  number of keys, then split it too by following the same steps.

### Example:

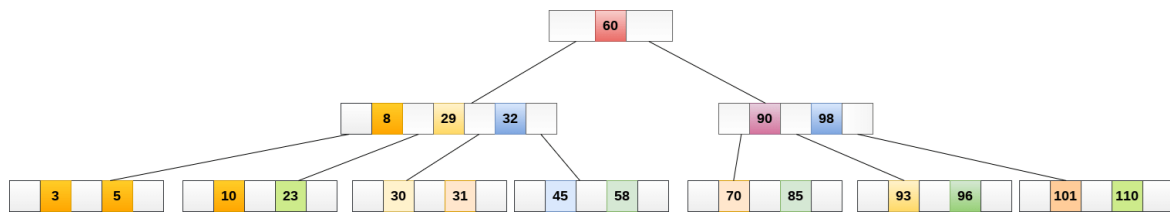
Insert the node 8 into the B Tree of order 5 shown in the following image.



8 will be inserted to the right of 5, therefore insert 8.



The node, now contains 5 keys which is greater than  $(5 - 1 = 4)$  keys. Therefore split the node from the median i.e. 8 and push it up to its parent node shown as follows.



## Deletion

Deletion is also performed at the leaf nodes. The node which is to be deleted can either be a leaf node or an internal node. Following algorithm needs to be followed in order to delete a node from a B tree.

Locate the leaf node.

If there are more than  $m/2$  keys in the leaf node then delete the desired key from the node.

If the leaf node doesn't contain  $m/2$  keys then complete the keys by taking the element from eight or left sibling.

If the left sibling contains more than  $m/2$  elements then push its largest element up to its parent and move the intervening element down to the node where the key is deleted.

If the right sibling contains more than  $m/2$  elements then push its smallest element up to the parent and move intervening element down to the node where the key is deleted.

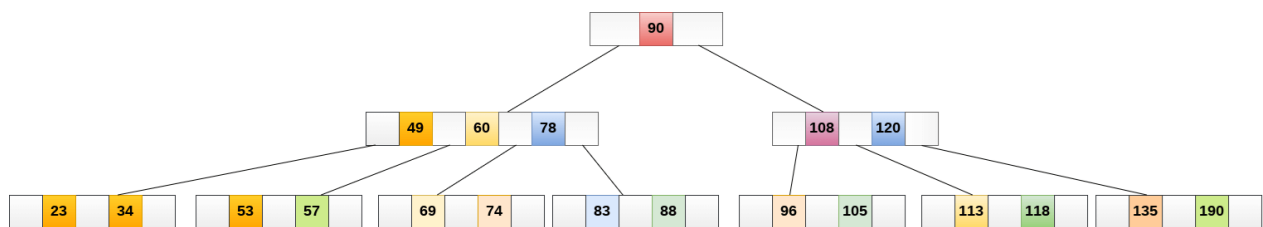
If neither of the sibling contain more than  $m/2$  elements then create a new leaf node by joining two leaf nodes and the intervening element of the parent node.

If parent is left with less than  $m/2$  nodes then, apply the above process on the parent too.

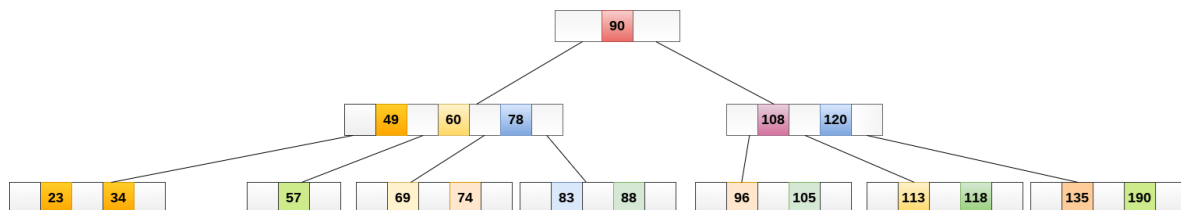
If the the node which is to be deleted is an internal node, then replace the node with its in-order successor or predecessor. Since, successor or predecessor will always be on the leaf node hence, the process will be similar as the node is being deleted from the leaf node.

### Example 1

Delete the node 53 from the B Tree of order 5 shown in the following figure.



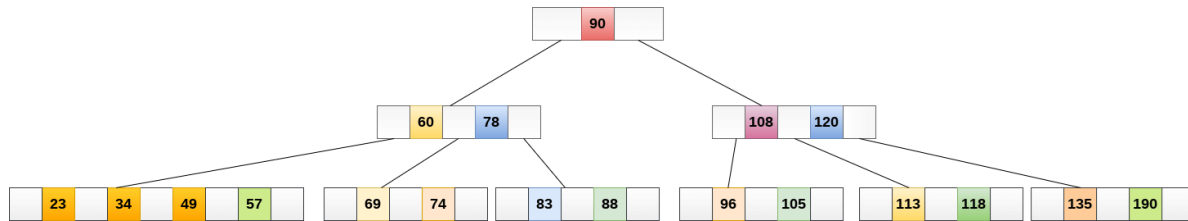
53 is present in the right child of element 49. Delete it.





Now, 57 is the only element which is left in the node, the minimum number of elements that must be present in a B tree of order 5, is 2. it is less than that, the elements in its left and right sub-tree are also not sufficient therefore, merge it with the left sibling and intervening element of parent i.e. 49.

The final B tree is shown as follows.



## Application of B tree

B tree is used to index the data and provides fast access to the actual data stored on the disks since, the access to value stored in a large database that is stored on a disk is a very time consuming process.

Searching an un-indexed and unsorted database containing  $n$  key values needs  $O(n)$  running time in worst case. However, if we use B Tree to index this database, it will be searched in  $O(\log n)$  time in worst case.

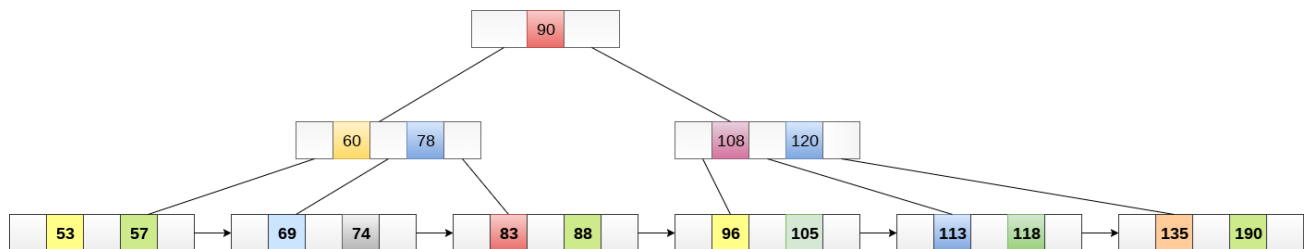
## B+ Tree

B+ Tree is an extension of B Tree which allows efficient insertion, deletion and search operations. In B Tree, Keys and records both can be stored in the internal as well as leaf nodes. Whereas, in B+ tree, records (data) can only be stored on the leaf nodes while internal nodes can only store the key values.

The leaf nodes of a B+ tree are linked together in the form of a singly linked lists to make the search queries more efficient.

B+ Tree are used to store the large amount of data which can not be stored in the main memory. Due to the fact that, size of main memory is always limited, the internal nodes (keys to access records) of the B+ tree are stored in the main memory whereas, leaf nodes are stored in the secondary memory.

The internal nodes of B+ tree are often called index nodes. A B+ tree of order 3 is shown in the following figure.



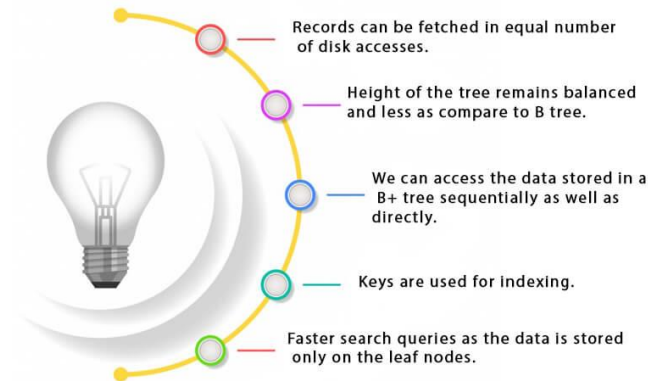
## Advantages of B+ Tree

Records can be fetched in equal number of disk accesses.

Height of the tree remains balanced and less as compare to B tree.

We can access the data stored in a B+ tree sequentially as well as directly.  
 Keys are used for indexing.  
 Faster search queries as the data is stored only on the leaf nodes.

### Advantages of B+ Tree



## B Tree VS B+ Tree

| SN | B Tree                                                                                                              | B+ Tree                                                                                              |
|----|---------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| 1  | Search keys can not be repeatedly stored.                                                                           | Redundant search keys can be present.                                                                |
| 2  | Data can be stored in leaf nodes as well as internal nodes                                                          | Data can only be stored on the leaf nodes.                                                           |
| 3  | Searching for some data is a slower process since data can be found on internal nodes as well as on the leaf nodes. | Searching is comparatively faster as data can only be found on the leaf nodes.                       |
| 4  | Deletion of internal nodes are so complicated and time consuming.                                                   | Deletion will never be a complexed process since element will always be deleted from the leaf nodes. |
| 5  | Leaf nodes can not be linked together.                                                                              | Leaf nodes are linked together to make the search operations more efficient.                         |

## Insertion in B+ Tree

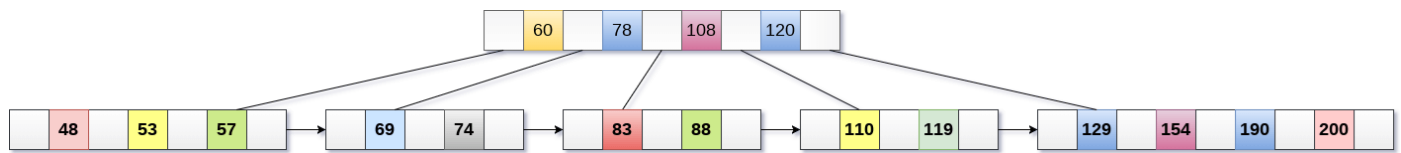
**Step 1:** Insert the new node as a leaf node

**Step 2:** If the leaf doesn't have required space, split the node and copy the middle node to the next index node.

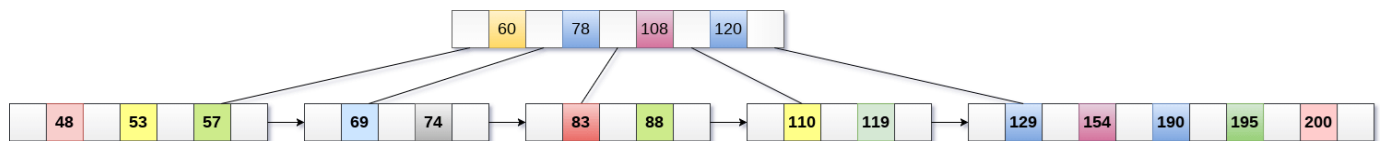
**Step 3:** If the index node doesn't have required space, split the node and copy the middle element to the next index page.

### Example :

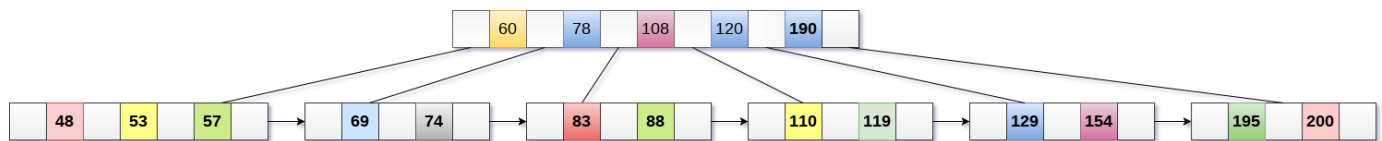
Insert the value 195 into the B+ tree of order 5 shown in the following figure.



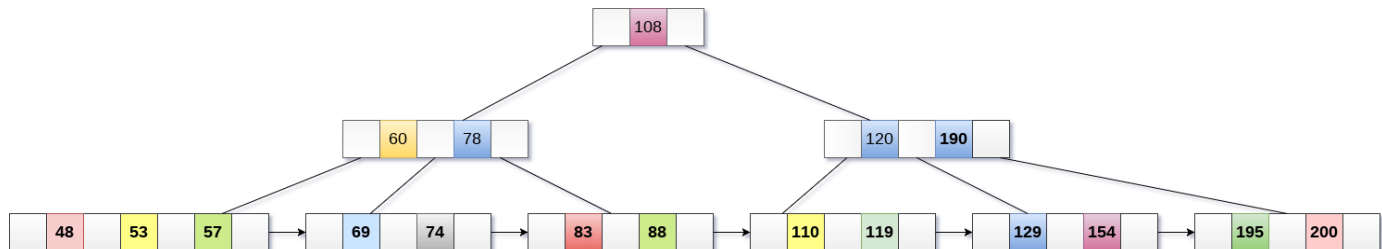
195 will be inserted in the right sub-tree of 120 after 190. Insert it at the desired position.



The node contains greater than the maximum number of elements i.e. 4, therefore split it and place the median node up to the parent.



Now, the index node contains 6 children and 5 keys which violates the B+ tree properties, therefore we need to split it, shown as follows.



## Deletion in B+ Tree

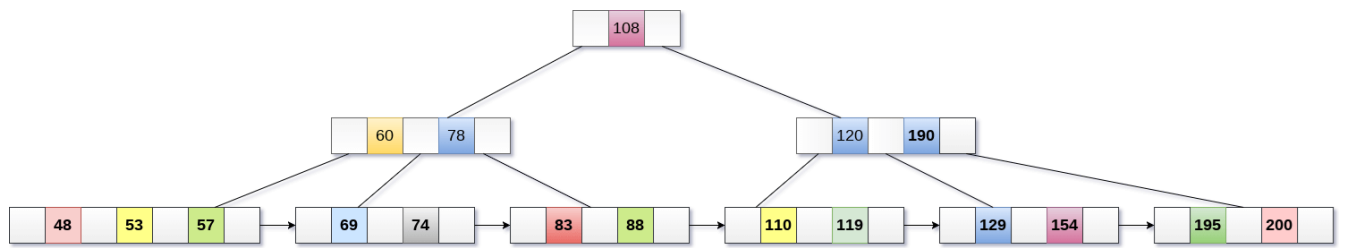
**Step 1:** Delete the key and data from the leaves.

**Step 2:** if the leaf node contains less than minimum number of elements, merge down the node with its sibling and delete the key in between them.

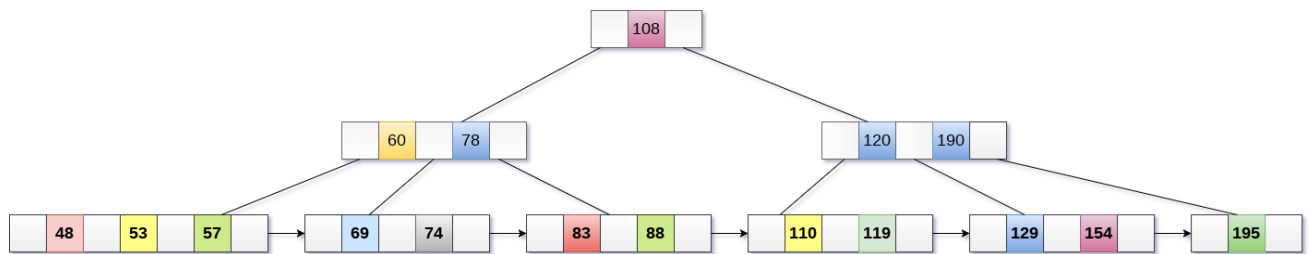
**Step 3:** if the index node contains less than minimum number of elements, merge the node with the sibling and move down the key in between them.

### Example

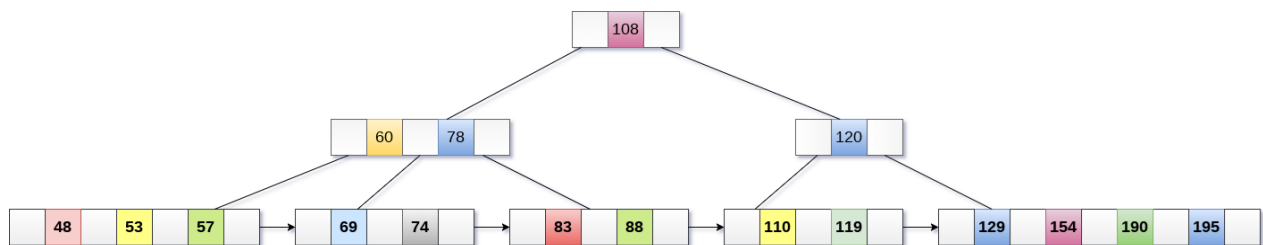
Delete the key 200 from the B+ Tree shown in the following figure.



200 is present in the right sub-tree of 190, after 195. delete it.

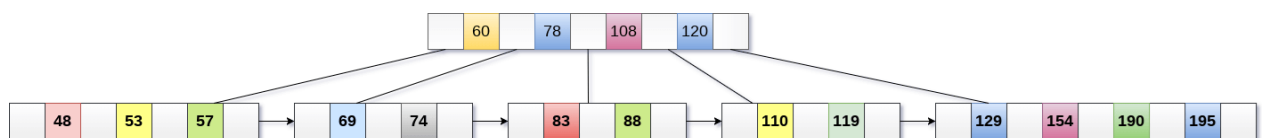


Merge the two nodes by using 195, 190, 154 and 129.



Now, element 120 is the single element present in the node which is violating the B+ Tree properties. Therefore, we need to merge it by using 60, 78, 108 and 120.

Now, the height of B+ tree will be decreased by 1.



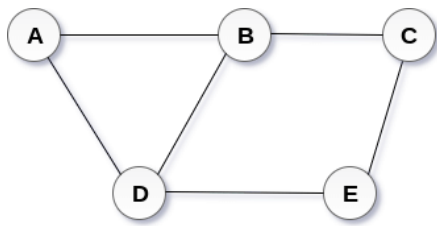
## Graph

A graph can be defined as group of vertices and edges that are used to connect these vertices. A graph can be seen as a cyclic tree, where the vertices (Nodes) maintain any complex relationship among them instead of having parent child relationship.

## Definition

A graph  $G$  can be defined as an ordered set  $G(V, E)$  where  $V(G)$  represents the set of vertices and  $E(G)$  represents the set of edges which are used to connect these vertices.

A Graph  $G(V, E)$  with 5 vertices (A, B, C, D, E) and six edges ((A,B), (B,C), (C,E), (E,D), (D,B), (D,A)) is shown in the following figure.

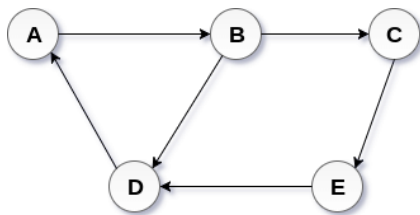


**Undirected Graph**

## Directed and Undirected Graph

A graph can be directed or undirected. However, in an undirected graph, edges are not associated with the directions with them. An undirected graph is shown in the above figure since its edges are not attached with any of the directions. If an edge exists between vertex A and B then the vertices can be traversed from B to A as well as A to B. In a directed graph, edges form an ordered pair. Edges represent a specific path from some vertex A to another vertex B. Node A is called initial node while node B is called terminal node.

A directed graph is shown in the following figure.



**Directed Graph**

## Graph Terminology

### Path

A path can be defined as the sequence of nodes that are followed in order to reach some terminal node V from the initial node U.

### Closed Path

A path will be called as closed path if the initial node is same as terminal node. A path will be closed path if  $V_0 = V_N$ .

### Simple Path

If all the nodes of the graph are distinct with an exception  $V_0 = V_N$ , then such path P is called as closed simple path.

## Cycle

A cycle can be defined as the path which has no repeated edges or vertices except the first and last vertices.

## Connected Graph

A connected graph is the one in which some path exists between every two vertices ( $u, v$ ) in  $V$ . There are no isolated nodes in connected graph.

## Complete Graph

A complete graph is the one in which every node is connected with all other nodes. A complete graph contains  $n(n-1)/2$  edges where  $n$  is the number of nodes in the graph.

## Weighted Graph

In a weighted graph, each edge is assigned with some data such as length or weight. The weight of an edge  $e$  can be given as  $w(e)$  which must be a positive (+) value indicating the cost of traversing the edge.

## Digraph

A digraph is a directed graph in which each edge of the graph is associated with some direction and the traversing can be done only in the specified direction.

## Loop

An edge that is associated with the similar end points can be called as Loop.

## Adjacent Nodes

If two nodes  $u$  and  $v$  are connected via an edge  $e$ , then the nodes  $u$  and  $v$  are called as neighbours or adjacent nodes.

## Degree of the Node

A degree of a node is the number of edges that are connected with that node. A node with degree 0 is called as isolated node.

# Graph representation

In this article, we will discuss the ways to represent the graph. By Graph representation, we simply mean the technique to be used to store some graph into the computer's memory.

A graph is a data structure that consists of a set of vertices (called nodes) and edges. There are two ways to store Graphs into the computer's memory:

**Sequential representation** (or, Adjacency matrix representation)

**Linked list representation** (or, Adjacency list representation)

In sequential representation, an adjacency matrix is used to store the graph. Whereas in linked list representation, there is a use of an adjacency list to store the graph.

In this tutorial, we will discuss each one of them in detail.

Now, let's start discussing the ways of representing a graph in the data structure.

## Sequential representation

In sequential representation, there is a use of an adjacency matrix to represent the mapping between vertices and edges of the graph. We can use an adjacency matrix to represent the undirected graph, directed graph, weighted directed graph, and weighted undirected graph.

If  $\text{adj}[i][j] = w$ , it means that there is an edge exists from vertex  $i$  to vertex  $j$  with weight  $w$ .  
 An entry  $A_{ij}$  in the adjacency matrix representation of an undirected graph  $G$  will be 1 if an edge exists between  $V_i$  and  $V_j$ . If an Undirected Graph  $G$  consists of  $n$  vertices, then the adjacency matrix for that graph is  $n \times n$ , and the matrix  $A = [a_{ij}]$  can be defined as -

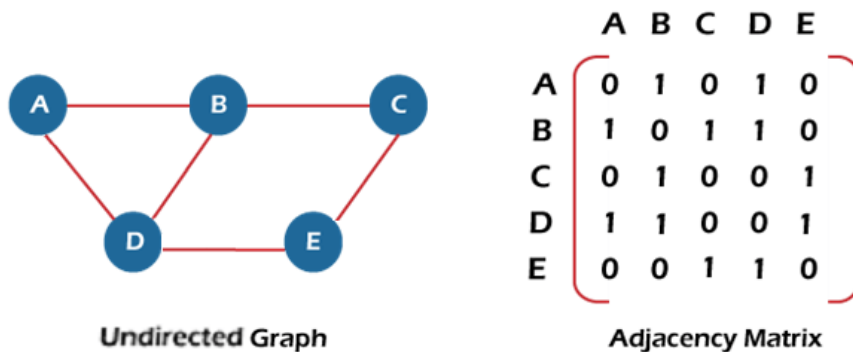
$a_{ij} = 1$  {if there is a path exists from  $V_i$  to  $V_j$ }

$a_{ij} = 0$  {Otherwise}

It means that, in an adjacency matrix, 0 represents that there is no association exists between the nodes, whereas 1 represents the existence of a path between two edges.

If there is no self-loop present in the graph, it means that the diagonal entries of the adjacency matrix will be 0.

Now, let's see the adjacency matrix representation of an undirected graph.



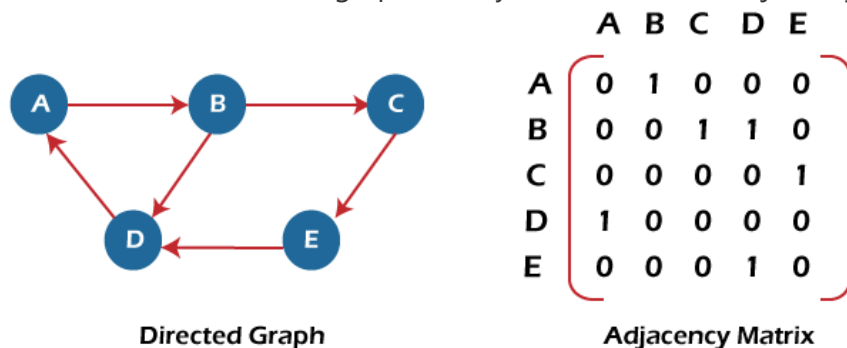
In the above figure, an image shows the mapping among the vertices (A, B, C, D, E), and this mapping is represented by using the adjacency matrix.

There exist different adjacency matrices for the directed and undirected graph. In a directed graph, an entry  $A_{ij}$  will be 1 only when there is an edge directed from  $V_i$  to  $V_j$ .

## Adjacency matrix for a directed graph

In a directed graph, edges represent a specific path from one vertex to another vertex. Suppose a path exists from vertex A to another vertex B; it means that node A is the initial node, while node B is the terminal node.

Consider the below-directed graph and try to construct the adjacency matrix of it.

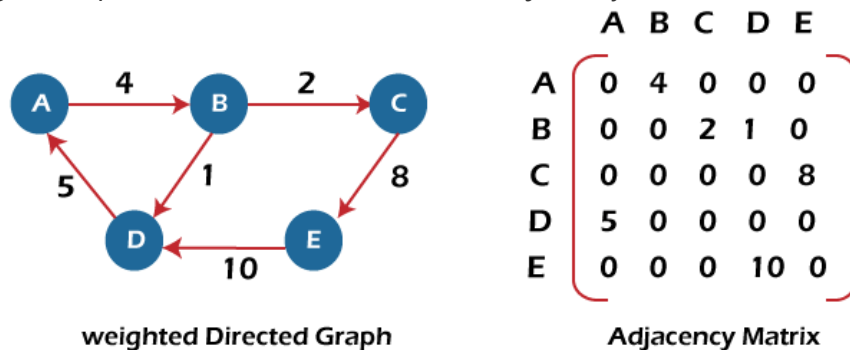


In the above graph, we can see there is no self-loop, so the diagonal entries of the adjacent matrix are 0.

## Adjacency matrix for a weighted directed graph

It is similar to an adjacency matrix representation of a directed graph except that instead of using the '1' for the existence of a path, here we have to use the weight associated with the

edge. The weights on the graph edges will be represented as the entries of the adjacency matrix. We can understand it with the help of an example. Consider the below graph and its adjacency matrix representation. In the representation, we can see that the weight associated with the edges is represented as the entries in the adjacency matrix.



In the above image, we can see that the adjacency matrix representation of the weighted directed graph is different from other representations. It is because, in this representation, the non-zero values are replaced by the actual weight assigned to the edges.

Adjacency matrix is easier to implement and follow. An adjacency matrix can be used when the graph is dense and a number of edges are large.

Though, it is advantageous to use an adjacency matrix, but it consumes more space. Even if the graph is sparse, the matrix still consumes the same space.

## Linked list representation

An adjacency list is used in the linked representation to store the Graph in the computer's memory. It is efficient in terms of storage as we only have to store the values for edges.

Let's see the adjacency list representation of an undirected graph.



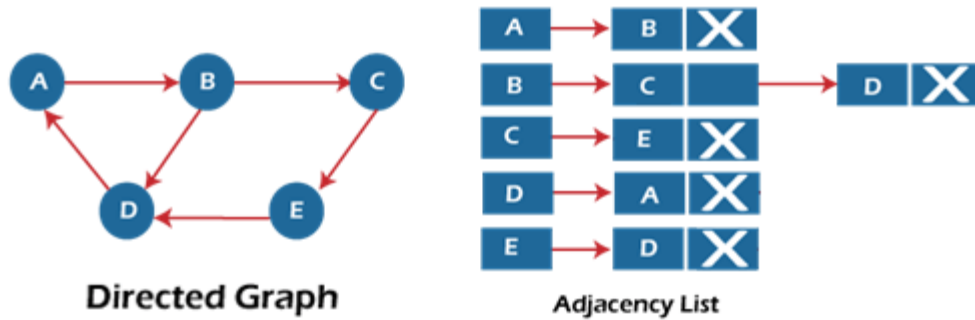
In the above figure, we can see that there is a linked list or adjacency list for every node of the graph. From vertex A, there are paths to vertex B and vertex D. These nodes are linked to nodes A in the given adjacency list.

An adjacency list is maintained for each node present in the graph, which stores the node value and a pointer to the next adjacent node to the respective node. If all the adjacent nodes are traversed, then store the NULL in the pointer field of the last node of the list.

The sum of the lengths of adjacency lists is equal to twice the number of edges present in an undirected graph.

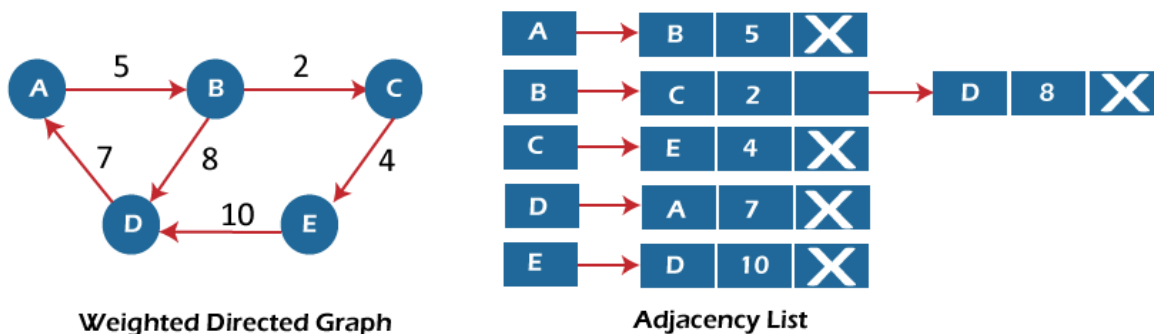
Now, consider the directed graph, and let's see the adjacency list representation of that graph.





For a directed graph, the sum of the lengths of adjacency lists is equal to the number of edges present in the graph.

Now, consider the weighted directed graph, and let's see the adjacency list representation of that graph.



In the case of a weighted directed graph, each node contains an extra field that is called the weight of the node.

In an adjacency list, it is easy to add a vertex. Because of using the linked list, it also saves space.

## Implementation of adjacency matrix representation of Graph

Now, let's see the implementation of adjacency matrix representation of graph in C.

In this program, there is an adjacency matrix representation of an undirected graph. It means that if there is an edge exists from vertex A to vertex B, there will also an edge exists from vertex B to vertex A.

Here, there are four vertices and five edges in the graph that are non-directed.

*/\* Adjacency Matrix representation of an undirected graph in C \*/*

```
#include <stdio.h>
#define V 4 /* number of vertices in the graph */

/* function to initialize the matrix to zero */
void init(int arr[][V]) {
    int i, j;
    for (i = 0; i < V; i++)
        for (j = 0; j < V; j++)
            arr[i][j] = 0;
}
```

```

/* function to add edges to the graph */
void insertEdge(int arr[][V], int i, int j) {
    arr[i][j] = 1;
    arr[j][i] = 1;
}

/* function to print the matrix elements */
void printAdjMatrix(int arr[][V]) {
    int i, j;
    for (i = 0; i < V; i++) {
        printf("%d: ", i);
        for (j = 0; j < V; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int adjMatrix[V][V];

    init(adjMatrix);
    insertEdge(adjMatrix, 0, 1);
    insertEdge(adjMatrix, 0, 2);
    insertEdge(adjMatrix, 1, 2);
    insertEdge(adjMatrix, 2, 0);
    insertEdge(adjMatrix, 2, 3);

    printAdjMatrix(adjMatrix);

    return 0;
}

```

### Output:

After the execution of the above code, the output will be -

```

0: 0 1 1 0
1: 1 0 1 0
2: 1 1 0 1
3: 0 0 1 0

```

## Implementation of adjacency list representation of Graph

Now, let's see the implementation of adjacency list representation of graph in C.

In this program, there is an adjacency list representation of an undirected graph. It means that if there is an edge exists from vertex A to vertex B, there will also an edge exists from vertex B to vertex A.

```

/* Adjacency list representation of a graph in C */
#include <stdio.h>
#include <stdlib.h>

/* structure to represent a node of adjacency list */
struct AdjNode {
    int dest;
    struct AdjNode* next;
};

/* structure to represent an adjacency list */
struct AdjList {
    struct AdjNode* head;
};

/* structure to represent the graph */
struct Graph {
    int V; /*number of vertices in the graph*/
    struct AdjList* array;
};

struct AdjNode* newAdjNode(int dest)
{
    struct AdjNode* newNode = (struct AdjNode*)malloc(sizeof(struct AdjNode));
    newNode->dest = dest;
    newNode->next = NULL;
    return newNode;
}

struct Graph* createGraph(int V)
{
    struct Graph* graph = (struct Graph*)malloc(sizeof(struct Graph));
    graph->V = V;
    graph->array = (struct AdjList*)malloc(V * sizeof(struct AdjList));

    /* Initialize each adjacency list as empty by making head as NULL */
    int i;
    for (i = 0; i < V; ++i)
        graph->array[i].head = NULL;
    return graph;
}

/* function to add an edge to an undirected graph */

```

```

void addEdge(struct Graph* graph, int src, int dest)
{
    /* Add an edge from src to dest. The node is added at the beginning */
    struct AdjNode* check = NULL;
    struct AdjNode* newNode = newAdjNode(dest);

    if (graph->array[src].head == NULL) {
        newNode->next = graph->array[src].head;
        graph->array[src].head = newNode;
    }
    else {

        check = graph->array[src].head;
        while (check->next != NULL) {
            check = check->next;
        }
        // graph->array[src].head = newNode;
        check->next = newNode;
    }

    /* Since graph is undirected, add an edge from dest to src also */
    newNode = newAdjNode(src);
    if (graph->array[dest].head == NULL) {
        newNode->next = graph->array[dest].head;
        graph->array[dest].head = newNode;
    }
    else {
        check = graph->array[dest].head;
        while (check->next != NULL) {
            check = check->next;
        }
        check->next = newNode;
    }
}

/* function to print the adjacency list representation of graph*/
void print(struct Graph* graph)
{
    int v;
    for (v = 0; v < graph->V; ++v) {
        struct AdjNode* pCrawl = graph->array[v].head;
        printf("\n The Adjacency list of vertex %d is: \n head ", v);
        while (pCrawl) {
            printf("-> %d", pCrawl->dest);
            pCrawl = pCrawl->next;
        }
    }
}

```

```

    }
    printf("\n");
}

int main()
{

    int V = 4;
    struct Graph* g = createGraph(V);
    addEdge(g, 0, 1);
    addEdge(g, 0, 3);
    addEdge(g, 1, 2);
    addEdge(g, 1, 3);
    addEdge(g, 2, 4);
    addEdge(g, 2, 3);
    addEdge(g, 3, 4);
    print(g);
    return 0;
}

```

### Output:

In the output, we will see the adjacency list representation of all the vertices of the graph. After the execution of the above code, the output will be -

```

The Adjacency list of vertex 0 is:
head -> 1-> 3

The Adjacency list of vertex 1 is:
head -> 0-> 2-> 3

The Adjacency list of vertex 2 is:
head -> 1-> 4-> 3

The Adjacency list of vertex 3 is:
head -> 0-> 1-> 2-> 4

```

## Conclusion

Here, we have seen the description of graph representation using the adjacency matrix and adjacency list. We have also seen their implementations in C programming language. So, that's all about the article. Hope, it will be helpful and informative to you.

## BFS algorithm

In this article, we will discuss the BFS algorithm in the data structure. Breadth-first search is a graph traversal algorithm that starts traversing the graph from the root node and explores all

the neighboring nodes. Then, it selects the nearest node and explores all the unexplored nodes. While using BFS for traversal, any node in the graph can be considered as the root node. There are many ways to traverse the graph, but among them, BFS is the most commonly used approach. It is a recursive algorithm to search all the vertices of a tree or graph data structure. BFS puts every vertex of the graph into two categories - visited and non-visited. It selects a single node in a graph and, after that, visits all the nodes adjacent to the selected node.

## Applications of BFS algorithm

The applications of breadth-first-algorithm are given as follows -

BFS can be used to find the neighboring locations from a given source location.

In a peer-to-peer network, BFS algorithm can be used as a traversal method to find all the neighboring nodes. Most torrent clients, such as BitTorrent, uTorrent, etc. employ this process to find "seeds" and "peers" in the network.

BFS can be used in web crawlers to create web page indexes. It is one of the main algorithms that can be used to index web pages. It starts traversing from the source page and follows the links associated with the page. Here, every web page is considered as a node in the graph.

BFS is used to determine the shortest path and minimum spanning tree.

BFS is also used in Cheney's technique to duplicate the garbage collection.

It can be used in ford-Fulkerson method to compute the maximum flow in a flow network.

## Algorithm

The steps involved in the BFS algorithm to explore a graph are given as follows -

**Step 1:** SET STATUS = 1 (ready state) for each node in G

**Step 2:** Enqueue the starting node A and set its STATUS = 2 (waiting state)

**Step 3:** Repeat Steps 4 and 5 until QUEUE is empty

**Step 4:** Dequeue a node N. Process it and set its STATUS = 3 (processed state).

**Step 5:** Enqueue all the neighbours of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2

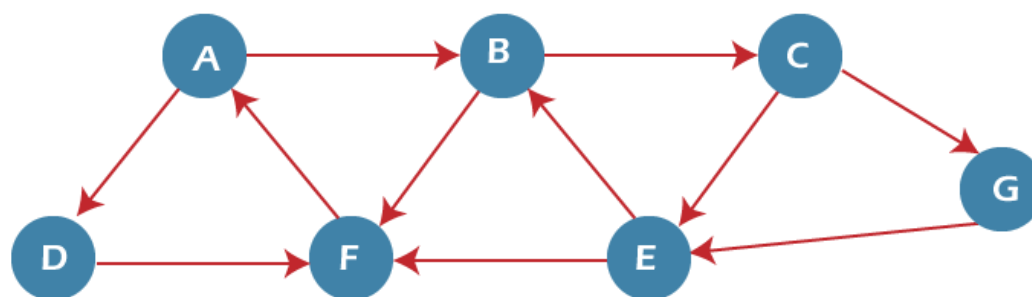
(waiting state)

[END OF LOOP]

**Step 6:** EXIT

## Example of BFS algorithm

Now, let's understand the working of BFS algorithm by using an example. In the example given below, there is a directed graph having 7 vertices.



### Adjacency Lists

A : B, D  
B : C, F  
C : E, G  
G : E  
E : B, F  
F : A  
D : F

In the above graph, minimum path 'P' can be found by using the BFS that will start from Node A and end at Node E. The algorithm uses two queues, namely QUEUE1 and QUEUE2. QUEUE1

holds all the nodes that are to be processed, while QUEUE2 holds all the nodes that are processed and deleted from QUEUE1.

Now, let's start examining the graph starting from Node A.

**Step 1** - First, add A to queue1 and NULL to queue2.

QUEUE1 = {A}

QUEUE2 = {NULL}

**Step 2** - Now, delete node A from queue1 and add it into queue2. Insert all neighbors of node A to queue1.

QUEUE1 = {B, D}

QUEUE2 = {A}

**Step 3** - Now, delete node B from queue1 and add it into queue2. Insert all neighbors of node B to queue1.

QUEUE1 = {D, C, F}

QUEUE2 = {A, B}

**Step 4** - Now, delete node D from queue1 and add it into queue2. Insert all neighbors of node D to queue1. The only neighbor of Node D is F since it is already inserted, so it will not be inserted again.

QUEUE1 = {C, F}

QUEUE2 = {A, B, D}

**Step 5** - Delete node C from queue1 and add it into queue2. Insert all neighbors of node C to queue1.

QUEUE1 = {F, E, G}

QUEUE2 = {A, B, D, C}

**Step 5** - Delete node F from queue1 and add it into queue2. Insert all neighbors of node F to queue1. Since all the neighbors of node F are already present, we will not insert them again.

QUEUE1 = {E, G}

QUEUE2 = {A, B, D, C, F}

**Step 6** - Delete node E from queue1. Since all of its neighbors have already been added, so we will not insert them again. Now, all the nodes are visited, and the target node E is encountered into queue2.

QUEUE1 = {G}

QUEUE2 = {A, B, D, C, F, E}

## Complexity of BFS algorithm

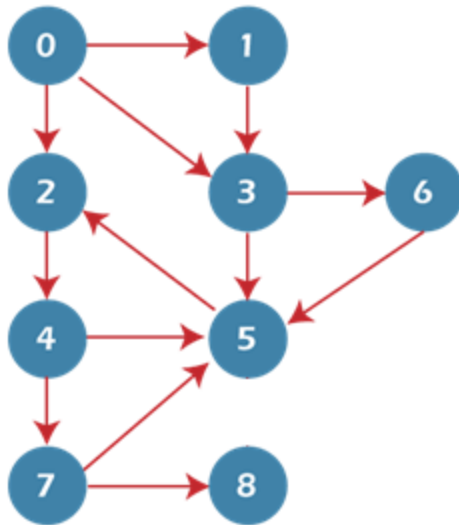
Time complexity of BFS depends upon the data structure used to represent the graph. The time complexity of BFS algorithm is  $O(V+E)$ , since in the worst case, BFS algorithm explores every node and edge. In a graph, the number of vertices is  $O(V)$ , whereas the number of edges is  $O(E)$ . The space complexity of BFS can be expressed as  $O(V)$ , where  $V$  is the number of vertices.

## Implementation of BFS algorithm

Now, let's see the implementation of BFS algorithm in java.

In this code, we are using the adjacency list to represent our graph. Implementing the Breadth-First Search algorithm in Java makes it much easier to deal with the adjacency list since we only have to travel through the list of nodes attached to each node once the node is dequeued from the head (or start) of the queue.

In this example, the graph that we are using to demonstrate the code is given as follows -



```

import java.io.*;
import java.util.*;
public class BFSTraversal
{
    private int vertex;    /* total number number of vertices in the graph */
    private LinkedList<Integer> adj[];    /* adjacency list */
    private Queue<Integer> que;    /* maintaining a queue */
    BFSTraversal(int v)
    {
        vertex = v;
        adj = new LinkedList[vertex];
        for (int i=0; i<v; i++)
        {
            adj[i] = new LinkedList<>();
        }
        que = new LinkedList<Integer>();
    }
    void insertEdge(int v,int w)
    {
        adj[v].add(w);    /* adding an edge to the adjacency list (edges are bidirectional in this example) */
    }
    void BFS(int n)
    {
        boolean nodes[] = new boolean[vertex];    /* initialize boolean array for holding the data */
        int a = 0;
        nodes[n]=true;
        que.add(n);    /* root node is added to the top of the queue */
    }
}

```



```

while (que.size() != 0)
{
    n = que.poll();    /* remove the top element of the queue */
    System.out.print(n+ " "); /* print the top element of the queue */
    for (int i = 0; i < adj[n].size(); i++) /* iterate through the linked list and push all neighbor
s into queue */
    {
        a = adj[n].get(i);
        if (!nodes[a]) /* only insert nodes into queue if they have not been explored already */
        {
            nodes[a] = true;
            que.add(a);
        }
    }
}
}

public static void main(String args[])
{
    BFSTraversal graph = new BFSTraversal(10);
    graph.insertEdge(0, 1);
    graph.insertEdge(0, 2);
    graph.insertEdge(0, 3);
    graph.insertEdge(1, 3);
    graph.insertEdge(2, 4);
    graph.insertEdge(3, 5);
    graph.insertEdge(3, 6);
    graph.insertEdge(4, 7);
    graph.insertEdge(4, 5);
    graph.insertEdge(5, 2);
    graph.insertEdge(6, 5);
    graph.insertEdge(7, 5);
    graph.insertEdge(7, 8);
    System.out.println("Breadth First Traversal for the graph is:");
    graph.BFS(2);
}
}

```

### Output

```

Breadth First Traversal for the graph is:
2 4 7 5 8

```

### Conclusion

In this article, we have discussed the Breadth-first search technique along with its example, complexity, and implementation in java programming language. Here, we have also seen the real-life applications of BFS that show it the important data structure algorithm.

So, that's all about the article. Hope, it will be helpful and informative to you.

## DFS (Depth First Search) algorithm

In this article, we will discuss the DFS algorithm in the data structure. It is a recursive algorithm to search all the vertices of a tree data structure or a graph. The depth-first search (DFS) algorithm starts with the initial node of graph G and goes deeper until we find the goal node or the node with no children.

Because of the recursive nature, stack data structure can be used to implement the DFS algorithm. The process of implementing the DFS is similar to the BFS algorithm.

The step by step process to implement the DFS traversal is given as follows -

First, create a stack with the total number of vertices in the graph.

Now, choose any vertex as the starting point of traversal, and push that vertex into the stack.

After that, push a non-visited vertex (adjacent to the vertex on the top of the stack) to the top of the stack.

Now, repeat steps 3 and 4 until no vertices are left to visit from the vertex on the stack's top.

If no vertex is left, go back and pop a vertex from the stack.

Repeat steps 2, 3, and 4 until the stack is empty.

## Applications of DFS algorithm

The applications of using the DFS algorithm are given as follows -

DFS algorithm can be used to implement the topological sorting.

It can be used to find the paths between two vertices.

It can also be used to detect cycles in the graph.

DFS algorithm is also used for one solution puzzles.

DFS is used to determine if a graph is bipartite or not.

## Algorithm

**Step 1:** SET STATUS = 1 (ready state) for each node in G

**Step 2:** Push the starting node A on the stack and set its STATUS = 2 (waiting state)

**Step 3:** Repeat Steps 4 and 5 until STACK is empty

**Step 4:** Pop the top node N. Process it and set its STATUS = 3 (processed state)

**Step 5:** Push on the stack all the neighbors of N that are in the ready state (whose STATUS = 1) and set their STATUS = 2 (waiting state)

[END OF LOOP]

**Step 6:** EXIT

## Pseudocode

DFS(G,v) ( v is the vertex where the search starts )

Stack S := {}; ( start with an empty stack )

**for** each vertex u, set visited[u] := **false**;

push S, v;

**while** (S is not empty) **do**

u := pop S;

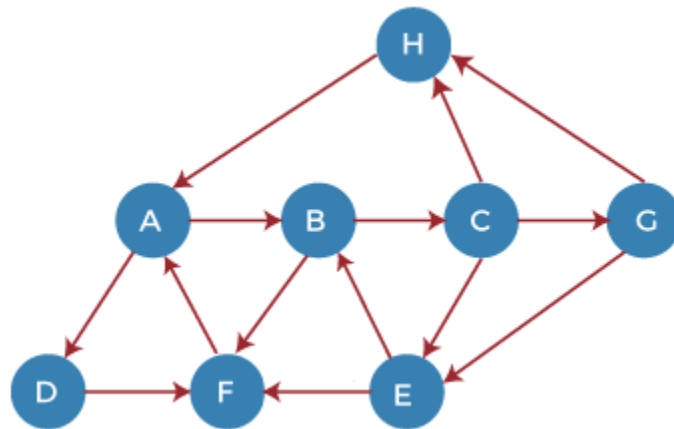
```

    if (not visited[u]) then
        visited[u] := true;
        for each unvisited neighbour w of uu
            push S, w;
        end if
    end while
END DFS()

```

## Example of DFS algorithm

Now, let's understand the working of the DFS algorithm by using an example. In the example given below, there is a directed graph having 7 vertices.



### Adjacency Lists

```

A : B, D
B : C, F
C : E, G, H
G : E, H
E : B, F
F : A
D : F
H : A

```

Now, let's start examining the graph starting from Node H.

**Step 1** - First, push H onto the stack.

STACK: H

**Step 2** - POP the top element from the stack, i.e., H, and print it. Now, PUSH all the neighbors of H onto the stack that are in ready state.

Print: H]STACK: A

**Step 3** - POP the top element from the stack, i.e., A, and print it. Now, PUSH all the neighbors of A onto the stack that are in ready state.

Print: A

STACK: B, D

**Step 4** - POP the top element from the stack, i.e., D, and print it. Now, PUSH all the neighbors of D onto the stack that are in ready state.

Print: D

STACK: B, F

**Step 5** - POP the top element from the stack, i.e., F, and print it. Now, PUSH all the neighbors of F onto the stack that are in ready state.

Print: F

STACK: B

**Step 6** - POP the top element from the stack, i.e., B, and print it. Now, PUSH all the neighbors of B onto the stack that are in ready state.

Print: B

STACK: C

**Step 7** - POP the top element from the stack, i.e., C, and print it. Now, PUSH all the neighbors of C onto the stack that are in ready state.

Print: C

STACK: E, G

**Step 8** - POP the top element from the stack, i.e., G and PUSH all the neighbors of G onto the stack that are in ready state.

Print: G

STACK: E

**Step 9** - POP the top element from the stack, i.e., E and PUSH all the neighbors of E onto the stack that are in ready state.

Print: E

STACK:

Now, all the graph nodes have been traversed, and the stack is empty.

## Complexity of Depth-first search algorithm

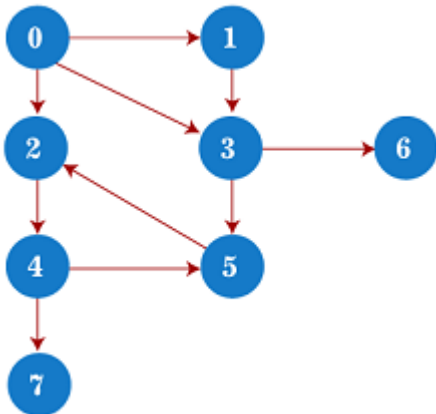
The time complexity of the DFS algorithm is  $O(V+E)$ , where V is the number of vertices and E is the number of edges in the graph.

The space complexity of the DFS algorithm is  $O(V)$ .

## Implementation of DFS algorithm

Now, let's see the implementation of DFS algorithm in Java.

In this example, the graph that we are using to demonstrate the code is given as follows -



/\*A sample java program to implement the DFS algorithm\*/

```
import java.util.*;
```

```
class DFSTraversal {  
    private LinkedList<Integer> adj[]; /*adjacency list representation*/  
    private boolean visited[];  
  
    /* Creation of the graph */  
    DFSTraversal(int V) /*'V' is the number of vertices in the graph*/  
    {  
        adj = new LinkedList[V];
```

```

visited = new boolean[V];

for (int i = 0; i < V; i++)
    adj[i] = new LinkedList<Integer>();
}

/* Adding an edge to the graph */
void insertEdge(int src, int dest) {
    adj[src].add(dest);
}

void DFS(int vertex) {
    visited[vertex] = true; /*Mark the current node as visited*/
    System.out.print(vertex + " ");

    Iterator<Integer> it = adj[vertex].listIterator();
    while (it.hasNext()) {
        int n = it.next();
        if (!visited[n])
            DFS(n);
    }
}

public static void main(String args[]) {
    DFSTraversal graph = new DFSTraversal(8);

    graph.insertEdge(0, 1);
    graph.insertEdge(0, 2);
    graph.insertEdge(0, 3);
    graph.insertEdge(1, 3);
    graph.insertEdge(2, 4);
    graph.insertEdge(3, 5);
    graph.insertEdge(3, 6);
    graph.insertEdge(4, 7);
    graph.insertEdge(4, 5);
    graph.insertEdge(5, 2);

    System.out.println("Depth First Traversal for the graph is:");
    graph.DFS(0);
}
}

```

## Output

```
Depth First Traversal for the graph is:  
0 1 3 5 2 4 7 6
```

## Conclusion

In this article, we have discussed the depth-first search technique, its example, complexity, and implementation in the java programming language. Along with that, we have also seen the applications of the depth-first search algorithm.

So, that's all about the article. Hope it will be helpful and informative to you.

## Spanning tree

In this article, we will discuss the spanning tree and the minimum spanning tree. But before moving directly towards the spanning tree, let's first see a brief description of the graph and its types.

### Graph

A graph can be defined as a group of vertices and edges to connect these vertices. The types of graphs are given as follows -

**Undirected graph:** An undirected graph is a graph in which all the edges do not point to any particular direction, i.e., they are not unidirectional; they are bidirectional. It can also be defined as a graph with a set of  $V$  vertices and a set of  $E$  edges, each edge connecting two different vertices.

**Connected graph:** A connected graph is a graph in which a path always exists from a vertex to any other vertex. A graph is connected if we can reach any vertex from any other vertex by following edges in either direction.

**Directed graph:** Directed graphs are also known as digraphs. A graph is a directed graph (or digraph) if all the edges present between any vertices or nodes of the graph are directed or have a defined direction.

Now, let's move towards the topic spanning tree.

### What is a spanning tree?

A spanning tree can be defined as the subgraph of an undirected connected graph. It includes all the vertices along with the least possible number of edges. If any vertex is missed, it is not a spanning tree. A spanning tree is a subset of the graph that does not have cycles, and it also cannot be disconnected.

A spanning tree consists of  $(n-1)$  edges, where ' $n$ ' is the number of vertices (or nodes). Edges of the spanning tree may or may not have weights assigned to them. All the possible spanning trees created from the given graph  $G$  would have the same number of vertices, but the number of edges in the spanning tree would be equal to the number of vertices in the given graph minus 1.

A complete undirected graph can have  $n^{n-2}$  number of spanning trees where  $n$  is the number of vertices in the graph. Suppose, if  $n = 5$ , the number of maximum possible spanning trees would be  $5^{5-2} = 125$ .

### Applications of the spanning tree

Basically, a spanning tree is used to find a minimum path to connect all nodes of the graph. Some of the common applications of the spanning tree are listed as follows -

Cluster Analysis

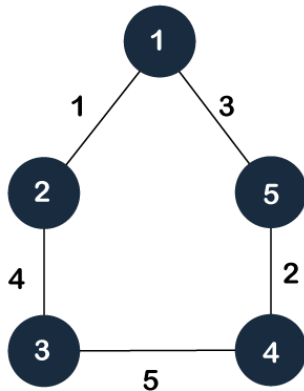
Civil network planning

Computer network routing protocol

Now, let's understand the spanning tree with the help of an example.

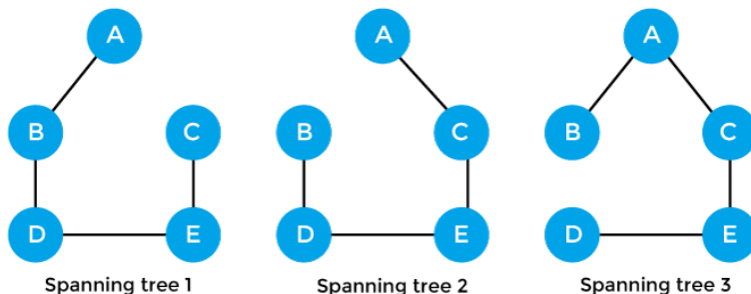
## Example of Spanning tree

Suppose the graph be -



As discussed above, a spanning tree contains the same number of vertices as the graph, the number of vertices in the above graph is 5; therefore, the spanning tree will contain 5 vertices. The edges in the spanning tree will be equal to the number of vertices in the graph minus 1. So, there will be 4 edges in the spanning tree.

Some of the possible spanning trees that will be created from the above graph are given as follows -



## Properties of spanning-tree

Some of the properties of the spanning tree are given as follows -

There can be more than one spanning tree of a connected graph G.

A spanning tree does not have any cycles or loop.

A spanning tree is **minimally connected**, so removing one edge from the tree will make the graph disconnected.

A spanning tree is **maximally acyclic**, so adding one edge to the tree will create a loop.

There can be a maximum  $n^{n-2}$  number of spanning trees that can be created from a complete graph.

A spanning tree has  $n-1$  edges, where 'n' is the number of nodes.

If the graph is a complete graph, then the spanning tree can be constructed by removing maximum  $(e-n+1)$  edges, where 'e' is the number of edges and 'n' is the number of vertices.

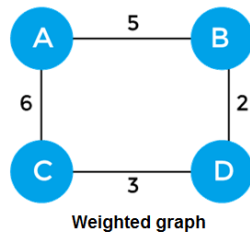
So, a spanning tree is a subset of connected graph G, and there is no spanning tree of a disconnected graph.

## Minimum Spanning tree

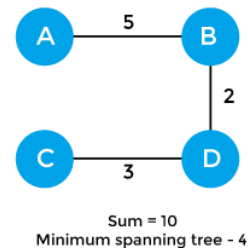
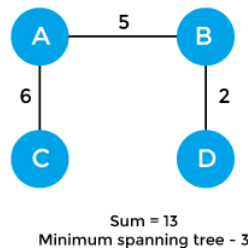
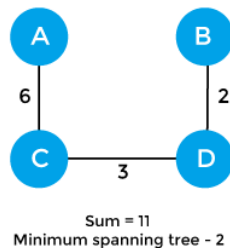
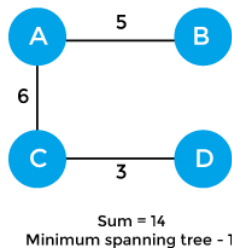
A minimum spanning tree can be defined as the spanning tree in which the sum of the weights of the edge is minimum. The weight of the spanning tree is the sum of the weights given to the edges of the spanning tree. In the real world, this weight can be considered as the distance, traffic load, congestion, or any random value.

### Example of minimum spanning tree

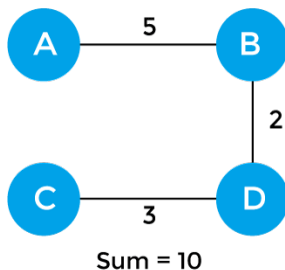
Let's understand the minimum spanning tree with the help of an example.



The sum of the edges of the above graph is 16. Now, some of the possible spanning trees created from the above graph are -



So, the minimum spanning tree that is selected from the above spanning trees for the given weighted graph is -



## Applications of minimum spanning tree

The applications of the minimum spanning tree are given as follows -

Minimum spanning tree can be used to design water-supply networks, telecommunication networks, and electrical grids.

It can be used to find paths in the map.

## Algorithms for Minimum spanning tree

A minimum spanning tree can be found from a weighted graph by using the algorithms given below -

Prim's Algorithm



## Kruskal's Algorithm

Let's see a brief description of both of the algorithms listed above.

**Prim's algorithm** - It is a greedy algorithm that starts with an empty spanning tree. It is used to find the minimum spanning tree from the graph. This algorithm finds the subset of edges that includes every vertex of the graph such that the sum of the weights of the edges can be minimized.

To learn more about the prim's algorithm, you can click the below link

- <https://www.javatpoint.com/prim-algorithm>

**Kruskal's algorithm** - This algorithm is also used to find the minimum spanning tree for a connected weighted graph. Kruskal's algorithm also follows greedy approach, which finds an optimum solution at every stage instead of focusing on a global optimum.

To learn more about the prim's algorithm, you can click the below link

- <https://www.javatpoint.com/kruskal-algorithm>

So, that's all about the article. Hope the article will be helpful and informative to you. Here, we have discussed spanning tree and minimum spanning tree along with their properties, examples, and applications.

## Linear Search Algorithm

In this article, we will discuss the Linear Search Algorithm. Searching is the process of finding some particular element in the list. If the element is present in the list, then the process is called successful, and the process returns the location of that element; otherwise, the search is called unsuccessful.

Two popular search methods are Linear Search and Binary Search. So, here we will discuss the popular searching technique, i.e., Linear Search Algorithm.

Linear search is also called as **sequential search algorithm**. It is the simplest searching algorithm. In Linear search, we simply traverse the list completely and match each element of the list with the item whose location is to be found. If the match is found, then the location of the item is returned; otherwise, the algorithm returns NULL.

It is widely used to search an element from the unordered list, i.e., the list in which items are not sorted. The worst-case time complexity of linear search is **O(n)**.

The steps used in the implementation of Linear Search are listed as follows -

First, we have to traverse the array elements using a **for** loop.

In each iteration of **for loop**, compare the search element with the current array element, and -  
If the element matches, then return the index of the corresponding array element.

If the element does not match, then move to the next element.

If there is no match or the search element is not present in the given array, return **-1**.

Now, let's see the algorithm of linear search.

### Algorithm

Linear\_Search(a, n, val) // 'a' is the given array, 'n' is the size of given array, 'val' is the value to search

Step 1: set **pos** = -1

Step 2: set **i** = 1

Step 3: repeat step 4 while **i** <= n

Step 4: if **a[i] == val**

```
set pos = i
print pos
go to step 6
[end of if]
set ii = i + 1
[end of loop]
Step 5: if pos = -1
print "value is not present in the array "
[end of if]
Step 6: exit
```

## Working of Linear search

Now, let's see the working of the linear search Algorithm.

To understand the working of linear search algorithm, let's take an unsorted array. It will be easy to understand the working of linear search with an example.

Let the elements of array are -

Let the element to be searched is **K = 41**

Now, start from the first element and compare **K** with each element of the array.

The value of **K**, i.e., **41**, is not matched with the first element of the array. So, move to the next element. And follow the same process until the respective element is found.

Now, the element to be searched is found. So algorithm will return the index of the element matched.

## Linear Search complexity

Now, let's see the time complexity of linear search in the best case, average case, and worst case. We will also see the space complexity of linear search.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(1)$          |
| Average Case | $O(n)$          |
| Worst Case   | $O(n)$          |

**Best Case Complexity** - In Linear search, best case occurs when the element we are finding is at the first position of the array. The best-case time complexity of linear search is  **$O(1)$** .

**Average Case Complexity** - The average case time complexity of linear search is  **$O(n)$** .

**Worst Case Complexity** - In Linear search, the worst case occurs when the element we are looking is present at the end of the array. The worst-case in linear search could be when the

target element is not present in the given array, and we have to traverse the entire array. The worst-case time complexity of linear search is  **$O(n)$** .

The time complexity of linear search is  **$O(n)$**  because every element in the array is compared only once.

## 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(1)$ |
|------------------|--------|

The space complexity of linear search is  $O(1)$ .

## Implementation of Linear Search

Now, let's see the programs of linear search in different programming languages.

**Program:** Write a program to implement linear search in C language.

```
#include <stdio.h>
int linearSearch(int a[], int n, int val) {
    // Going through array sequentially
    for (int i = 0; i < n; i++)
    {
        if (a[i] == val)
            return i+1;
    }
    return -1;
}
int main() {
    int a[] = {70, 40, 30, 11, 57, 41, 25, 14, 52}; // given array
    int val = 41; // value to be searched
    int n = sizeof(a) / sizeof(a[0]); // size of array
    int res = linearSearch(a, n, val); // Store result
    printf("The elements of the array are - ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\nElement to be searched is - %d", val);
    if (res == -1)
        printf("\nElement is not present in the array");
    else
        printf("\nElement is present at %d position of array", res);
    return 0;
}
```

### Output

**Program:** Write a program to implement linear search in C++.

```
#include <iostream>
using namespace std;
int linearSearch(int a[], int n, int val) {
```

```

// Going through array linearly
for (int i = 0; i < n; i++)
{
    if (a[i] == val)
        return i+1;
}
return -1;
}

int main() {
    int a[] = {69, 39, 29, 10, 56, 40, 24, 13, 51}; // given array
    int val = 56; // value to be searched
    int n = sizeof(a) / sizeof(a[0]); // size of array
    int res = linearSearch(a, n, val); // Store result
    cout<<"The elements of the array are - ";
    for (int i = 0; i < n; i++)
        cout<<a[i]<<" ";
    cout<<"\nElement to be searched is - "<<val;
    if (res == -1)
        cout<<"\nElement is not present in the array";
    else
        cout<<"\nElement is present at "<<res<<" position of array";
    return 0;
}

```

## Output

**Program:** Write a program to implement linear search in C#.

```

using System;
class LinearSearch {
static int linearSearch(int[] a, int n, int val) {
    // Going through array sequentially
    for (int i = 0; i < n; i++)
    {
        if (a[i] == val)
            return i+1;
    }
    return -1;
}

static void Main() {
    int[] a = {56, 30, 20, 41, 67, 31, 22, 14, 52}; // given array
    int val = 14; // value to be searched
    int n = a.Length; // size of array
    int res = linearSearch(a, n, val); // Store result
    Console.WriteLine("The elements of the array are - ");
}
}

```

```

for (int i = 0; i < n; i++)
    Console.Write(" " + a[i]);
Console.WriteLine();
Console.WriteLine("Element to be searched is - " + val);
if (res == -1)
    Console.WriteLine("Element is not present in the array");
else
    Console.WriteLine("Element is present at " + res + " position of array");
}
}

```

## Output

**Program:** Write a program to implement linear search in Java.

```

class LinearSearch {
static int linearSearch(int a[], int n, int val) {
    // Going through array sequentially
    for (int i = 0; i < n; i++)
    {
        if (a[i] == val)
            return i + 1;
    }
    return -1;
}

public static void main(String args[]) {
    int a[] = {55, 29, 10, 40, 57, 41, 20, 24, 45}; // given array
    int val = 10; // value to be searched
    int n = a.length; // size of array
    int res = linearSearch(a, n, val); // Store result
    System.out.println();
    System.out.print("The elements of the array are - ");
    for (int i = 0; i < n; i++)
        System.out.print(" " + a[i]);
    System.out.println();
    System.out.println("Element to be searched is - " + val);
    if (res == -1)
        System.out.println("Element is not present in the array");
    else
        System.out.println("Element is present at " + res + " position of array");
}
}

```

## Output

**Program:** Write a program to implement linear search in JavaScript.

```

<html>
<head>
</head>
<body>
<script>
    var a = [54, 26, 9, 80, 47, 71, 10, 24, 45]; // given array
    var val = 71; // value to be searched
    var n = a.length; // size of array
    function linearSearch(a, n, val) {
        // Going through array sequentially
        for (var i = 0; i < n; i++)
        {
            if (a[i] == val)
                return i+1;
        }
        return -1
    }
    var res = linearSearch(a, n, val); // Store result
    document.write("The elements of the array are: ");
    for (i = 0; i < n; i++)
        document.write(" " + a[i]);
    document.write("<br>" + "Element to be searched is: " + val);
    if (res == -1)
        document.write("<br>" + "Element is not present in the array");
    else
        document.write("<br>" + "Element is present at " + res + " position of array");
</script>
</body>
</html>

```

## Output

**Program:** Write a program to implement linear search in PHP.

```

<?php
    $a = array(45, 24, 8, 80, 62, 71, 10, 23, 43); // given array
    $val = 62; // value to be searched
    $n = sizeof($a); //size of array
    function linearSearch($a, $n, $val) {
        // Going through array sequentially
        for ($i = 0; $i < $n; $i++)
        {
            if ($a[$i] == $val)
                return $i+1;
        }
    }

```

```

    return -1;
}
$res = linearSearch($a, $n, $val); // Store result
echo "The elements of the array are: ";
for ($i = 0; $i < $n; $i++)
    echo " ", $a[$i];
echo "<br>", "Element to be searched is: ", $val;
if ($res == -1)
    echo "<br>", "Element is not present in the array";
else
    echo "<br>", "Element is present at ", $res, " position of array";
?>

```

### Output

So, that's all about the article. Hope the article will be helpful and informative to you.

## Binary Search Algorithm

In this article, we will discuss the Binary Search Algorithm. Searching is the process of finding some particular element in the list. If the element is present in the list, then the process is called successful, and the process returns the location of that element. Otherwise, the search is called unsuccessful.

Linear Search and Binary Search are the two popular searching techniques. Here we will discuss the Binary Search Algorithm.

Binary search is the search technique that works efficiently on sorted lists. Hence, to search an element into some list using the binary search technique, we must ensure that the list is sorted. Binary search follows the divide and conquer approach in which the list is divided into two halves, and the item is compared with the middle element of the list. If the match is found then, the location of the middle element is returned. Otherwise, we search into either of the halves depending upon the result produced through the match.

**NOTE: Binary search can be implemented on sorted array elements. If the list elements are not arranged in a sorted manner, we have first to sort them.**

Now, let's see the algorithm of Binary Search.

### Algorithm

Binary\_Search(a, lower\_bound, upper\_bound, val) // 'a' is the given array, 'lower\_bound' is the index of the first array element, 'upper\_bound' is the index of the last array element, 'val' is the value to search

Step 1: set **beg** = **lower\_bound**, **end** = **upper\_bound**, **pos** = - 1

Step 2: repeat steps 3 and 4 while **beg** <= **end**

Step 3: set **mid** = (**beg** + **end**)/2

Step 4: if **a[mid]** = **val**

set **pos** = **mid**

print **pos**

go to step 6

```
else if a[mid] > val
set end = mid - 1
else
set beg = mid + 1
[end of if]
[end of loop]
Step 5: if pos = -1
print "value is not present in the array"
[end of if]
Step 6: exit
```

## Working of Binary search

Now, let's see the working of the Binary Search Algorithm.

To understand the working of the Binary search algorithm, let's take a sorted array. It will be easy to understand the working of Binary search with an example.

There are two methods to implement the binary search algorithm -

Iterative method

Recursive method

The recursive method of binary search follows the divide and conquer approach.

Let the elements of array are -

| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  |
|----|----|----|----|----|----|----|----|----|
| 10 | 12 | 24 | 29 | 39 | 40 | 51 | 56 | 69 |

Let the element to search is, **K = 56**

We have to use the below formula to calculate the **mid** of the array -

**mid** = (beg + end)/2

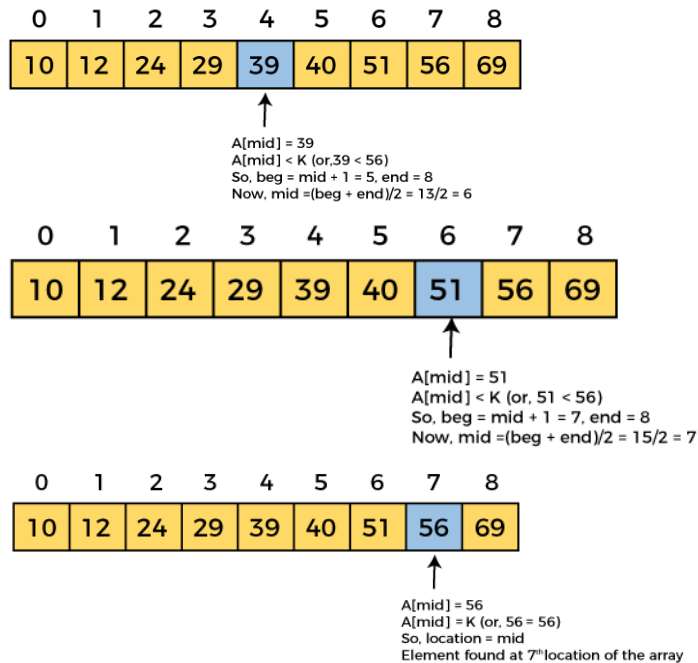
So, in the given array -

**beg** = 0

**end** = 8

**mid** = (0 + 8)/2 = 4. So, 4 is the mid of the array.





Now, the element to search is found. So algorithm will return the index of the element matched.

## Binary Search complexity

Now, let's see the time complexity of Binary search in the best case, average case, and worst case. We will also see the space complexity of Binary search.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(1)$          |
| Average Case | $O(\log n)$     |
| Worst Case   | $O(\log n)$     |

**Best Case Complexity** - In Binary search, best case occurs when the element to search is found in first comparison, i.e., when the first middle element itself is the element to be searched. The best-case time complexity of Binary search is  **$O(1)$** .

**Average Case Complexity** - The average case time complexity of Binary search is  **$O(\log n)$** .

**Worst Case Complexity** - In Binary search, the worst case occurs, when we have to keep reducing the search space till it has only one element. The worst-case time complexity of Binary search is  **$O(\log n)$** .

### 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(1)$ |
|------------------|--------|

The space complexity of binary search is  $O(1)$ .

## Implementation of Binary Search

Now, let's see the programs of Binary search in different programming languages.

**Program:** Write a program to implement Binary search in C language.

```
#include <stdio.h>
```

```
int binarySearch(int a[], int beg, int end, int val)
```

```
{
    int mid;
    if(end >= beg)
    {
        mid = (beg + end)/2;
        /* if the item to be searched is present at middle */
        if(a[mid] == val)
        {
            return mid+1;
        }
        /* if the item to be searched is smaller than middle, then it can only be in left subarray */

        else if(a[mid] < val)
        {
            return binarySearch(a, mid+1, end, val);
        }
        /* if the item to be searched is greater than middle, then it can only be in right subarray */
    }
    else
    {
        return binarySearch(a, beg, mid-1, val);
    }
}
return -1;
```

```
int main() {
    int a[] = {11, 14, 25, 30, 40, 41, 52, 57, 70}; // given array
    int val = 40; // value to be searched
    int n = sizeof(a) / sizeof(a[0]); // size of array
    int res = binarySearch(a, 0, n-1, val); // Store result
    printf("The elements of the array are - ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\nElement to be searched is - %d", val);
    if (res == -1)
        printf("\nElement is not present in the array");
    else
        printf("\nElement is present at %d position of array", res);
    return 0;
}
```

**Output**

```
The elements of the array are - 11 14 25 30 40 41 52 57 70
Element to be searched is - 40
Element is present at 5 position of array
```

**Program:** Write a program to implement Binary search in C++.

```
#include <iostream>
using namespace std;
int binarySearch(int a[], int beg, int end, int val)
{
    int mid;
    if(end >= beg)
    {
        mid = (beg + end)/2;
        /* if the item to be searched is present at middle */
        if(a[mid] == val)
        {
            return mid+1;
        }
        /* if the item to be searched is smaller than middle, then it can only be in left subarray */

        else if(a[mid] < val)
        {
            return binarySearch(a, mid+1, end, val);
        }
        /* if the item to be searched is greater than middle, then it can only be in right subarray */

    else
    {
        return binarySearch(a, beg, mid-1, val);
    }
}
return -1;
}

int main() {
    int a[] = {10, 12, 24, 29, 39, 40, 51, 56, 70}; // given array
    int val = 51; // value to be searched
    int n = sizeof(a) / sizeof(a[0]); // size of array
    int res = binarySearch(a, 0, n-1, val); // Store result
    cout<<"The elements of the array are - ";
    for (int i = 0; i < n; i++)
        cout<<a[i]<<" ";
    cout<<"\nElement to be searched is - "<<val;
    if (res == -1)
        cout<<"\nElement is not present in the array";
    else
```

```

        cout<<"\nElement is present at "<<res<<" position of array";
        return 0;
    }

```

### Output

```

The elements of the array are - 10 12 24 29 39 40 51 56 70
Element to be searched is - 51
Element is present at 7 position of array

```

**Program:** Write a program to implement Binary search in C#.

**using** System;

**class** BinarySearch {

**static int** binarySearch(**int**[] a, **int** beg, **int** end, **int** val)

{

**int** mid;

**if**(end >= beg)

{

mid = (beg + end)/2;

**if**(a[mid] == val)

{

**return** mid+1; /\* if the item to be searched is present at middle \*/

}

/\* if the item to be searched is smaller than middle, then it can only be in left subarray \*/

**else if**(a[mid] < val)

{

**return** binarySearch(a, mid+1, end, val);

}

/\* if the item to be searched is greater than middle, then it can only be in right subarray \*/

/

**else**

{

**return** binarySearch(a, beg, mid-1, val);

}

}

**return** -1;

}

**static void** Main() {

**int**[] a = {9, 11, 23, 28, 38, 45, 50, 56, 70}; // given array

**int** val = 70; // value to be searched

**int** n = a.Length; // size of array

**int** res = binarySearch(a, 0, n-1, val); // Store result

Console.Write("The elements of the array are - ");

**for** (**int** i = 0; i < n; i++)

{

Console.Write(a[i] + " ");

```

    }
    Console.WriteLine();
    Console.WriteLine("Element to be searched is - " + val);
    if (res == -1)
        Console.WriteLine("Element is not present in the array");
    else
        Console.WriteLine("Element is present at " + res + " position of array");
}
}

```

### Output

```

The elements of the array are - 9 11 23 28 38 45 50 56 70
Element to be searched is - 70
Element is present at 9 position of array

```

**Program:** Write a program to implement Binary search in Java.

```

class BinarySearch {
    static int binarySearch(int a[], int beg, int end, int val)
    {
        int mid;
        if(end >= beg)
        {
            mid = (beg + end)/2;
            if(a[mid] == val)
            {
                return mid+1; /* if the item to be searched is present at middle
*/
            }
            /* if the item to be searched is smaller than middle, then it can only
be in left subarray */
            else if(a[mid] < val)
            {
                return binarySearch(a, mid+1, end, val);
            }
            /* if the item to be searched is greater than middle, then it can only be
in right subarray */
            else
            {
                return binarySearch(a, beg, mid-1, val);
            }
        }
        return -1;
    }
    public static void main(String args[]) {
        int a[] = {8, 10, 22, 27, 37, 44, 49, 55, 69}; // given array
    }
}

```

```

int val = 37; // value to be searched
int n = a.length; // size of array
int res = binarySearch(a, 0, n-1, val); // Store result
System.out.print("The elements of the array are: ");
for (int i = 0; i < n; i++)
{
    System.out.print(a[i] + " ");
}
System.out.println();
System.out.println("Element to be searched is: " + val);
if (res == -1)
    System.out.println("Element is not present in the array");
else
    System.out.println("Element is present at " + res + " position of array");
}
}

```

### Output

```

D:\JTP>javac BinarySearch.java
D:\JTP>java BinarySearch
The elements of the array are: 8 10 22 27 37 44 49 55 69
Element to be searched is: 37
Element is present at 5 position of array
D:\JTP>

```

**Program:** Write a program to implement Binary search in PHP.

```
<?php
```

```

function binarySearch($a, $beg, $end, $val)
{
    if($end >= $beg)
    {
        $mid = floor(($beg + $end)/2);
        if($a[$mid] == $val)
        {
            return $mid+1; /* if the item to be searched is present at middle */
        }
        /* if the item to be searched is smaller than middle, then it can only be in left subarray */
        else if($a[$mid] < $val)
        {
            return binarySearch($a, $mid+1, $end, $val);
        }
        /* if the item to be searched is greater than middle, then it can only be in right subarray */
        else
        {
            return binarySearch($a, $beg, $mid-1, $val);
        }
    }
}

```

```

}
return -1;
}

$a = array(7, 9, 21, 26, 36, 43, 48, 54, 68); // given array
$val = 68; // value to be searched
$n = sizeof($a); // size of array
$res = binarySearch($a, 0, $n-1, $val); // Store result
echo "The elements of the array are: ";
for ($i = 0; $i < $n; $i++)
    echo " ", $a[$i];
echo "<br>", "Element to be searched is: ", $val;
if ($res == -1)
    echo "<br>", "Element is not present in the array";
else
    echo "<br>", "Element is present at ", $res, " position of array";

```

?>

## Output



So, that's all about the article. Hope the article will be helpful and informative to you.

## Bubble sort Algorithm

In this article, we will discuss the Bubble sort Algorithm. The working procedure of bubble sort is simplest. This article will be very helpful and interesting to students as they might face bubble sort as a question in their examinations. So, it is important to discuss the topic.

Bubble sort works on the repeatedly swapping of adjacent elements until they are not in the intended order. It is called bubble sort because the movement of array elements is just like the movement of air bubbles in the water. Bubbles in water rise up to the surface; similarly, the array elements in bubble sort move to the end in each iteration.

Although it is simple to use, it is primarily used as an educational tool because the performance of bubble sort is poor in the real world. It is not suitable for large data sets. The average and worst-case complexity of Bubble sort is  $O(n^2)$ , where  $n$  is a number of items.

Bubble sort is majorly used where -  
 complexity does not matter  
 simple and short code is preferred

## Algorithm

In the algorithm given below, suppose **arr** is an array of **n** elements. The assumed **swap** function in the algorithm will swap the values of given array elements.

```

begin BubbleSort(arr)
    for all array elements
        if arr[i] > arr[i+1]

```

```

        swap(arr[i], arr[i+1])
    end if
end for
return arr
end BubbleSort

```

## Working of Bubble sort Algorithm

Now, let's see the working of Bubble sort Algorithm.

To understand the working of bubble sort algorithm, let's take an unsorted array. We are taking a short and accurate array, as we know the complexity of bubble sort is  $O(n^2)$ .

Let the elements of array are -

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 32 | 26 | 35 | 10 |
|----|----|----|----|----|

### First Pass

Sorting will start from the initial two elements. Let compare them to check which is greater.

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 32 | 26 | 35 | 10 |
|----|----|----|----|----|

Here, 32 is greater than 13 ( $32 > 13$ ), so it is already sorted. Now, compare 32 with 26.

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 32 | 26 | 35 | 10 |
|----|----|----|----|----|

Here, 26 is smaller than 32. So, swapping is required. After swapping new array will look like -

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 32 | 35 | 10 |
|----|----|----|----|----|

Now, compare 32 and 35.

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 32 | 35 | 10 |
|----|----|----|----|----|

Here, 35 is greater than 32. So, there is no swapping required as they are already sorted.

Now, the comparison will be in between 35 and 10.

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 32 | 35 | 10 |
|----|----|----|----|----|

Here, 10 is smaller than 35 that are not sorted. So, swapping is required. Now, we reach at the end of the array. After first pass, the array will be -

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 32 | 10 | 35 |
|----|----|----|----|----|

Now, move to the second iteration.

### Second Pass

The same process will be followed for second iteration.



|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 32 | 10 | 35 |
|----|----|----|----|----|

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 32 | 10 | 35 |
|----|----|----|----|----|

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 32 | 10 | 35 |
|----|----|----|----|----|

Here, 10 is smaller than 32. So, swapping is required. After swapping, the array will be -

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 10 | 32 | 35 |
|----|----|----|----|----|

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 10 | 32 | 35 |
|----|----|----|----|----|

Now, move to the third iteration.

### Third Pass

The same process will be followed for third iteration.

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 10 | 32 | 35 |
|----|----|----|----|----|

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 26 | 10 | 32 | 35 |
|----|----|----|----|----|

Here, 10 is smaller than 26. So, swapping is required. After swapping, the array will be -

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 10 | 26 | 32 | 35 |
|----|----|----|----|----|

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 10 | 26 | 32 | 35 |
|----|----|----|----|----|

|    |    |    |    |    |
|----|----|----|----|----|
| 13 | 10 | 26 | 32 | 35 |
|----|----|----|----|----|

Now, move to the fourth iteration.

### Fourth pass

Similarly, after the fourth iteration, the array will be -

|    |    |    |    |    |
|----|----|----|----|----|
| 10 | 13 | 26 | 32 | 35 |
|----|----|----|----|----|

Hence, there is no swapping required, so the array is completely sorted.

## Bubble sort complexity

Now, let's see the time complexity of bubble sort in the best case, average case, and worst case.

We will also see the space complexity of bubble sort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(n)$          |
| Average Case | $O(n^2)$        |
| Worst Case   | $O(n^2)$        |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of bubble sort is  **$O(n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of bubble sort is  **$O(n^2)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of bubble sort is  **$O(n^2)$** .

## 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(1)$ |
| Stable           | YES    |

The space complexity of bubble sort is  $O(1)$ . It is because, in bubble sort, an extra variable is required for swapping.

The space complexity of optimized bubble sort is  $O(2)$ . It is because two extra variables are required in optimized bubble sort.

Now, let's discuss the optimized bubble sort algorithm.

## Optimized Bubble sort Algorithm

In the bubble sort algorithm, comparisons are made even when the array is already sorted. Because of that, the execution time increases.

To solve it, we can use an extra variable **swapped**. It is set to **true** if swapping requires; otherwise, it is set to **false**.

It will be helpful, as suppose after an iteration, if there is no swapping required, the value of variable **swapped** will be **false**. It means that the elements are already sorted, and no further iterations are required.

This method will reduce the execution time and also optimizes the bubble sort.

## Algorithm for optimized bubble sort

```

bubbleSort(array)
n = length(array)
repeat
    swapped = false
    for i = 1 to n - 1
        if array[i - 1] > array[i], then
            swap(array[i - 1], array[i])

```

```

        swapped = true
    end if
end for
n = n - 1
until not swapped
end bubbleSort

```

## Implementation of Bubble sort

Now, let's see the programs of Bubble sort in different programming languages.

**Program:** Write a program to implement bubble sort in C language.

```
#include<stdio.h>
```

```
void print(int a[], int n) //function to print array elements
```

```

{
    int i;
    for(i = 0; i < n; i++)
    {
        printf("%d ",a[i]);
    }
}

```

```
void bubble(int a[], int n) // function to implement bubble sort
```

```

{
    int i, j, temp;
    for(i = 0; i < n; i++)
    {
        for(j = i+1; j < n; j++)
        {
            if(a[j] < a[i])
            {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
}

```

```
void main ()
```

```

{
    int i, j, temp;
    int a[5] = { 10, 35, 32, 13, 26};
    int n = sizeof(a)/sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    print(a, n);
    bubble(a, n);
    printf("\nAfter sorting array elements are - \n");
}

```

```
    print(a, n);  
}
```

### Output

```
Before sorting array elements are -  
10 35 32 13 26  
After sorting array elements are -  
10 13 26 32 35
```

**Program:** Write a program to implement bubble sort in C++ language.

```
#include<iostream>
```

```
using namespace std;
```

```
void print(int a[], int n) //function to print array elements
```

```
{  
    int i;  
    for(i = 0; i < n; i++)  
    {  
        cout<<a[i]<<" ";  
    }  
}
```

```
void bubble(int a[], int n) // function to implement bubble sort
```

```
{  
    int i, j, temp;  
    for(i = 0; i < n; i++)  
    {  
        for(j = i+1; j < n; j++)  
        {  
            if(a[j] < a[i])  
            {  
                temp = a[i];  
                a[i] = a[j];  
                a[j] = temp;  
            }  
        }  
    }  
}
```

```
}
```

```
int main()
```

```
{  
    int i, j, temp;  
    int a[5] = {45, 1, 32, 13, 26};  
    int n = sizeof(a)/sizeof(a[0]);  
    cout<<"Before sorting array elements are - \n";  
    print(a, n);  
    bubble(a, n);  
    cout<<"\nAfter sorting array elements are - \n";  
}
```

```
    print(a, n);  
    return 0;  
}
```

### Output

```
Before sorting array elements are -  
45 1 32 13 26  
After sorting array elements are -  
1 13 26 32 45
```

**Program:** Write a program to implement bubble sort in C# language.

```
using System;  
public class Bubble  
{  
    static void print (int[]a) //function to print array elements  
    {  
        int n = a.Length;  
        int i;  
        for (i = 0; i < n; i++)  
        {  
            Console.Write (" " + a[i]);  
        }  
    }  
    static void bubble (int[]a) // function to implement bubble sort  
    {  
        int n = a.Length;  
        int i, j, temp;  
        for (i = 0; i < n; i++)  
        {  
            for (j = i + 1; j < n; j++)  
            {  
                if (a[j] < a[i])  
                {  
                    temp = a[i];  
                    a[i] = a[j];  
                    a[j] = temp;  
                }  
            }  
        }  
    }  
}  
public static void Main ()  
{  
    int[] a = { 45, 1, 32, 13, 26 };  
    Console.WriteLine ("\n Before sorting array elements are - \n");  
    print (a);
```

```

        bubble (a);
        Console.WriteLine ();
        Console.WriteLine ("\n After sorting array elements are - \n");
        print (a);
    }
}

```

### Output

```

Before sorting array elements are -
45 1 32 13 26

After sorting array elements are -
1 13 26 32 45

```

**Program:** Write a program to implement bubble sort in Java.

```

public class Bubble {
    static void print (int a[]) //function to print array elements
    {
        int n = a.length;
        int i;
        for (i = 0; i < n; i++)
        {
            System.out.print(a[i] + " ");
        }
    }
    static void bubbleSort (int a[]) // function to implement bubble sort
    {
        int n = a.length;
        int i, j, temp;
        for (i = 0; i < n; i++)
        {
            for (j = i + 1; j < n; j++)
            {
                if (a[j] < a[i])
                {
                    temp = a[i];
                    a[i] = a[j];
                    a[j] = temp;
                }
            }
        }
    }
}

public static void main(String[] args) {
    int a[] = {35, 10, 31, 11, 26};
    Bubble b1 = new Bubble();
    System.out.println("Before sorting array elements are - ");
}

```

```

        b1.print(a);
        b1.bubbleSort(a);
        System.out.println();
        System.out.println("After sorting array elements are - ");
        b1.print(a);
    }
}

```

### Output

```

D:\JTP>javac Bubble.java
D:\JTP>java Bubble
Before sorting array elements are -
35 10 31 11 26
After sorting array elements are -
10 11 26 31 35

```

**Program:** Write a program to implement bubble sort in JavaScript.

```

<html>
<head>
</head>
<body>
<script>
    var a = [35, 10, 31, 11, 26];
    function print() //function to print array elements
    {
        for(i = 0; i < 5; i++)
        {
            document.writeln(a[i]);
        }
    }
    document.write("Before sorting array elements are - " + "<br>");
    print();

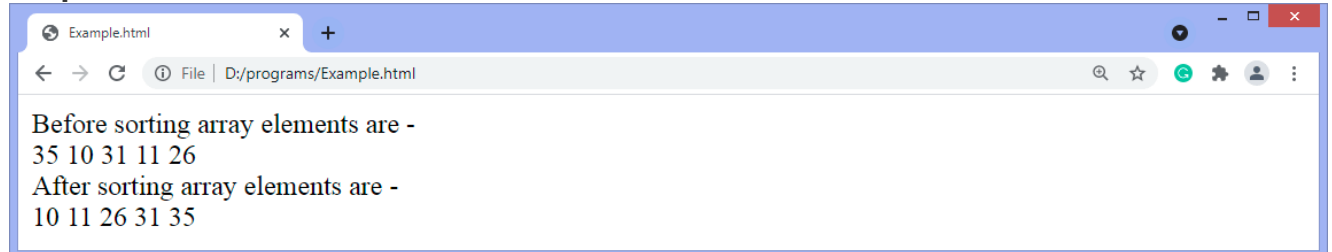
    for(i = 0; i < 5; i++)
    {
        for(j = 0; j < 5; j++)
        {
            if(a[i] < a[j])
            {
                temp = a[i];
                a[i]=a[j];
                a[j] = temp;
            }
        }
    }

    document.write("<br> After sorting array elements are - " + "<br>");

```

```
print();
</script>
</body>
</html>
```

### Output

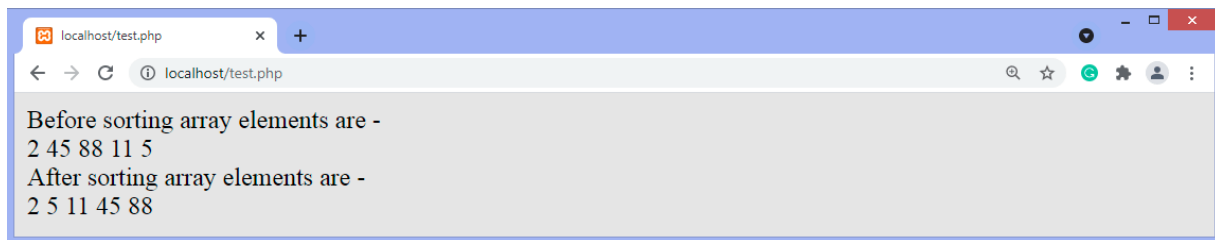


**Program:** Write a program to implement bubble sort in PHP.

```
<?php
$a = array(2, 45, 88, 11, 5);
function printArray($a)
{
    for($i = 0; $i < 5; $i++)
    {
        print_r($a[$i]);
        echo " ";
    }
}
echo "Before sorting array elements are - <br>";
printArray($a);
for($i = 0; $i < 5; $i++)
{
    for ($j = 0; $j < 5; $j++)
    {
        if($a[$i] < $a[$j])
        {
            $temp = $a[$i];
            $a[$i]=$a[$j];
            $a[$j] = $temp;
        }
    }
}
echo "<br> After sorting array elements are - <br>";
printArray($a);
?>
```

### Output



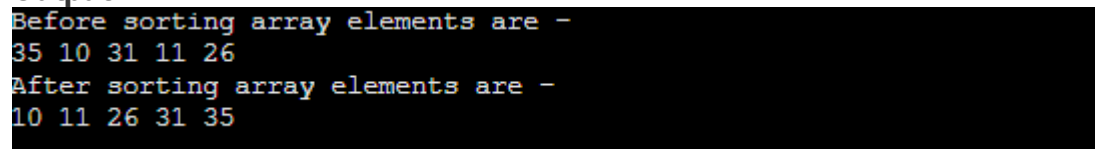


```
Before sorting array elements are -
2 45 88 11 5
After sorting array elements are -
2 5 11 45 88
```

**Program:** Write a program to implement bubble sort in python.

```
a = [35, 10, 31, 11, 26]
print("Before sorting array elements are - ")
for i in a:
    print(i, end = " ")
for i in range(0,len(a)):
    for j in range(i+1,len(a)):
        if a[j]<a[i]:
            temp = a[j]
            a[j]=a[i]
            a[i]=temp
print("\nAfter sorting array elements are - ")
for i in a:
    print(i, end = " ")
```

#### Output



```
Before sorting array elements are -
35 10 31 11 26
After sorting array elements are -
10 11 26 31 35
```

So, that's all about the article. Hope the article will be helpful and informative to you. This article was not only limited to the algorithm. We have also discussed the algorithm's complexity, working, optimized form, and implementation in different programming languages.

## Bucket Sort Algorithm

In this article, we will discuss the bucket sort Algorithm. The data items in the bucket sort are distributed in the form of buckets. In coding or technical interviews for software engineers, sorting algorithms are widely asked. So, it is important to discuss the topic.

Bucket sort is a sorting algorithm that separate the elements into multiple groups said to be buckets. Elements in bucket sort are first uniformly divided into groups called buckets, and then they are sorted by any other sorting algorithm. After that, elements are gathered in a sorted manner.

The basic procedure of performing the bucket sort is given as follows -

First, partition the range into a fixed number of buckets.

Then, toss every element into its appropriate bucket.

After that, sort each bucket individually by applying a sorting algorithm.

And at last, concatenate all the sorted buckets.

The advantages of bucket sort are -

Bucket sort reduces the no. of comparisons.

It is asymptotically fast because of the uniform distribution of elements.

The limitations of bucket sort are -

It may or may not be a stable sorting algorithm.

It is not useful if we have a large array because it increases the cost.

It is not an in-place sorting algorithm, because some extra space is required to sort the buckets.

The best and average-case complexity of bucket sort is  $O(n + k)$ , and the worst-case complexity of bucket sort is  $O(n^2)$ , where  $n$  is the number of items.

Bucket sort is commonly used -

With floating-point values.

When input is distributed uniformly over a range.

The basic idea to perform the bucket sort is given as follows -

bucketSort(a[], n)

1. Create ' $n$ ' empty buckets

2. Do **for** each array element  $a[i]$

    2.1. Put array elements into buckets, i.e. insert  $a[i]$  into bucket  $[n*a[i]]$

3. Sort the elements of individual buckets by using the insertion sort.

4. At last, gather or concatenate the sorted buckets.

End bucketSort

Now, let's see the algorithm of bucket sort.

## Algorithm

Bucket Sort(A[])

1. Let  $B[0 \dots n-1]$  be a **new** array

2.  $n = \text{length}[A]$

3. **for**  $i=0$  to  $n-1$

4. make  $B[i]$  an empty list

5. **for**  $i=1$  to  $n$

6. **do** insert  $A[i]$  into list  $B[n*a[i]]$

7. **for**  $i=0$  to  $n-1$

8. **do** sort list  $B[i]$  with insertion-sort

9. Concatenate lists  $B[0], B[1], \dots, B[n-1]$  together in order

End

## Scatter-gather approach

We can understand the Bucket sort algorithm via scatter-gather approach. Here, the given elements are first scattered into buckets. After scattering, elements in each bucket are sorted using a stable sorting algorithm. At last, the sorted elements will be gathered in order.

Let's take an unsorted array to understand the process of bucket sort. It will be easier to understand the bucket sort via an example.

Let the elements of array are -

Now, create buckets with a range from 0 to 25. The buckets range are 0-5, 5-10, 10-15, 15-20, 20-25. Elements are inserted in the buckets according to the bucket range. Suppose the value of an item is 16, so it will be inserted in the bucket with the range 15-20. Similarly, every item of the array will insert accordingly.

This phase is known to be the **scattering of array elements**.

Now, **sort** each bucket individually. The elements of each bucket can be sorted by using any of the stable sorting algorithms.

At last, **gather** the sorted elements from each bucket in order

Now, the array is completely sorted.

## Bucket sort complexity

Now, let's see the time complexity of bucket sort in best case, average case, and in worst case. We will also see the space complexity of the bucket sort.

### 1. Time Complexity

| Case         | Time       | Complexity |
|--------------|------------|------------|
| Best Case    | $O(n + k)$ |            |
| Average Case | $O(n + k)$ |            |
| Worst Case   | $O(n^2)$   |            |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. In Bucket sort, best case occurs when the elements are uniformly distributed in the buckets. The complexity will be better if the elements are already sorted in the buckets.

If we use the insertion sort to sort the bucket elements, the overall complexity will be linear, i.e.,  $O(n + k)$ , where  $O(n)$  is for making the buckets, and  $O(k)$  is for sorting the bucket elements using algorithms with linear time complexity at best case.

The best-case time complexity of bucket sort is  **$O(n + k)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. Bucket sort runs in the linear time, even when the elements are uniformly distributed. The average case time complexity of bucket sort is  **$O(n + K)$** .

**Worst Case Complexity** - In bucket sort, worst case occurs when the elements are of the close range in the array, because of that, they have to be placed in the same bucket. So, some buckets have more number of elements than others.

The complexity will get worse when the elements are in the reverse order.

The worst-case time complexity of bucket sort is  **$O(n^2)$** .

### 2. Space Complexity

|                  |          |
|------------------|----------|
| Space Complexity | $O(n*k)$ |
| Stable           | YES      |

The space complexity of bucket sort is  $O(n*k)$ .

## Implementation of bucket sort

Now, let's see the programs of bucket sort in different programming languages.

**Program:** Write a program to implement bucket sort in C language.

```
#include <stdio.h>
```

```
int getMax(int a[], int n) // function to get maximum element from the given array
```

```
{
    int max = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];
    return max;
}
```

```
void bucket(int a[], int n) // function to implement bucket sort
```

```
{
    int max = getMax(a, n); //max is the maximum element of array
    int bucket[max], i;
    for (int i = 0; i <= max; i++)
    {
        bucket[i] = 0;
    }
    for (int i = 0; i < n; i++)
    {
        bucket[a[i]]++;
    }
    for (int i = 0, j = 0; i <= max; i++)
    {
        while (bucket[i] > 0)
        {
            a[j++] = i;
            bucket[i]--;
        }
    }
}
```

```
void printArr(int a[], int n) // function to print array elements
```

```
{
    for (int i = 0; i < n; ++i)
        printf("%d ", a[i]);
}
```

```
int main()
```

```
{
    int a[] = {54, 12, 84, 57, 69, 41, 9, 5};
    int n = sizeof(a) / sizeof(a[0]); // n is the size of array
```

```

printf("Before sorting array elements are - \n");
printArr(a, n);
bucket(a, n);
printf("\nAfter sorting array elements are - \n");
printArr(a, n);
}

```

### Output

After the execution of above code, the output will be -

**Program:** Write a program to implement bucket sort in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int getMax(int a[], int n) // function to get maximum element from the given array
```

```
{
    int max = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];
    return max;
}
```

```
void bucket(int a[], int n) // function to implement bucket sort
```

```
{
    int max = getMax(a, n); //max is the maximum element of array
    int bucket[max], i;
    for (int i = 0; i <= max; i++)
    {
        bucket[i] = 0;
    }
    for (int i = 0; i < n; i++)
    {
        bucket[a[i]]++;
    }
    for (int i = 0, j = 0; i <= max; i++)
    {
        while (bucket[i] > 0)
        {
            a[j++] = i;
            bucket[i]--;
        }
    }
}
```

```

void printArr(int a[], int n) // function to print array elements
{
    for (int i = 0; i < n; ++i)
        cout<<a[i]<<" ";
}
int main()
{
    int a[] = {34, 42, 74, 57, 99, 84, 9, 5};
    int n = sizeof(a) / sizeof(a[0]); // n is the size of array
    cout<<"Before sorting array elements are - \n";
    printArr(a, n);
    bucket(a, n);
    cout<<"\nAfter sorting array elements are - \n";
    printArr(a, n);
}

```

### Output

After the execution of above code, the output will be -

**Program:** Write a program to implement bucket sort in C#.  
using System;

```

class Bucket {
static int getMax(int[] a) // function to get maximum element from the given array
{
    int n = a.Length;
    int max = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > max)
            max = a[i];
    return max;
}
static void bucket(int[] a) // function to implement bucket sort
{
    int n = a.Length;
    int max = getMax(a); //max is the maximum element of array
    int[] bucket = new int[max+1];
    for (int i = 0; i <= max; i++)
    {
        bucket[i] = 0;
    }
    for (int i = 0; i < n; i++)
    {
        bucket[a[i]]++;
    }
}
}

```

```

    }
    for (int i = 0, j = 0; i <= max; i++)
    {
        while (bucket[i] > 0)
        {
            a[j++] = i;
            bucket[i]--;
        }
    }
}

static void printArr(int[] a) /* function to print the array */
{
    int i;
    int n = a.Length;
    for (i = 0; i < n; i++)
        Console.Write(a[i] + " ");
}

static void Main() {
    int[] a = { 95, 50, 45, 15, 20, 10 };
    Console.WriteLine("Before sorting array elements are - \n");
    printArr(a);
    bucket(a);
    Console.WriteLine("\nAfter sorting array elements are - \n");
    printArr(a);
}
}

```

### Output

After the execution of above code, the output will be -

**Program:** Write a program to implement bucket sort in Java.

```

public class Bucket
{
    int getMax(int a[]) // function to get maximum element from the given
array
    {
        int n = a.length;
        int max = a[0];
        for (int i = 1; i < n; i++)
            if (a[i] > max)
                max = a[i];
        return max;
    }
}

```

```

void bucket(int a[]) // function to implement bucket sort
{
    int n = a.length;
    int max = getMax(a); //max is the maximum element of array
    int bucket[] = new int[max+1];
    for (int i = 0; i <= max; i++)
    {
        bucket[i] = 0;
    }
    for (int i = 0; i < n; i++)
    {
        bucket[a[i]]++;
    }
    for (int i = 0, j = 0; i <= max; i++)
    {
        while (bucket[i] > 0)
        {
            a[j++] = i;
            bucket[i]--;
        }
    }
}

void printArr(int a[]) /* function to print the array */
{
    int i;
    int n = a.length;
    for (i = 0; i < n; i++)
        System.out.print(a[i] + " ");
}

public static void main(String[] args) {
    int a[] = { 90, 40, 5, 15, 30, 9 };
    Bucket b1 = new Bucket();
    System.out.print("Before sorting array elements are - \n");
    b1.printArr(a);
    b1.bucket(a);
    System.out.print("\nAfter sorting array elements are - \n");
    b1.printArr(a);
}

```

### Output

After the execution of above code, the output will be -



So, that's all about the article. Hope the article will be helpful and informative to you. This article was not only limited to the algorithm. Along with the algorithm, we have also discussed the bucket sort complexity, working, and implementation in different programming languages.

## Comb Sort Algorithm

In this article, we will discuss the comb sort Algorithm. Comb Sort is the advanced form of bubble Sort. Bubble Sort compares all the adjacent values while comb sort removes all the turtle values or small values near the end of the list.

It is a comparison-based sorting algorithm that is mainly an improvement in bubble sort. In bubble sort, there is a comparison between the adjacent elements to sort the given array. So, in bubble sort, the gap size between the elements that are compared is 1. Comb sort improves the bubble sort by using a gap of size more than 1. The gap in the comb sort starts with the larger value and then shrinks by a factor of 1.3. It means that after the completion of each phase, the gap is divided by the shrink factor 1.3. The iteration continues until the gap is 1.

The shrink factor is found to be 1.3 by testing comb sort on 200,000 random lists. Comb sort works better than the bubble sort, but its time complexity in average case and worst case remains  $O(n^2)$ .

The process of performing the comb sort is given as follows -

**STEP 1** START

**STEP 2** Calculate the gap value if gap value==1 goto step 5 else goto step 3

**STEP 3** Iterate over data set and compare each item with gap item then goto step 4.

**STEP 4** Swap the element if require else goto step 2

**STEP 5** Print the sorted array.

**STEP 6** STOP

Now, let's see the algorithm of comb sort.

### Algorithm

```
combSort(array, n) // n is the size of array
```

```
    gap = n // Initialize gap size equal to the size of array
```

```
    shrink = 1.3 // gap shrink factor
```

```
    swapped = true
```

```
while (gap != 1 || swapped == true)
```

```
    gap = floor(gap/1.3);
```

```
    if gap ≤ 1
```

```
        gap = 1
```

```
    end if
```

```
    swapped = false
```

```
for every element range from 0 to n-gap, do the following -
```

```
    if array[i] > array[i+gap]
```

```
        swap(array[i], array[i+gap])
```

```
        swapped = true
```

```
    end if
```

```
end for loop
```

end **while** loop  
end combSort

## Working of comb Sort Algorithm

Now, let's see the working of the comb sort Algorithm.

To understand the working of the comb sort algorithm, let's take an unsorted array. It will be easier to understand the comb sort via an example.

Let the elements of array are -

| 0  | 1  | 2  | 3  | 4  | 5  | 6 | 7  |
|----|----|----|----|----|----|---|----|
| 49 | 11 | 24 | 44 | 29 | 27 | 2 | 22 |

Now, initialize

**n** = 8 // size of array

**gap** = n

**shrink** = 1.3

**swapped** = true

### First iteration

**gap** = floor(gap/shrink) = floor(8/1.3) = 6

**swapped** = false

|       | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |                  |
|-------|----|----|----|----|----|----|----|----|------------------|
| i = 0 | 49 | 11 | 24 | 44 | 29 | 27 | 2  | 22 | 49 > 2, swap     |
| i = 1 | 2  | 11 | 24 | 44 | 29 | 27 | 49 | 22 | 11 < 22, no swap |

This iteration ends here, because at i = 2, the value of i + gap = 2 + 6 = 8, and there is no element at 8<sup>th</sup> position of the array. So, after first iteration, the elements of array will be -

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|---|----|----|----|----|----|----|----|
| 2 | 11 | 24 | 44 | 29 | 27 | 49 | 22 |

Now, move to iteration 2.

### Second iteration

**gap** = floor(gap/shrink) = floor(6/1.3) = 4

**swapped** = false

|       | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |                |
|-------|---|----|----|----|----|----|----|----|----------------|
| i = 0 | 2 | 11 | 24 | 44 | 29 | 27 | 49 | 22 | 2<29, No swap  |
| i = 1 | 2 | 11 | 24 | 44 | 29 | 27 | 49 | 22 | 11<27, No swap |
| i = 2 | 2 | 11 | 24 | 44 | 29 | 27 | 49 | 22 | 24<49, No swap |
| i = 3 | 2 | 11 | 24 | 44 | 29 | 27 | 49 | 22 | 44>22, swap    |

This iteration ends here, because at i =4, the value of i + gap = 4 + 4 = 8, and there is no element at 8<sup>th</sup> position of the array. So, after second iteration, the elements of array will be -

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|---|----|----|----|----|----|----|----|
| 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 |

Now, move to iteration 3.

### Third iteration

gap = floor(gap/shrink) = floor(4/1.3) = 3

swapped = false

|       | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |                |
|-------|---|----|----|----|----|----|----|----|----------------|
| i = 0 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 2<22, No swap  |
| i = 1 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 11<29, No swap |
| i = 2 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 24<27, No swap |
| i = 3 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 22<49, No swap |
| i = 4 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 29<44, No swap |

This iteration ends here, because at i =5, the value of i + gap = 5 + 3 = 8, and there is no element at 8<sup>th</sup> position of the array. So, after third iteration, the elements of array will be -

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|---|----|----|----|----|----|----|----|
| 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 |

Now, move to iteration 4.

### Fourth iteration

gap = floor(gap/shrink) = floor(3/1.3) = 2

swapped = false

|       | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |                |
|-------|---|----|----|----|----|----|----|----|----------------|
| i = 0 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 2<24, no swap  |
| i = 1 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 11<22, no swap |
| i = 2 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 24<29, no swap |
| i = 3 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 22<27, no swap |
| i = 4 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 29<49, no swap |
| i = 5 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 27<44, no swap |

This iteration ends here, because at i = 6, the value of  $i + \text{gap} = 6 + 2 = 8$ , and there is no element at 8<sup>th</sup> position of the array. So, after fourth iteration, the elements of array will be -

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|---|----|----|----|----|----|----|----|
| 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 |

Now, move to iteration 5.

### Fifth iteration

**gap** = floor(gap/shrink) = floor(2/1.3) = 1

**swapped** = false

|       | 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |                |
|-------|---|----|----|----|----|----|----|----|----------------|
| i = 0 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 2<11, no swap  |
| i = 1 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 11<24, no swap |
| i = 2 | 2 | 11 | 24 | 22 | 29 | 27 | 49 | 44 | 24>22, swap    |
| i = 3 | 2 | 11 | 22 | 24 | 29 | 27 | 49 | 44 | 24<29, no swap |
| i = 4 | 2 | 11 | 22 | 24 | 29 | 27 | 49 | 44 | 29>27, swap    |
| i = 5 | 2 | 11 | 22 | 24 | 27 | 29 | 49 | 44 | 29<49, no swap |
| i = 6 | 2 | 11 | 22 | 24 | 27 | 29 | 49 | 44 | 49>44, swap    |

After the fifth iteration, the sorted array is -

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|---|----|----|----|----|----|----|----|
| 2 | 11 | 22 | 24 | 27 | 29 | 44 | 49 |

Hence, the iterations end here, and now the sorting is completed. Now, the final sorted array is -

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
| 2 | 11 | 22 | 24 | 27 | 29 | 44 | 49 |

## Comb sort complexity

Now, let's see the time complexity of Comb sort in the best case, average case, and worst case. We will also see the space complexity of Comb sort.

### 1. Time Complexity

| Case         | Time Complexity                                      |
|--------------|------------------------------------------------------|
| Best Case    | $\theta(n \log n)$                                   |
| Average Case | $\Omega(n^2/2^p)$ where p is a number of increments. |
| Worst Case   | $O(n^2)$                                             |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of comb sort is  **$\theta(n \log n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of comb sort is  **$\Omega(n^2/2^p)$ , where p is a number of increments..**

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of comb sort is  **$O(n^2)$** .

### 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(1)$ |
| Stable           | YES    |

The space complexity of comb sort is  $O(1)$ .

## Implementation of Comb sort

Now, let's see the programs of Comb sort in different programming languages.

**Program:** Write a program to implement comb sort in C language.

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int updatedGap(int gap)
{
    // Shrink gap by Shrink factor
    gap = floor(gap/1.3);

    if (gap < 1)
```

```

    return 1;
    return gap;
}

```

// Function to sort array elements using Comb Sort

```

void combSort(int a[], int n)
{
    int gap = n; /* Initialize gap size equal to the size of array */

    int swapped = 1;

    while (gap != 1 || swapped == 1)
    {
        gap = updatedGap(gap); // find updated gap
        // Initialize swapped as false so that we can
        // check if swap happened or not
        swapped = 0;

        for (int i = 0; i < n-gap; i++) /* Compare all elements with current gap */
        {
            if (a[i] > a[i+gap]) //swap a[i] with a[i+gap]
            {
                int temp = a[i];
                a[i] = a[i+gap];
                a[i+gap] = temp;
                swapped = 1;
            }
        }
    }
}

void printArr(int a[], int n) /* function to print array elements */
{
    for (int i=0; i<n; i++)
        printf("%d ", a[i]);
}

int main()
{
    int a[] = {49, 11, 24, 44, 29, 27, 2};
    int n = sizeof(a)/sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArr(a, n);
    combSort(a, n);
}

```

```

printf("\nAfter sorting array elements are - \n");
printArr(a, n);
return 0;
}

```

### Output

```

Before sorting array elements are -
49 11 24 44 29 27 2
After sorting array elements are -
2 11 24 27 29 44 49

```

**Program:** Write a program to implement comb sort in C++.

```
#include <iostream>
```

```
#include<cmath>
```

```
using namespace std;
```

```
int updatedGap(int gap)
```

```
{
    // Shrink gap by Shrink factor
    gap = floor(gap/1.3);
```

```
    if (gap < 1)
        return 1;
    return gap;
}
```

```
// Function to sort array elements using Comb Sort
```

```
void combSort(int a[], int n)
```

```
{
    int gap = n; /* Initialize gap size equal to the size of array */
```

```
    int swapped = 1;
```

```
    while (gap != 1 || swapped == 1)
```

```
{
        gap = updatedGap(gap); // find updated gap
        // Initialize swapped as false so that we can
        // check if swap happened or not
        swapped = 0;
```

```
        for (int i = 0; i < n-gap; i++) /* Compare all elements with current gap */
```

```
{
            if (a[i] > a[i+gap]) //swap a[i] with a[i+gap]
            {
                int temp = a[i];
                a[i] = a[i+gap];
```

```

        a[i+gap] = temp;
        swapped = 1;
    }
}
}

void printArr(int a[], int n) /* function to print array elements */
{
    for (int i=0; i<n; i++)
        cout<<a[i]<<" ";
}

int main()
{
    int a[] = {48, 10, 23, 43, 28, 26, 1};
    int n = sizeof(a)/sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    printArr(a, n);
    combSort(a, n);
    cout<<"\nAfter sorting array elements are - \n";
    printArr(a, n);
    return 0;
}

```

### Output

```

Before sorting array elements are -
48 10 23 43 28 26 1
After sorting array elements are -
1 10 23 26 28 43 48

```

**Program:** Write a program to implement comb sort in C#.

using System;

```

class CombSort {
    static int updatedGap(int gap)
    {
        // Shrink gap by Shrink factor
        gap = (gap*10)/13;

        if (gap < 1)
            return 1;
        return gap;
    }
}

```

// Function to sort array elements using Comb Sort

```

static void combSort(int[] a, int n)
{

```



```

int gap = n; /* Initialize gap size equal to the size of array */

int swapped = 1;

while (gap != 1 || swapped == 1)
{
    gap = updatedGap(gap); // find updated gap
    // Initialize swapped as false so that we can
    // check whether swap happened or not
    swapped = 0;

    for (int i = 0; i < n-gap; i++) /* Compare all elements with current gap */
    {
        if (a[i] > a[i+gap]) //swap a[i] with a[i+gap]
        {
            int temp = a[i];
            a[i] = a[i+gap];
            a[i+gap] = temp;
            swapped = 1;
        }
    }
}

static void printArr(int[] a, int n) /* function to print array elements */
{
    for (int i=0; i<n; i++)
        Console.Write(a[i] + " ");
}

static void Main() {

    int[] a = {44, 40, 14, 43, 28, 26, 1, 5, 9};
    int n = a.Length;
    Console.Write("Before sorting array elements are - \n");
    printArr(a, n);
    combSort(a, n);
    Console.Write("\nAfter sorting array elements are - \n");
    printArr(a, n);
}
}

```

## Output

```
Before sorting array elements are -  
44 40 14 43 28 26 1 5 9  
After sorting array elements are -  
1 5 9 14 26 28 40 43 44
```

**Program:** Write a program to implement comb sort in Java.

```
class CombSort {  
    static int updatedGap(int gap)  
    {  
        // Shrink gap by Shrink factor  
        gap = (gap*10)/13;  
  
        if (gap < 1)  
            return 1;  
        return gap;  
    }  
  
    // Function to sort array elements using Comb Sort  
    static void combSort(int a[], int n)  
    {  
        int gap = n; /* Initialize gap size equal to the size of array */  
  
        int swapped = 1;  
  
        while (gap != 1 || swapped == 1)  
        {  
            gap = updatedGap(gap); // find updated gap  
            // Initialize swapped as false so that we can  
            // check whether swap happened or not  
            swapped = 0;  
  
            for (int i = 0; i < n-gap; i++) /* Compare all elements  
with current gap */  
            {  
                if (a[i] > a[i+gap]) //swap a[i] with a[i+gap]  
                {  
                    int temp = a[i];  
                    a[i] = a[i+gap];  
                    a[i+gap] = temp;  
                    swapped = 1;  
                }  
            }  
        }  
    }  
}
```

```
static void printArr(int a[], int n) /* function to print array elements */
{
    for (int i=0; i<n; i++)
        System.out.print(a[i] + " ");
}
```

```
public static void main(String args[]) {

    int[] a = {43, 39, 13, 42, 27, 25, 0, 4, 8};
    int n = a.length;
    System.out.print("\nBefore sorting array elements are - ");
    printArr(a, n);
    combSort(a, n);
    System.out.print("\nAfter sorting array elements are - ");
    printArr(a, n);
    System.out.println();

}
}
```

### Output

```
D:\JTP>javac CombSort.java
D:\JTP>java CombSort
Before sorting array elements are - 43 39 13 42 27 25 0 4 8
After sorting array elements are - 0 4 8 13 25 27 39 42 43
```

**Program:** Write a program to implement comb sort in PHP.

**<?php**

```
function updatedGap($gap)
{
    // Shrink gap by Shrink factor
    $gap = ($gap*10)/13;

    if ($gap < 1)
        return 1;
    return $gap;
}
```

// Function to sort array elements using Comb Sort

```
function combSort(&$a, $n)
{
    $gap = $n; /* Initialize gap size equal to the size of array */

    $swapped = 1;
```

```

while ($gap != 1 || $swapped == 1)
{
    $gap = updatedGap($gap); // find updated gap
    // Initialize swapped as false so that we can
    // check whether swap happened or not
    $swapped = 0;

    for ($i = 0; $i < $n-$gap; $i++) /* Compare all elements with current gap */
    {
        if ($a[$i] > $a[$i+$gap]) //swap a[i] with a[i+gap]
        {
            $temp = $a[$i];
            $a[$i] = $a[$i+$gap];
            $a[$i+$gap] = $temp;
            $swapped = 1;
        }
    }
}

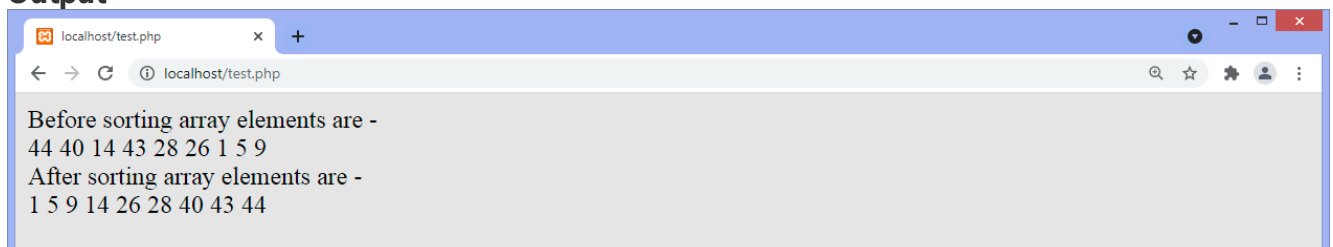
function printArr($a, $n) /* function to print array elements */
{
    for($i = 0; $i < $n; $i++)
    {
        print_r($a[$i]);
        echo " ";
    }
}

$a = array(44, 40, 14, 43, 28, 26, 1, 5, 9);
$n = count($a);
echo "Before sorting array elements are - <br>";
printArr($a, $n);
combSort($a,$n);
echo "<br> After sorting array elements are - <br>";
printArr($a, $n);

```

?>

## Output



So, that's all about the article. Hope the article will be helpful and informative to you. This article was not only limited to the algorithm. Along with the algorithm, we have also discussed the Comb Sort's complexity, working, and implementation in different programming languages.

## Counting Sort Algorithm

In this article, we will discuss the counting sort Algorithm. Counting sort is a sorting technique that is based on the keys between specific ranges. In coding or technical interviews for software engineers, sorting algorithms are widely asked. So, it is important to discuss the topic.

This sorting technique doesn't perform sorting by comparing elements. It performs sorting by counting objects having distinct key values like hashing. After that, it performs some arithmetic operations to calculate each object's index position in the output sequence. Counting sort is not used as a general-purpose sorting algorithm.

Counting sort is effective when range is not greater than number of objects to be sorted. It can be used to sort the negative input values.

Now, let's see the algorithm of counting sort.

### Algorithm

countingSort(array, n) // 'n' is the size of array

max = find maximum element in the given array

create count array with size maximum + 1

Initialize count array with all 0's

**for** i = 0 to n

find the count of every unique element and

store that count at ith position in the count array

**for** j = 1 to max

Now, find the cumulative sum and store it in count array

**for** i = n to 1

Restore the array elements

Decrease the count of every restored element by 1

end countingSort

### Working of counting sort Algorithm

Now, let's see the working of the counting sort Algorithm.

To understand the working of the counting sort algorithm, let's take an unsorted array. It will be easier to understand the counting sort via an example.

Let the elements of array are -

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 2 | 9 | 7 | 4 | 1 | 8 | 4 |
|---|---|---|---|---|---|---|

1. Find the maximum element from the given array. Let **max** be the maximum element.

max

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 9 | 2 | 7 | 4 | 1 | 8 | 4 |
|---|---|---|---|---|---|---|

2. Now, initialize array of length **max + 1** having all 0 elements. This array will be used to store the count of the elements in the given array.

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Count array

3. Now, we have to store the count of each array element at their corresponding index in the count array.

The count of an element will be stored as - Suppose array element '4' is appeared two times, so the count of element 4 is 2. Hence, 2 is stored at the 4<sup>th</sup> position of the count array. If any element is not present in the array, place 0, i.e. suppose element '3' is not present in the array, so, 0 will be stored at 3<sup>rd</sup> position.

|             |   |   |   |   |   |   |   |   |   |   |
|-------------|---|---|---|---|---|---|---|---|---|---|
| Given array | 2 | 9 | 7 | 4 | 1 | 8 | 4 |   |   |   |
|             | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
| Count array | 0 | 1 | 1 | 0 | 2 | 0 | 0 | 1 | 1 | 1 |

Count of each stored element

Now, store the cumulative sum of **count** array elements. It will help to place the elements at the correct index of the sorted array.

| 0     | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|-------|---|---|---|---|---|---|---|---|---|
| 0     | 1 | 2 | 0 | 2 | 0 | 0 | 1 | 1 | 1 |
| 1+1=2 |   |   |   |   |   |   |   |   |   |
| 0     | 1 | 2 | 2 | 2 | 0 | 0 | 1 | 1 | 1 |
| 2+0=2 |   |   |   |   |   |   |   |   |   |

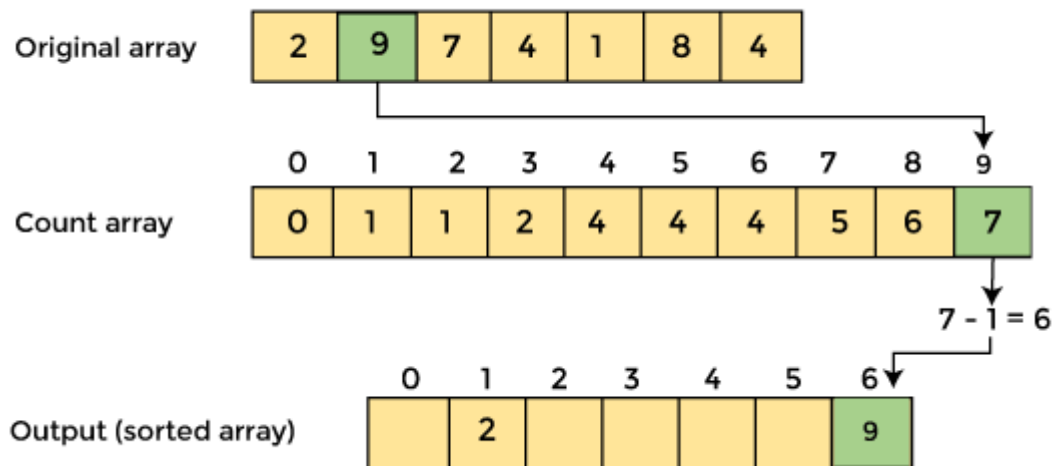
Similarly, the cumulative count of the count array is -

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 2 | 4 | 4 | 4 | 5 | 6 | 7 |

Cumulative count

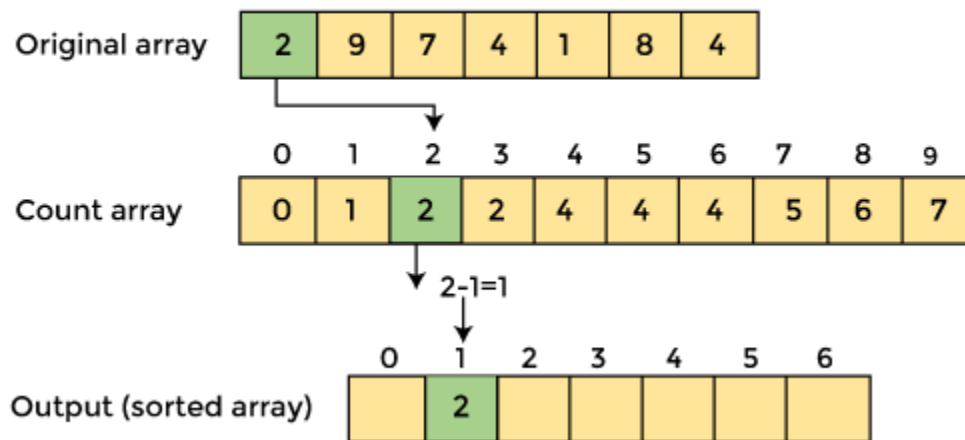
4. Now, find the index of each element of the original array

### For element 9



After placing element at its place, decrease its count by one. Before placing element 2, its count was 2, but after placing it at its correct position, the new count for element 2 is 1.

### For element 2



Similarly, after sorting, the array elements are -



Now, the array is completely sorted.

## Counting sort complexity

Now, let's see the time complexity of counting sort in best case, average case, and in worst case. We will also see the space complexity of the counting sort.

### 1. Time Complexity

| Case | Time | Complexity |
|------|------|------------|
|------|------|------------|

|                     |            |  |
|---------------------|------------|--|
| <b>Best Case</b>    | $O(n + k)$ |  |
| <b>Average Case</b> | $O(n + k)$ |  |
| <b>Worst Case</b>   | $O(n + k)$ |  |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of counting sort is  **$O(n + k)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of counting sort is  **$O(n + k)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of counting sort is  **$O(n + k)$** .

In all above cases, the time complexity of counting sort is same. This is because the algorithm goes through  **$n+k$**  times, regardless of how the elements are placed in the array.

Counting sort is better than the comparison-based sorting techniques because there is no comparison between elements in counting sort. But, when the integers are very large the counting sort is bad because arrays of that size have to be created.

## 2. Space Complexity

|                         |                 |
|-------------------------|-----------------|
| <b>Space Complexity</b> | $O(\text{max})$ |
| <b>Stable</b>           | YES             |

The space complexity of counting sort is  $O(\text{max})$ . The larger the range of elements, the larger the space complexity.

## Implementation of counting sort

Now, let's see the programs of counting sort in different programming languages.

**Program:** Write a program to implement counting sort in C language.

```
#include<stdio.h>
```

```
int getMax(int a[], int n) {
    int max = a[0];
    for(int i = 1; i < n; i++) {
        if(a[i] > max)
            max = a[i];
    }
    return max; //maximum element from the array
}
```

```
void countSort(int a[], int n) // function to perform counting sort
{
    int output[n+1];
    int max = getMax(a, n);
```



```

int count[max+1]; //create count array with size [max+1]

for (int i = 0; i <= max; ++i)
{
    count[i] = 0; // Initialize count array with all zeros
}

for (int i = 0; i < n; i++) // Store the count of each element
{
    count[a[i]]++;
}

for(int i = 1; i <= max; i++)
    count[i] += count[i-1]; //find cumulative frequency

/* This loop will find the index of each element of the original array in count array, and
place the elements in output array*/
for (int i = n - 1; i >= 0; i--) {
    output[count[a[i]] - 1] = a[i];
    count[a[i]]--; // decrease count for same numbers
}

for(int i = 0; i < n; i++) {
    a[i] = output[i]; //store the sorted elements into main array
}

void printArr(int a[], int n) /* function to print the array */
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}

int main() {
    int a[] = { 11, 30, 24, 7, 31, 16 };
    int n = sizeof(a)/sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArr(a, n);
    countSort(a, n);
    printf("\nAfter sorting array elements are - \n");
    printArr(a, n);
    return 0;
}

```

```
}
```

## Output

After the execution of above code, the output will be -

```
Before sorting array elements are -  
11 30 24 7 31 16  
After sorting array elements are -  
7 11 16 24 30 31
```

**Program:** Write a program to implement counting sort in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int getMax(int a[], int n) {
```

```
    int max = a[0];
```

```
    for(int i = 1; i < n; i++) {
```

```
        if(a[i] > max)
```

```
            max = a[i];
```

```
    }
```

```
    return max; //maximum element from the array
```

```
}
```

```
void countSort(int a[], int n) // function to perform counting sort
```

```
{
```

```
    int output[n+1];
```

```
    int max = getMax(a, n);
```

```
    int count[max+1]; //create count array with size [max+1]
```

```
    for (int i = 0; i <= max; ++i)
```

```
    {
```

```
        count[i] = 0; // Initialize count array with all zeros
```

```
    }
```

```
    for (int i = 0; i < n; i++) // Store the count of each element
```

```
    {
```

```
        count[a[i]]++;
```

```
    }
```

```
    for(int i = 1; i <= max; i++)
```

```
        count[i] += count[i-1]; //find cumulative frequency
```

```
    /* This loop will find the index of each element of the original array in count array, and  
    place the elements in output array*/
```

```

for (int i = n - 1; i >= 0; i--) {
    output[count[a[i]] - 1] = a[i];
    count[a[i]]--; // decrease count for same numbers
}

for(int i = 0; i < n; i++) {
    a[i] = output[i]; //store the sorted elements into main array
}

void printArr(int a[], int n) /* function to print the array */
{
    int i;
    for (i = 0; i < n; i++)
        cout << a[i] << " ";
}

int main() {
    int a[] = { 31, 11, 42, 7, 30, 11 };
    int n = sizeof(a)/sizeof(a[0]);
    cout << "Before sorting array elements are - \n";
    printArr(a, n);
    countSort(a, n);
    cout << "\nAfter sorting array elements are - \n";
    printArr(a, n);
    return 0;
}

```

## Output

After the execution of above code, the output will be -

```

Before sorting array elements are -
31 11 42 7 30 11
After sorting array elements are -
7 11 11 30 31 42

```

**Program:** Write a program to implement counting sort in C#.

**using** System;

**class** CountingSort {

**static int** getMax(int[] a, int n) {

int max = a[0];

for(int i = 1; i < n; i++) {

if(a[i] > max)

max = a[i];

}

return max; //maximum element from the array

```
}
```

```
static void countSort(int[] a, int n) // function to perform counting sort
```

```
{
```

```
    int[] output = new int [n+1];
```

```
    int max = getMax(a, n);
```

```
    int[] count = new int [max+1]; //create count array with size [max+1]
```

```
    for (int i = 0; i <= max; ++i)
```

```
    {
```

```
        count[i] = 0; // Initialize count array with all zeros
```

```
    }
```

```
    for (int i = 0; i < n; i++) // Store the count of each element
```

```
    {
```

```
        count[a[i]]++;
```

```
    }
```

```
    for(int i = 1; i <= max; i++)
```

```
        count[i] += count[i-1]; //find cumulative frequency
```

```
    /* This loop will find the index of each element of the original array in count array, and  
    place the elements in output array*/
```

```
    for (int i = n - 1; i >= 0; i--) {
```

```
        output[count[a[i]] - 1] = a[i];
```

```
        count[a[i]]--; // decrease count for same numbers
```

```
    }
```

```
    for(int i = 0; i < n; i++) {
```

```
        a[i] = output[i]; //store the sorted elements into main array
```

```
    }
```

```
}
```

```
static void printArr(int[] a, int n) /* function to print the array */
```

```
{
```

```
    int i;
```

```
    for (i = 0; i < n; i++)
```

```
        Console.Write(a[i] + " ");
```

```
}
```

```
static void Main() {
```

```
    int[] a = { 43, 31, 2, 7, 10, 1, 5, 6, 3 };
```

```
    int n = a.Length;
```

```
    Console.WriteLine("Before sorting array elements are - \n");
```

```

    printArr(a,n);
    countSort(a,n);
    Console.WriteLine("\nAfter sorting array elements are - \n");
    printArr(a,n);
}
}

```

### Output

After the execution of above code, the output will be -

```

Before sorting array elements are -
43 31 2 7 10 1 5 6 3
After sorting array elements are -
1 2 3 5 6 7 10 31 43

```

**Program:** Write a program to implement counting sort in Java.

```
class CountingSort {
```

```

    int getMax(int[] a, int n) {
        int max = a[0];
        for(int i = 1; i < n; i++) {
            if(a[i] > max)
                max = a[i];
        }
        return max; //maximum element from the array
    }

```

```

    void countSort(int[] a, int n) // function to perform counting sort
    {
        int[] output = new int [n+1];
        int max = getMax(a, n);
        //int max = 42;
        int[] count = new int [max+1]; //create count array with size [max+1]

        for (int i = 0; i <= max; ++i)
        {
            count[i] = 0; // Initialize count array with all zeros
        }

        for (int i = 0; i < n; i++) // Store the count of each element
        {
            count[a[i]]++;
        }

        for(int i = 1; i <= max; i++)
            count[i] += count[i-1]; //find cumulative frequency
    }

```

```
/* This loop will find the index of each element of the original array in
```

```
count array, and
```

```
place the elements in output array*/
```

```
for (int i = n - 1; i >= 0; i--) {  
    output[count[a[i]] - 1] = a[i];  
    count[a[i]]--; // decrease count for same numbers  
}
```

```
for(int i = 0; i < n; i++) {  
    a[i] = output[i]; //store the sorted elements into main array  
}
```

```
/* Function to print the array elements */
```

```
void printArray(int a[], int n)  
{  
    int i;  
    for (i = 0; i < n; i++)  
        System.out.print(a[i] + " ");  
}
```

```
public static void main(String args[])  
{  
    int a[] = { 11, 30, 24, 7, 31, 16, 39, 41 };  
    int n = a.length;  
    CountingSort c1 = new CountingSort();  
    System.out.println("\nBefore sorting array elements are - ");  
    c1.printArray(a, n);  
    c1.countSort(a,n);  
    System.out.println("\nAfter sorting array elements are - ");  
    c1.printArray(a, n);  
    System.out.println();  
}
```

### Output

```
D:\JTP>javac CountingSort.java  
D:\JTP>java CountingSort  
Before sorting array elements are -  
11 30 24 7 31 16 39 41  
After sorting array elements are -  
7 11 16 24 30 31 39 41
```

**Program:** Write a program to implement counting sort in PHP.

```
<?php
```

```

function getMax($a, $n) {
    $max = $a[0];
    for($i = 1; $i < $n; $i++) {
        if($a[$i] > $max)
            $max = $a[$i];
    }
    return $max; //maximum element from the array
}

function countSort(&$a, $n) // function to perform counting sort
{
    $LeftArray = array($n + 1);
    $max = getMax($a, $n);
    $count = array($max + 1); //create count array with size [max+1]

    for ($i = 0; $i <= $max; ++$i)
    {
        $count[$i] = 0; // Initialize count array with all zeros
    }

    for ($i = 0; $i < $n; $i++) // Store the count of each element
    {
        $count[$a[$i]]++;
    }

    for($i = 1; $i <=$max; $i++)
        $count[$i] += $count[$i-1]; //find cumulative frequency

    /* This loop will find the index of each element of the original array in
    count array, and
    place the elements in output array*/
    for ($i = $n - 1; $i >= 0; $i--) {
        $output[$count[$a[$i]] - 1] = $a[$i];
        $count[$a[$i]]--; // decrease count for same numbers
    }

    for($i = 0; $i <$n; $i++) {
        $a[$i] = $output[$i]; //store the sorted elements into main array
    }
}

/* Function to print the array elements */
function printArray($a, $n)

```

```

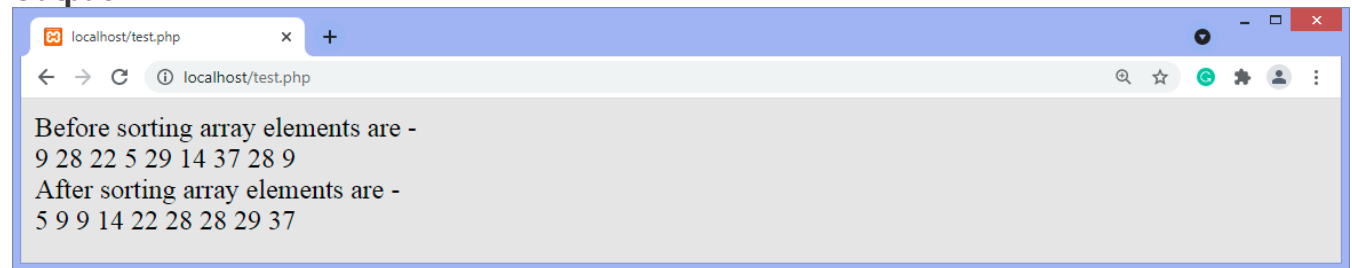
{
    for($i = 0; $i < $n; $i++)
    {
        print_r($a[$i]);
        echo " ";
    }
}

$a = array( 9, 28, 22, 5, 29, 14, 37, 28, 9 );
$n = count($a);
echo "Before sorting array elements are - <br>";
printArray($a, $n);
countSort($a,$n);
echo "<br> After sorting array elements are - <br>";
printArray($a, $n);

```

?>

## Output



So, that's all about the article. Hope the article will be helpful and informative to you. This article was not only limited to the algorithm. We have also discussed counting sort complexity, working, and implementation in different programming languages.

## Heap Sort Algorithm

In this article, we will discuss the Heapsort Algorithm. Heap sort processes the elements by creating the min-heap or max-heap using the elements of the given array. Min-heap or max-heap represents the ordering of array in which the root element represents the minimum or maximum element of the array.

Heap sort basically recursively performs two main operations -

Build a heap H, using the elements of array.

Repeatedly delete the root element of the heap formed in 1<sup>st</sup> phase.

Before knowing more about the heap sort, let's first see a brief description of **Heap**.

### What is a heap?

A heap is a complete binary tree, and the binary tree is a tree in which the node can have the utmost two children. A complete binary tree is a binary tree in which all the levels except the last level, i.e., leaf node, should be completely filled, and all the nodes should be left-justified.

### What is heap sort?



Heapsort is a popular and efficient sorting algorithm. The concept of heap sort is to eliminate the elements one by one from the heap part of the list, and then insert them into the sorted part of the list.

Heapsort is the in-place sorting algorithm.

Now, let's see the algorithm of heap sort.

## Algorithm

```
HeapSort(arr)
BuildMaxHeap(arr)
for i = length(arr) to 2
    swap arr[1] with arr[i]
    heap_size[arr] = heap_size[arr] - 1
    MaxHeapify(arr,1)
End
BuildMaxHeap(arr)
BuildMaxHeap(arr)
    heap_size(arr) = length(arr)
    for i = length(arr)/2 to 1
        MaxHeapify(arr,i)
    End
MaxHeapify(arr,i)
MaxHeapify(arr,i)
    L = left(i)
    R = right(i)
    if L < heap_size[arr] and arr[L] > arr[i]
        largest = L
    else
        largest = i
    if R < heap_size[arr] and arr[R] > arr[largest]
        largest = R
    if largest != i
        swap arr[i] with arr[largest]
        MaxHeapify(arr,largest)
    End
```

## Working of Heap sort Algorithm

Now, let's see the working of the Heapsort Algorithm.

In heap sort, basically, there are two phases involved in the sorting of elements. By using the heap sort algorithm, they are as follows -

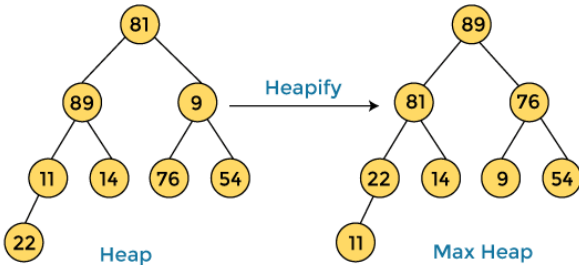
The first step includes the creation of a heap by adjusting the elements of the array.

After the creation of heap, now remove the root element of the heap repeatedly by shifting it to the end of the array, and then store the heap structure with the remaining elements.

Now let's see the working of heap sort in detail by using an example. To understand it more clearly, let's take an unsorted array and try to sort it using heap sort. It will make the explanation clearer and easier.

|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| 81 | 89 | 9 | 11 | 14 | 76 | 54 | 22 |
|----|----|---|----|----|----|----|----|

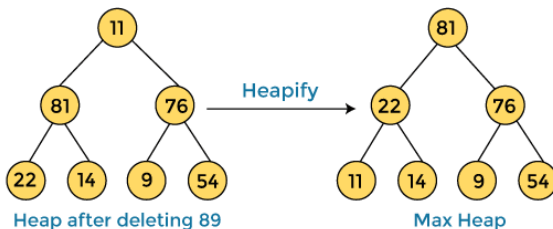
First, we have to construct a heap from the given array and convert it into max heap.



After converting the given heap into max heap, the array elements are -

|    |    |    |    |    |   |    |    |
|----|----|----|----|----|---|----|----|
| 89 | 81 | 76 | 22 | 14 | 9 | 54 | 11 |
|----|----|----|----|----|---|----|----|

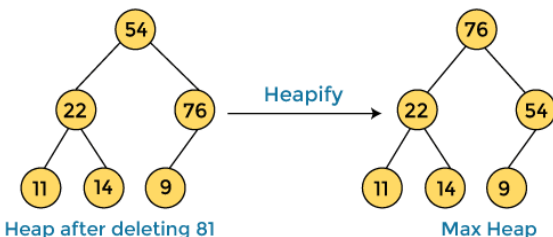
Next, we have to delete the root element (**89**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**11**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **89** with **11**, and converting the heap into max-heap, the elements of array are -

|    |    |    |    |    |   |    |    |
|----|----|----|----|----|---|----|----|
| 81 | 22 | 76 | 11 | 14 | 9 | 54 | 89 |
|----|----|----|----|----|---|----|----|

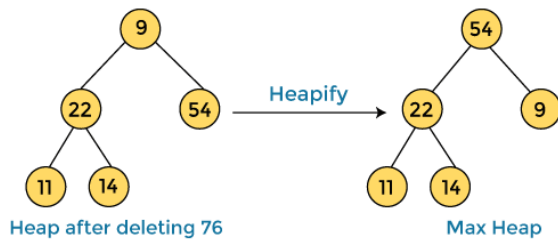
In the next step, again, we have to delete the root element (**81**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**54**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **81** with **54** and converting the heap into max-heap, the elements of array are -

|    |    |    |    |    |   |    |    |
|----|----|----|----|----|---|----|----|
| 76 | 22 | 54 | 11 | 14 | 9 | 81 | 89 |
|----|----|----|----|----|---|----|----|

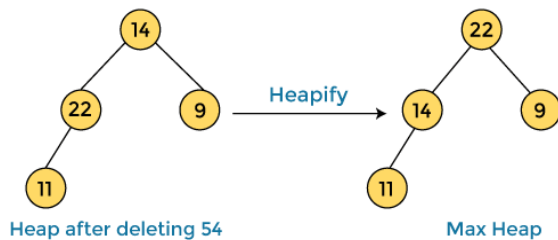
In the next step, we have to delete the root element (**76**) from the max heap again. To delete this node, we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **76** with **9** and converting the heap into max-heap, the elements of array are -

|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| 54 | 22 | 9 | 11 | 14 | 76 | 81 | 89 |
|----|----|---|----|----|----|----|----|

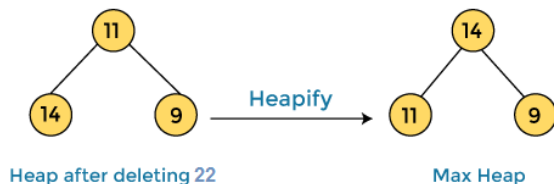
In the next step, again we have to delete the root element (**54**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**14**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **54** with **14** and converting the heap into max-heap, the elements of array are -

|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| 22 | 14 | 9 | 11 | 54 | 76 | 81 | 89 |
|----|----|---|----|----|----|----|----|

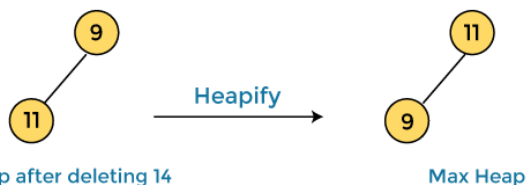
In the next step, again we have to delete the root element (**22**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**11**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **22** with **11** and converting the heap into max-heap, the elements of array are -

|    |    |   |    |    |    |    |    |
|----|----|---|----|----|----|----|----|
| 14 | 11 | 9 | 22 | 54 | 76 | 81 | 89 |
|----|----|---|----|----|----|----|----|

In the next step, again we have to delete the root element (**14**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **14** with **9** and converting the heap into max-heap, the elements of array are -

|    |   |    |    |    |    |    |    |
|----|---|----|----|----|----|----|----|
| 11 | 9 | 14 | 22 | 54 | 76 | 81 | 89 |
|----|---|----|----|----|----|----|----|

In the next step, again we have to delete the root element (**11**) from the max heap. To delete this node, we have to swap it with the last node, i.e. (**9**). After deleting the root element, we again have to heapify it to convert it into max heap.



After swapping the array element **11** with **9**, the elements of array are -

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 22 | 54 | 76 | 81 | 89 |
|---|----|----|----|----|----|----|----|

Now, heap has only one element left. After deleting it, heap will be empty.



After completion of sorting, the array elements are -

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 9 | 11 | 14 | 22 | 54 | 76 | 81 | 89 |
|---|----|----|----|----|----|----|----|

Now, the array is completely sorted.

## Heap sort complexity

Now, let's see the time complexity of Heap sort in the best case, average case, and worst case. We will also see the space complexity of Heapsort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(n \log n)$   |
| Average Case | $O(n \log n)$   |
| Worst Case   | $O(n \log n)$   |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of heap sort is  **$O(n \log n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of heap sort is  **$O(n \log n)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of heap sort is  **$O(n \log n)$** . The time complexity of heap sort is  **$O(n \log n)$**  in all three cases (best case, average case, and worst case). The height of a complete binary tree having  $n$  elements is  **$\log n$** .

## 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(1)$ |
| Stable           | NO     |

The space complexity of Heap sort is  $O(1)$ .

## Implementation of Heapsort

Now, let's see the programs of Heap sort in different programming languages.

**Program:** Write a program to implement heap sort in C language.

```
#include <stdio.h>
```

```
/* function to heapify a subtree. Here 'i' is the  
index of root node in array a[], and 'n' is the size of heap. */
```

```
void heapify(int a[], int n, int i)  
{  
    int largest = i; // Initialize largest as root  
    int left = 2 * i + 1; // left child  
    int right = 2 * i + 2; // right child  
    // If left child is larger than root  
    if (left < n && a[left] > a[largest])  
        largest = left;  
    // If right child is larger than root  
    if (right < n && a[right] > a[largest])  
        largest = right;  
    // If root is not largest  
    if (largest != i) {  
        // swap a[i] with a[largest]  
        int temp = a[i];  
        a[i] = a[largest];  
        a[largest] = temp;  
  
        heapify(a, n, largest);  
    }  
}
```

```
/*Function to implement the heap sort*/  
void heapSort(int a[], int n)
```

```

{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);
    // One by one extract an element from heap
    for (int i = n - 1; i >= 0; i--) {
        /* Move current root element to end*/
        // swap a[0] with a[i]
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}
/* function to print the array elements */
void printArr(int arr[], int n)
{
    for (int i = 0; i < n; ++i)
    {
        printf("%d", arr[i]);
        printf(" ");
    }
}

int main()
{
    int a[] = {48, 10, 23, 43, 28, 26, 1};
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArr(a, n);
    heapSort(a, n);
    printf("\nAfter sorting array elements are - \n");
    printArr(a, n);
    return 0;
}

```

### Output

```

Before sorting array elements are -
48 10 23 43 28 26 1
After sorting array elements are -
1 10 23 26 28 43 48

```

**Program:** Write a program to implement heap sort in C++.

#include <iostream>

using namespace std;

/\* function to heapify a subtree. Here 'i' is the

index of root node in array a[], and 'n' is the size of heap. \*/  
void heapify(int a[], int n, int i)

```
{  
    int largest = i; // Initialize largest as root  
    int left = 2 * i + 1; // left child  
    int right = 2 * i + 2; // right child  
    // If left child is larger than root  
    if (left < n && a[left] > a[largest])  
        largest = left;  
    // If right child is larger than root  
    if (right < n && a[right] > a[largest])  
        largest = right;  
    // If root is not largest  
    if (largest != i) {  
        // swap a[i] with a[largest]  
        int temp = a[i];  
        a[i] = a[largest];  
        a[largest] = temp;  
  
        heapify(a, n, largest);  
    }  
}
```

/\*Function to implement the heap sort\*/

```
void heapSort(int a[], int n)  
{
```

```
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(a, n, i);  
    // One by one extract an element from heap  
    for (int i = n - 1; i >= 0; i--) {  
        /* Move current root element to end */  
        // swap a[0] with a[i]  
        int temp = a[0];  
        a[0] = a[i];  
        a[i] = temp;  
  
        heapify(a, i, 0);  
    }  
}
```

/\* function to print the array elements \*/

```
void printArr(int a[], int n)  
{  
    for (int i = 0; i < n; ++i)  
    {
```

```

        cout<<a[i]<<" ";
    }

}

int main()
{
    int a[] = {47, 9, 22, 42, 27, 25, 0};
    int n = sizeof(a) / sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    printArr(a, n);
    heapSort(a, n);
    cout<<"\nAfter sorting array elements are - \n";
    printArr(a, n);
    return 0;
}

```

### Output

```

Before sorting array elements are -
47 9 22 42 27 25 0
After sorting array elements are -
0 9 22 25 27 42 47

```

**Program:** Write a program to implement heap sort in C#.

```

using System;
class HeapSort {
/* function to heapify a subtree. Here 'i' is the
index of root node in array a[], and 'n' is the size of heap. */
static void heapify(int[] a, int n, int i)
{
    int largest = i; // Initialize largest as root
    int left = 2 * i + 1; // left child
    int right = 2 * i + 2; // right child
    // If left child is larger than root
    if (left < n && a[left] > a[largest])
        largest = left;
    // If right child is larger than root
    if (right < n && a[right] > a[largest])
        largest = right;
    // If root is not largest
    if (largest != i) {
        // swap a[i] with a[largest]
        int temp = a[i];
        a[i] = a[largest];
        a[largest] = temp;

        heapify(a, n, largest);
    }
}
}

```



```

    }
}
/*Function to implement the heap sort*/
static void heapSort(int[] a, int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);

    // One by one extract an element from heap
    for (int i = n - 1; i >= 0; i--) {
        /* Move current root element to end*/
        // swap a[0] with a[i]
        int temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}
/* function to print the array elements */
static void printArr(int[] a, int n)
{
    for (int i = 0; i < n; ++i)
        Console.Write(a[i] + " ");
}
static void Main()
{
    int[] a = {46, 8, 21, 41, 26, 24, -1};
    int n = a.Length;
    Console.WriteLine("Before sorting array elements are - \n");
    printArr(a, n);
    heapSort(a, n);
    Console.WriteLine("\nAfter sorting array elements are - \n");
    printArr(a, n);
}
}

```

### Output

```

Before sorting array elements are -
46 8 21 41 26 24 -1
After sorting array elements are -
-1 8 21 24 26 41 46

```

**Program:** Write a program to implement heap sort in Java.

```

class HeapSort
{

```

/\* function to heapify a subtree. Here 'i' is the index of root node in array a[], and 'n' is the size of heap. \*/  
static void heapify(int a[], int n, int i)

```
{  
    int largest = i; // Initialize largest as root  
    int left = 2 * i + 1; // left child  
    int right = 2 * i + 2; // right child  
    // If left child is larger than root  
    if (left < n && a[left] > a[largest])  
        largest = left;  
    // If right child is larger than root  
    if (right < n && a[right] > a[largest])  
        largest = right;  
    // If root is not largest  
    if (largest != i) {  
        // swap a[i] with a[largest]  
        int temp = a[i];  
        a[i] = a[largest];  
        a[largest] = temp;  
  
        heapify(a, n, largest);  
    }  
}
```

/\*Function to implement the heap sort\*/

static void heapSort(int a[], int n)

```
{  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(a, n, i);  
  
    // One by one extract an element from heap  
    for (int i = n - 1; i >= 0; i--) {  
        /* Move current root element to end*/  
        // swap a[0] with a[i]  
        int temp = a[0];  
        a[0] = a[i];  
        a[i] = temp;  
  
        heapify(a, i, 0);  
    }  
}
```

/\* function to print the array elements \*/

static void printArr(int a[], int n)

```
{  
    for (int i = 0; i < n; ++i)
```

```

        System.out.print(a[i] + " ");
    }
    public static void main(String args[])
    {
        int a[] = {45, 7, 20, 40, 25, 23, -2};
        int n = a.length;
        System.out.print("Before sorting array elements are - \n");
        printArr(a, n);
        heapSort(a, n);
        System.out.print("\nAfter sorting array elements are - \n");
        printArr(a, n);
    }
}

```

### Output

```

D:\JTP>javac HeapSort.java
D:\JTP>java HeapSort
Before sorting array elements are -
45 7 20 40 25 23 -2
After sorting array elements are -
-2 7 20 23 25 40 45
D:\JTP>

```

So, that's all about the article. Hope the article will be helpful and informative to you.

## Insertion Sort Algorithm

In this article, we will discuss the Insertion sort Algorithm. The working procedure of insertion sort is also simple. This article will be very helpful and interesting to students as they might face insertion sort as a question in their examinations. So, it is important to discuss the topic. Insertion sort works similar to the sorting of playing cards in hands. It is assumed that the first card is already sorted in the card game, and then we select an unsorted card. If the selected unsorted card is greater than the first card, it will be placed at the right side; otherwise, it will be placed at the left side. Similarly, all unsorted cards are taken and put in their exact place. The same approach is applied in insertion sort. The idea behind the insertion sort is that first take one element, iterate it through the sorted array. Although it is simple to use, it is not appropriate for large data sets as the time complexity of insertion sort in the average case and worst case is  $O(n^2)$ , where  $n$  is the number of items. Insertion sort is less efficient than the other sorting algorithms like heap sort, quick sort, merge sort, etc.

Insertion sort has various advantages such as -

- Simple implementation

- Efficient for small data sets

- Adaptive, i.e., it is appropriate for data sets that are already substantially sorted.

Now, let's see the algorithm of insertion sort.

## Algorithm

The simple steps of achieving the insertion sort are listed as follows -

**Step 1** - If the element is the first element, assume that it is already sorted. Return 1.

**Step 2** - Pick the next element, and store it separately in a **key**.

**Step 3** - Now, compare the **key** with all elements in the sorted array.

**Step 4** - If the element in the sorted array is smaller than the current element, then move to the next element. Else, shift greater elements in the array towards the right.

**Step 5** - Insert the value.

**Step 6** - Repeat until the array is sorted.

## Working of Insertion sort Algorithm

Now, let's see the working of the insertion sort Algorithm.

To understand the working of the insertion sort algorithm, let's take an unsorted array. It will be easier to understand the insertion sort via an example.

Let the elements of array are -

|    |    |    |   |    |    |
|----|----|----|---|----|----|
| 12 | 31 | 25 | 8 | 32 | 17 |
|----|----|----|---|----|----|

Initially, the first two elements are compared in insertion sort.

|    |    |    |   |    |    |
|----|----|----|---|----|----|
| 12 | 31 | 25 | 8 | 32 | 17 |
|----|----|----|---|----|----|

Here, 31 is greater than 12. That means both elements are already in ascending order. So, for now, 12 is stored in a sorted sub-array.

|    |    |    |   |    |    |
|----|----|----|---|----|----|
| 12 | 31 | 25 | 8 | 32 | 17 |
|----|----|----|---|----|----|

Now, move to the next two elements and compare them.

|    |    |    |   |    |    |
|----|----|----|---|----|----|
| 12 | 31 | 25 | 8 | 32 | 17 |
|----|----|----|---|----|----|

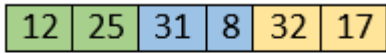
|    |    |    |   |    |    |
|----|----|----|---|----|----|
| 12 | 31 | 25 | 8 | 32 | 17 |
|----|----|----|---|----|----|

Here, 25 is smaller than 31. So, 31 is not at correct position. Now, swap 31 with 25. Along with swapping, insertion sort will also check it with all elements in the sorted array.

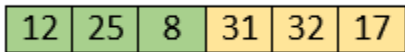
For now, the sorted array has only one element, i.e. 12. So, 25 is greater than 12. Hence, the sorted array remains sorted after swapping.

|    |    |    |   |    |    |
|----|----|----|---|----|----|
| 12 | 25 | 31 | 8 | 32 | 17 |
|----|----|----|---|----|----|

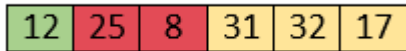
Now, two elements in the sorted array are 12 and 25. Move forward to the next elements that are 31 and 8.



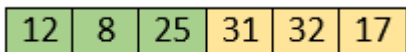
Both 31 and 8 are not sorted. So, swap them.



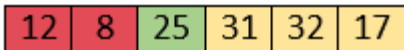
After swapping, elements 25 and 8 are unsorted.



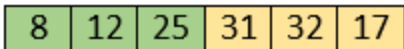
So, swap them.



Now, elements 12 and 8 are unsorted.



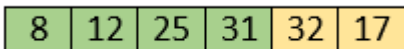
So, swap them too.



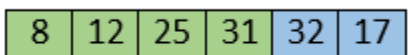
Now, the sorted array has three items that are 8, 12 and 25. Move to the next items that are 31 and 32.



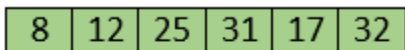
Hence, they are already sorted. Now, the sorted array includes 8, 12, 25 and 31.



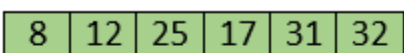
Move to the next elements that are 32 and 17.



17 is smaller than 32. So, swap them.



Swapping makes 31 and 17 unsorted. So, swap them too.



Now, swapping makes 25 and 17 unsorted. So, perform swapping again.

|   |    |    |    |    |    |
|---|----|----|----|----|----|
| 8 | 12 | 17 | 25 | 31 | 32 |
|---|----|----|----|----|----|

Now, the array is completely sorted.

## Insertion sort complexity

Now, let's see the time complexity of insertion sort in best case, average case, and in worst case. We will also see the space complexity of insertion sort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(n)$          |
| Average Case | $O(n^2)$        |
| Worst Case   | $O(n^2)$        |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of insertion sort is  **$O(n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of insertion sort is  **$O(n^2)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of insertion sort is  **$O(n^2)$** .

### 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(1)$ |
| Stable           | YES    |

The space complexity of insertion sort is  $O(1)$ . It is because, in insertion sort, an extra variable is required for swapping.

## Implementation of insertion sort

Now, let's see the programs of insertion sort in different programming languages.

**Program:** Write a program to implement insertion sort in C language.

```
#include <stdio.h>
```

```
void insert(int a[], int n) /* function to sort an aay with insertion sort */
{
    int i, j, temp;
    for (i = 1; i < n; i++) {
        temp = a[i];
        j = i - 1;

        while(j >= 0 && temp <= a[j]) /* Move the elements greater than temp to one position ahead from their current position*/
        {
```

```

        a[j+1] = a[j];
        j = j-1;
    }
    a[j+1] = temp;
}
}

void printArr(int a[], int n) /* function to print the array */
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}

int main()
{
    int a[] = { 12, 31, 25, 8, 32, 17 };
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArr(a, n);
    insert(a, n);
    printf("\nAfter sorting array elements are - \n");
    printArr(a, n);

    return 0;
}

```

#### Output:

```

Before sorting array elements are -
12 31 25 8 32 17
After sorting array elements are -
8 12 17 25 31 32

```

**Program:** Write a program to implement insertion sort in python.

```

def insertionSort(a): # Function to implement insertion sort
    for i in range(1, len(a)):
        temp = a[i]

        # Move the elements greater than temp to one position
        # ahead from their current position
        j = i-1
        while j >= 0 and temp < a[j]:
            a[j + 1] = a[j]
            j = j-1
        a[j + 1] = temp

```

```
def printArr(a): # function to print the array
```

```
    for i in range(len(a)):
        print (a[i], end = " ")
```

```
a = [70, 15, 2, 51, 60]
print("Before sorting array elements are - ")
printArr(a)
insertionSort(a)
print("\nAfter sorting array elements are - ")
printArr(a)
```

**Output:**

```
Before sorting array elements are -
70 15 2 51 60
After sorting array elements are -
2 15 51 60 70
```

**Program:** Write a program to implement insertion sort in C++ language.

```
#include <iostream>
```

```
using namespace std;
```

```
void insert(int a[], int n) /* function to sort an aay with insertion sort */
```

```
{
    int i, j, temp;
    for (i = 1; i < n; i++) {
        temp = a[i];
        j = i - 1;

        while(j >= 0 && temp <= a[j]) /* Move the elements greater than temp to one position ahead from their current position */
        {
            a[j+1] = a[j];
            j = j-1;
        }
        a[j+1] = temp;
    }
}
```

```
void printArr(int a[], int n) /* function to print the array */
```

```
{
    int i;
    for (i = 0; i < n; i++)
        cout << a[i] << " ";
}
```



```

int main()
{
    int a[] = { 89, 45, 35, 8, 12, 2 };
    int n = sizeof(a) / sizeof(a[0]);
    cout<<"Before sorting array elements are - "<<endl;
    printArr(a, n);
    insert(a, n);
    cout<<"\nAfter sorting array elements are - "<<endl;
    printArr(a, n);

    return 0;
}

```

**Output:**

```

Before sorting array elements are -
89 45 35 8 12 2
After sorting array elements are -
2 8 12 35 45 89

```

**Program:** Write a program to implement insertion sort in C# language.

```

using System;
class Insertion {
static void insert(int[] a) /* function to sort an aay with insertion sort */
{
    int i, j, temp;
    int n = a.Length;
    for (i = 1; i < n; i++) {
        temp = a[i];
        j = i - 1;

        while(j>=0 && temp <= a[j]) /* Move the elements greater than temp to one position ahead from their current position*/
        {
            a[j+1] = a[j];
            j = j-1;
        }
        a[j+1] = temp;
    }
}

static void printArr(int[] a) /* function to print the array */
{
    int i;
    int n = a.Length;
    for (i = 0; i < n; i++)
        Console.Write(a[i] + " ");
}
}

```

```

}
static void Main() {
    int[] a = { 98, 54, 53, 18, 21, 12 };
    Console.WriteLine("Before sorting array elements are - \n");
    printArr(a);
    insert(a);
    Console.WriteLine("\nAfter sorting array elements are - \n");
    printArr(a);
}
}

```

### Output:

```

Before sorting array elements are -
98 54 53 18 21 12
After sorting array elements are -
12 18 21 53 54 98

```

**Program:** Write a program to implement insertion sort in Java.

```

public class Insert
{
    void insert(int a[]) /* function to sort an aay with insertion sort */
    {
        int i, j, temp;
        int n = a.length;
        for (i = 1; i < n; i++) {
            temp = a[i];
            j = i - 1;

            while(j >= 0 && temp <= a[j]) /* Move the elements greater than temp to one position ahe
ad from their current position*/
            {
                a[j+1] = a[j];
                j = j-1;
            }
            a[j+1] = temp;
        }
    }

    void printArr(int a[]) /* function to print the array */
    {
        int i;
        int n = a.length;
        for (i = 0; i < n; i++)
            System.out.print(a[i] + " ");
    }

    public static void main(String[] args) {

```

```

int a[] = { 92, 50, 5, 20, 11, 22 };
Insert i1 = new Insert();
System.out.println("\nBefore sorting array elements are - ");
i1.printArr(a);
i1.insert(a);
System.out.println("\n\nAfter sorting array elements are - ");
i1.printArr(a);
System.out.println();
}
}

```

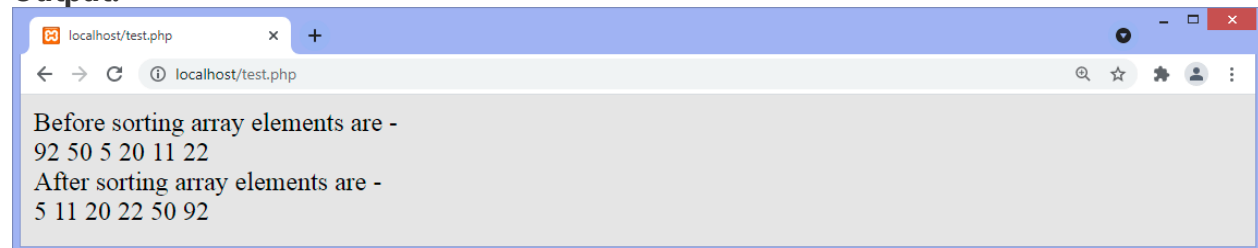
**Output:**

```

D:\JTP>javac Insert.java
D:\JTP>java Insert
Before sorting array elements are -
92 50 5 20 11 22
After sorting array elements are -
5 11 20 22 50 92
D:\JTP>_

```

**Output:**



So, that's all about the article. Hope the article will be helpful and informative to you.

This article was not only limited to the algorithm. We have also discussed the algorithm's complexity, working, and implementation in different programming languages.

## Merge Sort Algorithm

In this article, we will discuss the merge sort Algorithm. Merge sort is the sorting technique that follows the divide and conquer approach. This article will be very helpful and interesting to students as they might face merge sort as a question in their examinations. In coding or technical interviews for software engineers, sorting algorithms are widely asked. So, it is important to discuss the topic.

Merge sort is similar to the quick sort algorithm as it uses the divide and conquer approach to sort the elements. It is one of the most popular and efficient sorting algorithm. It divides the given list into two equal halves, calls itself for the two halves and then merges the two sorted halves. We have to define the **merge()** function to perform the merging.

The sub-lists are divided again and again into halves until the list cannot be divided further. Then we combine the pair of one element lists into two-element lists, sorting them in the process. The sorted two-element pairs is merged into the four-element lists, and so on until we get the sorted list.

Now, let's see the algorithm of merge sort.

## Algorithm

In the following algorithm, **arr** is the given array, **beg** is the starting element, and **end** is the last element of the array.

MERGE\_SORT(arr, beg, end)

```
if beg < end
set mid = (beg + end)/2
MERGE_SORT(arr, beg, mid)
MERGE_SORT(arr, mid + 1, end)
MERGE (arr, beg, mid, end)
end of if
```

END MERGE\_SORT

The important part of the merge sort is the **MERGE** function. This function performs the merging of two sorted sub-arrays that are **A[beg...mid]** and **A[mid+1...end]**, to build one sorted array **A[beg...end]**. So, the inputs of the **MERGE** function are **A[], beg, mid, and end**. The implementation of the **MERGE** function is given as follows -

```
/* Function to merge the subarrays of a[] */
void merge(int a[], int beg, int mid, int end)
{
    int i, j, k;
    int n1 = mid - beg + 1;
    int n2 = end - mid;

    int LeftArray[n1], RightArray[n2]; //temporary arrays

    /* copy data to temp arrays */
    for (int i = 0; i < n1; i++)
        LeftArray[i] = a[beg + i];
    for (int j = 0; j < n2; j++)
        RightArray[j] = a[mid + 1 + j];

    i = 0, /* initial index of first sub-array */
    j = 0, /* initial index of second sub-array */
    k = beg; /* initial index of merged sub-array */

    while (i < n1 && j < n2)
    {
        if(LeftArray[i] <= RightArray[j])
        {
            a[k] = LeftArray[i];
            i++;
        }
    }
}
```

```

    }
    else
    {
        a[k] = RightArray[j];
        j++;
    }
    k++;
}
while (i < n1)
{
    a[k] = LeftArray[i];
    i++;
    k++;
}

while (j < n2)
{
    a[k] = RightArray[j];
    j++;
    k++;
}
}

```

## Working of Merge sort Algorithm

Now, let's see the working of merge sort Algorithm.

To understand the working of the merge sort algorithm, let's take an unsorted array. It will be easier to understand the merge sort via an example.

Let the elements of array are -

|    |    |    |   |    |    |    |    |
|----|----|----|---|----|----|----|----|
| 12 | 31 | 25 | 8 | 32 | 17 | 40 | 42 |
|----|----|----|---|----|----|----|----|

According to the merge sort, first divide the given array into two equal halves. Merge sort keeps dividing the list into equal parts until it cannot be further divided.

As there are eight elements in the given array, so it is divided into two arrays of size 4.

|        |    |    |    |   |    |    |    |    |
|--------|----|----|----|---|----|----|----|----|
| divide | 12 | 31 | 25 | 8 | 32 | 17 | 40 | 42 |
|--------|----|----|----|---|----|----|----|----|

Now, again divide these two arrays into halves. As they are of size 4, so divide them into new arrays of size 2.

|        |    |    |    |   |    |    |    |    |
|--------|----|----|----|---|----|----|----|----|
| divide | 12 | 31 | 25 | 8 | 32 | 17 | 40 | 42 |
|--------|----|----|----|---|----|----|----|----|

Now, again divide these arrays to get the atomic value that cannot be further divided.

|        |    |    |    |   |    |    |    |    |
|--------|----|----|----|---|----|----|----|----|
| divide | 12 | 31 | 25 | 8 | 32 | 17 | 40 | 42 |
|--------|----|----|----|---|----|----|----|----|

Now, combine them in the same manner they were broken.

In combining, first compare the element of each array and then combine them into another array in sorted order.

So, first compare 12 and 31, both are in sorted positions. Then compare 25 and 8, and in the list of two values, put 8 first followed by 25. Then compare 32 and 17, sort them and put 17 first followed by 32. After that, compare 40 and 42, and place them sequentially.

|       |    |    |   |    |    |    |    |    |
|-------|----|----|---|----|----|----|----|----|
| merge | 12 | 31 | 8 | 25 | 17 | 32 | 40 | 42 |
|-------|----|----|---|----|----|----|----|----|

In the next iteration of combining, now compare the arrays with two data values and merge them into an array of found values in sorted order.

|       |   |    |    |    |    |    |    |    |
|-------|---|----|----|----|----|----|----|----|
| merge | 8 | 12 | 25 | 31 | 17 | 32 | 40 | 42 |
|-------|---|----|----|----|----|----|----|----|

Now, there is a final merging of the arrays. After the final merging of above arrays, the array will look like -

|   |    |    |    |    |    |    |    |
|---|----|----|----|----|----|----|----|
| 8 | 12 | 17 | 25 | 31 | 32 | 40 | 42 |
|---|----|----|----|----|----|----|----|

Now, the array is completely sorted.

## Merge sort complexity

Now, let's see the time complexity of merge sort in best case, average case, and in worst case. We will also see the space complexity of the merge sort.

### 1. Time Complexity

| Case         | Time Complexity     |
|--------------|---------------------|
| Best Case    | $O(n \cdot \log n)$ |
| Average Case | $O(n \cdot \log n)$ |
| Worst Case   | $O(n \cdot \log n)$ |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of merge sort is  **$O(n \cdot \log n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of merge sort is  **$O(n \cdot \log n)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of merge sort is  **$O(n \cdot \log n)$** .

### 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(n)$ |
|------------------|--------|

|        |     |
|--------|-----|
| Stable | YES |
|--------|-----|

The space complexity of merge sort is  $O(n)$ . It is because, in merge sort, an extra variable is required for swapping.

## Implementation of merge sort

Now, let's see the programs of merge sort in different programming languages.

**Program:** Write a program to implement merge sort in C language.

```
#include <stdio.h>
```

```
/* Function to merge the subarrays of a[] */
void merge(int a[], int beg, int mid, int end)
{
    int i, j, k;
    int n1 = mid - beg + 1;
    int n2 = end - mid;

    int LeftArray[n1], RightArray[n2]; //temporary arrays

    /* copy data to temp arrays */
    for (int i = 0; i < n1; i++)
        LeftArray[i] = a[beg + i];
    for (int j = 0; j < n2; j++)
        RightArray[j] = a[mid + 1 + j];

    i = 0; /* initial index of first sub-array */
    j = 0; /* initial index of second sub-array */
    k = beg; /* initial index of merged sub-array */

    while (i < n1 && j < n2)
    {
        if(LeftArray[i] <= RightArray[j])
        {
            a[k] = LeftArray[i];
            i++;
        }
        else
        {
            a[k] = RightArray[j];
            j++;
        }
        k++;
    }
    while (i < n1)
```

```

    {
        a[k] = LeftArray[i];
        i++;
        k++;
    }

    while (j < n2)
    {
        a[k] = RightArray[j];
        j++;
        k++;
    }
}

void mergeSort(int a[], int beg, int end)
{
    if (beg < end)
    {
        int mid = (beg + end) / 2;
        mergeSort(a, beg, mid);
        mergeSort(a, mid + 1, end);
        merge(a, beg, mid, end);
    }
}

/* Function to print the array */
void printArray(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int main()
{
    int a[] = { 12, 31, 25, 8, 32, 17, 40, 42 };
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArray(a, n);
    mergeSort(a, 0, n - 1);
    printf("After sorting array elements are - \n");
    printArray(a, n);
    return 0;
}

```



```
}
```

### Output:

```
Before sorting array elements are -  
12 31 25 8 32 17 40 42  
After sorting array elements are -  
8 12 17 25 31 32 40 42
```

**Program:** Write a program to implement merge sort in C++ language.

```
#include <iostream>
```

```
using namespace std;
```

```
/* Function to merge the subarrays of a[] */
```

```
void merge(int a[], int beg, int mid, int end)
```

```
{
```

```
    int i, j, k;
```

```
    int n1 = mid - beg + 1;
```

```
    int n2 = end - mid;
```

```
    int LeftArray[n1], RightArray[n2]; //temporary arrays
```

```
    /* copy data to temp arrays */
```

```
    for (int i = 0; i < n1; i++)
```

```
        LeftArray[i] = a[beg + i];
```

```
    for (int j = 0; j < n2; j++)
```

```
        RightArray[j] = a[mid + 1 + j];
```

```
    i = 0; /* initial index of first sub-array */
```

```
    j = 0; /* initial index of second sub-array */
```

```
    k = beg; /* initial index of merged sub-array */
```

```
    while (i < n1 && j < n2)
```

```
    {
```

```
        if(LeftArray[i] <= RightArray[j])
```

```
        {
```

```
            a[k] = LeftArray[i];
```

```
            i++;
```

```
        }
```

```
        else
```

```
        {
```

```
            a[k] = RightArray[j];
```

```
            j++;
```

```
        }
```

```
        k++;
```

```
    }
```

```

while (i<n1)
{
    a[k] = LeftArray[i];
    i++;
    k++;
}

while (j<n2)
{
    a[k] = RightArray[j];
    j++;
    k++;
}
}

void mergeSort(int a[], int beg, int end)
{
    if (beg < end)
    {
        int mid = (beg + end) / 2;
        mergeSort(a, beg, mid);
        mergeSort(a, mid + 1, end);
        merge(a, beg, mid, end);
    }
}

/* Function to print the array */
void printArray(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        cout<<a[i]<<" ";
}

int main()
{
    int a[] = { 11, 30, 24, 7, 31, 16, 39, 41 };
    int n = sizeof(a) / sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    printArray(a, n);
    mergeSort(a, 0, n - 1);
    cout<<"\nAfter sorting array elements are - \n";
    printArray(a, n);
    return 0;
}

```

```
}
```

### Output:

```
Before sorting array elements are -  
11 30 24 7 31 16 39 41  
After sorting array elements are -  
7 11 16 24 30 31 39 41
```

**Program:** Write a program to implement merge sort in Java.

**class** Merge {

/\* Function to merge the subarrays of a[] \*/

**void** merge(**int** a[], **int** beg, **int** mid, **int** end)

{

**int** i, j, k;

**int** n1 = mid - beg + 1;

**int** n2 = end - mid;

/\* temporary Arrays \*/

**int** LeftArray[] = **new int**[n1];

**int** RightArray[] = **new int**[n2];

/\* copy data to temp arrays \*/

**for** (i = 0; i < n1; i++)

        LeftArray[i] = a[beg + i];

**for** (j = 0; j < n2; j++)

        RightArray[j] = a[mid + 1 + j];

    i = 0; /\* initial index of first sub-array \*/

    j = 0; /\* initial index of second sub-array \*/

    k = beg; /\* initial index of merged sub-array \*/

**while** (i < n1 && j < n2)

    {

**if**(LeftArray[i] <= RightArray[j])

        {

            a[k] = LeftArray[i];

            i++;

        }

**else**

        {

            a[k] = RightArray[j];

            j++;

        }

        k++;

    }

```

while (i < n1)
{
    a[k] = LeftArray[i];
    i++;
    k++;
}

while (j < n2)
{
    a[k] = RightArray[j];
    j++;
    k++;
}
}

void mergeSort(int a[], int beg, int end)
{
    if (beg < end)
    {
        int mid = (beg + end) / 2;
        mergeSort(a, beg, mid);
        mergeSort(a, mid + 1, end);
        merge(a, beg, mid, end);
    }
}

/* Function to print the array */
void printArray(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        System.out.print(a[i] + " ");
}

public static void main(String args[])
{
    int a[] = { 11, 30, 24, 7, 31, 16, 39, 41 };
    int n = a.length;
    Merge m1 = new Merge();
    System.out.println("\nBefore sorting array elements are - ");
    m1.printArray(a, n);
    m1.mergeSort(a, 0, n - 1);
    System.out.println("\nAfter sorting array elements are - ");
    m1.printArray(a, n);
}

```

```

        System.out.println("");
    }

}

```

### Output:

```

D:\JTP>javac Merge.java
D:\JTP>java Merge
Before sorting array elements are -
11 30 24 7 31 16 39 41
After sorting array elements are -
7 11 16 24 30 31 39 41

```

**Program:** Write a program to implement merge sort in C#.

```
using System;
```

```
class Merge {
```

```
/* Function to merge the subarrays of a[] */
```

```
static void merge(int[] a, int beg, int mid, int end)
```

```
{
```

```
    int i, j, k;
```

```
    int n1 = mid - beg + 1;
```

```
    int n2 = end - mid;
```

```
//temporary arrays
```

```
    int[] LeftArray = new int [n1];
```

```
    int[] RightArray = new int [n2];
```

```
/* copy data to temp arrays */
```

```
for (i = 0; i < n1; i++)
```

```
LeftArray[i] = a[beg + i];
```

```
for (j = 0; j < n2; j++)
```

```
RightArray[j] = a[mid + 1 + j];
```

```
i = 0; /* initial index of first sub-array */
```

```
j = 0; /* initial index of second sub-array */
```

```
k = beg; /* initial index of merged sub-array */
```

```
while (i < n1 && j < n2)
```

```
{
```

```
    if(LeftArray[i] <= RightArray[j])
```

```
    {
```

```
        a[k] = LeftArray[i];
```

```
        i++;
```

```
    }
```

```
else
```

```

        {
            a[k] = RightArray[j];
            j++;
        }
        k++;
    }
    while (i < n1)
    {
        a[k] = LeftArray[i];
        i++;
        k++;
    }

    while (j < n2)
    {
        a[k] = RightArray[j];
        j++;
        k++;
    }
}

static void mergeSort(int[] a, int beg, int end)
{
    if (beg < end)
    {
        int mid = (beg + end) / 2;
        mergeSort(a, beg, mid);
        mergeSort(a, mid + 1, end);
        merge(a, beg, mid, end);
    }
}

/* Function to print the array */
static void printArray(int[] a, int n)
{
    int i;
    for (i = 0; i < n; i++)
        Console.Write(a[i] + " ");
}

static void Main()
{
    int[] a = { 10, 29, 23, 6, 30, 15, 38, 40 };
    int n = a.Length;

```

```

    Console.WriteLine("Before sorting array elements are - ");
    printArray(a, n);
    mergeSort(a, 0, n - 1);
    Console.WriteLine("\nAfter sorting array elements are - ");
    printArray(a, n);
}

}

```

### Output:

```

Before sorting array elements are - 10 29 23 6 30 15 38 40
After sorting array elements are - 6 10 15 23 29 30 38 40

```

**Program:** Write a program to implement merge sort in PHP.

```
<?php
```

```

/* Function to merge the subarrays of a[] */
function merge(&$a, $beg, $mid, $end)
{
    $n1 = ($mid - $beg) + 1;
    $n2 = $end - $mid;

    /* temporary Arrays */
    $LeftArray = array($n1);
    $RightArray = array($n2);

    /* copy data to temp arrays */
    for ($i = 0; $i < $n1; $i++)
        $LeftArray[$i] = $a[$beg + $i];
    for ($j = 0; $j < $n2; $j++)
        $RightArray[$j] = $a[$mid + 1 + $j];

    $i = 0; /* initial index of first sub-array */
    $j = 0; /* initial index of second sub-array */
    $k = $beg; /* initial index of merged sub-array */

    while ($i < $n1 && $j < $n2)
    {
        if ($LeftArray[$i] <= $RightArray[$j])
        {
            $a[$k] = $LeftArray[$i];
            $i++;
        }
        else
        {

```

```

        $a[$k] = $RightArray[$j];
        $j++;
    }
    $k++;
}
while ($i < $n1)
{
    $a[$k] = $LeftArray[$i];
    $i++;
    $k++;
}

while ($j < $n2)
{
    $a[$k] = $RightArray[$j];
    $j++;
    $k++;
}
}

function mergeSort(&$a, $beg, $end)
{
    if ($beg < $end)
    {
        $mid = (int)(( $beg + $end) / 2);
        mergeSort($a, $beg, $mid);
        mergeSort($a, $mid + 1, $end);
        merge($a, $beg, $mid, $end);
    }
}

/* Function to print array elements */
function printArray($a, $n)
{
    for($i = 0; $i < $n; $i++)
    {
        print_r($a[$i]);
        echo " ";
    }
}

$a = array( 10, 29, 23, 6, 30, 15, 38, 40 );
$n = count($a);
echo "Before sorting array elements are - <br>";

```

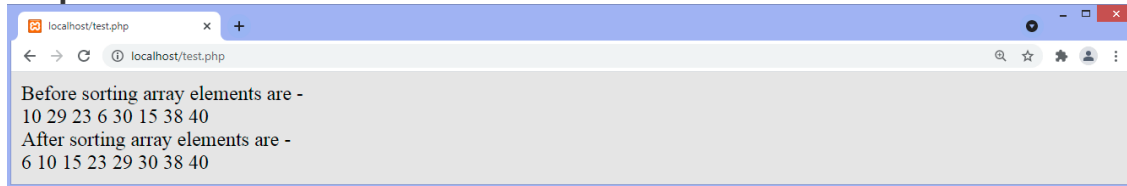


```

printArray($a, $n);
mergeSort($a, 0, $n - 1);
echo "<br> After sorting array elements are - <br>";
printArray($a, $n);
?>

```

### Output:



So, that's all about the article. Hope the article will be helpful and informative to you. This article was not only limited to the algorithm. We have also discussed the Merge sort complexity, working, and implementation in different programming languages.

## Quick Sort Algorithm

In this article, we will discuss the Quicksort Algorithm. The working procedure of Quicksort is also simple. This article will be very helpful and interesting to students as they might face quicksort as a question in their examinations. So, it is important to discuss the topic. Sorting is a way of arranging items in a systematic manner. Quicksort is the widely used sorting algorithm that makes  $n \log n$  comparisons in average case for sorting an array of  $n$  elements. It is a faster and highly efficient sorting algorithm. This algorithm follows the divide and conquer approach. Divide and conquer is a technique of breaking down the algorithms into subproblems, then solving the subproblems, and combining the results back together to solve the original problem.

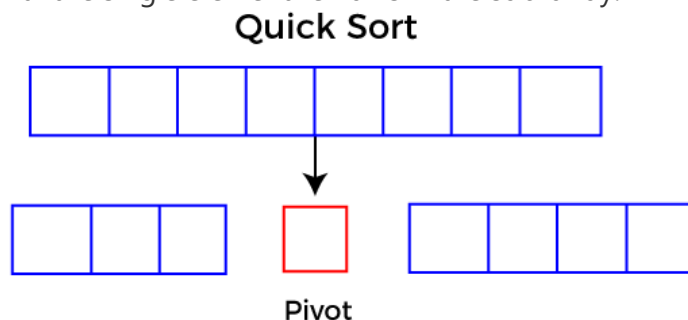
**Divide:** In Divide, first pick a pivot element. After that, partition or rearrange the array into two sub-arrays such that each element in the left sub-array is less than or equal to the pivot element and each element in the right sub-array is larger than the pivot element.

**Conquer:** Recursively, sort two subarrays with Quicksort.

**Combine:** Combine the already sorted array.

Quicksort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.

After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.



## Choosing the pivot

Picking a good pivot is necessary for the fast implementation of quicksort. However, it is typical to determine a good pivot. Some of the ways of choosing a pivot are as follows -

Pivot can be random, i.e. select the random pivot from the given array.

Pivot can either be the rightmost element or the leftmost element of the given array.

Select median as the pivot element.

## Algorithm

### Algorithm:

QUICKSORT (array A, start, end)

```
{
  1 if (start < end)
  2 {
  3 p = partition(A, start, end)
  4 QUICKSORT (A, start, p - 1)
  5 QUICKSORT (A, p + 1, end)
  6 }
}
```

### Partition Algorithm:

The partition algorithm rearranges the sub-arrays in a place.

PARTITION (array A, start, end)

```
{
  1 pivot = A[end]
  2 i = start - 1
  3 for j = start to end - 1 {
  4 do if (A[j] < pivot) {
  5 then i = i + 1
  6 swap A[i] with A[j]
  7 }}
  8 swap A[i + 1] with A[end]
  9 return i + 1
}
```

## Working of Quick Sort Algorithm

Now, let's see the working of the Quicksort Algorithm.

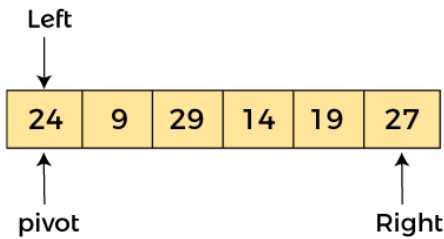
To understand the working of quick sort, let's take an unsorted array. It will make the concept more clear and understandable.

Let the elements of array are -

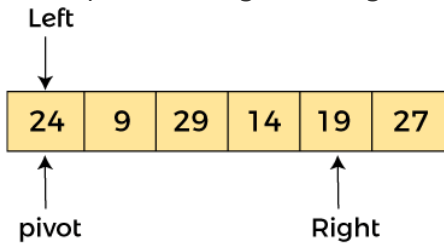
|    |   |    |    |    |    |
|----|---|----|----|----|----|
| 24 | 9 | 29 | 14 | 19 | 27 |
|----|---|----|----|----|----|

In the given array, we consider the leftmost element as pivot. So, in this case,  $a[\text{left}] = 24$ ,  $a[\text{right}] = 27$  and  $a[\text{pivot}] = 24$ .

Since, pivot is at left, so algorithm starts from right and move towards left.

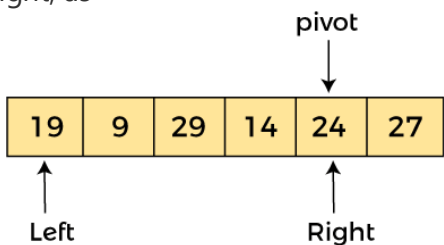


Now,  $a[\text{pivot}] < a[\text{right}]$ , so algorithm moves forward one position towards left, i.e. -



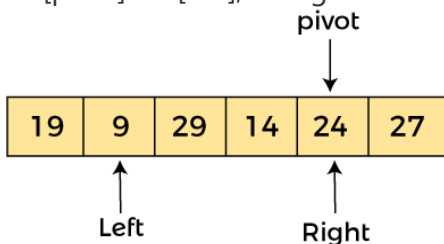
Now,  $a[\text{left}] = 24$ ,  $a[\text{right}] = 19$ , and  $a[\text{pivot}] = 24$ .

Because,  $a[\text{pivot}] > a[\text{right}]$ , so, algorithm will swap  $a[\text{pivot}]$  with  $a[\text{right}]$ , and pivot moves to right, as -

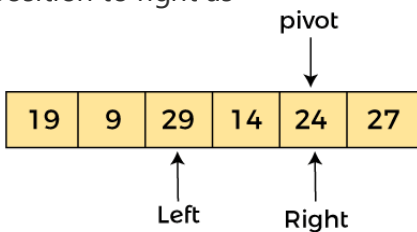


Now,  $a[\text{left}] = 19$ ,  $a[\text{right}] = 24$ , and  $a[\text{pivot}] = 24$ . Since, pivot is at right, so algorithm starts from left and moves to right.

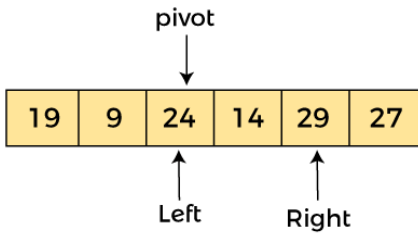
As  $a[\text{pivot}] > a[\text{left}]$ , so algorithm moves one position to right as -



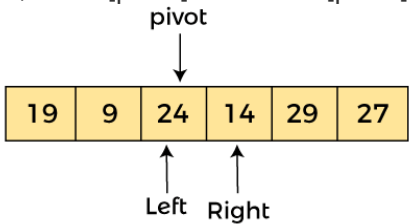
Now,  $a[\text{left}] = 9$ ,  $a[\text{right}] = 24$ , and  $a[\text{pivot}] = 24$ . As  $a[\text{pivot}] > a[\text{left}]$ , so algorithm moves one position to right as -



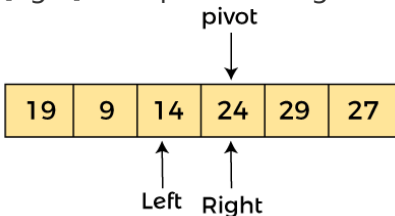
Now,  $a[\text{left}] = 29$ ,  $a[\text{right}] = 24$ , and  $a[\text{pivot}] = 24$ . As  $a[\text{pivot}] < a[\text{left}]$ , so, swap  $a[\text{pivot}]$  and  $a[\text{left}]$ , now pivot is at left, i.e. -



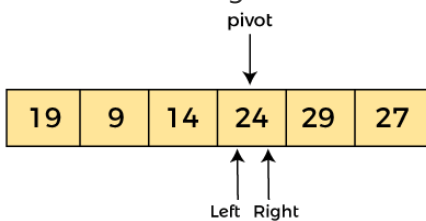
Since, pivot is at left, so algorithm starts from right, and move to left. Now,  $a[\text{left}] = 24$ ,  $a[\text{right}] = 29$ , and  $a[\text{pivot}] = 24$ . As  $a[\text{pivot}] < a[\text{right}]$ , so algorithm moves one position to left, as -



Now,  $a[\text{pivot}] = 24$ ,  $a[\text{left}] = 24$ , and  $a[\text{right}] = 14$ . As  $a[\text{pivot}] > a[\text{right}]$ , so, swap  $a[\text{pivot}]$  and  $a[\text{right}]$ , now pivot is at right, i.e. -



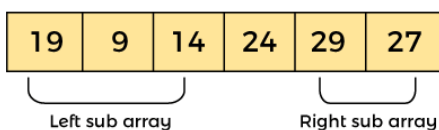
Now,  $a[\text{pivot}] = 24$ ,  $a[\text{left}] = 14$ , and  $a[\text{right}] = 24$ . Pivot is at right, so the algorithm starts from left and move to right.



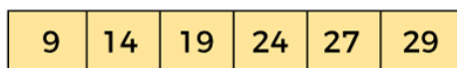
Now,  $a[\text{pivot}] = 24$ ,  $a[\text{left}] = 24$ , and  $a[\text{right}] = 24$ . So, pivot, left and right are pointing the same element. It represents the termination of procedure.

Element 24, which is the pivot element is placed at its exact position.

Elements that are right side of element 24 are greater than it, and the elements that are left side of element 24 are smaller than it.



Now, in a similar manner, quick sort algorithm is separately applied to the left and right sub-arrays. After sorting gets done, the array will be -



# Quicksort complexity

Now, let's see the time complexity of quicksort in best case, average case, and in worst case. We will also see the space complexity of quicksort.

## 1. Time Complexity

| Case         | Time Complexity     |
|--------------|---------------------|
| Best Case    | $O(n \cdot \log n)$ |
| Average Case | $O(n \cdot \log n)$ |
| Worst Case   | $O(n^2)$            |

**Best Case Complexity** - In Quicksort, the best-case occurs when the pivot element is the middle element or near to the middle element. The best-case time complexity of quicksort is  **$O(n \cdot \log n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of quicksort is  **$O(n \cdot \log n)$** .

**Worst Case Complexity** - In quick sort, worst case occurs when the pivot element is either greatest or smallest element. Suppose, if the pivot element is always the last element of the array, the worst case would occur when the given array is sorted already in ascending or descending order. The worst-case time complexity of quicksort is  **$O(n^2)$** .

Though the worst-case complexity of quicksort is more than other sorting algorithms such as **Merge sort** and **Heap sort**, still it is faster in practice. Worst case in quick sort rarely occurs because by changing the choice of pivot, it can be implemented in different ways. Worst case in quicksort can be avoided by choosing the right pivot element.

## 2. Space Complexity

|                  |                     |
|------------------|---------------------|
| Space Complexity | $O(n \cdot \log n)$ |
| Stable           | NO                  |

The space complexity of quicksort is  $O(n \cdot \log n)$ .

## Implementation of quicksort

Now, let's see the programs of quicksort in different programming languages.

**Program:** Write a program to implement quicksort in C language.

```
#include <stdio.h>
```

```
/* function that consider last element as pivot,  
place the pivot at its exact position, and place  
smaller elements to left of pivot and greater  
elements to right of pivot. */
```

```
int partition (int a[], int start, int end)
```

```
{
```

```

int pivot = a[end]; // pivot element
int i = (start - 1);

for (int j = start; j <= end - 1; j++)
{
    // If current element is smaller than the pivot
    if (a[j] < pivot)
    {
        i++; // increment index of smaller element
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
}
int t = a[i+1];
a[i+1] = a[end];
a[end] = t;
return (i + 1);
}

/* function to implement quick sort */
void quick(int a[], int start, int end) /* a[] = array to be sorted, start = Starting index, end = Ending index */
{
    if (start < end)
    {
        int p = partition(a, start, end); //p is the partitioning index
        quick(a, start, p - 1);
        quick(a, p + 1, end);
    }
}

/* function to print an array */
void printArr(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}

int main()
{
    int a[] = { 24, 9, 29, 14, 19, 27 };
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");

```

```

printArr(a, n);
quick(a, 0, n - 1);
printf("\nAfter sorting array elements are - \n");
printArr(a, n);

return 0;
}

```

### Output:

```

Before sorting array elements are -
24 9 29 14 19 27
After sorting array elements are -
9 14 19 24 27 29

```

**Program:** Write a program to implement quick sort in C++ language.

```
#include <iostream>
```

```
using namespace std;
```

```
/* function that consider last element as pivot,
place the pivot at its exact position, and place
smaller elements to left of pivot and greater
elements to right of pivot. */
```

```
int partition (int a[], int start, int end)
{
    int pivot = a[end]; // pivot element
    int i = (start - 1);

    for (int j = start; j <= end - 1; j++)
    {
        // If current element is smaller than the pivot
        if (a[j] < pivot)
        {
            i++; // increment index of smaller element
            int t = a[i];
            a[i] = a[j];
            a[j] = t;
        }
    }
    int t = a[i+1];
    a[i+1] = a[end];
    a[end] = t;
    return (i + 1);
}

```

```
/* function to implement quick sort */
```

```
void quick(int a[], int start, int end) /* a[] = array to be sorted, start = Starting index, end = Ending index */
```

```
{
    if (start < end)
    {
        int p = partition(a, start, end); //p is the partitioning index
        quick(a, start, p - 1);
        quick(a, p + 1, end);
    }
}
```

```
/* function to print an array */
```

```
void printArr(int a[], int n)
```

```
{
    int i;
    for (i = 0; i < n; i++)
        cout<<a[i]<< " ";
}
```

```
int main()
```

```
{
    int a[] = { 23, 8, 28, 13, 18, 26 };
    int n = sizeof(a) / sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    printArr(a, n);
    quick(a, 0, n - 1);
    cout<<"\nAfter sorting array elements are - \n";
    printArr(a, n);

    return 0;
}
```

**Output:**

```
Before sorting array elements are -
23 8 28 13 18 26
After sorting array elements are -
8 13 18 23 26 28
```

**Program:** Write a program to implement quicksort in python.

#function that consider last element as pivot,  
#place the pivot at its exact position, and place  
#smaller elements to left of pivot and greater  
#elements to right of pivot.

```
def partition (a, start, end):
```

```
    i = (start - 1)
```

```
    pivot = a[end] # pivot element
```



```

for j in range(start, end):
    # If current element is smaller than or equal to the pivot
    if (a[j] <= pivot):
        i = i + 1
        a[i], a[j] = a[j], a[i]

a[i+1], a[end] = a[end], a[i+1]

return (i + 1)

```

```

# function to implement quick sort
def quick(a, start, end): # a[] = array to be sorted, start = Starting index, end = Ending index
    if (start < end):
        p = partition(a, start, end) # p is partitioning index
        quick(a, start, p - 1)
        quick(a, p + 1, end)

```

```

def printArr(a): # function to print the array
    for i in range(len(a)):
        print (a[i], end = " ")

```

```

a = [68, 13, 1, 49, 58]
print("Before sorting array elements are - ")
printArr(a)
quick(a, 0, len(a)-1)
print("\nAfter sorting array elements are - ")
printArr(a)

```

**Output:**

```

Before sorting array elements are -
68 13 1 49 58
After sorting array elements are -
1 13 49 58 68

```

**Program:** Write a program to implement quicksort in Java.

```

public class Quick
{
    /* function that consider last element as pivot,
    place the pivot at its exact position, and place
    smaller elements to left of pivot and greater
    elements to right of pivot. */
    int partition (int a[], int start, int end)
    {

```

```

int pivot = a[end]; // pivot element
int i = (start - 1);

for (int j = start; j <= end - 1; j++)
{
    // If current element is smaller than the pivot
    if (a[j] < pivot)
    {
        i++; // increment index of smaller element
        int t = a[i];
        a[i] = a[j];
        a[j] = t;
    }
}
int t = a[i+1];
a[i+1] = a[end];
a[end] = t;
return (i + 1);
}

/* function to implement quick sort */
void quick(int a[], int start, int end) /* a[] = array to be sorted, start = Starting index, end = Ending index */
{
    if (start < end)
    {
        int p = partition(a, start, end); //p is partitioning index
        quick(a, start, p - 1);
        quick(a, p + 1, end);
    }
}

/* function to print an array */
void printArr(int a[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        System.out.print(a[i] + " ");
}

public static void main(String[] args) {
    int a[] = { 13, 18, 27, 2, 19, 25 };
    int n = a.length;
    System.out.println("\nBefore sorting array elements are - ");
    Quick q1 = new Quick();

```

```

q1.printArr(a, n);
q1.quick(a, 0, n - 1);
System.out.println("\nAfter sorting array elements are - ");
q1.printArr(a, n);
System.out.println();
}
}

```

### Output

After the execution of above code, the output will be -

```

D:\JTP>javac Quick.java
D:\JTP>java Quick
Before sorting array elements are -
13 18 27 2 19 25
After sorting array elements are -
2 13 18 19 25 27

```

**Program:** Write a program to implement quick sort in php.

**<?php**

```

/* function that consider last element as pivot,
place the pivot at its exact position, and place
smaller elements to left of pivot and greater
elements to right of pivot. */
function partition (&$a, $start, $end)
{
    $pivot = $a[$end]; // pivot element
    $i = ($start - 1);

    for ($j = $start; $j <= $end - 1; $j++)
    {
        // If current element is smaller than the pivot
        if ($a[$j] < $pivot)
        {
            $i++; // increment index of smaller element
            $t = $a[$i];
            $a[$i] = $a[$j];
            $a[$j] = $t;
        }
    }
    $t = $a[$i+1];
    $a[$i+1] = $a[$end];
    $a[$end] = $t;
    return ($i + 1);
}

```

/\* function to implement quick sort \*/

```

function quick(&$a, $start, $end) /* a[] = array to be sorted, start = Starting index, end = Ending
index */
{
    if ($start < $end)
    {
        $p = partition($a, $start, $end); //p is partitioning index
        quick($a, $start, $p - 1);
        quick($a, $p + 1, $end);
    }
}

function printArray($a, $n)
{
    for($i = 0; $i < $n; $i++)
    {
        print_r($a[$i]);
        echo " ";
    }
}

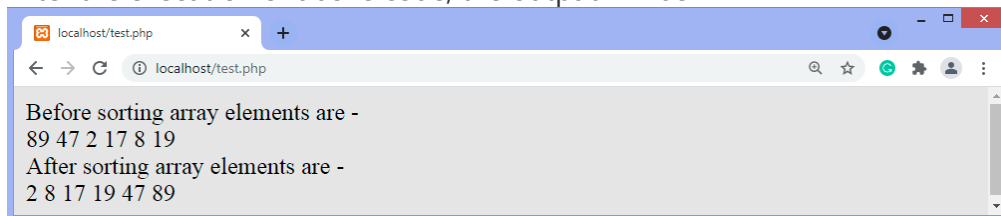
$a = array( 89, 47, 2, 17, 8, 19 );
$n = count($a);
echo "Before sorting array elements are - <br>";
printArray($a, $n);
quick($a, 0, $n - 1);
echo "<br> After sorting array elements are - <br>";
printArray($a, $n);

```

?>

## Output

After the execution of above code, the output will be -



So, that's all about the article. Hope the article will be helpful and informative to you.

This article was not only limited to the algorithm. Along with the algorithm, we have also discussed the quick sort complexity, working, and implementation in different programming languages.

## Radix Sort Algorithm

In this article, we will discuss the Radix sort Algorithm. Radix sort is the linear sorting algorithm that is used for integers. In Radix sort, there is digit by digit sorting is performed that is started from the least significant digit to the most significant digit.

The process of radix sort works similar to the sorting of students names, according to the alphabetical order. In this case, there are 26 radix formed due to the 26 alphabets in English. In the first pass, the names of students are grouped according to the ascending order of the first letter of their names. After that, in the second pass, their names are grouped according to the ascending order of the second letter of their name. And the process continues until we find the sorted list.

Now, let's see the algorithm of Radix sort.

## Algorithm

radixSort(arr)

max = largest element in the given array

d = number of digits in the largest element (or, max)

Now, create d buckets of size 0 - 9

for i -> 0 to d

sort the array elements using counting sort (or any stable sort) according to the digits at the ith place

## Working of Radix sort Algorithm

Now, let's see the working of Radix sort Algorithm.

The steps used in the sorting of radix sort are listed as follows -

First, we have to find the largest element (suppose **max**) from the given array. Suppose '**x**' be the number of digits in **max**. The '**x**' is calculated because we need to go through the significant places of all elements.

After that, go through one by one each significant place. Here, we have to use any stable sorting algorithm to sort the digits of each significant place.

Now let's see the working of radix sort in detail by using an example. To understand it more clearly, let's take an unsorted array and try to sort it using radix sort. It will make the explanation clearer and easier.

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 181 | 289 | 390 | 121 | 145 | 736 | 514 | 212 |
|-----|-----|-----|-----|-----|-----|-----|-----|

In the given array, the largest element is **736** that have **3** digits in it. So, the loop will run up to three times (i.e., to the **hundreds place**). That means three passes are required to sort the array. Now, first sort the elements on the basis of unit place digits (i.e., **x = 0**). Here, we are using the counting sort algorithm to sort the elements.

### Pass 1:

In the first pass, the list is sorted on the basis of the digits at 0's place.

|     |  |  |     |   |
|-----|--|--|-----|---|
| 181 |  |  | 390 | 0 |
| 289 |  |  | 181 | 1 |
| 390 |  |  | 121 | 2 |
| 121 |  |  | 212 | 3 |
| 145 |  |  |     | 4 |
| 736 |  |  | 514 | 5 |
| 514 |  |  | 145 | 6 |
| 212 |  |  | 736 | 7 |
|     |  |  |     | 8 |
|     |  |  | 289 | 9 |

After the first pass, the array elements are -

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 390 | 181 | 121 | 212 | 514 | 145 | 736 | 289 |
|-----|-----|-----|-----|-----|-----|-----|-----|

### Pass 2:

In this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 10<sup>th</sup> place).

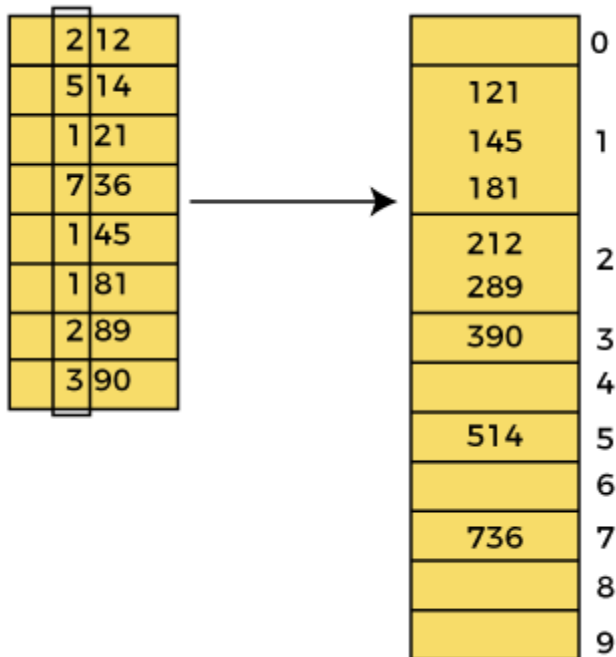
|     |  |  |  |  |  |  |  |  |  |
|-----|--|--|--|--|--|--|--|--|--|
| 390 |  |  |  |  |  |  |  |  |  |
| 181 |  |  |  |  |  |  |  |  |  |
| 121 |  |  |  |  |  |  |  |  |  |
| 212 |  |  |  |  |  |  |  |  |  |
| 514 |  |  |  |  |  |  |  |  |  |
| 145 |  |  |  |  |  |  |  |  |  |
| 736 |  |  |  |  |  |  |  |  |  |
| 289 |  |  |  |  |  |  |  |  |  |

After the second pass, the array elements are -

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 212 | 514 | 121 | 736 | 145 | 181 | 289 | 390 |
|-----|-----|-----|-----|-----|-----|-----|-----|

### Pass 3:

In this pass, the list is sorted on the basis of the next significant digits (i.e., digits at 100<sup>th</sup> place).



After the third pass, the array elements are -

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 121 | 145 | 181 | 212 | 289 | 390 | 514 | 736 |
|-----|-----|-----|-----|-----|-----|-----|-----|

Now, the array is sorted in ascending order.

## Radix sort complexity

Now, let's see the time complexity of Radix sort in best case, average case, and worst case. We will also see the space complexity of Radix sort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $\Omega(n+k)$   |
| Average Case | $\theta(nk)$    |
| Worst Case   | $O(nk)$         |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of Radix sort is  **$\Omega(n+k)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of Radix sort is  **$\theta(nk)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of Radix sort is  **$O(nk)$** .

Radix sort is a non-comparative sorting algorithm that is better than the comparative sorting algorithms. It has linear time complexity that is better than the comparative algorithms with complexity  $O(n \log n)$ .

## 2. Space Complexity

|                  |            |
|------------------|------------|
| Space Complexity | $O(n + k)$ |
| Stable           | YES        |

The space complexity of Radix sort is  $O(n + k)$ .

## Implementation of Radix sort

Now, let's see the programs of Radix sort in different programming languages.

**Program:** Write a program to implement Radix sort in C language.

`#include <stdio.h>`

```
int getMax(int a[], int n) {
```

```
    int max = a[0];
```

```
    for(int i = 1; i < n; i++) {
```

```
        if(a[i] > max)
```

```
            max = a[i];
```

```
    }
```

```
    return max; //maximum element from the array
```

```
}
```

```
void countingSort(int a[], int n, int place) // function to implement counting sort
```

```
{
```

```
    int output[n + 1];
```

```
    int count[10] = {0};
```

```
    // Calculate count of elements
```

```
    for (int i = 0; i < n; i++)
```

```
        count[(a[i] / place) % 10]++;
```

```
    // Calculate cumulative frequency
```

```
    for (int i = 1; i < 10; i++)
```

```
        count[i] += count[i - 1];
```

```
    // Place the elements in sorted order
```

```
    for (int i = n - 1; i >= 0; i--) {
```

```
        output[count[(a[i] / place) % 10] - 1] = a[i];
```

```
        count[(a[i] / place) % 10]--;
```

```
    }
```

```
    for (int i = 0; i < n; i++)
```



```

    a[i] = output[i];
}

// function to implement radix sort
void radixsort(int a[], int n) {

    // get maximum element from array
    int max = getMax(a, n);

    // Apply counting sort to sort elements based on place value
    for (int place = 1; max / place > 0; place *= 10)
        countingSort(a, n, place);
}

// function to print array elements
void printArray(int a[], int n) {
    for (int i = 0; i < n; ++i) {
        printf("%d ", a[i]);
    }
    printf("\n");
}

int main() {
    int a[] = {181, 289, 390, 121, 145, 736, 514, 888, 122};
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArray(a, n);
    radixsort(a, n);
    printf("After applying Radix sort, the array elements are - \n");
    printArray(a, n);
}

```

### Output:

After the execution of the above code, the output will be -

```

Before sorting array elements are -
181 289 390 121 145 736 514 888 122
After applying Radix sort, the array elements are -
121 122 145 181 289 390 514 736 888

```

**Program:** Write a program to implement Radix sort in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int getMax(int a[], int n) {
```

```

int max = a[0];
for(int i = 1; i < n; i++) {
    if(a[i] > max)
        max = a[i];
}
return max; //maximum element from the array
}

```

```

void countingSort(int a[], int n, int place) // function to implement counting sort
{

```

```

    int output[n + 1];
    int count[10] = {0};

```

```

    // Calculate count of elements

```

```

    for (int i = 0; i < n; i++)
        count[(a[i] / place) % 10]++;

```

```

    // Calculate cumulative frequency

```

```

    for (int i = 1; i < 10; i++)
        count[i] += count[i - 1];

```

```

    // Place the elements in sorted order

```

```

    for (int i = n - 1; i >= 0; i--) {
        output[count[(a[i] / place) % 10] - 1] = a[i];
        count[(a[i] / place) % 10]--;
    }

```

```

    for (int i = 0; i < n; i++)
        a[i] = output[i];
}

```

```

// function to implement radix sort

```

```

void radixsort(int a[], int n) {

```

```

    // get maximum element from array

```

```

    int max = getMax(a, n);

```

```

    // Apply counting sort to sort elements based on place value

```

```

    for (int place = 1; max / place > 0; place *= 10)
        countingSort(a, n, place);
}

```

```

// function to print array elements

```

```

void printArray(int a[], int n) {

```

```

    for (int i = 0; i < n; ++i)
        cout<<a[i]<<" ";
}

int main() {
    int a[] = {171, 279, 380, 111, 135, 726, 504, 878, 112};
    int n = sizeof(a) / sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    printArray(a,n);
    radixsort(a, n);
    cout<<"\n\nAfter applying Radix sort, the array elements are - \n";
    printArray(a, n);
    return 0;
}

```

**Output:**

```

Before sorting array elements are -
171 279 380 111 135 726 504 878 112

After applying Radix sort, the array elements are -
111 112 135 171 279 380 504 726 878

```

**Program:** Write a program to implement Radix sort in C#.

**using** System;

**class** RadixSort {

**static int** getMax(int[] a, int n) {

int max = a[0];

for(int i = 1; i<n; i++) {

if(a[i] > max)

max = a[i];

}

return max; //maximum element from the array

}

**static void** countingSort(int[] a, int n, int place) // function to implement counting sort

{

int[] output = new int[n+1];

int[] count = new int[10];

// Calculate count of elements

for (int i = 0; i < n; i++)

count[(a[i] / place) % 10]++;

// Calculate cumulative frequency

for (int i = 1; i < 10; i++)

```

    count[i] += count[i - 1];

    // Place the elements in sorted order
    for (int i = n - 1; i >= 0; i--) {
        output[count[(a[i] / place) % 10] - 1] = a[i];
        count[(a[i] / place) % 10]--;
    }

    for (int i = 0; i < n; i++)
        a[i] = output[i];
}

// function to implement radix sort
static void radixsort(int[] a, int n) {

    // get maximum element from array
    int max = getMax(a, n);

    // Apply counting sort to sort elements based on place value
    for (int place = 1; max / place > 0; place *= 10)
        countingSort(a, n, place);
}

// function to print array elements
static void printArray(int[] a, int n) {
    for (int i = 0; i < n; ++i)
        Console.Write(a[i] + " ");
}

static void Main() {
    int[] a = {161, 269, 370, 101, 125, 716, 54, 868, 12};
    int n = a.Length;
    Console.WriteLine("Before sorting array elements are - \n");
    printArray(a,n);
    radixsort(a, n);
    Console.WriteLine("\n\nAfter applying Radix sort, the array elements are - \n");
    printArray(a, n);
}
}

```

**Output:**

```
Before sorting array elements are -  
161 269 370 101 125 716 54 868 12
```

```
After applying Radix sort, the array elements are -  
12 54 101 125 161 269 370 716 868
```

**Program:** Write a program to implement Radix sort in Java.

```
class RadixSort {
```

```
int getMax(int a[], int n) {
```

```
    int max = a[0];
```

```
    for(int i = 1; i < n; i++) {
```

```
        if(a[i] > max)
```

```
            max = a[i];
```

```
    }
```

```
    return max; //maximum element from the array
```

```
}
```

```
void countingSort(int a[], int n, int place) // function to implement counting
```

```
sort
```

```
{
```

```
    int[] output = new int[n+1];
```

```
    int[] count = new int[10];
```

```
    // Calculate count of elements
```

```
    for (int i = 0; i < n; i++)
```

```
        count[(a[i] / place) % 10]++;
```

```
    // Calculate cumulative frequency
```

```
    for (int i = 1; i < 10; i++)
```

```
        count[i] += count[i - 1];
```

```
    // Place the elements in sorted order
```

```
    for (int i = n - 1; i >= 0; i--) {
```

```
        output[count[(a[i] / place) % 10] - 1] = a[i];
```

```
        count[(a[i] / place) % 10]--;
```

```
    }
```

```
    for (int i = 0; i < n; i++)
```

```
        a[i] = output[i];
```

```
}
```

```
// function to implement radix sort
```

```
void radixsort(int a[], int n) {
```

```

// get maximum element from array
int max = getMax(a, n);

// Apply counting sort to sort elements based on place value
for (int place = 1; max / place > 0; place *= 10)
    countingSort(a, n, place);
}

// function to print array elements
void printArray(int a[], int n) {
    for (int i = 0; i < n; ++i)
        System.out.print(a[i] + " ");
}

public static void main(String args[]) {
    int a[] = {151, 259, 360, 91, 115, 706, 34, 858, 2};
    int n = a.length;
    RadixSort r1 = new RadixSort();
    System.out.print("Before sorting array elements are - \n");
    r1.printArray(a,n);
    r1.radixsort(a, n);
    System.out.print("\n\nAfter applying Radix sort, the array elements are -
\n");
    r1.printArray(a, n);
}
}

```

#### Output:

```

D:\JTP>javac RadixSort.java
D:\JTP>java RadixSort
Before sorting array elements are -
151 259 360 91 115 706 34 858 2
After applying Radix sort, the array elements are -
2 34 91 115 151 259 360 706 858
D:\JTP>_

```

So, that's all about the article. Hope the article will be helpful and informative to you.

## Selection Sort Algorithm

In this article, we will discuss the Selection sort Algorithm. The working procedure of selection sort is also simple. This article will be very helpful and interesting to students as they might face selection sort as a question in their examinations. So, it is important to discuss the topic. In selection sort, the smallest value among the unsorted elements of the array is selected in every pass and inserted to its appropriate position into the array. It is also the simplest

algorithm. It is an in-place comparison sorting algorithm. In this algorithm, the array is divided into two parts, first is sorted part, and another one is the unsorted part. Initially, the sorted part of the array is empty, and unsorted part is the given array. Sorted part is placed at the left, while the unsorted part is placed at the right.

In selection sort, the first smallest element is selected from the unsorted array and placed at the first position. After that second smallest element is selected and placed in the second position. The process continues until the array is entirely sorted.

The average and worst-case complexity of selection sort is  $O(n^2)$ , where  $n$  is the number of items. Due to this, it is not suitable for large data sets.

Selection sort is generally used when -

- A small array is to be sorted

- Swapping cost doesn't matter

- It is compulsory to check all elements

Now, let's see the algorithm of selection sort.

## Algorithm

SELECTION SORT(arr, n)

Step 1: Repeat Steps 2 **and** 3 **for**  $i = 0$  to  $n-1$

Step 2: CALL SMALLEST(arr, i, n, pos)

Step 3: SWAP arr[i] with arr[pos]

[END OF LOOP]

Step 4: EXIT

SMALLEST (arr, i, n, pos)

Step 1: [INITIALIZE] SET SMALL = arr[i]

Step 2: [INITIALIZE] SET pos = i

Step 3: Repeat **for**  $j = i+1$  to  $n$

**if** (SMALL > arr[j])

    SET SMALL = arr[j]

SET pos = j

[END OF **if**]

[END OF LOOP]

Step 4: RETURN pos

## Working of Selection sort Algorithm

Now, let's see the working of the Selection sort Algorithm.

To understand the working of the Selection sort algorithm, let's take an unsorted array. It will be easier to understand the Selection sort via an example.

Let the elements of array are -

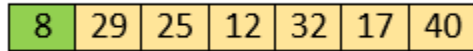
|    |    |    |   |    |    |    |
|----|----|----|---|----|----|----|
| 12 | 29 | 25 | 8 | 32 | 17 | 40 |
|----|----|----|---|----|----|----|

Now, for the first position in the sorted array, the entire array is to be scanned sequentially.

At present, **12** is stored at the first position, after searching the entire array, it is found that **8** is the smallest value.



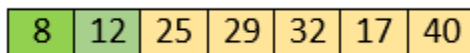
So, swap 12 with 8. After the first iteration, 8 will appear at the first position in the sorted array.



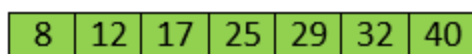
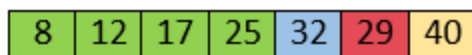
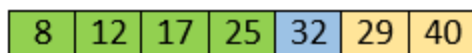
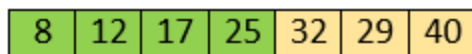
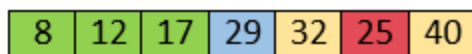
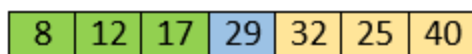
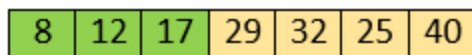
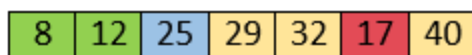
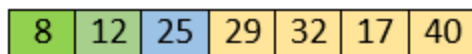
For the second position, where 29 is stored presently, we again sequentially scan the rest of the items of unsorted array. After scanning, we find that 12 is the second lowest element in the array that should be appeared at second position.



Now, swap 29 with 12. After the second iteration, 12 will appear at the second position in the sorted array. So, after two iterations, the two smallest values are placed at the beginning in a sorted way.



The same process is applied to the rest of the array elements. Now, we are showing a pictorial representation of the entire sorting process.



Now, the array is completely sorted.



# Selection sort complexity

Now, let's see the time complexity of selection sort in best case, average case, and in worst case. We will also see the space complexity of the selection sort.

## 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(n^2)$        |
| Average Case | $O(n^2)$        |
| Worst Case   | $O(n^2)$        |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of selection sort is  $O(n^2)$ .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of selection sort is  $O(n^2)$ .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of selection sort is  $O(n^2)$ .

## 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(1)$ |
| Stable           | YES    |

The space complexity of selection sort is  $O(1)$ . It is because, in selection sort, an extra variable is required for swapping.

## Implementation of selection sort

Now, let's see the programs of selection sort in different programming languages.

**Program:** Write a program to implement selection sort in C language.

```
#include <stdio.h>
```

```
void selection(int arr[], int n)
{
    int i, j, small;

    for (i = 0; i < n-1; i++) // One by one move boundary of unsorted subarray
    {
        small = i; //minimum element in unsorted array

        for (j = i+1; j < n; j++)
            if (arr[j] < arr[small])
                small = j;

        // Swap the minimum element with the first element
        int temp = arr[small];
        arr[small] = arr[i];
        arr[i] = temp;
    }
}
```

```

    }
}

void printArr(int a[], int n) /* function to print the array */
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}

int main()
{
    int a[] = { 12, 31, 25, 8, 32, 17 };
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArr(a, n);
    selection(a, n);
    printf("\nAfter sorting array elements are - \n");
    printArr(a, n);
    return 0;
}

```

### Output:

After the execution of above code, the output will be -

```

Before sorting array elements are -
12 31 25 8 32 17
After sorting array elements are -
8 12 17 25 31 32

```

**Program:** Write a program to implement selection sort in C++ language.

```
#include <iostream>
```

```
using namespace std;
```

```

void selection(int arr[], int n)
{
    int i, j, small;

    for (i = 0; i < n-1; i++) // One by one move boundary of unsorted subarray
    {
        small = i; //minimum element in unsorted array

        for (j = i+1; j < n; j++)
            if (arr[j] < arr[small])
                small = j;
        // Swap the minimum element with the first element
    }
}

```

```

    int temp = arr[small];
    arr[small] = arr[i];
    arr[i] = temp;
}

void printArr(int a[], int n) /* function to print the array */
{
    int i;
    for (i = 0; i < n; i++)
        cout << a[i] << " ";
}

int main()
{
    int a[] = { 80, 10, 29, 11, 8, 30, 15 };
    int n = sizeof(a) / sizeof(a[0]);
    cout << "Before sorting array elements are - " << endl;
    printArr(a, n);
    selection(a, n);
    cout << "\nAfter sorting array elements are - " << endl;
    printArr(a, n);

    return 0;
}

```

### Output:

After the execution of above code, the output will be -

```

Before sorting array elements are -
80 10 29 11 8 30 15
After sorting array elements are -
8 10 11 15 29 30 80

```

**Program:** Write a program to implement selection sort in C# language.

```

using System;
class Selection {
    static void selection(int[] arr)
    {
        int i, j, small;
        int n = arr.Length;
        for (i = 0; i < n-1; i++) // One by one move boundary of unsorted subarray
        {
            small = i; //minimum element in unsorted array

            for (j = i+1; j < n; j++)
                if (arr[j] < arr[small])

```

```

        small = j;
// Swap the minimum element with the first element
        int temp = arr[small];
        arr[small] = arr[i];
        arr[i] = temp;
    }
}

static void printArr(int[] a) /* function to print the array */
{
    int i;
    int n = a.Length;
    for (i = 0; i < n; i++)
        Console.Write(a[i] + " ");
}

static void Main() {
    int[] a = { 85, 50, 29, 18, 7, 30, 3};
    Console.WriteLine("Before sorting array elements are - ");
    printArr(a);
    selection(a);
    Console.WriteLine("\nAfter sorting array elements are - ");
    printArr(a);
}
}

```

#### Output:

```

Before sorting array elements are - 85 50 29 18 7 30 3
After sorting array elements are - 3 7 18 29 30 50 85

```

**Program:** Write a program to implement selection sort in python.

```

def selection(a): # Function to implement selection sort
    for i in range(len(a)): # Traverse through all array elements
        small = i # minimum element in unsorted array
        for j in range(i+1, len(a)):
            if a[small] > a[j]:
                small = j
        # Swap the found minimum element with
        # the first element
        a[i], a[small] = a[small], a[i]

def printArr(a): # function to print the array

    for i in range(len(a)):
        print (a[i], end = " ")

```

```
a = [69, 14, 1, 50, 59]
print("Before sorting array elements are - ")
printArr(a)
selection(a)
print("\nAfter sorting array elements are - ")
selection(a)
printArr(a)
```

**Output:**

```
Before sorting array elements are -
69 14 1 50 59
After sorting array elements are -
1 14 50 59 69
```

**Program:** Write a program to implement selection sort in Java.

```
public class Selection
{
    void selection(int a[]) /* function to sort an array with selection sort */
    {
        int i, j, small;
        int n = a.length;
        for (i = 0; i < n-1; i++)
        {
            small = i; //minimum element in unsorted array

            for (j = i+1; j < n; j++)
                if (a[j] < a[small])
                    small = j;
            // Swap the minimum element with the first element
            int temp = a[small];
            a[small] = a[i];
            a[i] = temp;
        }
    }

    void printArr(int a[]) /* function to print the array */
    {
        int i;
        int n = a.length;
        for (i = 0; i < n; i++)
            System.out.print(a[i] + " ");
    }

    public static void main(String[] args) {
```

```

int a[] = { 91, 49, 4, 19, 10, 21 };
Selection i1 = new Selection();
System.out.println("\nBefore sorting array elements are - ");
i1.printArr(a);
i1.selection(a);
System.out.println("\nAfter sorting array elements are - ");
i1.printArr(a);
System.out.println();
}
}

```

### Output:

```

D:\JTP>javac Selection.java
D:\JTP>java Selection
Before sorting array elements are -
91 49 4 19 10 21
After sorting array elements are -
4 10 19 21 49 91

```

**Program:** Write a program to implement selection sort in PHP.

**<?php**

```

function selection(&$a, $n) /* function to sort an array with selection sort */
{
    for ($i = 0; $i < $n; $i++)
    {
        $small = $i; //minimum element in unsorted array

        for ($j = $i+1; $j < $n; $j++)
            if ($a[$j] < $a[$small])
                $small = $j;
        // Swap the minimum element with the first element
        $temp = $a[$small];
        $a[$small] = $a[$i];
        $a[$i] = $temp;
    }
}

function printArray($a, $n)
{
    for($i = 0; $i < $n; $i++)
    {
        print_r($a[$i]);
        echo " ";
    }
}

$a = array( 90, 48, 3, 18, 9, 20 );

```

```

$n = count($a);
echo "Before sorting array elements are - <br>";
printArray($a, $n);
selection($a, $n);
echo "<br> After sorting array elements are - <br>";
printArray($a, $n);
?>

```

### Output:

After the execution of above code, the output will be -



So, that's all about the article. Hope the article will be helpful and informative to you.

This article was not only limited to the algorithm. We have also discussed the Selection sort complexity, working, and implementation in different programming languages.

## Shell Sort Algorithm

In this article, we will discuss the shell sort algorithm. Shell sort is the generalization of insertion sort, which overcomes the drawbacks of insertion sort by comparing elements separated by a gap of several positions.

It is a sorting algorithm that is an extended version of insertion sort. Shell sort has improved the average time complexity of insertion sort. As similar to insertion sort, it is a comparison-based and in-place sorting algorithm. Shell sort is efficient for medium-sized data sets.

In insertion sort, at a time, elements can be moved ahead by one position only. To move an element to a far-away position, many movements are required that increase the algorithm's execution time. But shell sort overcomes this drawback of insertion sort. It allows the movement and swapping of far-away elements as well.

This algorithm first sorts the elements that are far away from each other, then it subsequently reduces the gap between them. This gap is called as **interval**. This interval can be calculated by using the **Knuth's** formula given below -

$$h_{n+1} = h_n * 3 + 1$$

where, 'h' is the interval having initial value 1.

Now, let's see the algorithm of shell sort.

### Algorithm

The simple steps of achieving the shell sort are listed as follows -

ShellSort(a, n) // 'a' is the given array, 'n' is the size of array

for (interval = n/2; interval > 0; interval /= 2)

for (i = interval; i < n; i += 1)

temp = a[i];

for (j = i; j >= interval && a[j - interval] > temp; j -= interval)

a[j] = a[j - interval];

```
a[j] = temp;  
End ShellSort
```

## Working of Shell sort Algorithm

Now, let's see the working of the shell sort Algorithm.

To understand the working of the shell sort algorithm, let's take an unsorted array. It will be easier to understand the shell sort via an example.

Let the elements of array are -

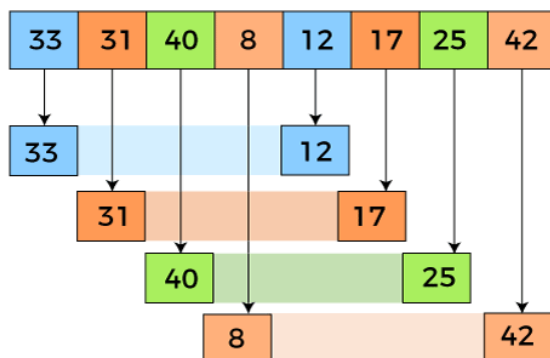
|    |    |    |   |    |    |    |    |
|----|----|----|---|----|----|----|----|
| 33 | 31 | 40 | 8 | 12 | 17 | 25 | 42 |
|----|----|----|---|----|----|----|----|

We will use the original sequence of shell sort, i.e.,  $N/2$ ,  $N/4$ , ..., 1 as the intervals.

In the first loop,  $n$  is equal to 8 (size of the array), so the elements are lying at the interval of 4 ( $n/2 = 4$ ). Elements will be compared and swapped if they are not in order.

Here, in the first loop, the element at the 0<sup>th</sup> position will be compared with the element at 4<sup>th</sup> position. If the 0<sup>th</sup> element is greater, it will be swapped with the element at 4<sup>th</sup> position. Otherwise, it remains the same. This process will continue for the remaining elements.

At the interval of 4, the sublists are {33, 12}, {31, 17}, {40, 25}, {8, 42}.

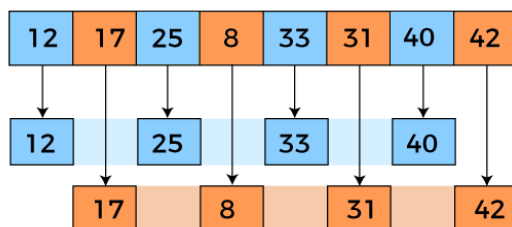


Now, we have to compare the values in every sub-list. After comparing, we have to swap them if required in the original array. After comparing and swapping, the updated array will look as follows -

|    |    |    |   |    |    |    |    |
|----|----|----|---|----|----|----|----|
| 12 | 17 | 25 | 8 | 33 | 31 | 40 | 42 |
|----|----|----|---|----|----|----|----|

In the second loop, elements are lying at the interval of 2 ( $n/4 = 2$ ), where  $n = 8$ .

Now, we are taking the interval of 2 to sort the rest of the array. With an interval of 2, two sublists will be generated - {12, 25, 33, 40}, and {17, 8, 31, 42}.





Now, we again have to compare the values in every sub-list. After comparing, we have to swap them if required in the original array. After comparing and swapping, the updated array will look as follows -

|    |   |    |    |    |    |    |    |
|----|---|----|----|----|----|----|----|
| 12 | 8 | 25 | 17 | 33 | 31 | 40 | 42 |
|----|---|----|----|----|----|----|----|

In the third loop, elements are lying at the interval of 1 ( $n/8 = 1$ ), where  $n = 8$ . At last, we use the interval of value 1 to sort the rest of the array elements. In this step, shell sort uses insertion sort to sort the array elements.

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 12 | 8  | 25 | 17 | 33 | 31 | 40 | 42 |
| 12 | 8  | 25 | 17 | 33 | 31 | 40 | 42 |
| 8  | 12 | 25 | 17 | 33 | 31 | 40 | 42 |
| 8  | 12 | 25 | 17 | 33 | 31 | 40 | 42 |
| 8  | 12 | 17 | 25 | 33 | 31 | 40 | 42 |
| 8  | 12 | 17 | 25 | 33 | 31 | 40 | 42 |
| 8  | 12 | 17 | 25 | 33 | 31 | 40 | 42 |
| 8  | 12 | 17 | 25 | 31 | 33 | 40 | 42 |
| 8  | 12 | 17 | 25 | 31 | 33 | 40 | 42 |
| 8  | 12 | 17 | 25 | 31 | 33 | 40 | 42 |

Now, the array is sorted in ascending order.

## Shell sort complexity

Now, let's see the time complexity of Shell sort in the best case, average case, and worst case. We will also see the space complexity of the Shell sort.

### 1. Time Complexity

| Case | Time Complexity |
|------|-----------------|
|------|-----------------|

|                     |                        |
|---------------------|------------------------|
| <b>Best Case</b>    | $O(n \cdot \log n)$    |
| <b>Average Case</b> | $O(n \cdot \log(n)^2)$ |
| <b>Worst Case</b>   | $O(n^2)$               |

**Best Case Complexity** - It occurs when there is no sorting required, i.e., the array is already sorted. The best-case time complexity of Shell sort is  **$O(n \cdot \log n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of Shell sort is  **$O(n \cdot \log n)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of Shell sort is  **$O(n^2)$** .

## 2. Space Complexity

|                         |        |
|-------------------------|--------|
| <b>Space Complexity</b> | $O(1)$ |
| <b>Stable</b>           | NO     |

The space complexity of Shell sort is  $O(1)$ .

## Implementation of Shell sort

Now, let's see the programs of Shell sort in different programming languages.

**Program:** Write a program to implement Shell sort in C language.

```
#include <stdio.h>
```

```
/* function to implement shellSort */
```

```
int shell(int a[], int n)
```

```
{
```

```
    /* Rearrange the array elements at n/2, n/4, ..., 1 intervals */
```

```
    for (int interval = n/2; interval > 0; interval /= 2)
```

```
    {
```

```
        for (int i = interval; i < n; i += 1)
```

```
        {
```

```
            /* store a[i] to the variable temp and make the ith position empty */
```

```
            int temp = a[i];
```

```
            int j;
```

```
            for (j = i; j >= interval && a[j - interval] > temp; j -= interval)
```

```
                a[j] = a[j - interval];
```

```
            // put temp (the original a[i]) in its correct position
```

```
            a[j] = temp;
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

```

void printArr(int a[], int n) /* function to print the array elements */
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", a[i]);
}
int main()
{
    int a[] = { 33, 31, 40, 8, 12, 17, 25, 42 };
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArr(a, n);
    shell(a, n);
    printf("\nAfter applying shell sort, the array elements are - \n");
    printArr(a, n);
    return 0;
}

```

### Output

After the execution of above code, the output will be -

```

Before sorting array elements are -
33 31 40 8 12 17 25 42
After applying shell sort, the array elements are -
8 12 17 25 31 33 40 42

```

**Program:** Write a program to implement Shell sort in C++.

```

#include <iostream>
using namespace std;
/* function to implement shellSort */
int shell(int a[], int n)
{
    /* Rearrange the array elements at n/2, n/4, ..., 1 intervals */
    for (int interval = n/2; interval > 0; interval /= 2)
    {
        for (int i = interval; i < n; i += 1)
        {
            /* store a[i] to the variable temp and make the ith position empty */
            int temp = a[i];
            int j;
            for (j = i; j >= interval && a[j - interval] > temp; j -= interval)
                a[j] = a[j - interval];

            // put temp (the original a[i]) in its correct position
            a[j] = temp;
        }
    }
}

```

```

    return 0;
}
void printArr(int a[], int n) /* function to print the array elements */
{
    int i;
    for (i = 0; i < n; i++)
        cout<<a[i]<<" ";
}
int main()
{
    int a[] = { 32, 30, 39, 7, 11, 16, 24, 41 };
    int n = sizeof(a) / sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    printArr(a, n);
    shell(a, n);
    cout<<"\nAfter applying shell sort, the array elements are - \n";
    printArr(a, n);
    return 0;
}

```

### Output

After the execution of the above code, the output will be -

```

Before sorting array elements are -
32 30 39 7 11 16 24 41
After applying shell sort, the array elements are -
7 11 16 24 30 32 39 41

```

**Program:** Write a program to implement Shell sort in C#.

```

using System;
class ShellSort {
/* function to implement shellSort */
static void shell(int[] a, int n)
{
    /* Rearrange the array elements at n/2, n/4, ..., 1 intervals */
    for (int interval = n/2; interval > 0; interval /= 2)
    {
        for (int i = interval; i < n; i += 1)
        {
            /* store a[i] to the variable temp and make the ith position empty */
            int temp = a[i];
            int j;
            for (j = i; j >= interval && a[j - interval] > temp; j -= interval)
                a[j] = a[j - interval];

            /* put temp (the original a[i]) in its correct position */
            a[j] = temp;
        }
    }
}
}

```

```

    }
}
static void printArr(int[] a, int n) /* function to print the array elements */
{
    int i;
    for (i = 0; i < n; i++)
        Console.Write(a[i] + " ");
}
static void Main()
{
    int[] a = { 31, 29, 38, 6, 10, 15, 23, 40 };
    int n = a.Length;
    Console.WriteLine("Before sorting array elements are - \n");
    printArr(a, n);
    shell(a, n);
    Console.WriteLine("\nAfter applying shell sort, the array elements are - \n");
    printArr(a, n);
}
}

```

### Output

After the execution of the above code, the output will be -

```

Before sorting array elements are -
31 29 38 6 10 15 23 40
After applying shell sort, the array elements are -
6 10 15 23 29 31 38 40

```

**Program:** Write a program to implement Shell sort in Java.

```

class ShellSort {
/* function to implement shellSort */
static void shell(int a[], int n)
{
    /* Rearrange the array elements at n/2, n/4, ..., 1 intervals */
    for (int interval = n/2; interval > 0; interval /= 2)
    {
        for (int i = interval; i < n; i += 1)
        {
            /* store a[i] to the variable temp and make
the ith position empty */
            int temp = a[i];
            int j;
            for (j = i; j >= interval && a[j - interval] >
temp; j -= interval)
                a[j] = a[j - interval];

```

```

        /* put temp (the original a[i]) in its correct
position */
        a[j] = temp;
    }
}
}
static void printArr(int a[], int n) /* function to print the array elements */
{
    int i;
    for (i = 0; i < n; i++)
        System.out.print(a[i] + " ");
}
public static void main(String args[])
{
    int a[] = { 30, 28, 37, 5, 9, 14, 22, 39 };
    int n = a.length;
    System.out.print("Before sorting array elements are - \n");
    printArr(a, n);
    shell(a, n);
    System.out.print("\nAfter applying shell sort, the array elements are - \n");
    printArr(a, n);
}
}

```

### Output

After the execution of the above code, the output will be -

```

D:\JTP>javac ShellSort.java

D:\JTP>java ShellSort
Before sorting array elements are -
30 28 37 5 9 14 22 39
After applying shell sort, the array elements are -
5 9 14 22 28 30 37 39
D:\JTP>

```

So, that's all about the article. Hope the article will be helpful and informative to you.

## Bitonic Sort Algorithm

In this article, we will discuss the Bitonic sort Algorithm. Bitonic sort is a parallel sorting algorithm that performs  $O(n^2 \log n)$  comparisons. Although the number of comparisons is more than that in any other popular sorting algorithm, it performs better for the parallel implementation because elements are compared in a predefined sequence that must not depend upon the data being sorted. The predefined sequence is called the Bitonic sequence. To understand the bitonic sort, we first have to understand the **Bitonic sequence**.

In **Bitonic sequence**, elements are first arranged in increasing order, and then after some particular index, they start decreasing.

An array with  $A[0 \dots i \dots n-1]$  is said to be bitonic, if there is an index  $i$ , such that -

$A[0] < A[1] < A[2] \dots A[i-1] < A[i] > A[i+1] > A[i+2] > A[i+3] > \dots > A[n-1]$

Where,  $0 \leq i \leq n-1$ .

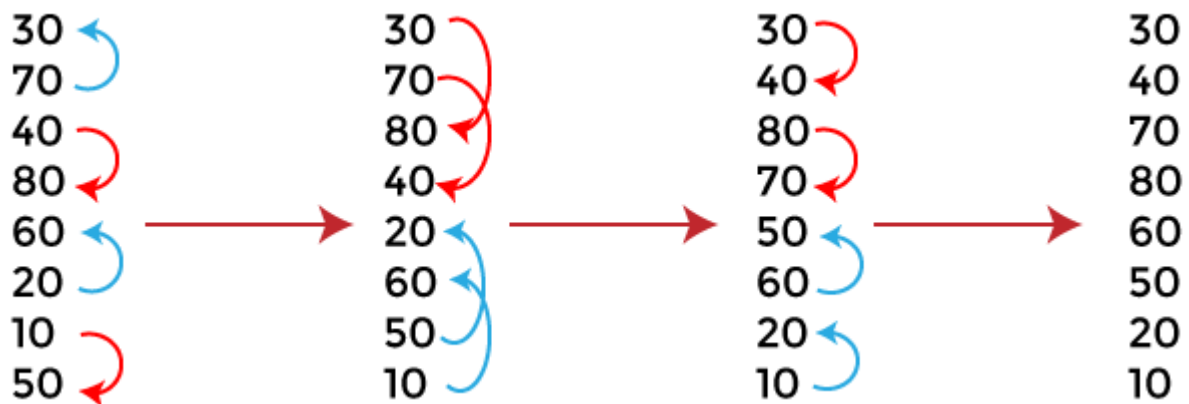
Before moving directly towards the algorithm of bitonic sort, first, understand the conversion of any random sequence into a bitonic sequence.

## How to convert the random sequence into a bitonic sequence?

Consider a sequence  $A[0 \dots n-1]$  of  $n$  elements. First, start constructing a Bitonic sequence by using 4 elements of the sequence. Sort the first 2 elements in ascending order and the last 2 elements in descending order, concatenate this pair to form a Bitonic sequence of 4 elements. Repeat this process for the remaining pairs of the element until we find a Bitonic sequence.

Let's understand the process to convert the random sequence into a bitonic sequence using an example.

Suppose the elements of array are - **{30, 70, 40, 80, 60, 20, 10, 50}**



After conversion, the bitonic sequence that we will get is -

**30, 40, 70, 80, 60, 50, 20, 10**

Now, move towards the steps of performing the bitonic sort.

## Steps to perform Bitonic sort

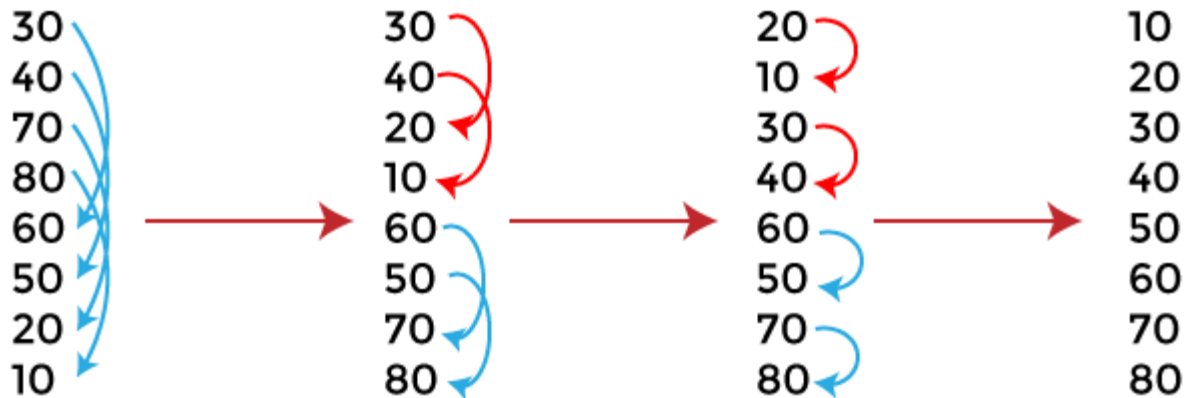
The steps used to perform the bitonic sort are listed as follows -

In first step, we have to create the bitonic sequence from the given random sequence as we have done above. It is considered as the first step of the process. After this step, we will get a sequence in which the elements in the first half are arranged in ascending order, while the elements of the second half are arranged in descending order.

Now, in the second step, we have to compare the first element of first half with the first element of second half, then second element of first half with the second element of second half, and so on. Here, we have to swap the elements if any element in the second half is found to be smaller. After the above step, we got all the elements in the first half smaller than all the elements in the second half. The compare and swap results into the two sequences of  $n/2$  length each. Repeat

the process performed in the second step recursively onto every sequence until we get a single sorted sequence of length  $n$ .

Now, let's see the entire procedure of bitonic sort with an example. It will be easier to understand the bitonic sort with an example as it makes the explanation clearer and easier. In the below example, we are using the bitonic sequence given above that we created from a random sequence.



Now, the given array is completely sorted.

## Bitonic sort complexity

Now, let's see the time complexity of bitonic sort in the best case, average case, and worst case. We will also see the space complexity of bitonic sort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(\log^2 n)$   |
| Average Case | $O(\log^2 n)$   |
| Worst Case   | $O(\log^2 n)$   |

The time complexity of bitonic sort is  **$O(\log^2 n)$**  in all three cases.

### 2. Space Complexity

|                  |                 |
|------------------|-----------------|
| Space Complexity | $O(n \log^2 n)$ |
| Stable           | No              |

The space complexity of bitonic sort is  **$O(n \log^2 n)$** .

## Implementation of Bitonic sort

Now, let's see the programs of Bitonic sort in different programming languages.

**Program:** Write a program to implement bitonic sort in C language.

*/\* This program works only when the size of input is in the power of 2. \*/*

*#include<stdio.h>*



/\*In this function the parameter 'd' represents the sorting direction\*/

**void** exchange(**int** a[], **int** i, **int** j, **int** d)

```
{  
    int temp;  
    if (d==(a[i]>a[j]))  
    {  
        temp = a[i];  
        a[i] = a[j];  
        a[j] = temp;  
    }  
}
```

**void** merge(**int** a[], **int** beg, **int** c, **int** d)

```
{  
    int k, i;  
    if (c > 1)  
    {  
        k = c/2;  
        for (i = beg; i < beg+k; i++)  
            exchange(a, i, i+k, d);  
        merge(a, beg, k, d);  
        merge(a, beg+k, k, d);  
    }  
}
```

**void** bitonicSort(**int** a[],**int** beg, **int** c, **int** d)

```
{  
    int k;  
    if (c>1)  
    {  
        k = c/2;  
        bitonicSort(a, beg, k, 1); // sort in ascending order  
        bitonicSort(a, beg+k, k, 0); // sort in descending order  
        merge(a,beg, c, d); //merge the sequence in ascending order  
    }  
}
```

/\* function to call the bitonicSort() function to sort the given array in ascending order \*/

**void** sort(**int** a[], **int** n, **int** order)

```
{  
    bitonicSort(a, 0, n, order);  
}  
  
void print(int a[], int n) //function to print array elements  
{  
    int i;  
    for(i = 0; i < n; i++)
```

```

    {
        printf("%d ",a[i]);
    }
}

int main()
{
    int a[] = {30, 70, 40, 80, 60, 20, 10, 50};
    int n = sizeof(a)/sizeof(a[0]);
    int order = 1; //It means sorting in increasing order
    printf("Before sorting array elements are - \n");
    print(a, n);
    sort(a, n, order);
    printf("\nAfter sorting array elements are - \n");
    print(a, n);
    return 0;
}

```

### Output

After the execution of the above code, the output will be -

```

Before sorting array elements are -
30 70 40 80 60 20 10 50
After sorting array elements are -
10 20 30 40 50 60 70 80

```

**Program:** Write a program to implement bitonic sort in C++.

/\* This program works only when the size of input is in the power of 2. \*/

#include <iostream>

using namespace std;

/\*In this function the parameter 'd' represents the sorting direction\*/

void exchange(int a[], int i, int j, int d)

```

{
    int temp;
    if (d==(a[i]>a[j]))
    {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}

```

void merge(int a[], int beg, int c, int d)

```

{
    int k, i;
    if (c > 1)

```

```

    {
        k = c/2;
        for (i = beg; i < beg+k; i++)
            exchange(a, i, i+k, d);
        merge(a, beg, k, d);
        merge(a, beg+k, k, d);
    }
}

void bitonicSort(int a[], int beg, int c, int d)
{
    int k;
    if (c > 1)
    {
        k = c/2;
        bitonicSort(a, beg, k, 1); // sort in ascending order
        bitonicSort(a, beg+k, k, 0); // sort in descending order
        merge(a, beg, c, d); //merge the sequence in ascending order
    }
}

/* function to call the bitonicSort() function to sort the given array in ascending order */
void sort(int a[], int n, int order)
{
    bitonicSort(a, 0, n, order);
}

void print(int a[], int n) //function to print array elements
{
    int i;
    for (i = 0; i < n; i++)
    {
        cout << a[i] << " ";
    }
}

int main()
{
    int a[] = {20, 60, 30, 70, 50, 10, 5, 40};
    int n = sizeof(a)/sizeof(a[0]);
    int order = 1; //It means sorting in increasing order
    cout << "Before sorting array elements are - \n";
    print(a, n);
    sort(a, n, order);
    cout << "\nAfter sorting array elements are - \n";
    print(a, n);
}

```

```
    return 0;
}
```

### Output

After the execution of the above code, the output will be -

```
Before sorting array elements are -
20 60 30 70 50 10 5 40
After sorting array elements are -
5 10 20 30 40 50 60 70
```

**Program:** Write a program to implement bitonic sort in C#.

*/\* This program works only when the size of input is in the power of 2. \*/*

**using** System;

**class** BitonicSort {

*/\*In this function the parameter 'd' represents the sorting direction\*/*

**static void** exchange(**int**[] a, **int** i, **int** j, **int** d)

```
{
    int temp;
    if ( (a[i] > a[j] && d == 1) || (a[i] < a[j] && d == 0))
    {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
    }
}
```

**static void** merge(**int**[] a, **int** beg, **int** c, **int** d)

```
{
    int k, i;
    if (c > 1)
    {
        k = c/2;
        for (i = beg; i < beg+k; i++)
            exchange(a, i, i+k, d);
        merge(a, beg, k, d);
        merge(a, beg+k, k, d);
    }
}
```

**static void** bitonicSort(**int**[] a, **int** beg, **int** c, **int** d)

```
{
    int k;
    if (c > 1)
    {
        k = c/2;
        bitonicSort(a, beg, k, 1); // sort in ascending order
        bitonicSort(a, beg+k, k, 0); // sort in descending order
    }
}
```

```

        merge(a,beg, c, d); //merge the sequence in ascending order
    }
}

/* function to call the bitonicSort() function to sort the given array in
ascending order */
static void sort(int[] a, int n, int order)
{
    bitonicSort(a, 0, n, order);
}

static void print(int[] a, int n) //function to print array elements
{
    int i;
    for(i = 0; i < n; i++)
    {
        Console.Write(a[i] + " ");
    }
}

static void Main() {
    int[] a= {19, 59, 29, 69, 49, 9, 4, 39};
    int n = a.Length;
    int order = 1; //It means sorting in increasing order
    Console.WriteLine("Before sorting array elements are - \n");
    print(a, n);
    sort(a, n, order);
    Console.WriteLine("\nAfter sorting array elements are - \n");
    print(a, n);
}
}

```

### Output

After the execution of the above code, the output will be -

```

Before sorting array elements are -
19 59 29 69 49 9 4 39
After sorting array elements are -
4 9 19 29 39 49 59 69

```

**Program:** Write a program to implement bitonic sort in Java.

/\* This program works only when the size of input is in the power of 2. \*/

**class** BitonicSort {

/\*In this function the parameter 'd' represents the sorting direction\*/

**static void** exchange(int a[], int i, int j, int d)

```

{
    int temp;

```

```

        if ( (a[i] > a[j] && d == 1) || (a[i] < a[j] && d == 0))
        {
            temp = a[i];
            a[i] = a[j];
            a[j] = temp;
        }
    }
}

static void merge(int a[], int beg, int c, int d)
{
    int k, i;
    if (c > 1)
    {
        k = c/2;
        for (i = beg; i < beg+k; i++)
            exchange(a, i, i+k, d);
        merge(a, beg, k, d);
        merge(a, beg+k, k, d);
    }
}

static void bitonicSort(int a[], int beg, int c, int d)
{
    int k;
    if (c > 1)
    {
        k = c/2;
        bitonicSort(a, beg, k, 1); // sort in ascending order
        bitonicSort(a, beg+k, k, 0); // sort in descending order
        merge(a, beg, c, d); //merge the sequence in ascending order
    }
}

```

/\* function to call the bitonicSort() function to sort the given array in

ascending order \*/

```

static void sort(int a[], int n, int order)
{
    bitonicSort(a, 0, n, order);
}

static void print(int a[], int n) //function to print array elements
{
    int i;
    for(i = 0; i < n; i++)
    {
        System.out.print(a[i] + " ");
    }
}

```

```

    }
}

public static void main(String args[]) {
    int[] a= {20, 60, 30, 70, 50, 10, 5, 40};
    int n = a.length;
    int order = 1; //It means sorting in increasing order
    System.out.print("Before sorting array elements are - \n");
    print(a, n);
    sort(a, n, order);
    System.out.print("\nAfter sorting array elements are - \n");
    print(a, n);
}
}

```

### Output

After the execution of the above code, the output will be -

```

D:\JTP>javac BitonicSort.java

D:\JTP>java BitonicSort
Before sorting array elements are -
20 60 30 70 50 10 5 40
After sorting array elements are -
5 10 20 30 40 50 60 70
D:\JTP>

```

So, that's all about the article. Hope the article will be helpful and informative to you.

## Cocktail Sort Algorithm

In this article, we will discuss the cocktail sort algorithm. Cocktail sort is the variation of Bubble Sort, which traverses the list in both directions alternatively. It is different from bubble sort in the sense that bubble sort traverses the list in the forward direction only, while this algorithm traverses in forward as well as backward direction in one iteration.

Cocktail sort is also called as **bi-directional** bubble sort. In bubble sort, elements are traversed from left to right, i.e., in one direction. In first iteration, bubble sort first moves the largest element to its correct position, then the second-largest element in its exact position, and so on. But cocktail sort traverses in both directions alternatively.

As similar to the bubble sort, the worst and average case complexities of cocktail sort is  **$O(n^2)$** . Cocktail sort is faster than bubble sort.

There are two stages of cocktail sort in one iteration that are listed as follows -

In first stage, similar to the bubble sort, loop through the array from left to right. The adjacent elements are compared, and if the left element is greater than the right one, we swap those elements. The largest element of the list is placed at the end of the array in the forward pass.

In second stage, loop through the array from the rightmost unsorted element to the left. The adjacent elements are compared, and if the right element is smaller than the left element then, we swap those elements. The smallest element of the list is placed at the beginning of the array in the backward pass.

This process continues until the array elements are not sorted.

Now, let's see the algorithm of cocktail sort algorithm.

## Algorithm

cocktailSort(a, n) // 'a' is the given array, 'n' is the size of given array

swapped = true

beg = 0

end = n-1

**while**(swapped)

swapped = false

**for** i **in** range **from** beg to end

**if** (a[i] > a[i + 1])

swap(a[i], a[i+1])

swapped = true

End **if**

End **for** loop

**if**(!swapped)

**break**

end **if**

swapped = false

end = end - 1

**for** i **in** range **from** end - 1 to beg

**if** (a[i] > a[i + 1])

swap(a[i], a[i+1])

swapped = true

End **if**

End **for** loop

start = start + 1

End **while** loop

End cocktailSort

## Working of Cocktail Sort Algorithm

Now, let's see the working of the cocktail sort Algorithm.

To understand the working of the cocktail sort algorithm, let's take an unsorted array. We are taking a short and accurate array, as we know the complexity of the cocktail sort is  $O(n^2)$ .

Let the elements of array are -

**Array = {4, 0, 3, 1, 7, 1, 2}**

### Iteration 1:

#### **First Forward pass:**

In first iteration, the forward pass is similar to the bubble sort. The comparisons of forward pass in first iteration are given as follows -

Sorting will start from the initial two elements. Let compare them to check which is greater.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 4 | 0 | 3 | 1 | 7 | 1 | 2 |
|---|---|---|---|---|---|---|

 (0 < 4), swap

After swapping new array will look like -



|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 4 | 3 | 1 | 7 | 1 | 2 |
|---|---|---|---|---|---|---|

Now compare 4 and 3.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 4 | 3 | 1 | 7 | 1 | 2 |
|---|---|---|---|---|---|---|

(3 < 4), swap

After swapping new array will look like -

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 4 | 1 | 7 | 1 | 2 |
|---|---|---|---|---|---|---|

Now, compare 4 and 1.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 4 | 1 | 7 | 1 | 2 |
|---|---|---|---|---|---|---|

(1 < 4), swap

After swapping new array will look like -

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 4 | 7 | 1 | 2 |
|---|---|---|---|---|---|---|

Now, compare 4 and 7.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 4 | 7 | 1 | 2 |
|---|---|---|---|---|---|---|

(7 > 4), no swap

Now, the comparison will be between 7 and 1.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 4 | 7 | 1 | 2 |
|---|---|---|---|---|---|---|

(1 < 7), swap

After swapping new array will look like -

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 4 | 1 | 7 | 2 |
|---|---|---|---|---|---|---|

Now, compare 7 and 2.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 4 | 1 | 7 | 2 |
|---|---|---|---|---|---|---|

(2 < 7), swap

Now, we reach at the end of the array. After swapping and first forward pass, the array elements will look like -

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 4 | 1 | 2 | 7 |
|---|---|---|---|---|---|---|

After the first forward pass, the largest element of the array is stored at the last position of the array.

### First Backward pass:

Now, the first backward pass is started. It will be started from the last index of the array except for the index where the largest element is stored.

So, from the backward direction, first array elements **2** and **1** will be compared.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 4 | 1 | 2 | 7 |
|---|---|---|---|---|---|---|

 ( $1 < 2$ ) no swap

Now compare, 1 and 4.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 4 | 1 | 2 | 7 |
|---|---|---|---|---|---|---|

 ( $1 < 4$ ) swap

After swapping, the array will be -

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 1 | 4 | 2 | 7 |
|---|---|---|---|---|---|---|

Now compare 1 and 1.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 1 | 4 | 2 | 7 |
|---|---|---|---|---|---|---|

Now compare, 3 and 1.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 3 | 1 | 1 | 4 | 2 | 7 |
|---|---|---|---|---|---|---|

 ( $3 > 1$ ) swap

After swapping, the array will be -

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 1 | 4 | 2 | 7 |
|---|---|---|---|---|---|---|

Now compare, 0 and 1.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 1 | 4 | 2 | 7 |
|---|---|---|---|---|---|---|

 ( $0 < 1$ ) no swap

So, after the first backward pass, smallest element from the array is stored at the first index of array. So, after first iteration, the array elements will be -

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 1 | 4 | 2 | 7 |
|---|---|---|---|---|---|---|

## Iteration 2:

### Second Forward pass:

Now, the second forward pass is started. It will be started from the first index of the array except for the index where the smallest element is stored.

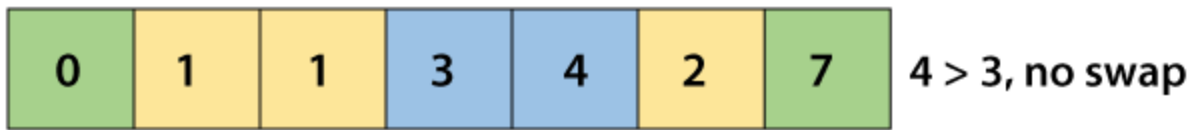
So, from the forward direction, first array elements **1** and **3** will be compared.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 1 | 4 | 2 | 7 |
|---|---|---|---|---|---|---|

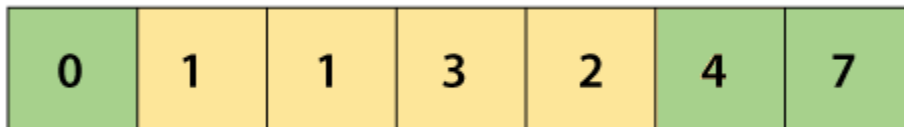
 $3 > 1$ , no swap

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | 1 | 3 | 1 | 4 | 2 | 7 |
|---|---|---|---|---|---|---|

 $1 < 3$ , swap



Now, we reach at the end of the array. After the second forward pass, the second largest array element will be stored at its exact position. After swapping and second forward pass, the array elements will look like -



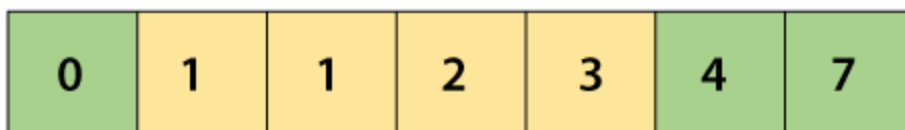
#### Second Backward pass:

Now, the second backward pass is started.

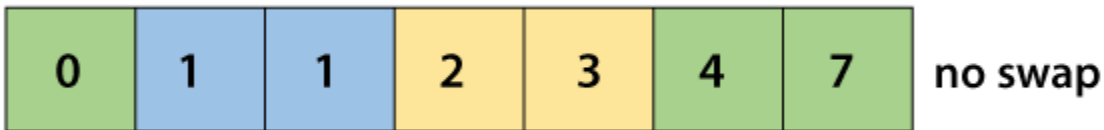
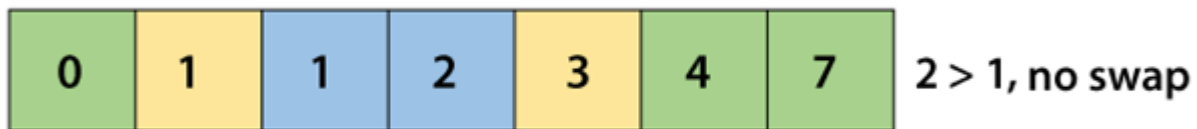
So, from backward direction, array elements 3 and 2 will be compared.



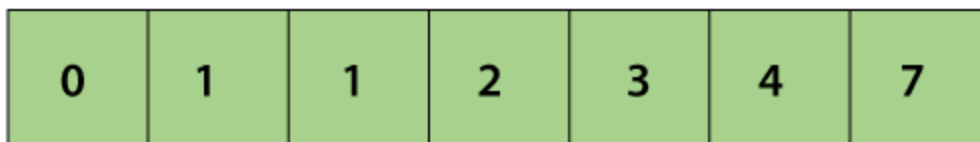
After swapping, the array will be -



Now, the array is completely sorted, but the algorithm has to complete the entire pass without any **swap** to know that the array is sorted.



So, after sorting the array elements will be -



Now, the array is completely sorted.

Now, the array is completely sorted.

## Cocktail sort complexity

Now, let's see the time complexity of cocktail sort in the best case, average case, and worst case.

We will also see the space complexity of the cocktail sort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Cased   | $O(n)$          |
| Average Case | $O(n^2)$        |
| Worst Case   | $O(n^2)$        |

**Best Case Complexity** - It occurs when there is no sorting required, i.e., the array is already sorted. The best-case time complexity of cocktail sort is  **$O(n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of cocktail sort is  **$O(n^2)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of cocktail sort is  **$O(n^2)$** .

### 2. Space Complexity

|                  |        |
|------------------|--------|
| Space Complexity | $O(1)$ |
|------------------|--------|

|        |     |
|--------|-----|
| Stable | YES |
|--------|-----|

The space complexity of the cocktail sort is  $O(1)$ .

## Implementation of cocktail sort

Now, let's see the programs of cocktail sort in different programming languages.

**Program:** Write a program to implement cocktail sort in C language.

```
#include <stdio.h>
```

```
/* function to implement Cocktail sort */
```

```
void cocktail(int a[], int n)
```

```
{
```

```
    int swap = 1;
```

```
    int beg = 0;
```

```
    int end = n - 1;
```

```
    int i, temp;
```

```
    while (swap)
```

```
    {
```

```
        swap = 0;
```

```
/* Loop similar to bubble sort to compare and swap array elements starting from left to right */
```

```
    for (i = beg; i < end; ++i)
```

```
    {
```

```
        if (a[i] > a[i + 1]) {
```

```
            temp = a[i];
```

```
            a[i] = a[i + 1];
```

```
            a[i + 1] = temp;
```

```
            swap = 1;
```

```
        }
```

```
    }
```

```
    if (!swap)
```

```
        break;
```

```
    swap = 0;
```

```
/* decrease the 'end' point by one position.
```

```
It is because the item at the last position is at its correct position */
```

```
--end;
```

```
/* This loop starts from right to left to perform the same comparison as in the previous loop */
```

```
    for (int i = end - 1; i >= beg; --i)
```

```
    {
```

```

        if (a[i] > a[i + 1]) {
            temp = a[i];
            a[i] = a[i+1];
            a[i+1] = temp;
            swap = 1;
        }
    }

    /* increase the beg point by one position.
    It is because the item at the first position is at its correct position */
    ++beg;
}
}

```

```

void print(int a[], int n) //function to print array elements
{
    int i;
    for(i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }
}

```

```

int main()
{
    int a[] = { 40, 10, 30, 5, 70, 2, 20 };
    int n = sizeof(a)/sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    print(a, n);
    cocktail(a, n);
    printf("\nAfter sorting array elements are - \n");
    print(a, n);
    return 0;
}

```

**Output:**

```

Before sorting array elements are -
40 10 30 5 70 2 20
After sorting array elements are -
2 5 10 20 30 40 70

```

**Program:** Write a program to implement cocktail sort in C++.

```
#include <iostream>
```

```
using namespace std;
```

/\* function to implement Cocktail sort \*/

**void** cocktail(**int** a[], **int** n)

{

**int** swap = 1;

**int** beg = 0;

**int** end = n - 1;

**int** i, temp;

**while** (swap)

    {

        swap = 0;

/\* Loop similar to bubble sort to compare and swap array elements starting from left to right \*/

**for** (i = beg; i < end; ++i)

    {

**if** (a[i] > a[i + 1]) {

            temp = a[i];

            a[i] = a[i+1];

            a[i+1] = temp;

            swap = 1;

        }

    }

**if** (!swap)

**break**;

    swap = 0;

/\* decrease the 'end' point by one position.

It is because the item at the last position is at its correct position \*/

--end;

/\* This loop starts from right to left to perform the same comparison as in the previous loop \*/

**for** (**int** i = end - 1; i >= beg; --i)

    {

**if** (a[i] > a[i + 1]) {

            temp = a[i];

            a[i] = a[i+1];

            a[i+1] = temp;

            swap = 1;

        }

    }

```

        /* increase the beg point by one position.
        It is because the item at the first position is at its correct position */
        ++beg;
    }
}

```

```

void print(int a[], int n) //function to print array elements
{
    int i;
    for(i = 0; i < n; i++)
    {
        cout<<a[i]<<" ";
    }
}

```

```

int main()
{
    int a[] = { 39, 5, 29, 4, 69, 1, 19 };
    int n = sizeof(a)/sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    print(a, n);
    cocktail(a, n);
    cout<<"\nAfter sorting array elements are - \n";
    print(a, n);
    return 0;
}

```

**Output:**

```

Before sorting array elements are -
39 5 29 4 69 1 19
After sorting array elements are -
1 4 5 19 29 39 69

```

**Program:** Write a program to implement cocktail sort in C#.

```

using System;

```

```

class CocktailSort {

```

```

    /* function to implement Cocktail sort */

```

```

    static void cocktail(int[] a, int n)

```

```

    {
        int swap = 1;
        int beg = 0;
        int end = n - 1;
        int i, temp;
        while (swap == 1)

```



```

{
    swap = 0;

    /* Loop similar to bubble sort to compare and swap array elements starting from left to right */
    for (i = beg; i < end; ++i)
    {
        if (a[i] > a[i + 1]) {
            temp = a[i];
            a[i] = a[i+1];
            a[i+1] = temp;
            swap = 1;
        }
    }

    if (swap == 0)
        break;

    swap = 0;

    /* decrease the 'end' point by one position.
    It is because the item at the last position is at its correct position */
    --end;

    /* This loop starts from right to left to perform the same comparison as in the previous loop */
    for (i = end - 1; i >= beg; --i)
    {
        if (a[i] > a[i + 1]) {
            temp = a[i];
            a[i] = a[i+1];
            a[i+1] = temp;
            swap = 1;
        }
    }

    /* increase the beg point by one position.
    It is because the item at the first position is at its correct position */
    ++beg;
}

static void print(int[] a, int n) //function to print array elements
{
    int i;

```

```

for(i = 0; i < n; i++)
{
    Console.Write(a[i] + " ");
}
}

```

```

static void Main()
{
    int[] a = { 38, 4, 28, 3, 68, 0, 18 };
    int n = a.Length;
    Console.WriteLine("Before sorting array elements are - \n");
    print(a, n);
    cocktail(a, n);
    Console.WriteLine("\nAfter sorting array elements are - \n");
    print(a, n);
}
}

```

#### Output:

```

Before sorting array elements are -
38 4 28 3 68 0 18
After sorting array elements are -
0 3 4 18 28 38 68

```

**Program:** Write a program to implement cocktail sort in Java.

```

class CocktailSort {

```

```

/* function to implement Cocktail sort */

```

```

static void cocktail(int a[], int n)

```

```

{
    int swap = 1;
    int beg = 0;
    int end = n - 1;
    int i, temp;
    while (swap == 1)
    {
        swap = 0;

```

```

/* Loop similar to bubble sort to compare and swap array elements starting

```

```

from left to right */

```

```

    for (i = beg; i < end; ++i)
    {
        if (a[i] > a[i + 1]) {
            temp = a[i];

```

```

        a[i] = a[i+1];
        a[i+1] = temp;
        swap = 1;
    }
}

```

```

if (swap == 0)
    break;

```

```

swap = 0;

```

/\* decrease the 'end' point by one position.  
 It **is** because the item at the last position **is** at its

```

correct position */
--end;

```

/\* This loop starts **from** right to left to perform the

same comparison as **in** the previous loop \*/

```

for (i = end - 1; i >= beg; --i)
{
    if (a[i] > a[i + 1]) {
        temp = a[i];
        a[i] = a[i+1];
        a[i+1] = temp;
        swap = 1;
    }
}

```

/\* increase the beg point by one position.  
 It **is** because the item at the first position **is** at its

```

correct position */
++beg;
}
}

```

static void **print**(int a[], int n) //function to **print** array elements

```

{
    int i;
    for(i = 0; i < n; i++)
    {
        System.out.print(a[i] + " ");
    }
}

```

```
}  
}
```

```
public static void main(String args[])  
{  
    int[] a = { 37, 3, 27, 2, 67, 0, 17 };  
    int n = a.length;  
    System.out.print("Before sorting array elements are - \n");  
    print(a, n);  
    cocktail(a, n);  
    System.out.print("\nAfter sorting array elements are - \n");  
    print(a, n);  
}  
}
```

**Output:**

```
D:\JTP>javac CocktailSort.java  
D:\JTP>java CocktailSort  
Before sorting array elements are -  
37 3 27 2 67 0 17  
After sorting array elements are -  
0 2 3 17 27 37 67  
D:\JTP>
```

So, that's all about the article. Hope the article will be helpful and informative to you.

## Cycle sort algorithm

In this article, we will discuss the Cycle sort Algorithm. Cycle sort is a comparison sorting algorithm that forces array to be factored into the number of cycles where each of them can be rotated to produce a sorted array. It is theoretically optimal in the sense that it reduces the number of writes to the original array.

It is an in-place and unstable sorting algorithm. Cycle sort forces an array to be factored into a number of cycles where every element can rotate in order to produce a sorted array. The time complexity of cycle sort is  $O(n^2)$ , even in the best case.

Now, let's see the algorithm of cycle sort.

## Algorithm

Suppose there is an array **arr** with **n** distinct elements. Given an element **A**, we can find its index by counting the number of elements smaller than **A**.

If the element is at its correct position, simply leave it as it is.

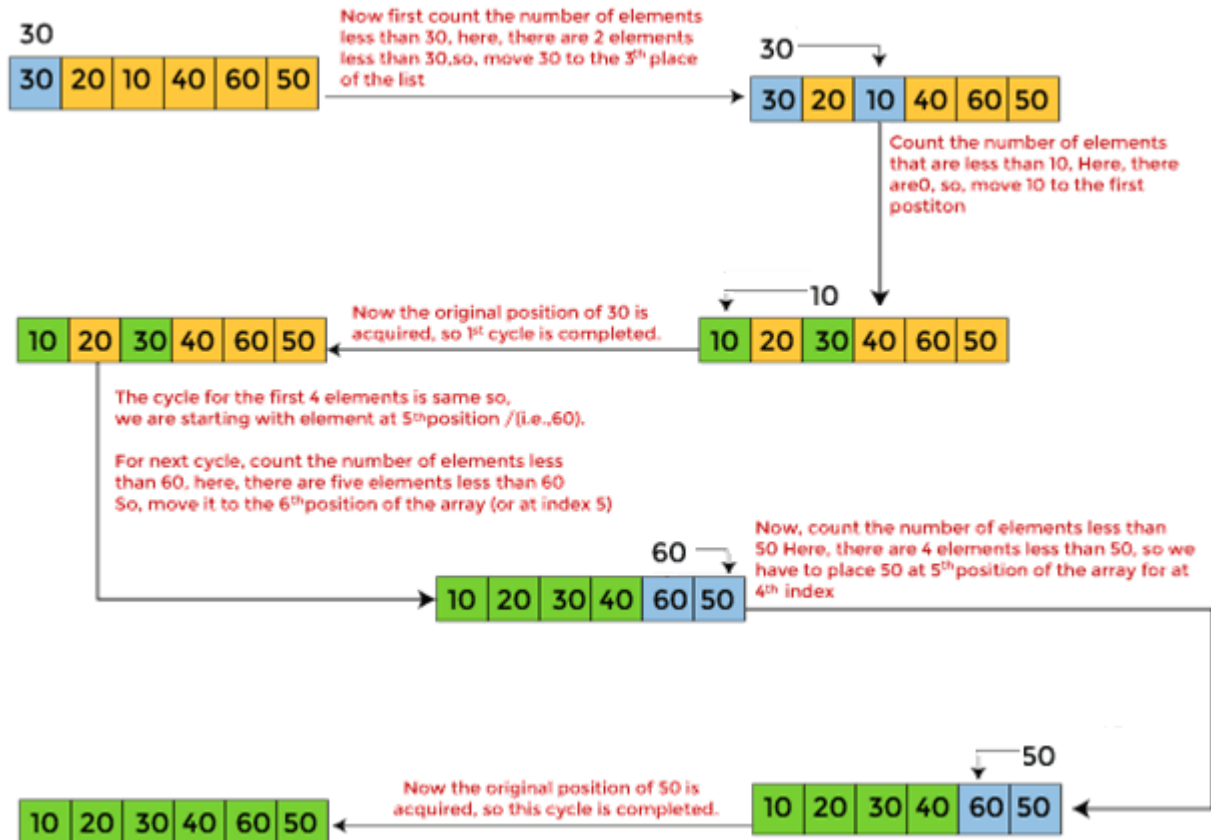
Else, we have to find the correct position of **A** by counting the number of elements smaller than it. Another element **B** is replaced to be moved to its correct position. This process continues until we get an element at the original position of **A**.

The above-illustrated process constitutes a cycle. Repeat this cycle for every element of the list until the list is sorted.

# Working of Cycle sort Algorithm

Now, let's see the working of Cycle sort Algorithm. To understand the working of cycle sort algorithm, let's take an unsorted array.

Let the elements of array are - {30, 20, 10, 40, 60, 50}



Now, the given array is completely sorted.

## Cycle sort complexity

Now, let's see the time complexity of cycle sort in the best case, average case, and worst case.

We will also see the space complexity of cycle sort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(n^2)$        |
| Average Case | $O(n^2)$        |
| Worst Case   | $O(n^2)$        |

The time complexity of cycle sort in all three cases is  $O(n^2)$ . Even in the best case, it takes  $O(n^2)$  time to sort the array elements. In Cycle sort, there is always the traversing of the entire subarray

from the current position in order to count the number of elements less than the current element.

So, in cycle sort, it doesn't matter whether the given array is already sorted or not. It has no consequence on the running time of the algorithm. So, the cycle sort must run in the quadratic time.

## 2. Space Complexity

|                  |      |
|------------------|------|
| Space Complexity | O(1) |
| Stable           | No   |

The space complexity of cycle sort is O(1).

## Implementation of Cycle sort

Now, let's see the programs of cycle sort in different programming languages.

**Program:** Write a program to implement cycle sort in C language.

```
#include<stdio.h>
```

```
/*function to implement to cycle sort*/
```

```
void cycleSort(int a[], int n)
```

```
{
```

```
    int start, element, pos, temp, i;
```

```
    /*Loop to traverse the array elements and place them on the correct position*/
```

```
    for (start = 0; start <= n - 2; start++) {  
        element = a[start];
```

```
        /*position to place the element*/
```

```
        pos = start;
```

```
        for (i = start + 1; i < n; i++)
```

```
            if (a[i] < element)
```

```
                pos++;
```

```
        if (pos == start) /*if the element is at exact position*/
```

```
            continue;
```

```
        while (element == a[pos])
```

```
            pos += 1;
```

```
        if (pos != start) /*put element at its exact position*/
```

```
        {
```

```
            //swap(element, a[pos]);
```

```
            temp = element;
```

```
            element = a[pos];
```

```
            a[pos] = temp;
```

```
        }
```

```
        /*Rotate rest of the elements*/
```

```

while (pos != start)
{
    pos = start;
    /*find position to put the element*/
    for (i = start + 1; i < n; i++)
        if (a[i] < element)
            pos += 1;

    /*Ignore duplicate elements*/
    while (element == a[pos])
        pos += 1;

    /*put element to its correct position*/
    if (element != a[pos])
    {
        temp = element;
        element = a[pos];
        a[pos] = temp;
    }
}

}

void print(int a[], int n) /*function to print array elements*/
{
    int i;
    for(i = 0; i < n; i++)
    {
        printf("%d ",a[i]);
    }
}

int main()
{
    int a[] = {90, 45, 30, 20, 10, 40, 60, 50, 5, 2};
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    print(a, n);
    cycleSort(a, n);
    printf("\nAfter applying cycle sort, array elements are - \n");
    print(a, n);
    return 0;
}

```

```
}
```

## Output

```
Before sorting array elements are -  
90 45 30 20 10 40 60 50 5 2  
After applying cycle sort, array elements are -  
2 5 10 20 30 40 45 50 60 90
```

**Program:** Write a program to implement cycle sort in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
/*function to implement to cycle sort*/
```

```
void cycleSort(int a[], int n)
```

```
{
```

```
    int start, element, pos, temp, i;
```

```
/*Loop to traverse the array elements and place them on the correct position*/
```

```
for (start = 0; start <= n - 2; start++) {
```

```
    element = a[start];
```

```
/*position to place the element*/
```

```
pos = start;
```

```
for (i = start + 1; i < n; i++)
```

```
    if (a[i] < element)
```

```
        pos++;
```

```
if (pos == start) /*if the element is at exact position*/
```

```
    continue;
```

```
while (element == a[pos])
```

```
    pos += 1;
```

```
if (pos != start) /*put element at its exact position*/
```

```
{
```

```
    //swap(element, a[pos]);
```

```
    temp = element;
```

```
    element = a[pos];
```

```
    a[pos] = temp;
```

```
}
```

```
/*Rotate rest of the elements*/
```

```
while (pos != start)
```

```
{
```

```
    pos = start;
```

```
/*find position to put the element*/
```

```
for (i = start + 1; i < n; i++)
```

```
    if (a[i] < element)
```



```

        pos += 1;

        /*Ignore duplicate elements*/
        while (element == a[pos])
            pos += 1;

        /*put element to its correct position*/
        if (element != a[pos])
        {
            temp = element;
            element = a[pos];
            a[pos] = temp;
        }
    }
}

void print(int a[], int n) /*function to print array elements*/
{
    int i;
    for(i = 0; i < n; i++)
    {
        cout<<a[i]<<" ";
    }
}

int main()
{
    int a[] = {89, 44, 29, 19, 9, 39, 59, 49, 4, 1};
    int n = sizeof(a) / sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    print(a, n);
    cycleSort(a, n);
    cout<<"\nAfter applying cycle sort, array elements are - \n";
    print(a, n);
    return 0;
}

```

### Output

```

Before sorting array elements are -
89 44 29 19 9 39 59 49 4 1
After applying cycle sort, array elements are -
1 4 9 19 29 39 44 49 59 89

```

**Program:** Write a program to implement cycle sort in C#.

**using** System;

**class** CycleSort {

/\*function to implement to cycle sort\*/

**static void** cycleSort(**int**[] a, **int** n)

{

**int** start, element, pos, temp, i;

/\*Loop to traverse the array elements and place them on the correct position\*/

**for** (start = 0; start <= n - 2; start++) {

        element = a[start];

/\*position to place the element\*/

        pos = start;

**for** (i = start + 1; i < n; i++)

**if** (a[i] < element)

                pos++;

**if** (pos == start) /\*if the element is at exact position\*/

**continue**;

**while** (element == a[pos])

            pos += 1;

**if** (pos != start) /\*put element at its exact position\*/

        {

            //swap(element, a[pos]);

            temp = element;

            element = a[pos];

            a[pos] = temp;

        }

/\*Rotate rest of the elements\*/

**while** (pos != start)

        {

            pos = start;

/\*find position to put the element\*/

**for** (i = start + 1; i < n; i++)

**if** (a[i] < element)

                    pos += 1;

/\*Ignore duplicate elements\*/

**while** (element == a[pos])

            pos += 1;

/\*put element to its correct position\*/

```

        if (element != a[pos])
        {
            temp = element;
            element = a[pos];
            a[pos] = temp;
        }
    }
}

static void print(int[] a, int n) /*function to print array elements*/
{
    int i;
    for(i = 0; i < n; i++)
    {
        Console.Write(a[i] + " ");
    }
}

static void Main()
{
    int[] a = {88, 43, 28, 18, 8, 38, 58, 48, 3, 0};
    int n = a.Length;
    Console.WriteLine("Before sorting array elements are - \n");
    print(a, n);
    cycleSort(a, n);
    Console.WriteLine("\nAfter applying cycle sort, array elements are - \n");
    print(a, n);
}
}

```

### Output

After the execution of the above code, the output will be -

```

Before sorting array elements are -
88 43 28 18 8 38 58 48 3 0
After applying cycle sort, array elements are -
0 3 8 18 28 38 43 48 58 88

```

**Program:** Write a program to implement cycle sort in Java.

```

class CycleSort
{
    /*function to implement to cycle sort*/
    static void cycleSort(int a[], int n)

```

```

{
    int start, element, pos, temp, i;

    /*Loop to traverse the array elements and place them on the correct
position*/
    for (start = 0; start <= n - 2; start++) {
        element = a[start];

        /*position to place the element*/
        pos = start;

        for (i = start + 1; i < n; i++)
            if (a[i] < element)
                pos++;
        if (pos == start) /*if the element is at exact position*/
            continue;
        while (element == a[pos])
            pos += 1;
        if (pos != start) /*put element at its exact position*/
        {
            //swap(element, a[pos]);
            temp = element;
            element = a[pos];
            a[pos] = temp;
        }
        /*Rotate rest of the elements*/
        while (pos != start)
        {
            pos = start;
            /*find position to put the element*/
            for (i = start + 1; i < n; i++)
                if (a[i] < element)
                    pos += 1;

            /*Ignore duplicate elements*/
            while (element == a[pos])
                pos += 1;

            /*put element to its correct position*/
            if (element != a[pos])
            {
                temp = element;
                element = a[pos];
            }
        }
    }
}

```

```

        a[pos] = temp;
    }
}

}

static void print(int a[], int n) /*function to print array elements*/
{
    int i;
    for(i = 0; i < n; i++)
    {
        System.out.print(a[i] + " ");
    }
}

public static void main(String args[])
{
    int[] a = {87, 42, 27, 17, 7, 37, 57, 47, 2, 1};
    int n = a.length;
    System.out.print("Before sorting array elements are - \n");
    print(a, n);
    cycleSort(a, n);
    System.out.print("\nAfter applying cycle sort, array elements are - \n");
    print(a, n);
}

}

```

## Output

After the execution of the above code, the output will be -

```

D:\JTP>javac CycleSort.java

D:\JTP>java CycleSort
Before sorting array elements are -
87 42 27 17 7 37 57 47 2 1
After applying cycle sort, array elements are -
1 2 7 17 27 37 42 47 57 87
D:\JTP>

```

So, that's all about the article. Hope, the article will be helpful and informative to you.

# Tim Sort Algorithm

In this article, we will discuss the Tim sort Algorithm. Tim-sort is a sorting algorithm derived from insertion sort and merge sort. It was designed to perform optimally on different kind of real-world data.

Tim sort is an adaptive sorting algorithm that needs  $O(n \log n)$  comparisons to sort an array of  $n$  elements. It was designed and implemented by Tim Peters in 2002 in a python programming language. It has been python's standard sorting algorithm since version 2.3. It is the fastest sorting algorithm.

The basic approach used in the Tim sort algorithm is - first sort small chunks by using the insertion sort and then merge all the big chunks using the merge function of the merge sort. Now, let's see the algorithm of Tim sort.

## Algorithm

Step 1 - Divide the array into the number of blocks known as run.

Step 2 - Consider the size of run, either 32 or 64.

Step 3 - Sort the individual elements of every run one by one using insertion sort.

Step 4 - Merge the sorted runs one by one using the merge function of merge sort.

Step 5 - Double the size of merged sub-arrays after every iteration.

## Working of Tim sort Algorithm

Now, let's see the working of the Tim sort Algorithm.

In Tim sort, first the array is divided into small chunks that are known as **RUN**. After dividing, each individual **RUN** is taken, and sorted using the insertion sort technique. After that, all sorted RUNs are merged using the merge() function of the merge sort algorithm.

In Tim sort, the advantage of using the insertion sort is that insertion sort works efficiently for the array with small size.

## Example of Tim sort Algorithm

Let's see the example of Tim sort algorithm. To understand the working of Tim sort algorithm, let's take an unsorted array. It will be easier to understand the Tim sort via an example.

Suppose the array elements are -

|    |    |    |    |    |    |   |    |
|----|----|----|----|----|----|---|----|
| 40 | 10 | 20 | 42 | 27 | 25 | 1 | 19 |
|----|----|----|----|----|----|---|----|

For the simple illustration, let's consider the size of RUN is 4.

Now, divide the given array into two sub-arrays that are -

|    |    |    |    |    |    |   |    |
|----|----|----|----|----|----|---|----|
| 40 | 10 | 20 | 42 | 27 | 25 | 1 | 19 |
|----|----|----|----|----|----|---|----|

The first sub-array is -

|    |    |    |    |
|----|----|----|----|
| 40 | 10 | 20 | 42 |
|----|----|----|----|

Sorted subarray

Unsorted subarray

Array

|      |              |                  |
|------|--------------|------------------|
| (40) | (10, 20, 42) | (40, 10, 20, 42) |
|------|--------------|------------------|

**First iteration**

**a[1] = 10**

| Sorted subarray | Unsorted subarray | Array            |
|-----------------|-------------------|------------------|
| (10, 40)        | (20, 42)          | (10, 40, 20, 42) |

**Second iteration**

**a[2] = 20**

| Sorted subarray | Unsorted subarray | Array            |
|-----------------|-------------------|------------------|
| (10, 20, 40)    | (42)              | (10, 20, 40, 42) |

**Third iteration**

**a[3] = 42**

| Sorted subarray  | Unsorted subarray | Final array      |
|------------------|-------------------|------------------|
| (10, 20, 40, 42) | ()                | (10, 20, 40, 42) |

**The second sub-array is -**

|           |           |          |           |
|-----------|-----------|----------|-----------|
| <b>27</b> | <b>25</b> | <b>1</b> | <b>19</b> |
|-----------|-----------|----------|-----------|

| Sorted subarray | Unsorted subarray | Final array     |
|-----------------|-------------------|-----------------|
| (27)            | (25, 1, 19)       | (27, 25, 1, 19) |

**First iteration**

**a[1] = 25**

| Sorted subarray | Unsorted subarray | Final array     |
|-----------------|-------------------|-----------------|
| (25, 27)        | (1, 19)           | (25, 27, 1, 19) |

**Second iteration**

**a[2] = 1**

| Sorted subarray | Unsorted subarray | Final array     |
|-----------------|-------------------|-----------------|
| (1, 25, 27)     | (19)              | (1, 25, 27, 19) |

### Third iteration

a[3] = 19

| Sorted subarray | Unsorted subarray | Final array     |
|-----------------|-------------------|-----------------|
| (1, 19, 25, 27) | ()                | (1, 19, 25, 27) |

Now, merge both sorted subarrays to get the final array as -

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 10 | 20 | 40 | 42 | 1  | 19 | 25 | 27 |
| 1  | 10 | 19 | 20 | 25 | 27 | 40 | 42 |

Now, the array is completely sorted.

## Tim sort complexity

Now, let's see the time complexity of Tim sort in the best case, average case, and worst case. We will also see the space complexity of Tim sort.

### 1. Time Complexity

| Case         | Time Complexity |
|--------------|-----------------|
| Best Case    | $O(n)$          |
| Average Case | $O(n \log n)$   |
| Worst Case   | $O(n \log n)$   |

**Best Case Complexity** - It occurs when there is no sorting required, i.e. the array is already sorted. The best-case time complexity of Tim sort is  **$O(n)$** .

**Average Case Complexity** - It occurs when the array elements are in jumbled order that is not properly ascending and not properly descending. The average case time complexity of Tim sort is  **$O(n \log n)$** .

**Worst Case Complexity** - It occurs when the array elements are required to be sorted in reverse order. That means suppose you have to sort the array elements in ascending order, but its elements are in descending order. The worst-case time complexity of Tim sort is  **$O(n \log n)$** .

### 2. Space Complexity



|                  |        |
|------------------|--------|
| Space Complexity | $O(n)$ |
| Stable           | YES    |

The space complexity of Tim sort is  $O(n)$ .

## Implementation of Tim sort

Now, let's see the programs of Tim sort in different programming languages.

**Program:** Write a program to implement Tim sort in C language.

```
#include<stdio.h>
const int RUN = 32;
int min(int a, int b)
{
    if(a<b)
        return a;
    else
        return b;
}
/* This function sorts array from starting index to ending index which is of size atmost RUN */
void insertionSort(int a[], int beg, int end) /* function to sort an array with insertion sort */
{
    int i, j, temp;
    for (i = beg + 1; i <= end; i++) {
        temp = a[i];
        j = i - 1;

        while(j>=0 && temp <= a[j])
        {
            a[j+1] = a[j];
            j = j-1;
        }
        a[j+1] = temp;
    }
}
/* Function to merge the sorted runs */
void merge(int a[], int beg, int mid, int end)
{
    int i, j, k;
    int n1 = mid - beg + 1;
    int n2 = end - mid;
    int LeftArray[n1], RightArray[n2]; //temporary arrays
    /* copy data to temp arrays */
    for (int i = 0; i < n1; i++)
        LeftArray[i] = a[beg + i];
    for (int j = 0; j < n2; j++)
```

```

RightArray[j] = a[mid + 1 + j];
i = 0;
j = 0;
k = beg;
while (i < n1 && j < n2)
{
    if(LeftArray[i] <= RightArray[j])
    {
        a[k] = LeftArray[i];
        i++;
    }
    else
    {
        a[k] = RightArray[j];
        j++;
    }
    k++;
}
while (i < n1)
{
    a[k] = LeftArray[i];
    i++;
    k++;
}
while (j < n2)
{
    a[k] = RightArray[j];
    j++;
    k++;
}
}
/* function to implement tim sort */
void timSort(int a[], int n)
{
    /* Sort individual subarrays of size RUN */
    for (int i = 0; i < n; i+=RUN)
        insertionSort(a, i, min((i+RUN-1), (n-1)));
    // Start merging from size RUN (or 32).
    for (int size = RUN; size < n; size = 2*size)
    {
        for (int beg = 0; beg < n; beg += 2*size)
        {
            /* find ending point of left sub array. The starting point of right sub array is mid + 1 */

```

```

    int mid = beg + size - 1;
    int end = min((beg + 2*size - 1), (n-1));

    /* Merge subarray a[beg...mid] and a[mid+1...end] */
    if (mid < end)
        merge(a, beg, mid, end);
    }
}
/* function to print the array elements */
void printArr(int a[], int n)
{
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}
int main()
{
    int a[] = { 40, 12, 31, 27, 25, 8, 1, 32, 17 };
    int n = sizeof(a) / sizeof(a[0]);
    printf("Before sorting array elements are - \n");
    printArr(a, n);
    timSort(a, n);
    printf("\nAfter sorting array elements are - \n");
    printArr(a, n);
    return 0;
}

```

### Output

After the execution of the above code, the output will be -

```

Before sorting array elements are -
40 12 31 27 25 8 1 32 17

After sorting array elements are -
1 8 12 17 25 27 31 32 40

```

**Program:** Write a program to implement Tim sort in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
const int RUN = 32;
```

```
int min(int a, int b)
```

```
{
```

```
    if(a<b)
```

```
        return a;
```

```
    else
```

```
        return b;
```

```

}
/* This function sorts array from starting index to ending index which is of size atmost RUN */
void insertionSort(int a[], int beg, int end) /* function to sort an array with insertion sort */
{
    int i, j, temp;
    for (i = beg + 1; i <= end; i++) {
        temp = a[i];
        j = i - 1;

        while(j >= 0 && temp <= a[j])
        {
            a[j+1] = a[j];
            j = j-1;
        }
        a[j+1] = temp;
    }
}

/* Function to merge the sorted runs */
void merge(int a[], int beg, int mid, int end)
{
    int i, j, k;
    int n1 = mid - beg + 1;
    int n2 = end - mid;
    int LeftArray[n1], RightArray[n2]; //temporary arrays
    /* copy data to temp arrays */
    for (int i = 0; i < n1; i++)
        LeftArray[i] = a[beg + i];
    for (int j = 0; j < n2; j++)
        RightArray[j] = a[mid + 1 + j];
    i = 0;
    j = 0;
    k = beg;
    while (i < n1 && j < n2)
    {
        if(LeftArray[i] <= RightArray[j])
        {
            a[k] = LeftArray[i];
            i++;
        }
        else
        {
            a[k] = RightArray[j];
            j++;
        }
    }
}

```

```

        k++;
    }
    while (i < n1)
    {
        a[k] = LeftArray[i];
        i++;
        k++;
    }
    while (j < n2)
    {
        a[k] = RightArray[j];
        j++;
        k++;
    }
}

/* function to implement tim sort */
void timSort(int a[], int n)
{
    /* Sort individual subarrays of size RUN */
    for (int i = 0; i < n; i += RUN)
        insertionSort(a, i, min((i + RUN - 1), (n - 1)));

    // Start merging from size RUN (or 32).
    for (int size = RUN; size < n; size = 2 * size)
    {
        for (int beg = 0; beg < n; beg += 2 * size)
        {
            /* find ending point of left sub array. The starting point of right sub array is mid + 1 */
            int mid = beg + size - 1;
            int end = min((beg + 2 * size - 1), (n - 1));

            /* Merge subarray a[beg...mid] and a[mid+1...end] */
            if (mid < end)
                merge(a, beg, mid, end);
        }
    }
}

/* function to print the array elements */
void printArr(int a[], int n)
{
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
}

int main()

```

```

{
    int a[] = { 39, 11, 30, 26, 24, 7, 0, 31, 16 };
    int n = sizeof(a) / sizeof(a[0]);
    cout<<"Before sorting array elements are - \n";
    printArr(a, n);
    timSort(a, n);
    cout<<"\nAfter sorting array elements are - \n";
    printArr(a, n);
    return 0;
}

```

### Output

```

Before sorting array elements are -
39 11 30 26 24 7 0 31 16
After sorting array elements are -
0 7 11 16 24 26 30 31 39

```

**Program:** Write a program to implement Tim sort in C#.

```

using System;
class TimSort {
    const int RUN = 32;
    static int min(int a, int b)
    {
        if(a<b)
            return a;
        else
            return b;
    }
    /* This function sorts array from starting index to ending index which is of size atmost RUN */
    static void insertionSort(int[] a, int beg, int end) /* function to sort an array with insertion sort */
    {
        int i, j, temp;
        for (i = beg + 1; i <= end; i++)
        {
            temp = a[i];
            j = i - 1;

            while(j >= beg && temp <= a[j])
            {
                a[j+1] = a[j];
                j = j-1;
            }
            a[j+1] = temp;
        }
    }
}

```

```

/* Function to merge the sorted runs */
public static void merge(int[] a, int beg, int mid, int end)
{
    int i, j, k;
    int n1 = mid - beg + 1;
    int n2 = end - mid;
    //temporary arrays
    int[] LeftArray = new int [n1];
    int[] RightArray = new int [n2];
    /* copy data to temp arrays */
    for (i = 0; i < n1; i++)
        LeftArray[i] = a[beg + i];
    for (j = 0; j < n2; j++)
        RightArray[j] = a[mid + 1 + j];
    i = 0;
    j = 0;
    k = beg;
    while (i < n1 && j < n2)
    {
        if(LeftArray[i] <= RightArray[j])
        {
            a[k] = LeftArray[i];
            i++;
        }
        else
        {
            a[k] = RightArray[j];
            j++;
        }
        k++;
    }
    while (i < n1)
    {
        a[k] = LeftArray[i];
        i++;
        k++;
    }
    while (j < n2)
    {
        a[k] = RightArray[j];
        j++;
        k++;
    }
}

```

```

/* function to implement tim sort */
public static void timSort(int[] a, int n)
{
    /* Sort individual subarrays of size RUN */
    for (int i = 0; i < n; i+=RUN)
        insertionSort(a, i, min((i+RUN-1), (n-1)));
    // Start merging from size RUN (or 32).
    for (int size = RUN; size < n; size = 2*size)
    {
        for (int beg = 0; beg < n; beg += 2*size)
        {
            /* find ending point of left sub array. The starting point of right sub array is mid + 1 */
            int mid = beg + size - 1;
            int end = min((beg + 2*size - 1), (n-1));

            /* Merge subarray a[beg...mid] and a[mid+1...end] */
            if (mid < end)
                merge(a, beg, mid, end);
        }
    }
}

/* function to print the array elements */
static void printArr(int[] a, int n)
{
    for (int i = 0; i < n; i++)
        Console.Write(a[i] + " ");
}

static void Main()
{
    int[] a = { 39, 11, 30, 26, 24, 7, 0, 31, 16 };
    int n = a.Length;
    Console.WriteLine("Before sorting array elements are - ");
    printArr(a, n);
    timSort(a, n);
    Console.WriteLine("\nAfter sorting array elements are - ");
    printArr(a, n);
}
}

```

### Output

```

Before sorting array elements are - 39 11 30 26 24 7 0 31 16
After sorting array elements are - 0 7 11 16 24 26 30 31 39

```

**Program:** Write a program to implement Tim sort in Java.

```

class TimSort {
    static int RUN = 32;

```



```

int min(int a, int b)
{
    if(a<b)
        return a;
    else
        return b;
}
/* This function sorts array from starting index to ending index which is of
size atmost RUN */
void insertionSort(int a[], int beg, int end) /* function to sort an array with
insertion sort */
{
    int i, j, temp;
    for (i = beg + 1; i <= end; i++)
    {
        temp = a[i];
        j = i - 1;

        while(j >= beg && temp <= a[j])
        {
            a[j+1] = a[j];
            j = j-1;
        }
        a[j+1] = temp;
    }
}
/* Function to merge the sorted runs */
void merge(int a[], int beg, int mid, int end)
{
    int i, j, k;
    int n1 = mid - beg + 1;
    int n2 = end - mid;
    //temporary arrays
    int[] LeftArray = new int [n1];
    int[] RightArray = new int [n2];
    /* copy data to temp arrays */
    for (i = 0; i < n1; i++)
        LeftArray[i] = a[beg + i];
    for (j = 0; j < n2; j++)
        RightArray[j] = a[mid + 1 + j];
    i = 0;
    j = 0;
    k = beg;
    while (i < n1 && j < n2)

```

```

{
    if(LeftArray[i] <= RightArray[j])
    {
        a[k] = LeftArray[i];
        i++;
    }
    else
    {
        a[k] = RightArray[j];
        j++;
    }
    k++;
}
while (i < n1)
{
    a[k] = LeftArray[i];
    i++;
    k++;
}
while (j < n2)
{
    a[k] = RightArray[j];
    j++;
    k++;
}
}
/* function to implement tim sort */
void timSort(int a[], int n)
{
    /* Sort individual subarrays of size RUN */
    for (int i = 0; i < n; i += RUN)
        insertionSort(a, i, min((i + RUN - 1), (n - 1)));
    // Start merging from size RUN (or 32).
    for (int size = RUN; size < n; size = 2 * size)
    {
        for (int beg = 0; beg < n; beg += 2 * size)
        {
            /* find ending point of left subarray. The
            starting point of right sub array is mid + 1 */
            int mid = beg + size - 1;
            int end = min((beg + 2 * size - 1), (n - 1));

            /* Merge subarray a[beg...mid] and a[mid
            + 1...end] */

```

```

        if(mid < end)
            merge(a, beg, mid, end);
    }
}
}
/* function to print the array elements */
void printArr(int[] a, int n)
{
    for (int i = 0; i < n; i++)
        System.out.print(a[i] + " ");
}
public static void main(String args[])
{
    int a[] = { 38, 10, 29, 25, 23, 6, 2, 30, 15 };
    int n = a.length;
    TimSort t1 = new TimSort();
    System.out.print("\nBefore sorting array elements are - ");
    t1.printArr(a, n);
    t1.timSort(a, n);
    System.out.print("\nAfter sorting array elements are - ");
    t1.printArr(a, n);
    System.out.println();
}
}

```

## Output

After the execution of above code, the output will be -

```

D:\JTP>javac TimSort.java
D:\JTP>java TimSort
Before sorting array elements are - 38 10 29 25 23 6 2 30 15
After sorting array elements are - 2 6 10 15 23 25 29 30 38

```

So, that's all about the article. Hope the article will be helpful and informative to you.

# Linear vs Non-Linear data structure

## What is Data structure?

A data structure is a technique of storing and organizing the data in such a way that the data can be utilized in an efficient manner. In [computer science](#), a data structure is designed in such a way that it can work with various algorithms. A data structure is classified into two categories:

Linear data structure

Non-linear data structure

Now let's have a brief look at both these data structures.

## What is the Linear data structure?

A linear [data structure](#) is a structure in which the elements are stored sequentially, and the elements are connected to the previous and the next element. As the elements are stored

sequentially, so they can be traversed or accessed in a single run. The implementation of linear data structures is easier as the elements are sequentially organized in memory. The data elements in an array are traversed one after another and can access only one element at a time. The types of linear data structures are Array, Queue, Stack, Linked List.

**Let's discuss each linear data structure in detail.**

**Array:** An array consists of data elements of a same data type. For example, if we want to store the roll numbers of 10 students, so instead of creating 10 integer type variables, we will create an array having size 10. Therefore, we can say that an array saves a lot of memory and reduces the length of the code.

**Stack:** It is linear data structure that uses the LIFO (Last In-First Out) rule in which the data added last will be removed first. The addition of data element in a stack is known as a push operation, and the deletion of data element from the list is known as pop operation.

**Queue:** It is a data structure that uses the FIFO rule (First In-First Out). In this rule, the element which is added first will be removed first. There are two terms used in the queue **front** end and **rear**. The insertion operation performed at the back end is known as enqueue, and the deletion operation performed at the front end is known as dequeue.

**Linked list:** It is a collection of nodes that are made up of two parts, i.e., data element and reference to the next node in the sequence.

## What is a Non-linear data structure?

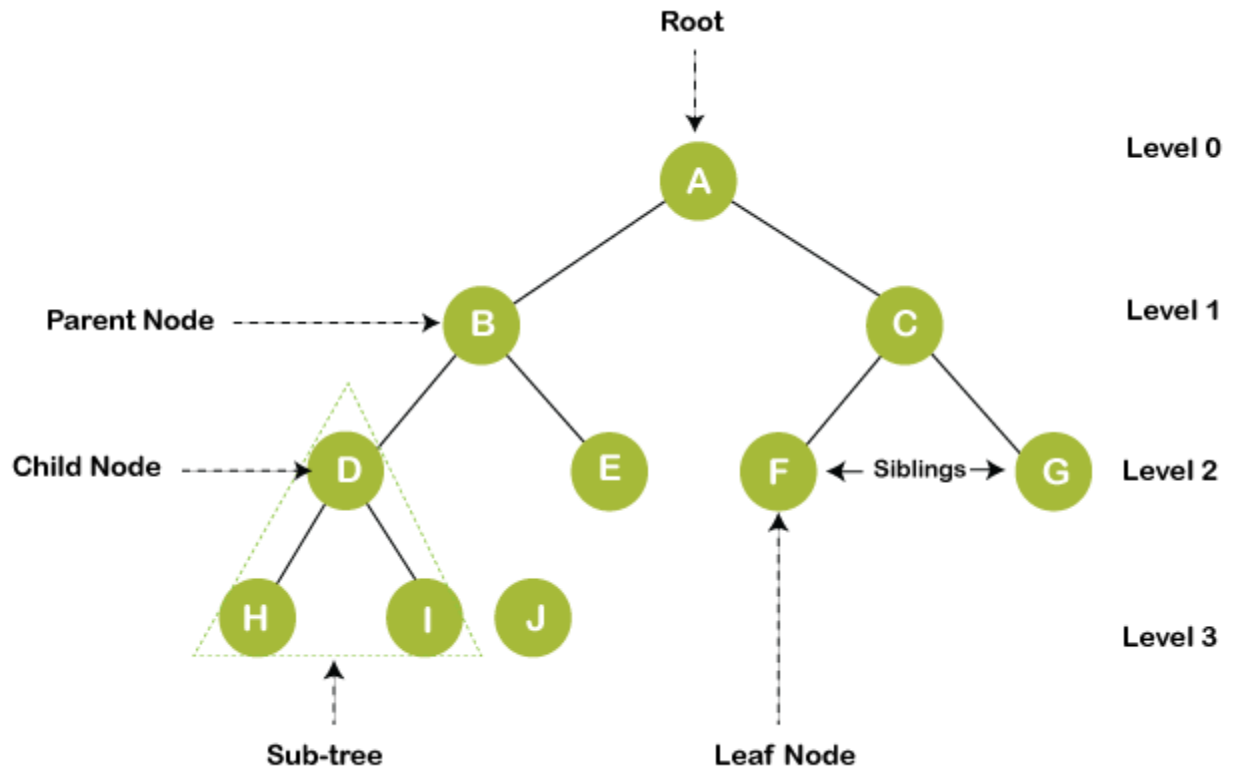
A non-linear data structure is also another type of data structure in which the data elements are not arranged in a contiguous manner. As the arrangement is nonsequential, so the data elements cannot be traversed or accessed in a single run. In the case of linear data structure, element is connected to two elements (previous and the next element), whereas, in the non-linear data structure, an element can be connected to more than two elements.

**Trees** and **Graphs** are the types of non-linear data structure.

**Let's discuss both the data structures in detail.**

### Tree

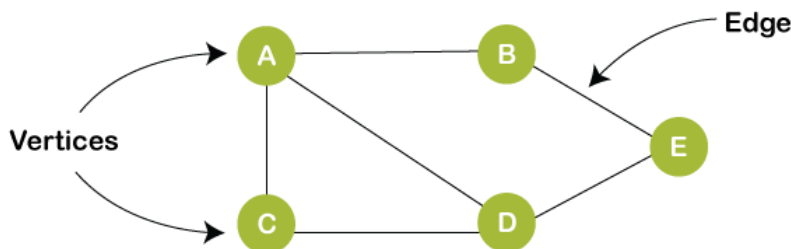
It is a non-linear data structure that consists of various linked nodes. It has a hierarchical tree structure that forms a parent-child relationship. The diagrammatic representation of a **tree** data structure is shown below:



**For example**, the posts of employees are arranged in a tree data structure like managers, officers, clerk. In the above figure, **A** represents a manager, **B** and **C** represent the officers, and other nodes represent the clerks.

### Graph

A graph is a non-linear data structure that has a finite number of vertices and edges, and these edges are used to connect the vertices. The vertices are used to store the data elements, while the edges represent the relationship between the vertices. A graph is used in various real-world problems like telephone networks, circuit networks, social networks like LinkedIn, Facebook. In the case of facebook, a single user can be considered as a node, and the connection of a user with others is known as edges.



**Differences between the Linear data structure and non-linear data structure.**

|                           | Linear Data structure                                                                                          | Non-Linear Data structure                                                                                                    |
|---------------------------|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>Basic</b>              | In this structure, the elements are arranged sequentially or linearly and attached to one another.             | In this structure, the elements are arranged hierarchically or non-linear manner.                                            |
| <b>Types</b>              | Arrays, linked list, stack, queue are the types of a linear data structure.                                    | Trees and graphs are the types of a non-linear data structure.                                                               |
| <b>implementation</b>     | Due to the linear organization, they are easy to implement.                                                    | Due to the non-linear organization, they are difficult to implement.                                                         |
| <b>Traversal</b>          | As linear data structure is a single level, so it requires a single run to traverse each data item.            | The data items in a non-linear data structure cannot be accessed in a single run. It requires multiple runs to be traversed. |
| <b>Arrangement</b>        | Each data item is attached to the previous and next items.                                                     | Each item is attached to many other items.                                                                                   |
| <b>Levels</b>             | This data structure does not contain any hierarchy, and all the data elements are organized in a single level. | In this, the data elements are arranged in multiple levels.                                                                  |
| <b>Memory utilization</b> | In this, the memory utilization is not efficient.                                                              | In this, memory is utilized in a very efficient manner.                                                                      |
| <b>Time complexity</b>    | The time complexity of linear data structure increases with the increase in the input size.                    | The time complexity of non-linear data structure often remains same with the increase in the input size.                     |
| <b>Applications</b>       | Linear data structures are mainly used for developing the software.                                            | Non-linear data structures are used in <b>image processing</b> and <b>Artificial Intelligence</b> .                          |

## Array vs Linked List

**Array** and **Linked list** are the two ways of organizing the data in the memory. Before understanding the differences between the **Array** and the **Linked List**, we first look at an **array** and a **linked list**.

### What is an array?

An [array](#) is a [data structure](#) that contains the elements of the same type. A data structure is a way of organizing the data; an array is a data structure because it sequentially organizes the data. An array is a big chunk of memory in which memory is divided into small-small blocks, and each block is capable of storing some value.

Suppose we have created an array that consists of 10 values, then each block will store the value of an integer type. If we try to store the value in an array of different types, then it is not a correct array and will throw a compile-time error.

## Declaration of array

**An array can be declared as:**

`data_type name of the array[no of elements]`

To declare an array, we first need to specify the type of the array and then the array's name. Inside the square brackets, we need to specify the number of elements that our array should contain.

**Let's understand through an example.**

```
int a[5];
```

In the above case, we have declared an array of 5 elements with 'a' name of an **integer** data type.

## What is Linked list?

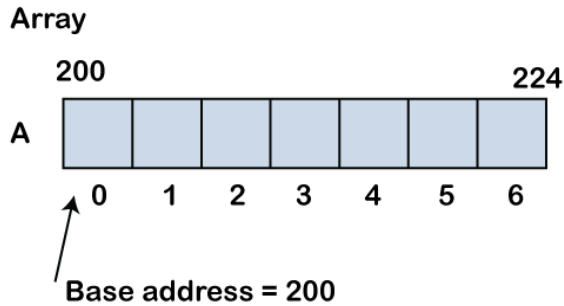
A linked list is the collection of nodes that are randomly stored. Each node consists of two fields, i.e., **data** and **link**. Here, data is the value stored at that particular node, and the link is the pointer that holds the address of the next node.

## Differences between Array and Linked list

We cannot say which data structure is better, i.e., array or [linked list](#). There can be a possibility that one data structure is better for one kind of requirement, while the other data structure is better for another kind of requirement. There are various factors like what are the frequent operations performed on the data structure or the size of the data, and other factors also on which basis the data structure is selected. Now we will see some differences between the array and the linked list based on some parameters.

### 1. Cost of accessing an element

In case of an array, irrespective of the size of an array, an array takes a constant time for accessing an element. In an array, the elements are stored in a contiguous manner, so if we know the base address of the element, then we can easily get the address of any element in an array. We need to perform a simple calculation to obtain the address of any element in an array. So, accessing the element in an array is **O(1)** in terms of time complexity.



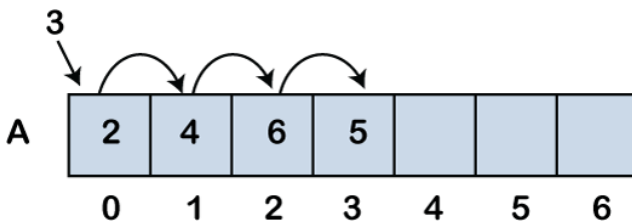
In the linked list, the elements are not stored in a contiguous manner. It consists of multiple blocks, and each block is represented as a node. Each node has two fields, i.e., one is for the data field, and another one stores the address of the next node. To find any node in the linked list, we first need to determine the first node known as the head node. If we have to find the second node in the list, then we need to traverse from the first node, and in the worst case, to find the last node, we will be traversing all the nodes. The average case for accessing the element is  $O(n)$ .

We conclude that the cost of accessing an element in array is less than the linked list. Therefore, if we have any requirement for accessing the elements, then array is a better choice.

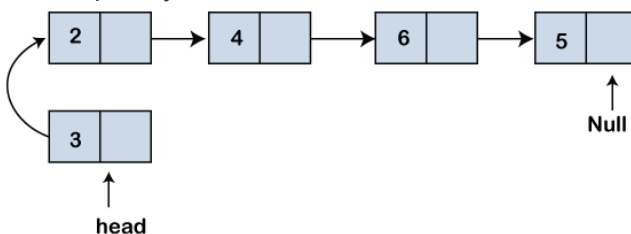
## 2. Cost of inserting an element

**There can be three scenarios in the insertion:**

**Inserting the element at the beginning:** To insert the new element at the beginning, we first need to shift the element towards the right to create a space in the first position. So, the time complexity will be proportional to the size of the list. If  $n$  is the size of the array, the time complexity would be  $O(n)$ .



In the case of a linked list, to insert an element at the starting of the linked list, we will create a new node, and the address of the first node is added to the new node. In this way, the new node becomes the first node. So, the time complexity is not proportional to the size of the list. The time complexity would be constant, i.e.,  $O(1)$ .



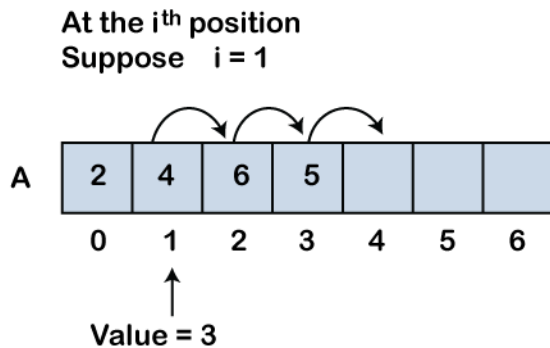
**Inserting an element at the end**



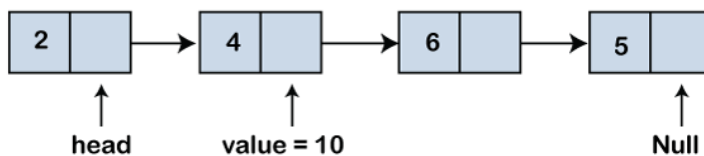
If the array is not full, then we can directly add the new element through the index. In this case, the time complexity would be constant, i.e.,  $O(1)$ . If the array is full, we first need to copy the array into another array and add a new element. In this case, the time complexity would be  $O(n)$ . To insert an element at the end of the linked list, we have to traverse the whole list. If the linked list consists of  $n$  elements, then the time complexity would be  $O(n)$ .

### Inserting an element at the mid

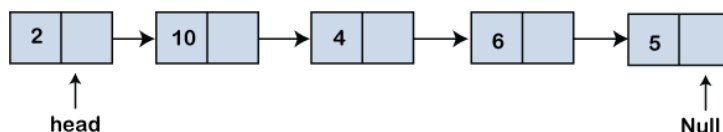
Suppose we want to insert the element at the  $i^{\text{th}}$  position of the array; we need to shift the  $n/2$  elements towards the right. Therefore, the time complexity is proportional to the number of the elements. The time complexity would be  $O(n)$  for the average case.



In the case of linked list, we have to traverse to that position where we have to insert the new element. Even though, we do not have to perform any kind of shifting, but we have to traverse to  $n/2$  position. The time taken is proportional to the  $n$  number of elements, and the time complexity for the average case would be  $O(n)$ .



The resultant linked list is:



### Ease of use

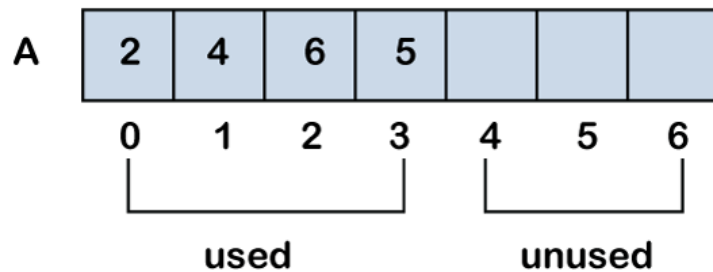
The implementation of an array is easy as compared to the linked list. While creating a program using a linked list, the program is more prone to errors like segmentation fault or memory leak. So, lots of care need to be taken while creating a program in the linked list.

### Dynamic in size

The linked list is dynamic in size whereas the array is static. Here, static doesn't mean that we cannot decide the size at the run time, but we cannot change it once the size is decided.

## 3. Memory requirements

As the elements in an array store in one contiguous block of memory, so array is of fixed size. Suppose we have an array of size 7, and the array consists of 4 elements then the rest of the space is unused. The memory occupied by the 7 elements:



**Memory space =  $7 \times 4 = 28$  bytes**

Where 7 is the number of elements in an array and 4 is the number of bytes of an integer type. In case of linked list, there is no unused memory but the extra memory is occupied by the pointer variables. If the data is of integer type, then total memory occupied by one node is 8 bytes, i.e., 4 bytes for data and 4 bytes for pointer variable. If the linked list consists of 4 elements, then the memory space occupied by the linked list would be:

**Memory space =  $8 \times 4 = 32$  bytes**

The linked list would be a better choice if the data part is larger in size. Suppose the data is of 16 bytes. The memory space occupied by the array would be  $16 \times 7 = 112$  bytes while the linked list occupies  $20 \times 4 = 80$ , here we have specified 20 bytes as 16 bytes for the size of the data plus 4 bytes for the pointer variable. If we are choosing the larger size of data, then the linked list would consume a less memory; otherwise, it depends on the factors that we are adopting to determine the size.

**Let's look at the differences between the array and linked list in a tabular form.**

| Array                                                                                                                           | Linked list                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| An array is a collection of elements of a similar data type.                                                                    | A linked list is a collection of objects known as a node where node consists of two parts, i.e., data and address.                                       |
| Array elements store in a contiguous memory location.                                                                           | Linked list elements can be stored anywhere in the memory or randomly stored.                                                                            |
| Array works with a static memory. Here static memory means that the memory size is fixed and cannot be changed at the run time. | The Linked list works with dynamic memory. Here, dynamic memory means that the memory size can be changed at the run time according to our requirements. |

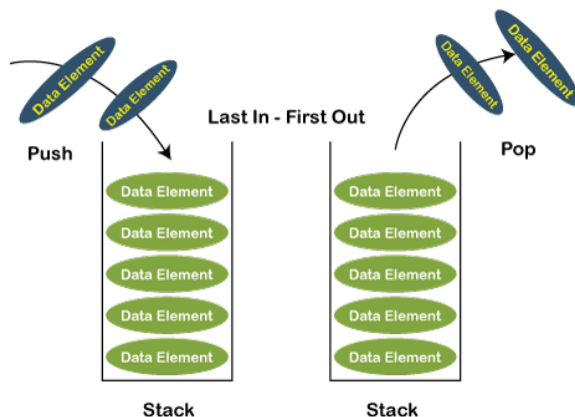
|                                                                                                                                                                              |                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Array elements are independent of each other.                                                                                                                                | Linked list elements are dependent on each other. As each node contains the address of the next node so to access the next node, we need to access its previous node. |
| Array takes more time while performing any operation like insertion, deletion, etc.                                                                                          | Linked list takes less time while performing any operation like insertion, deletion, etc.                                                                             |
| Accessing any element in an array is faster as the element in an array can be directly accessed through the index.                                                           | Accessing an element in a linked list is slower as it starts traversing from the first element of the linked list.                                                    |
| In the case of an array, memory is allocated at compile-time.                                                                                                                | In the case of a linked list, memory is allocated at run time.                                                                                                        |
| Memory utilization is inefficient in the array. For example, if the size of the array is 6, and array consists of 3 elements only then the rest of the space will be unused. | Memory utilization is efficient in the case of a linked list as the memory can be allocated or deallocated at the run time according to our requirement.              |

## Stack vs. Queue

First, we will look at **what is stack** and **what is queue** individually, and then we will discuss the differences between stack and queue.

### What is a Stack?

A Stack is a linear data structure. In case of an array, random access is possible, i.e., any element of an array can be accessed at any time, whereas in a stack, the sequential access is only possible. It is a container that follows the insertion and deletion rule. It follows the principle **LIFO (Last In First Out)** in which the insertion and deletion take place from one side known as a **top**. In stack, we can insert the elements of a similar data type, i.e., the different data type elements cannot be inserted in the same stack. The two operations are performed in LIFO, i.e., **push** and **pop** operation.



The following are the operations that can be performed on the stack:

**push(x):** It is an operation in which the elements are inserted at the top of the stack. In the **push** function, we need to pass an element which we want to insert in a stack.

**pop():** It is an operation in which the elements are deleted from the top of the stack. In the **pop()** function, we do not have to pass any argument.

**peek()/top():** This function returns the value of the topmost element available in the stack. Like **pop()**, it returns the value of the topmost element but does not remove that element from the stack.

**isEmpty():** If the stack is empty, then this function will return a true value or else it will return a false value.

**isFull():** If the stack is full, then this function will return a true value or else it will return a false value.

In stack, the **top** is a pointer which is used to keep track of the last inserted element. To implement the stack, we should know the size of the stack. We need to allocate the memory to get the size of the stack. There are two ways to implement the stack:

**Static:** The static implementation of the stack can be done with the help of arrays.

**Dynamic:** The dynamic implementation of the stack can be done with the help of a linked list.

## What is the Queue?

A Queue is a linear data structure. It is an ordered list that follows the principle FIFO (First In - First Out). A Queue is a structure that follows some restrictions on insertion and deletion. In the case of Queue, insertion is performed from one end, and that end is known as a rear end. The deletion is performed from another end, and that end is known as a front end. In Queue, the technical words for insertion and deletion are **enqueue()** and **dequeue()**, respectively whereas, in the case of the stack, the technical words for insertion and deletion are **push()** and **pop()**, respectively. Its structure contains two pointers **front pointer** and **rear pointer**, where the front pointer is a pointer that points to the element that was first added in the queue and the rear pointer that points to the element inserted last in the queue.



## Similarities between stack and queue.

There are two similarities between the stack and queue:

### Linear data structure

Both the stack and queue are the linear data structure, which means that the elements are stored sequentially and accessed in a single run.

### Flexible in size

Both the stack and queue are flexible in size, which means they can grow and shrink according to the requirements at the run-time.

## Differences between stack and queue

## The following are the differences between the stack and queue:

| Basis for comparison                      | Stack                                                                                                                                                 | Queue                                                                                                                                                                                                             |
|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Principle</b>                          | It follows the principle LIFO (Last In- First Out), which implies that the element which is inserted last would be the first one to be deleted.       | It follows the principle FIFO (First In -First Out), which implies that the element which is added first would be the first element to be removed from the list.                                                  |
| <b>Structure</b>                          | It has only one end from which both the insertion and deletion take place, and that end is known as a top.                                            | It has two ends, i.e., front and rear end. The front end is used for the deletion while the rear end is used for the insertion.                                                                                   |
| <b>Number of pointers used</b>            | It contains only one pointer known as a top pointer. The top pointer holds the address of the last inserted or the topmost element of the stack.      | It contains two pointers front and rear pointer. The front pointer holds the address of the first element, whereas the rear pointer holds the address of the last element in a queue.                             |
| <b>Operations performed</b>               | It performs two operations, push and pop. The push operation inserts the element in a list while the pop operation removes the element from the list. | It performs mainly two operations, enqueue and dequeue. The enqueue operation performs the insertion of the elements in a queue while the dequeue operation performs the deletion of the elements from the queue. |
| <b>Examination of the empty condition</b> | If $\text{top} == -1$ , which means that the stack is empty.                                                                                          | If $\text{front} == -1$ or $\text{front} = \text{rear} + 1$ , which means that the queue is empty.                                                                                                                |
| <b>Examination of full condition</b>      | If $\text{top} == \text{max} - 1$ , this condition implies that the stack is full.                                                                    | If $\text{rear} == \text{max} - 1$ , this condition implies that the stack is full.                                                                                                                               |
| <b>Variants</b>                           | It does not have any types.                                                                                                                           | It is of three types like priority queue, circular queue and double ended queue.                                                                                                                                  |

|                       |                                                 |                                                             |
|-----------------------|-------------------------------------------------|-------------------------------------------------------------|
| <b>Implementation</b> | It has a simpler implementation.                | It has a comparatively complex implementation than a stack. |
| <b>Visualization</b>  | A Stack is visualized as a vertical collection. | A Queue is visualized as a horizontal collection.           |

## Linear vs Circular Queue

### What is a Linear Queue?

A linear queue is a linear data structure that serves the request first, which has been arrived first. It consists of data elements which are connected in a linear fashion. It has two pointers, i.e., front and rear, where the insertion takes place from the front end, and deletion occurs from the front end.



### Operations on Linear Queue

There are two operations that can be performed on a linear queue:

**Enqueue:** The enqueue operation inserts the new element from the rear end.

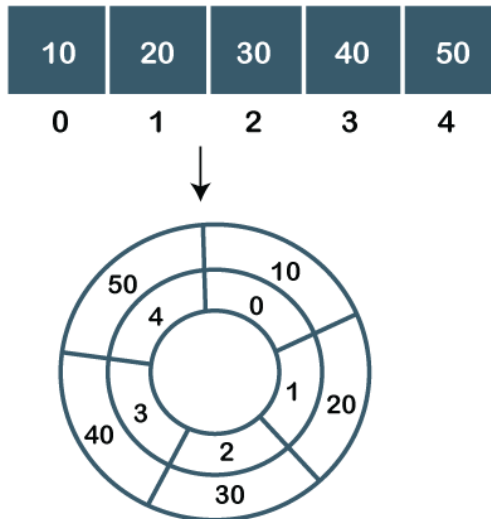
**Dequeue:** The dequeue operation is used to delete the existing element from the front end of the queue.

### What is a Circular Queue?

As we know that in a queue, the front pointer points to the first element while the rear pointer points to the last element of the queue. The problem that arises with the linear queue is that if some empty cells occur at the beginning of the queue then we cannot insert new element at the empty space as the rear cannot be further incremented.

A circular queue is also a linear data structure like a normal queue that follows the FIFO principle but it does not end the queue; it connects the last position of the queue to the first position of the queue. If we want to insert new elements at the beginning of the queue, we can insert it using the circular queue data structure.

In the circular queue, when the rear reaches the end of the queue, then rear is reset to zero. It helps in refilling all the free spaces. The problem of managing the circular queue is overcome if the first position of the queue comes after the last position of the queue.



### Conditions for the queue to be a circular queue

Front == 0 and rear = n-1

Front = rear + 1

If either of the above conditions is satisfied means that the queue is a circular queue.

## Operations on Circular Queue

The following are the two operations that can be performed on a circular queue are:

**Enqueue:** It inserts an element in a queue. The given below are the scenarios that can be considered while inserting an element:

If the queue is empty, then the front and rear are set to 0 to insert a new element.

If queue is not empty, then the value of the rear gets incremented.

If queue is not empty and rear is equal to n-1, then rear is set to 0.

**Dequeue:** It performs a deletion operation in the Queue. The following are the points or cases that can be considered while deleting an element:

If there is only one element in a queue, after the dequeue operation is performed on the queue, the queue will become empty. In this case, the front and rear values are set to -1.

If the value of the front is equal to n-1, after the dequeue operation is performed, the value of the front variable is set to 0.

If either of the above conditions is not fulfilled, then the front value is incremented.

## Differences between linear Queue and Circular Queue

| Basis of comparison | Linear Queue                                                                                           | Circular Queue                                                                                                                                       |
|---------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Meaning             | The linear queue is a type of linear data structure that contains the elements in a sequential manner. | The circular queue is also a linear data structure in which the last element of the Queue is connected to the first element, thus creating a circle. |

|                               |                                                                                                |                                                                            |
|-------------------------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| <b>Insertion and Deletion</b> | In linear queue, insertion is done from the rear end, and deletion is done from the front end. | In circular queue, the insertion and deletion can take place from any end. |
| <b>Memory space</b>           | The memory space occupied by the linear queue is more than the circular queue.                 | It requires less memory as compared to linear queue.                       |
| <b>Memory utilization</b>     | The usage of memory is inefficient.                                                            | The memory can be more efficiently utilized.                               |
| <b>Order of execution</b>     | It follows the FIFO principle in order to perform the tasks.                                   | It has no specific order for execution.                                    |

## Linear Search vs Binary Search

Before understanding the differences between the linear and binary search, we should first know the linear search and binary search separately.

### What is a linear search?

A linear search is also known as a sequential search that simply scans each element at a time. Suppose we want to search an element in an array or list; we simply calculate its length and do not jump at any item.

**Let's consider a simple example.**

**Suppose we have an array of 10 elements as shown in the below figure:**



The above figure shows an array of character type having 10 values. If we want to search 'E', then the searching begins from the 0<sup>th</sup> element and scans each element until the element, i.e., 'E' is not found. We cannot directly jump from the 0<sup>th</sup> element to the 4<sup>th</sup> element, i.e., each element is scanned one by one till the element is not found.

### Complexity of Linear search

As linear search scans each element one by one until the element is not found. If the number of elements increases, the number of elements to be scanned is also increased. We can say that the **time taken to search the elements is proportional to the number of elements**. Therefore, the worst-case complexity is  $O(n)$

### What is a Binary search?



The one ***pre-requisite of binary search*** is that an array should be in sorted order, whereas the linear search works on both sorted and unsorted array. The binary search algorithm is based on the divide and conquer technique, which means that it will divide the array recursively.

**Case 1:  $\text{data} < a[\text{mid}]$  then  $\text{left} = \text{mid} + 1$ .**

**Case 3: data = a[mid] // element is found**

**Let's understand the working of binary search through an example.**

We want to search for 70 element from the above array.

[illegible]
$$mid = \frac{left + right}{2}$$

|      |    |    |    |     |    |    |    |    |       |
|------|----|----|----|-----|----|----|----|----|-------|
| 10   | 20 | 30 | 40 | 50  | 60 | 70 | 80 | 90 | 100   |
| 0    | 1  | 2  | 3  | 4   | 5  | 6  | 7  | 8  | 9     |
| ↑    |    |    |    | ↑   |    |    |    |    | ↑     |
| Left |    |    |    | mid |    |    |    |    | Right |

**Step 2:** As `data>a[mid]`, so the value of `left` is incremented by `mid+1`, i.e., `left=mid+1`. The value of `mid` is 4, so the value of `left` becomes 5. Now, we have got a subarray as shown in the below figure:

|    |    |    |    |          |           |    |    |    |            |
|----|----|----|----|----------|-----------|----|----|----|------------|
| 10 | 20 | 30 | 40 | 50       | 60        | 70 | 80 | 90 | 100        |
| 0  | 1  | 2  | 3  | 4        | 5         | 6  | 7  | 8  | 9          |
|    |    |    |    | ↑<br>mid | ↑<br>Left |    |    |    | ↑<br>Right |

Now again, the mid-value is calculated by using the above formula, and the value of mid becomes 7. Now, the mid can be represented as:

|    |    |    |    |    |           |    |          |    |            |
|----|----|----|----|----|-----------|----|----------|----|------------|
| 10 | 20 | 30 | 40 | 50 | 60        | 70 | 80       | 90 | 100        |
| 0  | 1  | 2  | 3  | 4  | 5         | 6  | 7        | 8  | 9          |
|    |    |    |    |    | ↑<br>Left |    | ↑<br>mid |    | ↑<br>Right |

In the above figure, we can observe that  $a[mid] > data$ , so again, the value of  $mid$  will be calculated in the next step.

**Step 3:** As  $a[mid] > data$ , the value of `right` is decremented by `mid-1`. The value of `mid` is 7, so the value of `right` becomes 6. The array can be represented as:

|    |    |    |    |    |      |       |     |    |     |
|----|----|----|----|----|------|-------|-----|----|-----|
| 10 | 20 | 30 | 40 | 50 | 60   | 70    | 80  | 90 | 100 |
| 0  | 1  | 2  | 3  | 4  | 5    | 6     | 7   | 8  | 9   |
|    |    |    |    |    | ↑    | ↑     | ↑   |    |     |
|    |    |    |    |    | Left | Right | mid |    |     |

The value of mid will be calculated again. The values of left and right are 5 and 6, respectively. Therefore, the value of mid is 5. Now the mid can be represented in an array as shown below:

Diagram illustrating the initial state of the array and pointers for the binary search algorithm:

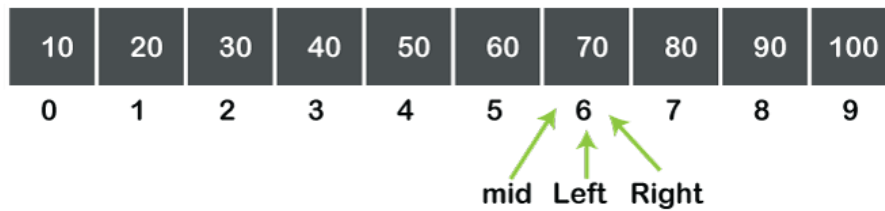
| 10 | 20 | 30 | 40 | 50 | 60 | 70 | 80 | 90 | 100 |
|----|----|----|----|----|----|----|----|----|-----|
| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9   |

Pointers: Left (points to index 5), mid (points to index 5), Right (points to index 6)

**In the above figure, we can observe that  $a[mid] < data$ .**

**Step 4:** As  $a[mid] < data$ , the left value is incremented by  $mid+1$ . The value of  $mid$  is 5, so the value of  $left$  becomes 6.

Now the value of mid is calculated again by using the formula which we have already discussed. The values of left and right are 6 and 6 respectively, so the value of mid becomes 6 as shown in the below figure:



We can observe in the above figure that  $a[\text{mid}] = \text{data}$ . Therefore, the search is completed, and the element is found successfully.

## Differences between Linear search and Binary search

The following are the differences between linear search and binary search:

### Description

Linear search is a search that finds an element in the list by searching the element sequentially until the element is found in the list. On the other hand, a binary search is a search that finds the middle element in the list recursively until the middle element is matched with a searched element.

### Working of both the searches

The linear search starts searching from the first element and scans one element at a time without jumping to the next element. On the other hand, binary search divides the array into half by calculating an array's middle element.

### Implementation

The linear search can be implemented on any linear data structure such as vector, singly linked list, double linked list. In contrast, the binary search can be implemented on those data structures with two-way traversal, i.e., forward and backward traversal.

### Complexity

The linear search is easy to use, or we can say that it is less complex as the elements for a linear search can be arranged in any order, whereas in a binary search, the elements must be arranged in a particular order.

### Sorted elements

The elements for a linear search can be arranged in random order. It is not mandatory in linear search that the elements are arranged in a sorted order. On the other hand, in a binary search, the elements must be arranged in sorted order. It can be arranged either in an increasing or in decreasing order, and accordingly, the algorithm will be changed. As binary search uses a sorted array, it is necessary to insert the element at the proper place. In contrast, the linear search does not need a sorted array, so that the new element can be easily inserted at the end of the array.

### Approach

The linear search uses an iterative approach to find the element, so it is also known as a sequential approach. In contrast, the binary search calculates the middle element of the array, so it uses the divide and conquer approach.

### **Data set**

Linear search is not suitable for the large data set. If we want to search the element, which is the last element of the array, a linear search will start searching from the first element and goes on till the last element, so the time taken to search the element would be large. On the other hand, binary search is suitable for a large data set as it takes less time.

### **Speed**

If the data set is large in linear search, then the computational cost would be high, and speed becomes slow. If the data set is large in binary search, then the computational cost would be less compared to a linear search, and speed becomes fast.

### **Dimensions**

Linear search can be used on both single and multidimensional array, whereas the binary search can be implemented only on the one-dimensional array.

### **Efficiency**

Linear search is less efficient when we consider the large data sets. Binary search is more efficient than the linear search in the case of large data sets.

**Let's look at the differences in a tabular form.**

| <b>Basis of comparison</b> | <b>Linear search</b>                                                                                                                       | <b>Binary search</b>                                                                                                               |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Definition</b>          | The linear search starts searching from the first element and compares each element with a searched element till the element is not found. | It finds the position of the searched element by finding the middle element of the array.                                          |
| <b>Sorted data</b>         | In a linear search, the elements don't need to be arranged in sorted order.                                                                | The pre-condition for the binary search is that the elements must be arranged in a sorted order.                                   |
| <b>Implementation</b>      | The linear search can be implemented on any linear data structure such as an array, linked list, etc.                                      | The implementation of binary search is limited as it can be implemented only on those data structures that have two-way traversal. |

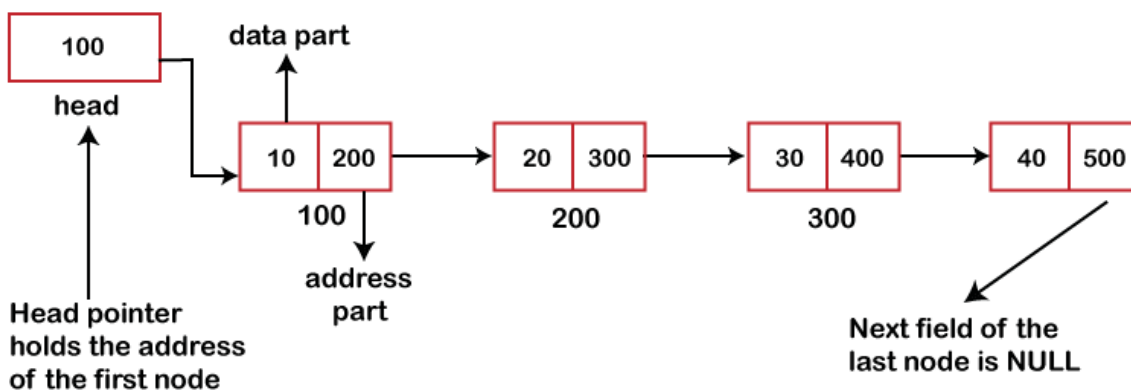
|                            |                                                                                                  |                                                                                                  |
|----------------------------|--------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>Approach</b>            | It is based on the sequential approach.                                                          | It is based on the divide and conquer approach.                                                  |
| <b>Size</b>                | It is preferable for the small-sized data sets.                                                  | It is preferable for the large-size data sets.                                                   |
| <b>Efficiency</b>          | It is less efficient in the case of large-size data sets.                                        | It is more efficient in the case of large-size data sets.                                        |
| <b>Worst-case scenario</b> | In a linear search, the worst- case scenario for finding the element is $O(n)$ .                 | In a binary search, the worst-case scenario for finding the element is $O(\log_2 n)$ .           |
| <b>Best-case scenario</b>  | In a linear search, the best-case scenario for finding the first element in the list is $O(1)$ . | In a binary search, the best-case scenario for finding the first element in the list is $O(1)$ . |
| <b>Dimensional array</b>   | It can be implemented on both a single and multidimensional array.                               | It can be implemented only on a multidimensional array.                                          |

## Singly Linked List vs Doubly Linked List

Before looking at the differences between the singly linked list and doubly linked list, we first understand what is singly linked list and doubly linked list separately.

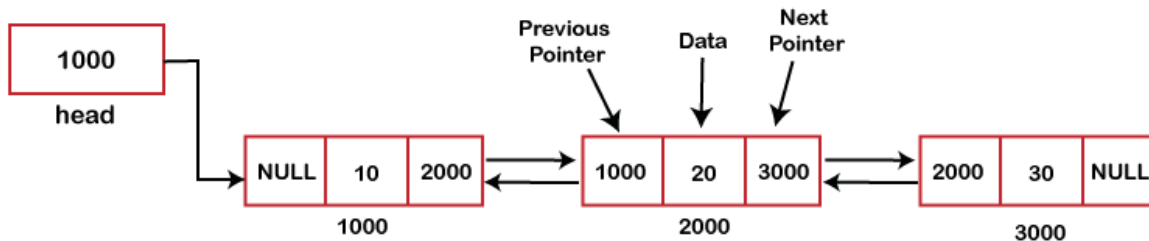
### What is a singly linked list?

A [singly linked list](#) can be simply called a [linked list](#). A singly linked list is a list that consists of a collection of nodes, and each node has two parts; one part is the data part, and another part is the address. The singly linked can also be called a chain as each node refers to another node through its address part. We can perform various operations on a singly linked list like insertion, deletion, and traversing.



### What is a doubly-linked list?

A doubly linked list is another type of the linked list. It is called a doubly linked list because it contains two addresses while a singly linked list contains a single address. It is a list that has total three parts, one is a data part, and others two are the pointers, i.e., previous and next. The previous pointer holds the address of the previous node, and the next pointer holds the address of the next node. Therefore, we can say that list has two references, i.e., forward and backward reference to traverse in either direction.



We can also perform various operations on a doubly-linked list like insertion, deletion, and traversing.

## Differences between the singly-linked list and doubly linked list.

The differences between the singly-linked list and doubly linked list are given below:

### Definition

The singly-linked is a linear data structure that consists of a collection of nodes in which one node consists of two parts, i.e., one is the data part, and another one is the address part. In contrast, a doubly-linked list is also a linear data structure in which the node consists of three parts, i.e., one is the data part, and the other two are the address parts.

### Direction

As we know that in a singly linked list, a node contains the address of the next node, so the elements can be traversed in only one direction, i.e., forward direction. In contrast, in a doubly-linked list, the node contains two pointers (previous pointer and next pointer) that hold the **address of the next node** and the **address of the previous node, respectively** so elements can be traversed in both directions.

### Memory space

The singly linked list occupies less memory space as it contains a single address. We know that the pointer variable stores the address, and the pointer variable occupies 4 bytes; therefore, the memory space occupied by the pointer variable in the singly linked list is also 4 bytes. The doubly linked list holds two addresses in a node, one is of the next node and the other one is of the previous node; therefore, the space occupied by the two pointer variables is 8 bytes.

### Insertion and Deletion

The insertion and deletion in a singly-linked list are less complex than a doubly linked list. If we insert an element in a singly linked list then we need to update the address of only next node. On the other hand, in the doubly linked list, we need to update the address of both the next and the previous node.

**Let's look at the differences in a tabular form.**

| Basis of comparison   | Singly linked list                                                                                                                                                                         | Doubly linked list                                                                                                                                                                                                                                                 |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Definition</b>     | A single linked list is a list of nodes in which node has two parts, the first part is the data part, and the next part is the pointer pointing to the next node in the sequence of nodes. | A doubly linked list is also a collection of nodes in which node has three fields, the first field is the pointer containing the address of the previous node, the second is the data field, and the third is the pointer containing the address of the next node. |
| <b>Access</b>         | The singly linked list can be traversed only in the forward direction.                                                                                                                     | The doubly linked list can be accessed in both directions.                                                                                                                                                                                                         |
| <b>List pointer</b>   | It requires only one list pointer variable, i.e., the head pointer pointing to the first node.                                                                                             | It requires two list pointer variables, <b>head</b> and <b>last</b> . The head pointer points to the first node, and the last pointer points to the last node of the list.                                                                                         |
| <b>Memory space</b>   | It utilizes less memory space.                                                                                                                                                             | It utilizes more memory space.                                                                                                                                                                                                                                     |
| <b>Efficiency</b>     | It is less efficient as compared to a doubly-linked list.                                                                                                                                  | It is more efficient.                                                                                                                                                                                                                                              |
| <b>Implementation</b> | It can be implemented on the stack.                                                                                                                                                        | It can be implemented on stack, heap and binary tree.                                                                                                                                                                                                              |
| <b>Complexity</b>     | In a singly linked list, the time complexity for inserting and deleting an element from the list is <b>O(n)</b> .                                                                          | In a doubly-linked list, the time complexity for inserting and deleting an element is <b>O(1)</b> .                                                                                                                                                                |

## Binary tree vs Binary Search tree

First, we will understand the **binary tree** and **binary search tree** separately, and then we will look at the differences between a binary tree and a binary search tree.

### What is a Binary tree?

A **Binary tree** is a non-linear data structure in which a node can have either **0, 1** or **maximum 2 nodes**. Each node in a binary tree is represented either as a parent node or a child node. There can be two children of the parent node, i.e., **left child** and **right child**.

There is only one way to reach from one node to its next node in a binary tree.

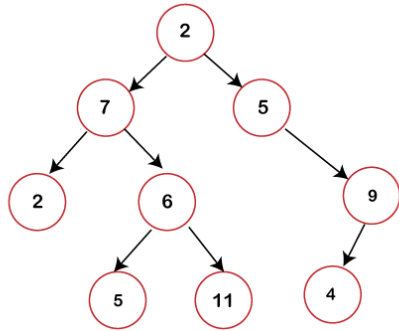
**A node in a binary tree has three fields:**

**Pointer to the left child:** It stores the reference of the left-child node.

**Pointer to the right child:** It stores the reference of the right-child node.

**Data element:** The data element is the value of the data which is stored by the node.

**The binary tree can be represented as:**



In the above figure, we can observe that each node contains utmost 2 children. If any node does not contain left or right child then the value of the pointer with respect to that child would be NULL.

**Basic terminologies used in a Binary tree are:**

**Root node:** The root node is the first or the topmost node in a binary tree.

**Parent node:** When a node is connected to another node through edges, then that node is known as a parent node. In a binary tree, parent node can have a maximum of 2 children.

**Child node:** If a node has its predecessor, then that node is known as a **child node**.

**Leaf node:** The node which does not contain any child known as a **leaf node**.

**Internal node:** The node that has atleast 2 children known as an **internal node**.

**Depth of a node:** The distance from the root node to the given node is known as a **depth of a node**. We provide labels to all the nodes like root node is labeled with 0 as it has no depth, children of the root nodes are labeled with 1, children of the root child are labeled with 2.

**Height:** The longest distance from the root node to the leaf node is the **height of the node**.

In a binary tree, there is one tree known as a **perfect binary tree**. It is a **tree** in which all the internal nodes must contain two nodes, and all the leaf nodes must be at the same depth. In the case of a perfect binary tree, the total number of nodes exist in a binary tree can be calculated by using the following equation:

$$n = 2^{m+1} - 1$$

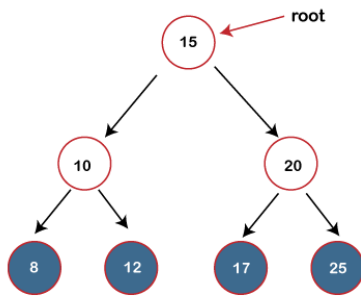
where **n** is the number of nodes, **m** is the depth of a node.

## What is a Binary Search tree?

A **Binary search tree** is a tree that follows some order to arrange the elements, whereas the binary tree does not follow any order. In a Binary search tree, the value of the left node must be smaller than the parent node, and the value of the right node must be greater than the parent node.

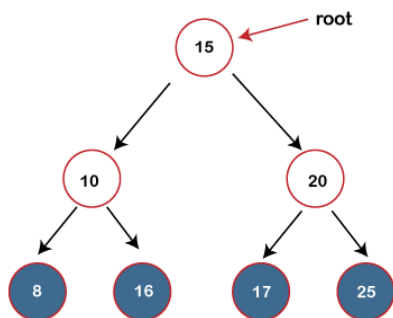
**Let's understand the concept of a binary search tree through examples.**





In the above figure, we can observe that the value of the root node is 15, which is greater than the value of all the nodes in the left subtree. The value of root node is less than the values of all the nodes in a right-subtree. Now, we move to the left-child of the root node. 10 is greater than 8 and lesser than 12; it also satisfies the property of the Binary search tree. Now, we move to the right-child of the root node; the value 20 is greater than 17 and lesser than 25; it also satisfies the property of binary search tree. Therefore, we can say that the tree shown above is the binary search tree.

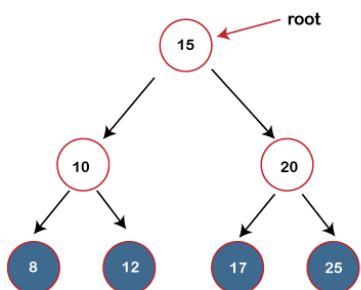
Now, if we change the value of 12 to 16 in the above binary tree, we have to find whether it is still a binary search tree or not.



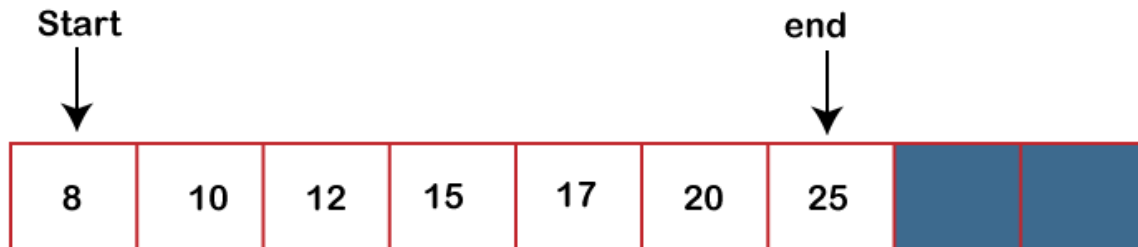
The value of the root node is 15 which is greater than 10 but lesser than 16, so it does not satisfy the property of the Binary search tree. Therefore, it is not a binary search tree.

## Operations on Binary search tree

We can perform insert, delete and search operations on the binary search tree. Let's understand how a search is performed on a binary search. The binary tree is shown below on which we have to perform the search operation:



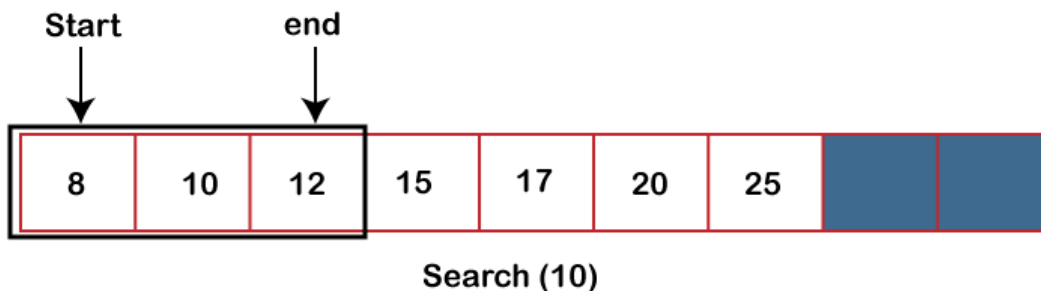
Suppose we have to search 10 in the above binary tree. To perform the binary search, we will consider all the integers in a sorted array. First, we create a complete list in a search space, and all the numbers will exist in the search space. The search space is marked by two pointers, i.e., start and end. The array of the above binary tree can be represented as



First, we will calculate the middle element and compare the middle element with the element, which is to be searched. The middle element is calculated by using  $n/2$ . The value of  $n$  is 7; therefore, the middle element is 15. The middle element is not equal to the searched element, i.e., 10.

**Note:** If the element is being searched is lesser than the mid element, then the searching will be performed in the left half; else, searching will be done on the right half. In the case of equality, the element is found.

As the element to be searched is lesser than the mid element, so searching will be performed on the left array. Now the search is reduced to half, as shown below:



The mid element in the left array is 10, which is equal to the searched element.

## Time complexity

In a binary search, there are  $n$  elements. If the middle element is not equal to the searched element, then the search space is reduced to  $n/2$ , and we will keep on reducing the search space by  $n/2$  until we found the element. In the whole reduction, if we move from  $n$  to  $n/2$  to  $n/4$  and so on, then it will take  $\log_2 n$  steps.

## Differences between Binary tree and Binary search tree

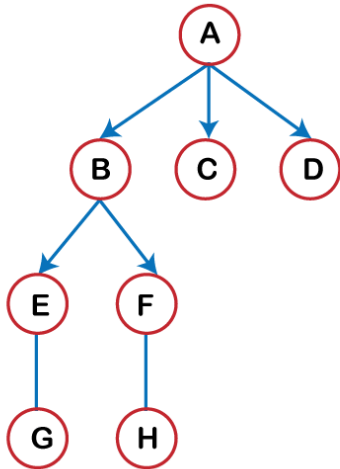
| Basis for comparison | Binary tree | Binary search tree |
|----------------------|-------------|--------------------|
|----------------------|-------------|--------------------|

|                   |                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Definition</b> | A binary tree is a non-linear data structure in which a node can have utmost two children, i.e., a node can have 0, 1 or maximum two children. A binary search tree is an ordered binary tree in which some order is followed to organize the nodes in a tree.                                             |                                                                                                                                                                                                                                                                |
| <b>Structure</b>  | The structure of the binary tree is that the first node or the topmost node is known as the root node. Each node in a binary tree contains the left pointer and the right pointer. The left pointer contains the address of the left subtree, whereas right pointer contains the address of right subtree. | The binary search tree is one of the types of binary tree that has the value of all the nodes in the left subtree lesser or equal to the root node, and the value of all the nodes in a right subtree are greater than or equal to the value of the root node. |
| <b>Operations</b> | The operations that can be implemented on a binary tree are insertion, deletion, and traversal.                                                                                                                                                                                                            | Binary search trees are the sorted binary trees that provide fast insertion, deletion and search. Lookups mainly implement binary search as all the keys are arranged in sorted order.                                                                         |
| <b>types</b>      | Four types of binary trees are Full Binary Tree, Complete Binary Tree, Perfect Binary Tree, and Extended Binary Tree.                                                                                                                                                                                      | There are different types of binary search trees such as AVL trees, Splay tree, Tango trees, etc.                                                                                                                                                              |

## Tree vs Graph data structure

Before knowing about the tree and graph data structure, we should know the linear and non-linear data structures. Linear data structure is a structure in which all the elements are stored sequentially and have only single level. In contrast, a non-linear data structure is a structure that follows a hierarchy, i.e., elements are arranged in multiple levels.

**Let's understand the structure that forms the hierarchy.**



In the above figure, we can assume the company hierarchy where A represents the CEO of the company, B, C and D represent the managers of the company, E and F represent the team leaders, and G and H represent the team members. This type of structure has more than one level, so it is known as a non-linear data structure.

## What is Tree?

A **tree** is a **non-linear** data structure that represents the hierarchy. A tree is a collection of nodes that are linked together to form a hierarchy.

Let's look at some terminologies used in a **tree** data structure.

**Root node:** The topmost node in a tree data structure is known as a root node. A root node is a node that does not have any parent.

**Parent of a node:** The immediate predecessor of a node is known as a parent of a node. Here predecessor means the previous node of that particular node.

**Child of a node:** The immediate successor of a node is known as a **child of a node**.

**Leaf node:** The leaf node is a node that does not have any child node. It is also known as an external node.

**Non-leaf node:** The non-leaf node is a node that has atleast one child node. It is also known as an **internal node**.

**Path:** It is a sequence of the consecutive edges from a source node to the destination node. Here edge is a link between two nodes.

**Ancestor:** The predecessor nodes that occur in the path from the root to that node is known as an ancestor.

**Descendant:** The successor nodes that exist in the path from that node to the leaf node.

**Sibling:** All the children that have the same parent node are known as siblings.

**Degree:** The number of children of a particular node is known as a degree.

**Depth of node:** The length of the path from the root to that node is known as a depth of a node.

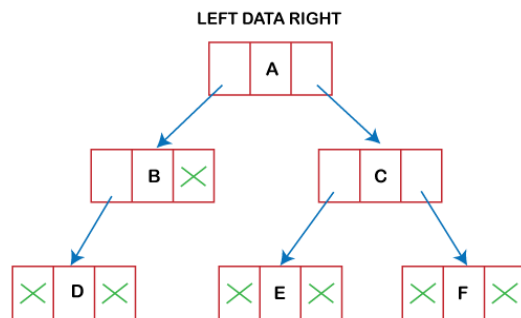
**Height of a node:** The number of edges that occur in the longest path from that node to the leaf node is known as the height of a node.

**Level of node:** The number of edges that exist from the root node to the given node is known as a level of a node.

**Note:** If there are  $n$  number of nodes in the tree, then there would be  $(n-1)$  number of edges.

## How is a tree represented in the memory?

Each node will contain three parts, data part, address of the left subtree, and address of the right subtree. If any node does not have the child, then both link parts will have NULL values.



## What is a Graph?

A graph is like a tree data structure is a collection of objects or entities known as nodes that are connected to each other through a set of edges. A tree follows some rule that determines the relationship between the nodes, whereas graph does not follow any rule that defines the relationship among the nodes. A graph contains a set of edges and nodes, and edges can connect the nodes in any possible way.

Mathematically, it can be defined as an ordered pair of a set of vertices, and a set of nodes where vertices are represented by 'V' and edges are represented by 'E'.

**G = (V, E)**

Here we are referring to an ordered pair because the first object must be the set of vertices, and the second object must be a set of edges.

In Graph, each node has a different name or index to uniquely identify each node in the graph. The graph shown below has eight vertices named as  $v_1, v_2, v_3, v_4, v_5, v_6, v_7$ , and  $v_8$ . There is no first node, a second node, a third node and so on. There is no ordering of the nodes. Now, we will see how can we represent the edges in a graph?. An edge can be represented by the two endpoints in the graph. We can write the name of the two endpoints as a pair, that represents the edge in a graph.

**There are two types of edges:**

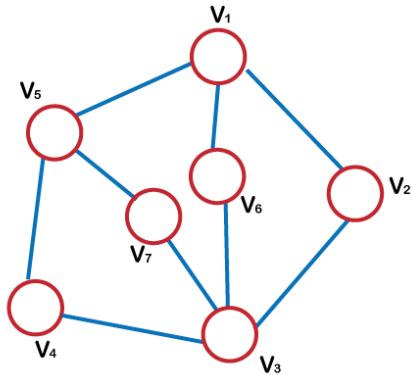
**Directed edge:** The directed edge represents one endpoint as an origin and another point as a destination. The directed edge is one-way. For example, there are two vertices U and V; then directed edge would represent the link or path from U to V, but no path exists from V to U. If we want to create a path from V to U, then we need to have one more directed edge from V to U. The directed edge can be represented as an ordered pair in which the first element is the origin, whereas the second element is the destination.

**Undirected edge:** The undirected edge is two-way means that there is no **origin** and **destination**. For example, there are two vertices U and V, then undirected would represent two paths, i.e., from U to V as well as from V to U. An undirected edge can be represented as an unordered pair because the edge is **bi-directional**.

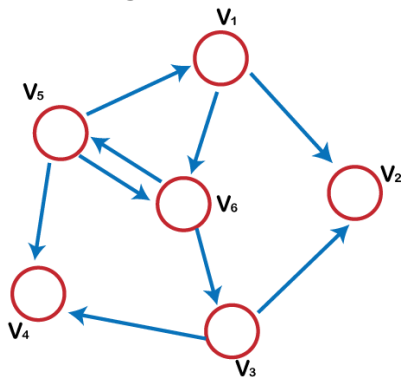
The tree data structure contains only directed edges, whereas the graph can have both types of edges, i.e., **directed as well as undirected**. But, we consider the graph in which all the edges are either directed edges or undirected edges.

**There are two types of graphs:**

**Directed graph:** The graph with the directed edges known as a **directed graph**.



**Undirected graph:** The graph with the undirected edges known as a **undirected graph**. The directed graph is a graph in which all the edges are uni-directional, whereas the undirected graph is a graph in which all the edges are bi-directional.



## Differences between tree and graph data structure.

| Basis for comparison | Tree                                                                                                                                                  | Graph                                                                                                                                                            |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Definition</b>    | Tree is a non-linear data structure in which elements are arranged in multiple levels.                                                                | A Graph is also a non-linear data structure.                                                                                                                     |
| <b>Structure</b>     | It is a collection of edges and nodes. For example, node is represented by N and edge is represented as E, so it can be written as:<br>$T = \{N, E\}$ | It is a collection of vertices and edges. For example, vertices are represented by V, and edge is represented as 'E', so it can be written as:<br>$T = \{V, E\}$ |

|                       |                                                                                                                                                                      |                                                                                                      |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Root node</b>      | In tree data structure, there is a unique node known as a parent node. It represents the topmost node in the tree data structure.                                    | In graph data structure, there is no unique node.                                                    |
| <b>Loop formation</b> | It does not create any loop or cycle.                                                                                                                                | In graph, loop or cycle can be formed.                                                               |
| <b>Model type</b>     | It is a hierarchical model because nodes are arranged in multiple level, and that creates a hierarchy. For example, any organization will have a hierarchical model. | It is a network model. For example, facebook is a social network that uses the graph data structure. |
| <b>Edges</b>          | If there are n nodes then there would be n-1 number of edges.                                                                                                        | The number of edges depends on the graph.                                                            |
| <b>Type of edge</b>   | Tree data structure will always have directed edges.                                                                                                                 | In graph data structure, all the edges can either be directed edges, undirected edges, or both.      |
| <b>Applications</b>   | It is used for inserting, deleting or searching any element in tree.                                                                                                 | It is mainly used for finding the shortest path in the network.                                      |

## Binary Search tree vs AVL tree

Before knowing about the Binary search tree and AVL tree differences, we should know about the binary search tree and AVL tree separately.

### What is a Binary Search Tree?

The [binary search tree](#) is a [tree data structure](#) that follows the condition of the [binary tree](#). As we know, that tree can have 'n' number of children, whereas; the binary tree can contain the utmost two children. So, the constraint of a binary tree is also followed by the binary search tree. Each node in a binary search tree should have the utmost two children; in other words, we can say that the binary search tree is a variant of the binary tree.

The binary search tree also follows the properties of the binary search. In binary search, all the elements in an array must be in sorted order. We calculate the middle element in the binary search in which the left part of the middle element contains the value lesser than the middle value, and the right part of the middle element contains the values greater than the middle value.

In Binary Search Tree, the middle element becomes the root node, the right part becomes the right subtree, and the left part becomes the left subtree. Therefore, we can say that the binary search tree is a combination of a **binary tree** and **binary search**.

**Note: Binary Search Tree can be defined as the binary tree in which all the elements of the left subtree are less than the root node, and all the elements of the right subtree are greater than the root node.**

### Time Complexity in Binary Search Tree

If the binary search tree is almost a balanced tree then all the operations will have a time complexity of  $O(\log n)$  because the search is divided either to the left or the right subtree.

If the binary search tree is either left or right-skewed, then all the operations will have the time complexity of  $O(n)$  because we need to traverse till the  $n$  elements.

### What is the AVL Tree?

An [AVL tree](#) is a self-balancing binary search tree where the difference between heights of left and right subtrees cannot be more than one. This difference is known as a balance factor. In the AVL tree, the values of balance factor could be either -1, 0 or 1.

#### How does the self-balancing of the binary search tree happen?

As we know that AVL tree is a self-balancing binary search tree. If the binary search tree is not balanced, it can be self-balanced with some re-arrangements. These re-arrangements can be done using some rotations.

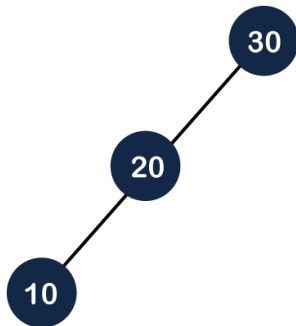
#### Let's understand self-balancing through an example.

Suppose we want to insert 10, 20, 30 in an AVL tree.

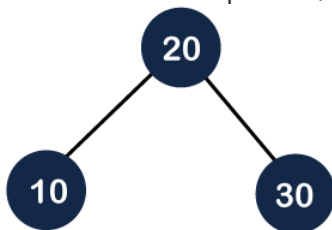
**The following are the ways of inserting 10, 20, 30 in an AVL tree:**

**If the order of insertion is 30, 20, 10.**

**Step 1:** First, we create a Binary search tree, as shown below:



**Step 2:** In the above figure, we can observe that tree is unbalanced because the balance factor of node 30 is 2. In order to make it an AVL tree, we need to perform some rotations. If we perform the right rotation on node 20 then the node 30 will move downwards, whereas the node 20 will move upwards, as shown below:



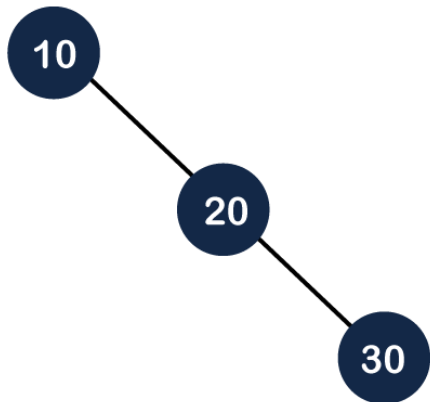
As we can observe, the final tree follows the property of the Binary Search tree and a balanced tree; therefore, it is an AVL tree.

In the case, the tree was **left unbalanced tree**, so we perform the right rotation on the node.

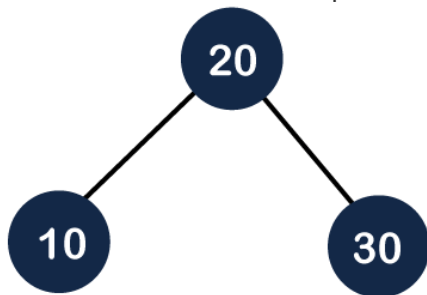
**If the order of insertion is 10, 20, 30.**

**Step 1:** First we create a Binary search tree as shown below:





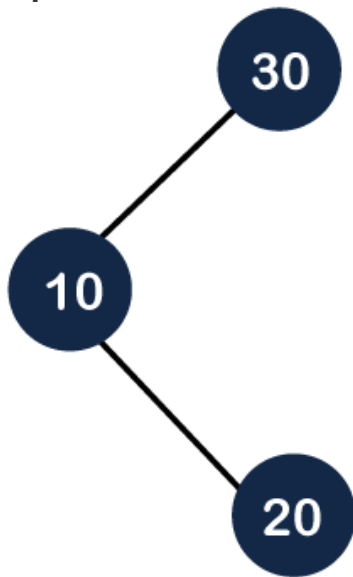
**Step 2:** In the above figure, we can observe that the tree is unbalanced because the balance factor of node 10 is -2. In order to make it an AVL tree, we need to perform some rotations. It is a right unbalanced tree, so we will perform left rotation. If we perform left rotation on node 20, then the node 20 will move upwards, and node 10 will move downwards, as shown below:



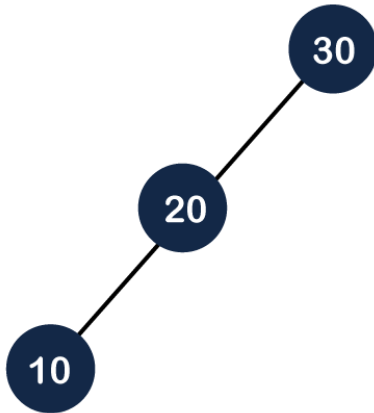
As we can observe, the final tree follows the property of the **Binary Search tree** and a **balanced tree**; therefore, it is an AVL tree.

**If the order of insertion is 30, 10, 20**

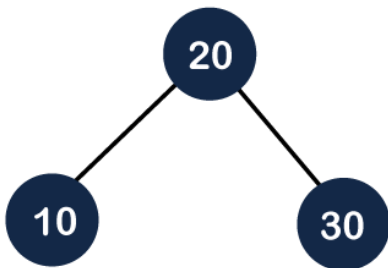
**Step 1:** First we create the Binary Search tree as shown below:



**Step 2:** In the above figure, we can observe that the tree is unbalanced because the balance factor of the root node is 2. In order to make it an AVL tree, we need to perform some rotations. The above scenario is left-right unbalanced as one node is the left of its parent node and another is the right of its parent node. First, we will perform the left rotation, and rotation happens between nodes 10 and 20. After left rotation, 20 will move upwards, and 10 will move downwards as shown below:



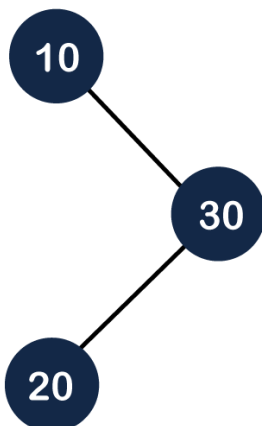
Still, the tree is unbalanced, so we perform the right rotation on the tree. Once the right rotation is performed on the tree, then the tree would like, as shown below:



As we can observe in the above tree, the tree follows the property of the Binary Search tree and a balanced tree; therefore, it is an AVL tree.

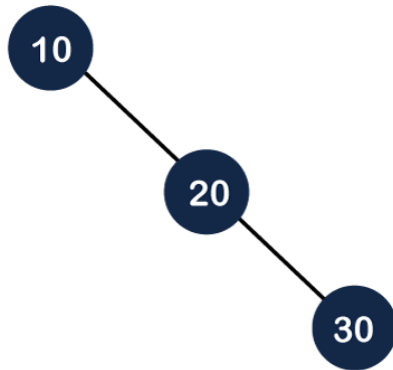
**If the order of the insertion is 10, 30, 20**

**Step 1:** First, we create the Binary Search tree, as shown below:

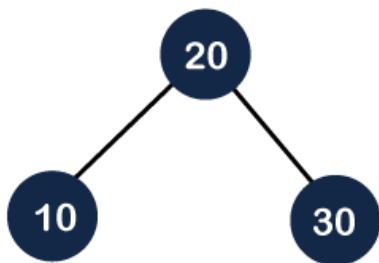


**Step 2:** In the above figure, we can observe that tree is unbalanced because the balance factor of the root node is 2. In order to make it an AVL tree, we need to perform some rotations. The

above scenario is right-left unbalanced as one node is right of its parent node, and another node is left of its parent node. First, we will perform the right rotation that happens between nodes 30 and 20. After right rotation, 20 will move upwards, and 30 will move downwards as shown below:



Still, the above tree is unbalanced, so we need to perform left rotation on the node. Once the left rotation is performed, the node 20 will move upwards, and node 10 will move downwards as shown below:



As we can observe in the above tree, the tree follows the property of the Binary Search tree and a balanced tree; therefore, it is an AVL tree.

## Differences between Binary Search tree and AVL tree

| Binary Search tree                                                                                        | AVL tree                                                                                                              |
|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Every binary search tree is a binary tree because both the trees contain the utmost two children.         | Every AVL tree is also a binary tree because AVL tree also has the utmost two children.                               |
| In BST, there is no term exists, such as balance factor.                                                  | In the AVL tree, each node contains a balance factor, and the value of the balance factor must be either -1, 0, or 1. |
| Every Binary Search tree is not an AVL tree because BST could be either a balanced or an unbalanced tree. | Every AVL tree is a binary search tree because the AVL tree follows the property of the BST.                          |

|                                                                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Each node in the Binary Search tree consists of three fields, i.e., left subtree, node value, and the right subtree.                                                                                                                                                                                                                                                                                              | Each node in the AVL tree consists of four fields, i.e., left subtree, node value, right subtree, and the balance factor.                                                                                                                                                                                                                           |
| In the case of Binary Search tree, if we want to insert any node in the tree then we compare the node value with the root value; if the value of node is greater than the root node value then the node is inserted to the right subtree otherwise the node is inserted to the left subtree. Once the node is inserted, there is no need of checking the height balance factor for the insertion to be completed. | In the case of AVL tree, first, we will find the suitable place to insert the node. Once the node is inserted, we will calculate the balance factor of each node. If the balance factor of each node is satisfied, the insertion is completed. If the balance factor is greater than 1, then we need to perform some rotations to balance the tree. |
| In Binary Search tree, the height or depth of the tree is $O(n)$ where $n$ is the number of nodes in the Binary Search tree.                                                                                                                                                                                                                                                                                      | In AVL tree, the height or depth of the tree is $O(\log n)$ .                                                                                                                                                                                                                                                                                       |
| It is simple to implement as we have to follow the Binary Search properties to insert the node.                                                                                                                                                                                                                                                                                                                   | It is complex to implement because in AVL tree, we have to first construct the AVL tree, and then we need to check height balance. If the height is imbalance then we need to perform some rotations to balance the tree.                                                                                                                           |
| BST is not a balanced tree because it does not follow the concept of the balance factor.                                                                                                                                                                                                                                                                                                                          | AVL tree is a height balanced tree because it follows the concept of the balance factor.                                                                                                                                                                                                                                                            |
| Searching is inefficient in BST when there are large number of nodes available in the tree because the height is not balanced.                                                                                                                                                                                                                                                                                    | Searching is efficient in AVL tree even when there are large number of nodes in the tree because the height is balanced.                                                                                                                                                                                                                            |

## Red Black Tree vs AVL tree

Before understanding the **Red-Black tree** and **AVL tree** differences, we should know about the Red-Black tree and AVL tree separately.

### What is a Red-Black tree?

The **Red-black tree** is a self-balanced **binary search tree** in which each node contains one extra bit of information that denotes the color of the node. The color of the node could be either Red or Black, depending on the bit information stored by the node.

#### Properties

**The following are the properties associated with a Red-Black tree:**

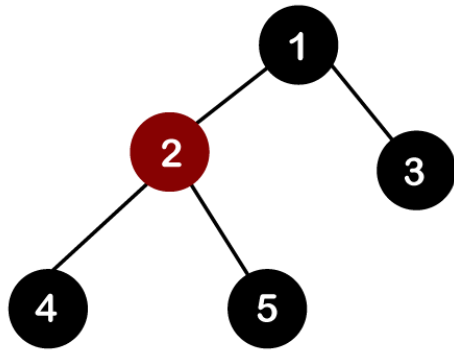
The root node of the tree should be Black.

A red node can have only black children. Or, we can say that two adjacent nodes cannot be Red in color.

If the node does not have a left or right child, then that node is said to be a leaf node. So, we put the left and right children below the leaf node known as **nil**

The black depth or black height of a leaf node can be defined as the number of black that we encounter when we move from the leaf node to the root node. One of the key properties of the Red-Black tree is that every leaf node should have the same black depth.

**Let's understand this property through an example.**



In the above tree, there are five nodes, in which one is a Red and the other four nodes are Black. There are three leaf nodes in the above tree. Now we calculate the black depth of each leaf node. As we can observe that the black depth of all the three leaf nodes is 2; therefore, it is a Red-Black tree.

If the tree does not obey any of the above three properties, then it is not a Red-Black tree.

### **Height of a red-black tree**

Consider  $h$  as the height of the tree having  $n$  nodes. What would be the smallest value of  $n$ ?

The value of  $n$  is the smallest when all the nodes are black. If the tree contains all the black nodes, then the tree would be a complete binary tree. If the height of a complete binary tree is  $h$ , then the number of nodes in a tree is:

$$n = 2^h - 1$$

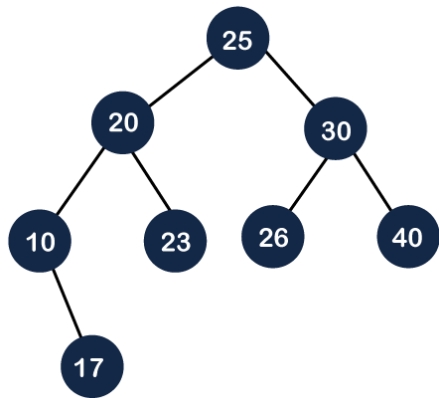
### **What would be the largest value of $n$ ?**

The value of  $n$  is largest when every black node has two red children, but the red node cannot have red children, so that it will have black children. In this way, there are alternate layers of black and red children. So, if the number of black layers is  $h$ , then the number of red layers is also  $h$ ; therefore, the tree's total height becomes  $2h$ . The total number of nodes is:

$$n = 2^{2h} - 1$$

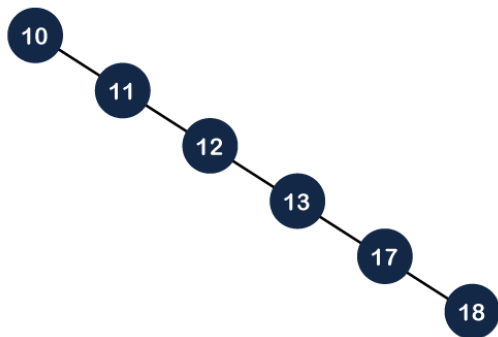
## **What is the AVL tree?**

An [AVL tree](#) is a self-balancing binary search tree if we observe the below case, which is a binary search tree and a balanced tree.



In the above tree, the worst-case time complexity for searching an element is  $O(h)$ , i.e., the height of the tree. In the above case, the number of comparisons made to search 17 element is 4, and the height of the tree is also 4.

If we consider the skewed binary tree, as shown below:



In the above right skewed tree, the number of comparisons made to search 17 element is 5, and the number of elements present in the tree is also 5. Therefore, we can say that if the tree is a skewed binary tree then the time complexity would be  $O(n)$ .

Now, we have to balance these skewed trees so that they will have the time complexity  $O(h)$ .

There is a term known as a **balance factor**, which is used to self -balance the binary search tree.

The balance factor can be computed as:

Balance factor = height of left subtree - height of right subtree

The value of the balance factor could be either 1, 0 or -1. If each node in the binary tree is having a value of either 1, 0, or -1, then that tree is said to be a balanced [binary tree](#) or AVL tree.

The tree is known as a perfectly balanced tree if the balance factor of each node is 0. The AVL tree is an almost balanced tree because each node in the tree has the value of balance factor either 1, 0 or -1.

## Differences between the Red-Black tree and AVL tree.

The following are the differences between the Red-Black tree and AVL tree:

### Binary search tree

The Red-Black tree is a binary search tree, and the AVL tree is also a binary search tree.

### Rules

The following rules are applied in a Red-Black Tree:

The node in a Red-Black tree is either red or black in color.

The color of the root node should be black.

The adjacent nodes should not be red. In other words, we can say that the red node cannot have red children, but the black node can have black children.

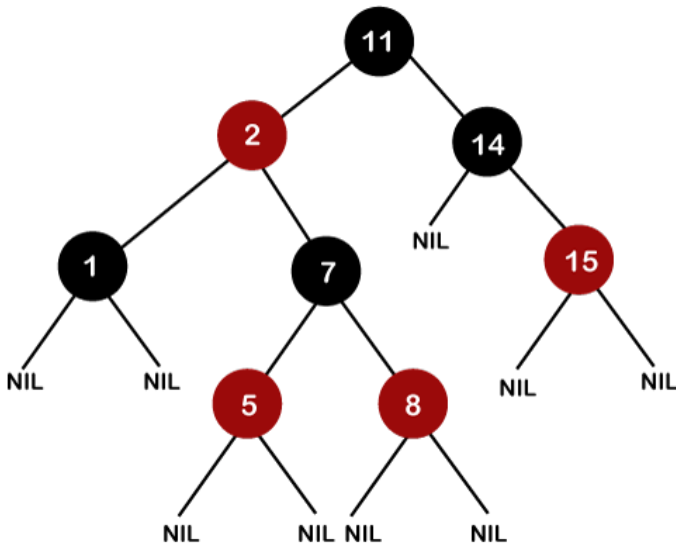
There should be the same number of black nodes in every path; then, only a tree can be considered a Red-Black tree.

The external nodes are the nil nodes, which are always black in color.

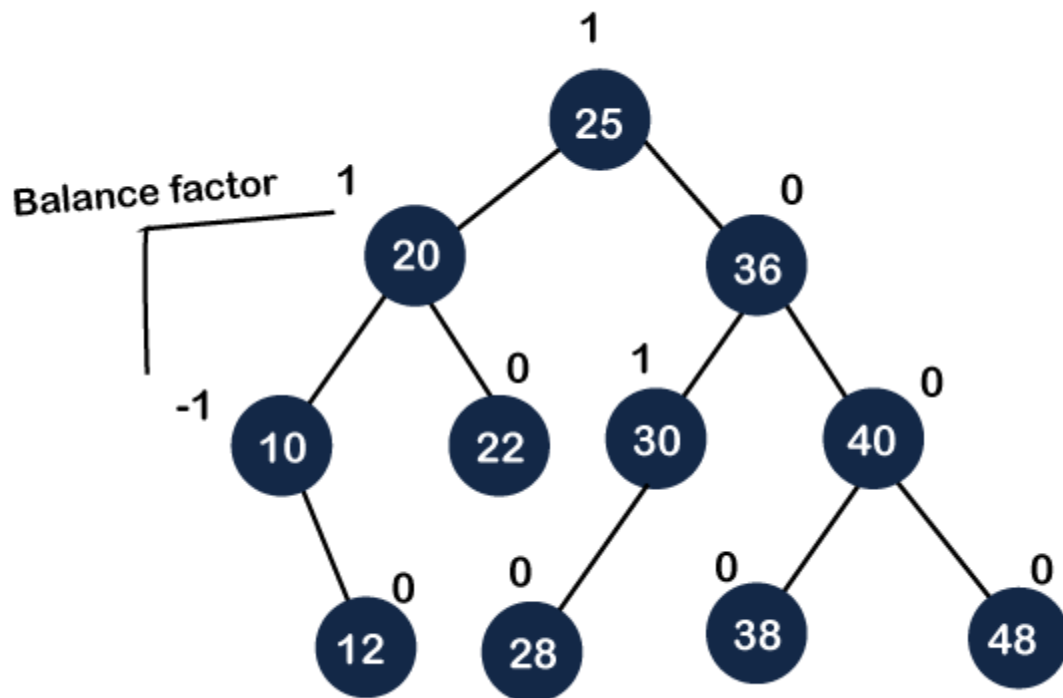
Rule of the AVL tree:

Every node should have the balance factor either as -1, 0 or 1.

### Example



In the above figure, we need to check whether the tree is a Red-Black tree or not. In order to check this, first, we need to check whether the tree is a binary search tree or not. As we can observe in the above figure that it satisfies all the properties of the binary search tree; therefore, it is a binary search tree. Secondly, we have to verify whether it satisfies the above-said rules or not. The above tree satisfies all the above five rules; therefore, it concludes that the above tree is a Red-Black tree.



In the above figure, we need to check whether the tree is an AVL tree or not. As each node has a value of balance factor either as -1, 0, or 1, so it is an AVL tree.

#### **How can the tree be considered as a balanced tree or not?**

In the case of a Red-Black tree, if all the above rules are satisfied, provided that a tree is a binary search tree, then the tree is said to be a Red-black tree.

In the case of the AVL tree, if the balance factor is -1, 0, or 1, then the above tree is said to be an AVL tree.

#### **Tools used for balancing**

If the tree is not balanced, then two tools are used for balancing the tree in a Red-Black tree:

##### **Recoloring**

##### **Rotation**

If the tree is not balanced, then one tool is used for balancing the tree in the AVL tree:

##### **Rotation**

#### **Efficient for which operation**

In the case of the Red-Black tree, the insertion and deletion operations are efficient. If the tree gets balanced through the recoloring, then insertion and deletion operations are efficient in the Red-Black tree.

In the case of the AVL tree, the searching operation is efficient as it requires only one tool to balance the tree.

#### **Time complexity**

In the Red-Black tree case, the time complexity for all the operations, i.e., insertion, deletion, and searching is  $O(\log n)$ .

In the case of AVL tree, the time complexity for all the operations, i.e., insertion, deletion, and searching is  $O(\log n)$ .



Let's understand the differences in the tabular form.

| Parameter                     | Red Black Tree                                                                                                                    | AVL Tree                                                                                                                                 |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Searching</b>              | Red Black tree does not provide efficient searching as Red Black Trees are roughly balanced.                                      | AVL trees provide efficient searching as it is strictly balanced tree.                                                                   |
| <b>Insertion and Deletion</b> | Insertion and Deletion are easier in Red Black tree as it requires fewer rotations to balance the tree.                           | Insertion and Deletion are complex in AVL tree as it requires multiple rotations to balance the tree.                                    |
| <b>Color of the node</b>      | In the Red-Black tree, the color of the node is either Red or Black.                                                              | In the case of AVL trees, there is no color of the node.                                                                                 |
| <b>Balance factor</b>         | It does not contain any balance factor. It stores only one bit of information that denotes either Red or Black color of the node. | Each node has a balance factor in AVL tree whose value can be 1, 0, or -1. It requires extra space to store the balance factor per node. |
| <b>Strictly balanced</b>      | Red-black trees are not strictly balanced.                                                                                        | AVL trees are strictly balanced, i.e., the left subtree's height and the height of the right subtree differ by at most 1.                |

## B tree vs B+ tree

Before understanding **B tree** and **B+ tree** differences, we should know the B tree and B+ tree separately.

### What is the B tree?

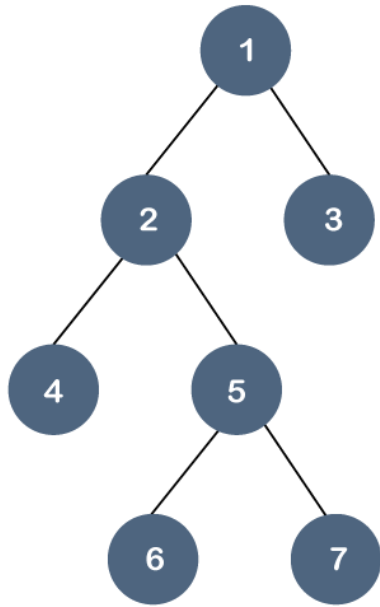
**B tree** is a self-balancing tree, and it is a m-way tree where m defines the order of the tree. **Btree** is a generalization of the [Binary Search tree](#) in which a node can have more than one key and more than two children depending upon the value of **m**. In the B tree, the data is specified in a sorted order having lower values on the left subtree and higher values in the right subtree.

#### Properties of B tree

**The following are the properties of the B tree:**

In the B tree, all the leaf nodes must be at the same level, whereas, in the case of a binary tree, the leaf nodes can be at different levels.

**Let's understand this property through an example.**



In the above tree, all the leaf nodes are not at the same level, but they have the utmost two children. Therefore, we can say that the above tree is a [binary tree](#) but not a B tree.

If the Btree has an order of  $m$ , then each node can have a maximum of  $m$ . In the case of minimum children, the leaf nodes have zero children, the root node has two children, and the internal nodes have a ceiling of  $m/2$ .

Each node can have maximum  $(m-1)$  keys. For example, if the value of  $m$  is 5 then the maximum value of keys is 4.

The root node has minimum one key, whereas all the other nodes except the root node have  $(\text{ceiling of } m/2 \text{ minus } 1)$  minimum keys.

If we perform insertion in the B tree, then the node is always inserted in the leaf node.

**Suppose we want to create a B tree of order 3 by inserting values from 1 to 10.**

**Step 1:** First, we create a node with 1 value as shown below:



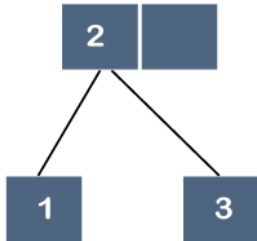
**Step 2:** The next element is 2, which comes after 1 as shown below:



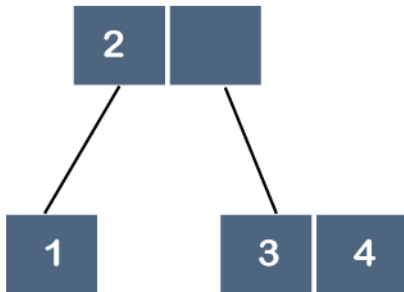
**Step 3:** The next element is 3, and it is inserted after 2 as shown below:



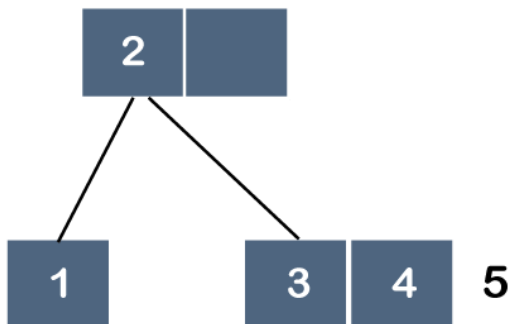
As we know that each node can have 2 maximum keys, so we will split this node through the middle element. The middle element is 2, so it moves to its parent. The node 2 does not have any parent, so it will become the root node as shown below:



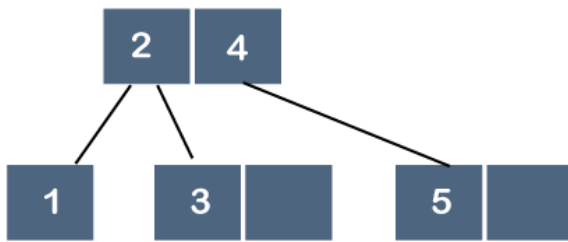
**Step 4:** The next element is 4. Since 4 is greater than 2 and 3, so it will be added after the 3 as shown below:



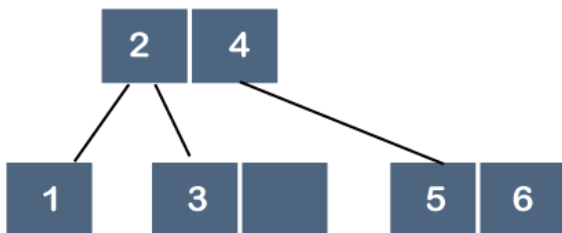
**Step 5:** The next element is 5. Since 5 is greater than 2, 3 and 4 so it will be added after 4 as shown below:



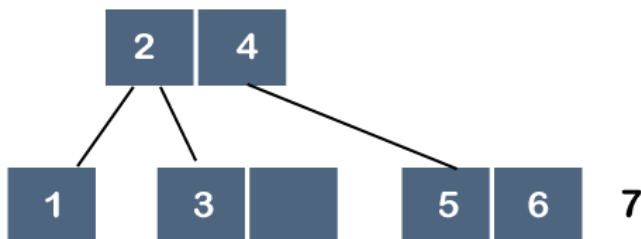
As we know that each node can have 2 maximum keys, so we will split this node through the middle element. The middle element is 4, so it moves to its parent. The parent is node 2; therefore, 4 will be added after 2 as shown below:



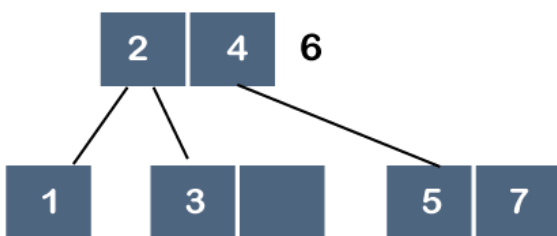
**Step 6:** The next element is 6. Since 6 is greater than 2, 4 and 5, so 6 will come after 5 as shown below:



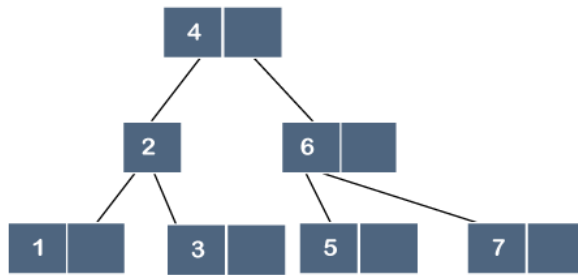
**Step 7:** The next element is 7. Since 7 is greater than 2, 4, 5 and 6, so 7 will come after 6 as shown below:



As we know that each node can have 2 maximum keys, so we will split this node through the middle element. The middle element is 6, so it moves to its parent as shown below:



But, 6 cannot be added after 4 because the node can have 2 maximum keys, so we will split this node through the middle element. The middle element is 4, so it moves to its parent. As node 4 does not have any parent, node 4 will become a root node as shown below:



## What is a B+ tree?

The **B+ tree** is also known as an advanced self-balanced tree because every path from the root of the tree to the leaf of the tree has the same length. Here, the same length means that all the leaf nodes occur at the same level. It will not happen that some of the leaf nodes occur at the third level and some of them at the second level.

A B+ tree index is considered a multi-level index, but the B+ tree structure is not similar to the multi-level index sequential files.

### Why is the B+ tree used?

A B+ tree is used to store the records very efficiently by storing the records in an indexed manner using the B+ tree indexed structure. Due to the multi-level indexing, the data accessing becomes faster and easier.

### B+ tree Node Structure

The node structure of the B+ tree contains pointers and key values shown in the below figure:



As we can observe in the above B+ tree node structure that it contains n-1 key values ( $k_1$  to  $k_{n-1}$ ) and n pointers ( $p_1$  to  $p_n$ ).

The search key values which are placed in the node are kept in sorted order. Thus, if  $i < j$  then  $k_i < k_j$ .

### Constraint on various types of nodes

Let 'b' be the order of the B+ tree.

#### Non-Leaf node

Let 'm' represents the number of children of a node, then the relation between the order of the tree and the number of children can be represented as:

$$\left\lceil \frac{b}{2} \right\rceil \leq m \leq b$$

Let k represents the search key values. The relation between the order of the tree and search key can be represented as:

As we know that the number of pointers is equal to the search key values plus 1, so mathematically, it can be written as:

**Number of Pointers (or children) = Number of Search keys + 1**

Therefore, the maximum number of pointers would be 'b', and the minimum number of pointers would be the ceiling function of  $b/2$ .

### Leaf Node

A leaf node is a node that occurs at the last level of the B+ tree, and each leaf node uses only one pointer to connect with each other to provide the sequential access at the leaf level.

In leaf node, the maximum number of children is:

$$\left\lceil \frac{b}{2} \right\rceil - 1 \leq k \leq b - 1$$

The maximum number of search keys is:

$$\left\lceil \frac{b}{2} \right\rceil - 1 \leq m \leq b - 1$$

### Root Node

The maximum number of children in the case of the root node is: b

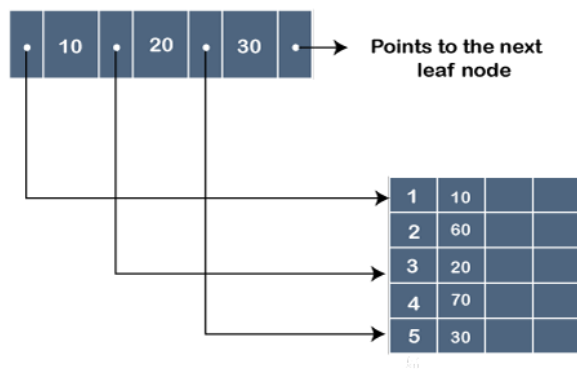
The minimum number of children is: 2

### Special cases in B+ tree

**Case 1:** If the root node is the only node in the tree. In this case, the root node becomes the leaf node.

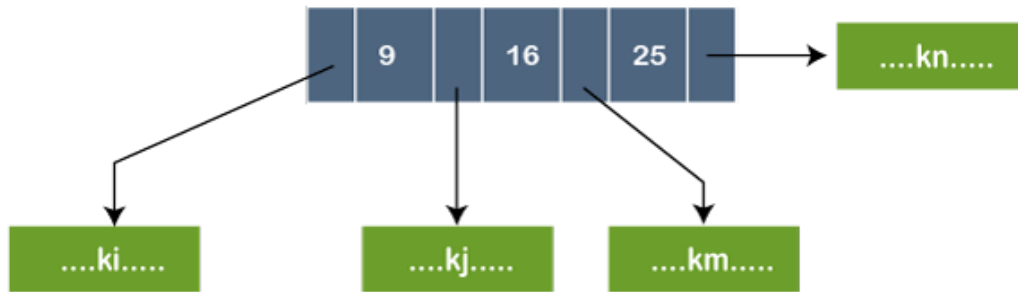
In this case, the maximum number of children is 1, i.e., the root node itself, whereas, the minimum number of children is b-1, which is the same as that of a leaf node.

### Representation of a leaf node in B+ tree



In the above figure, '.' represents the pointer, whereas the 10, 20 and 30 are the key values. The pointer contains the address at which the key value is stored, as shown in the above figure.

### Example of B+ tree



In the above figure, the node contains three key values, i.e., 9, 16, and 25. The pointer that appears before 9, contains the key values less than 9 represented by  $k_i$ . The pointer that appears before 16, contains the key values greater than or equal to 9 but less than 16 represented by  $k_j$ . The pointer that appears before 25, contains the key values greater than or equal to 16 but less than 25 represented by  $k_n$ .

## Differences between B tree and B+ tree

The following are the differences between the B tree and B+ tree:

| B tree                                                                                                                         | B+ tree                                                                                                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| In the B tree, all the keys and records are stored in both internal as well as leaf nodes.                                     | In the B+ tree, keys are the indexes stored in the internal nodes and records are stored in the leaf nodes.                                                                                                                               |
| In B tree, keys cannot be repeatedly stored, which means that there is no duplication of keys or records.                      | In the B+ tree, there can be redundancy in the occurrence of the keys. In this case, the records are stored in the leaf nodes, whereas the keys are stored in the internal nodes, so redundant keys can be present in the internal nodes. |
| In the Btree, leaf nodes are not linked to each other.                                                                         | In B+ tree, the leaf nodes are linked to each other to provide the sequential access.                                                                                                                                                     |
| In Btree, searching is not very efficient because the records are either stored in leaf or internal nodes.                     | In B+ tree, searching is very efficient or quicker because all the records are stored in the leaf nodes.                                                                                                                                  |
| Deletion of internal nodes is very slow and a time-consuming process as we need to consider the child of the deleted key also. | Deletion in B+ tree is very fast because all the records are stored in the leaf nodes so we do not have to consider the child of the node.                                                                                                |

|                                                                                                                                                      |                                                                                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| In Btree, sequential access is not possible.                                                                                                         | In the B+ tree, all the leaf nodes are connected to each other through a pointer, so sequential access is possible.                                                                    |
| In Btree, the more number of splitting operations are performed due to which height increases compared to width,                                     | B+ tree has more width as compared to height.                                                                                                                                          |
| In Btree, each node has atleast two branches and each node contains some records, so we do not need to traverse till the leaf nodes to get the data. | In B+ tree, internal nodes contain only pointers and leaf nodes contain records. All the leaf nodes are at the same level, so we need to traverse till the leaf nodes to get the data. |
| The root node contains atleast 2 to m children where m is the order of the tree.                                                                     | The root node contains atleast 2 to m children where m is the order of the tree.                                                                                                       |

## Difference Between Quick Sort and Merge Sort

A **sorting** is the arrangement of collectively data in particular format like ascending or descending order. Generally, it is used to arrange the homogeneous data in sorted manner. Using the **sorting** algorithms, we can arrange the data in a sequence order and search an element easily and faster. **Sorting techniques** depends on two situation such as total time and total space required to execute a program. In this section, we will discuss **quick sort** and **merge sort** and also compare them each other.

### Quick Sort

Quick sort is a comparison based sorting algorithm that follows the divide and conquer technique to sort the arrays. In quick sort, we usually use a **pivot (key)** element to compare and interchange the position of the element based on some condition. When a pivot element gets its fixed position in the array that shows the termination of comparison & interchange procedure. After that, the array divides into the two sub arrays. Where the first partition contains all those elements that are less than pivot (key) element and the other parts contains all those elements that are greater than pivot element. After that, it again selects a pivot element on each of the sub arrays and repeats the same process until all the elements in the arrays are sorted into an array.

### Algorithm of Quick Sort

**Partition (A, p, r)**

X <- A[r]



```

l <- p-1
For j <- p to r-1
Do if A[j] <= x
Then l <- l + 1
Exchange A[l] <-> A[j]
Exchange A[l + 1] <-> A[r]
Return l + 1

```

### **Quicksort (A, p, r)**

```

While (p < r)
Do q <- Partition (A, p, r)
R <- q-1
While (p < r)
Do q <- Partition (A, p, r)
P <- q + 1

```

### **Steps to sort an array using the quick sort algorithm**

Suppose, we have an array X having the elements X[1], X[2], X[3],..., X[n] that are to be sort. Let's follow the below steps to sort an array using the quick sort.

**Step 1:** Set the first element of the array as the **pivot** or key element. Here, we assume pivot as X[0], **left** pointer is placed at the first element and the **last** index of the array element as **right**.

**Step 2:** Now we starts the scanning of the array elements from right side index, then

If X[key] is less than X[right] or if X[key] < X[Right],

Continuously decreases the right end pointer variable until it becomes equal to the key.

If X[key] > X[right], interchange the position of the key element to the X[right] element.

Set, key = right and increment the left index by 1.

**Step 3:** Now we again start the scanning of the element from left side and compare each element with the key element. X[key] > X[left] or X[key] is greater than X[left], then it performs the following actions:

Continuously compare the left element with the X[key] and increment the left index by 1 until key becomes equal to the left.

If X[key] < X[left], interchange the position of the X[key] with X[left] and go to step 2.

**Step 4:** Repeat Step 2 and 3 until the X[left] becomes equal to X[key]. So, we can say that if X[left] = X[key], it shows the termination of the procedures.

**Step 5:** After that, all the elements at the left side will be smaller than the key element and the rest element of the right side will be larger than the key element. Thus indicating the array needs to partitioned into two sub arrays.

**Step 6:** Similarly, we need to repeatedly follow the above procedure to the sub arrays until the entire array becomes sorted.

Let's see an example of quick sort.

**Example: Consider an array of 6 elements. Sort the array using the quick sort.**

arr[] = {50, 20, 60, 30, 40, 56}

|           |    |    |    |    |       |
|-----------|----|----|----|----|-------|
| 50        | 20 | 60 | 30 | 40 | 56    |
| 0         | 1  | 2  | 3  | 4  | 5     |
| ↑         |    |    |    |    | ↑     |
| loc, left |    |    |    |    | right |

|           |    |    |    |    |       |
|-----------|----|----|----|----|-------|
| 50        | 20 | 60 | 30 | 40 | 56    |
| 0         | 1  | 2  | 3  | 4  | 5     |
| ↑         |    |    |    |    | ↑     |
| loc, left |    |    |    |    | right |

|      |    |    |    |            |    |
|------|----|----|----|------------|----|
| 40   | 20 | 60 | 30 | 40         | 56 |
| 0    | 1  | 2  | 3  | 4          | 5  |
| ↑    |    |    |    | ↑          |    |
| left |    |    |    | right, loc |    |

|    |    |      |    |            |    |
|----|----|------|----|------------|----|
| 40 | 20 | 60   | 30 | 50         | 56 |
| 0  | 1  | 2    | 3  | 4          | 5  |
|    |    | ↑    |    | ↑          |    |
|    |    | left |    | right, loc |    |

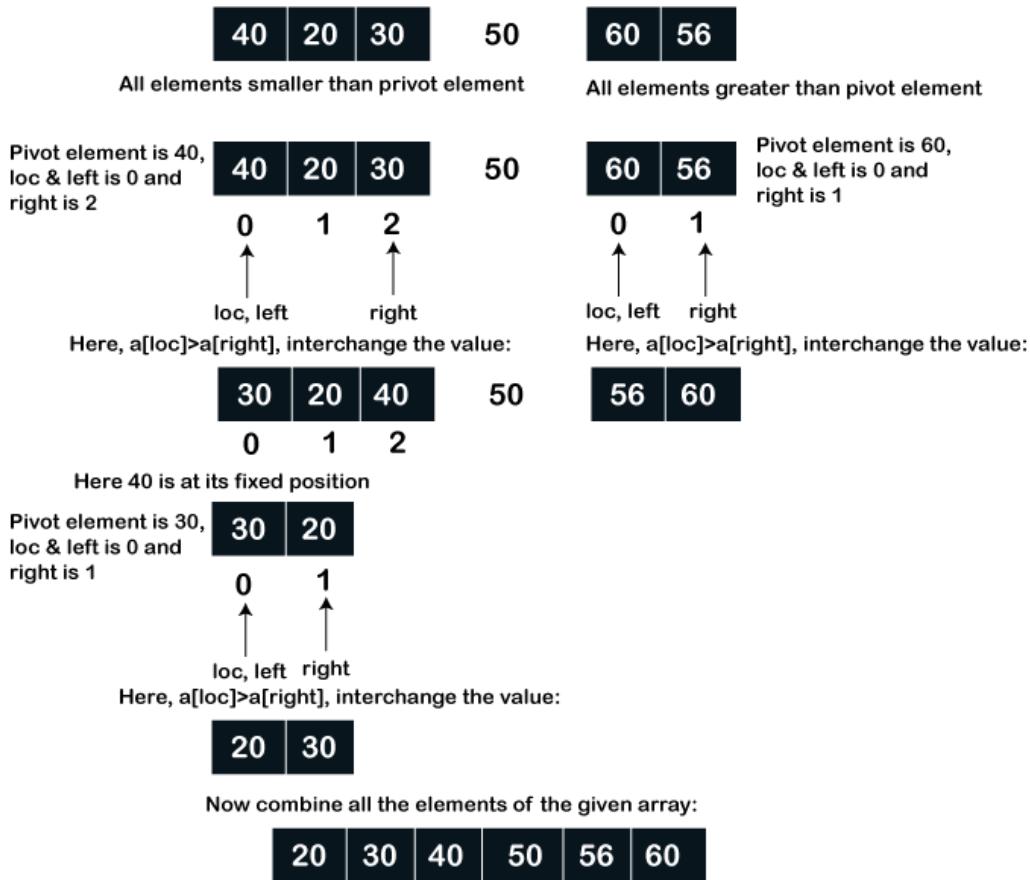
|    |    |           |    |       |    |
|----|----|-----------|----|-------|----|
| 40 | 20 | 50        | 30 | 60    | 56 |
| 0  | 1  | 2         | 3  | 4     | 5  |
|    |    | ↑         |    | ↑     |    |
|    |    | loc, left |    | right |    |

|    |    |           |       |    |    |
|----|----|-----------|-------|----|----|
| 40 | 20 | 50        | 30    | 60 | 56 |
| 0  | 1  | 2         | 3     | 4  | 5  |
|    |    | ↑         | ↑     |    |    |
|    |    | loc, left | right |    |    |

|    |    |      |           |    |    |
|----|----|------|-----------|----|----|
| 40 | 20 | 30   | 50        | 60 | 56 |
| 0  | 1  | 2    | 3         | 4  | 5  |
|    |    | ↑    | ↑         |    |    |
|    |    | left | right loc |    |    |

|    |    |    |           |       |    |
|----|----|----|-----------|-------|----|
| 40 | 20 | 30 | 50        | 60    | 56 |
| 0  | 1  | 2  | 3         | 4     | 5  |
|    |    |    | ↑         | ↑     |    |
|    |    |    | loc, left | right |    |

In the above array, 50 is in its right place. So, we divided the elements that are less than pivot in one sub array and the elements that are larger than the pivot element in another sub array.



Hence, we get the sorted array.

Let's implement the above logic in C program.

### Quick.c

```
#include <stdio.h>
```

```
int division_of_array(int x[], int st, int lt); // declaration of functions
```

```
void quick_sort(int x[], int st, int lt);
```

```
void main()
```

```
{
```

```
    int i;
```

```
    int ar[6] = {50, 20, 60, 30, 40, 56}; // given array elements
```

```
    quick_sort(ar, 0, 5); // it contains array, starting index and last index as an parameters.
```

```
    print(" Sorted Array is : \n");
```

```
    for(i=0; i<6; i++)
```

```
    {
```

```
        printf("Array = %d\t", ar[i]);
```

```
    }
```

```
    int division_of_array(int x[], int st, int lt)
```

```
    {
```

```
        int left, right, temp, key, flag;
```

```
        key = left = st; // initially these variables are in same place.
```

```

right = lt;
flag = 0;
while(flag != 1)
{
    while((x[key] <= x[right]) && (key != right))
        right--;
    if(key == right) // if pivot element is equal to the last index.
        flag = 1;
    else if(x[key] > x[right]) // if pivot is greater than last index.
    {
        temp = x[key];
        x[key] = x[right];
        x[right] = temp;
        key = right;
    }
    if (flag != 1)
    {
        /* Repeat the while loop until pivot is greater than left and equal to the left. */
        while((x[key] >= x[left]) && (key != left))
            left++;
        if(key == left)
            flag = 1;
        else if(x[key] < x[left])
        {
            temp = x[key];
            x[key] = x[left];
            x[left] = temp;
            key = left;
        }
    }
}
return key;
}
}
void quick_sort(int x[], int st, int lt)
{
    int key;
    if(st < lt) // left index value is greater than right index.
    {
        key = division_of_array(x, st, lt);
        quick_sort(x, st, key-1);
        quick_sort(x, key+1, lt);
    }
}
}

```

**Output:**

Sorted Array is:

Array = 50 20 60 30 40 56

## Merge sort

Merge sort is a most important sorting techniques that work on the divide and conquer strategies. It is the most popular sorting techniques used to sort data that is externally available in a file. The merge sort algorithm divides the given array into two halves ( $N/2$ ). And then, it recursively divides the set of two halves array elements into the single or individual elements or we can say that until no more division can take place. After that, it compares the corresponding element to sort the element and finally, all sub elements are combined to form the final sorted elements.

### Steps to sort an array using the Merge sort algorithm

Suppose we have a given array, then first we need to divide the array into sub array. Each sub array can store 5 elements.

Here we gave the first sub array name as A1 and divide into next two subarray as B1 and B2.

Similarly, the right sub array name as A2 and divide it into next two sub array as B3 and B4.

This process is repeated continuously until the sub array is divided into a single element and no more partitions may be possible.

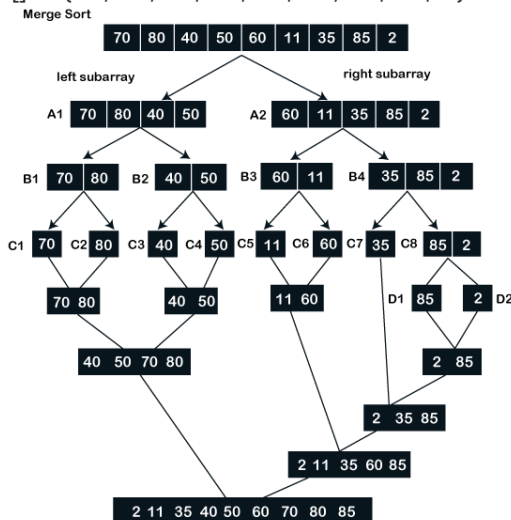
After that, compare each element with the corresponding one and then start the process of merging to arrange each element in such a way that they are placed in ascending order.

The merging process continues until all the elements are merged in ascending order.

Let's see an example of merge sort.

**Example: Consider an array of 9 elements. Sort the array using the merge sort.**

arr[] = {70, 80, 40, 50, 60, 11, 35, 85, 2}



Hence, we get the sorted array using the merge sort.

Let's implement the above logic in a C program.

### Merge.c

```
#include <stdio.h>
```

```
//#include <conio.h>
```

```
#include <stdlib.h>
```

```

void merge(int arr[], int l, int m, int end)
{
    int i, j, k;
    int a1 = m - l + 1;
    int a2 = end - m;

    // create temp subarray
    int sub1[a1], sub2[a2];

    // Store data to temp subarray subArr1[] and subArr2[]
    for (i = 0; i < a1; i++)
        sub1[i] = arr[l + i];
    for (j = 0; j < a2; j++)
        sub2[j] = arr[m + 1 + j];

    // Merge the temp array into the original array arr[]
    i = 0;
    j = 0;
    k = l;
    while (i < a1 && j < a2)
    {
        if (sub1[i] <= sub2[j])
        {
            arr[k] = sub1[i];
            i++;
        }
        else
        {
            arr[k] = sub2[j];
            j++;
        }
        k++;
    }

    // copy the rest element of subArr1[], if some element is left;
    while (i < a1)
    {
        arr[k] = sub1[i];
        i++;
        k++;
    }

    // copy the rest element of subArr2[], if some element is left;
    while (j < a2)

```

```

{
    arr[k] = sub2[j];
    j++;
    k++;
}
}

```

/\* st represents the first index and end represents the right index of the subarray of arr[] to be sorted using merge sort. \*/

**void** mergeSort(**int** arr[], **int** l, **int** end)

```

{
    if (l < end)
    {
        int m = l + (end - l) / 2;

        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, end);

        merge(arr, l, m, end);
    }
}

```

// Function to print the merge sort.

**void** mergePrint(**int** Ar[], **int** size)

```

{
    int i;
    for (i = 0; i < size; i++)
        printf("%d \t", Ar[i]);
    printf("\n");
}

```

**int** main()

```

{
    int arr[] = { 70, 80, 40, 50, 60, 11, 35, 85, 2 };
    int arr_size = sizeof(arr) / sizeof(arr[0]);

    printf(" Predefined Array is \n");
    mergePrint(arr, arr_size);

    mergeSort(arr, 0, arr_size - 1);

    printf("\n Sorted array using the Merge Sort algorithm \n");
    mergePrint(arr, arr_size);
    return 0;
}

```

}

### Output:

Predefined Array is

70 80 40 50 60 11 35 85 2

Sorted array using the Merge Sort algorithm

2 11 35 40 50 60 70 80 85

## Quick Sort vs. Merge Sort

| S.N. | Parameter                    | Quick Sort                                                                                                                                      | Merge Sort                                                                                                                                                                                       |
|------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.   | <b>Definition</b>            | It is a quick sort algorithm that arranges the given elements into ascending order by comparing and interchanging the position of the elements. | It is a merge sort algorithm that arranges the given sets of elements in ascending order using the divide and conquer technique, and then compare with corresponding elements to sort the array. |
| 2.   | <b>Principle</b>             | It works on divide and conquer techniques.                                                                                                      | It works on divide and conquer techniques.                                                                                                                                                       |
| 3.   | <b>Partition of elements</b> | In quick sort, the array can be divide into any ratio.                                                                                          | Merge sort partition an array into two sub array (N/2).                                                                                                                                          |
| 4.   | <b>Efficiency</b>            | It is more efficient and work faster in smaller size array, as compared to the merge sort.                                                      | It is more efficient and work faster in larger data sets or array, as compare to the quick sort.                                                                                                 |
| 5    | <b>Sorting method</b>        | It is an internal sorting method that sort the array or data available on main memory.                                                          | It is an external sorting method that sort the array or data sets available on external file.                                                                                                    |
| 6    | <b>Time complexity</b>       | Its worst time complexity is $O(n^2)$ .                                                                                                         | Whereas, it's worst time complexity is $O(n \log n)$ .                                                                                                                                           |
| 7    | <b>Preferred</b>             | It is a sorting algorithm that is applicable for large unsorted arrays.                                                                         | Whereas, the merge sort algorithm that is preferred to sort the linked lists.                                                                                                                    |



|     |                       |                                                                                                                |                                                                                                                                   |
|-----|-----------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 8   | <b>Stability</b>      | Quick sort is an unstable sort algorithm. But we can made it stable by using some changes in programming code. | Merge sort is a stable sort algorithm that contains two equal elements with same values in sorted output.                         |
| 9   | <b>Requires Space</b> | It does not require any additional space to perform the quick sort.                                            | It requires the additional space as temporary array to merge two sub arrays.                                                      |
| 10. | <b>Functionality</b>  | Compare each element with the pivot until all elements are arranged in ascending order.                        | Whereas, the merge sort splits the array into two parts ( $N/2$ ) and it continuously divides the array until an element is left. |

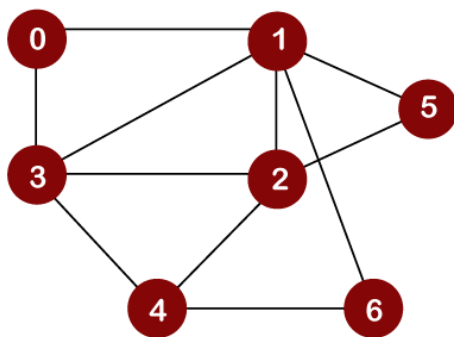
## BFS vs. DFS

Before looking at the differences between BFS and DFS, we first should know about BFS and DFS separately.

### What is BFS?

**BFS** stands for **Breadth First Search**. It is also known as **level order traversal**. The Queue data structure is used for the Breadth First Search traversal. When we use the BFS algorithm for the traversal in a graph, we can consider any node as a root node.

**Let's consider the below graph for the breadth first search traversal.**



Suppose we consider node 0 as a root node. Therefore, the traversing would be started from node 0.



Once node 0 is removed from the Queue, it gets printed and marked as a **visited node**. Once node 0 gets removed from the Queue, then the adjacent nodes of node 0 would be inserted in a Queue as shown below:

|   |  |  |  |  |
|---|--|--|--|--|
| 1 |  |  |  |  |
|---|--|--|--|--|

**Result : 0**

Now the node 1 will be removed from the Queue; it gets printed and marked as a visited node

Once node 1 gets removed from the Queue, then all the adjacent nodes of a node 1 will be added in a Queue. The adjacent nodes of node 1 are 0, 3, 2, 6, and 5. But we have to insert only unvisited nodes in a Queue. Since nodes 3, 2, 6, and 5 are unvisited; therefore, these nodes will be added in a Queue as shown below:

|   |   |   |   |  |
|---|---|---|---|--|
| 3 | 2 | 5 | 6 |  |
|---|---|---|---|--|

**Result : 0 , 1**

The next node is 3 in a Queue. So, node 3 will be removed from the Queue, it gets printed and marked as visited as shown below:

|   |   |   |  |  |
|---|---|---|--|--|
| 2 | 5 | 6 |  |  |
|---|---|---|--|--|

**Result : 0, 1, 3**

Once node 3 gets removed from the Queue, then all the adjacent nodes of node 3 except the visited nodes will be added in a Queue. The adjacent nodes of node 3 are 0, 1, 2, and 4. Since nodes 0, 1 are already visited, and node 2 is present in a Queue; therefore, we need to insert only node 4 in a Queue.

|   |   |   |   |  |
|---|---|---|---|--|
| 2 | 5 | 6 | 4 |  |
|---|---|---|---|--|

**Result : 0, 1, 3**

Now, the next node in the Queue is 2. So, 2 would be deleted from the Queue. It gets printed and marked as visited as shown below:

|   |   |   |  |  |
|---|---|---|--|--|
| 5 | 6 | 4 |  |  |
|---|---|---|--|--|

**Result : 0, 1, 3, 2,**

Once node 2 gets removed from the Queue, then all the adjacent nodes of node 2 except the visited nodes will be added in a Queue. The adjacent nodes of node 2 are 1, 3, 5, 6, and 4. Since the nodes 1 and 3 have already been visited, and 4, 5, 6 are already added in the Queue; therefore, we do not need to insert any node in the Queue.

The next element is 5. So, 5 would be deleted from the Queue. It gets printed and marked as visited as shown below:



**Result : 0, 1, 3, 2, 5**

Once node 5 gets removed from the Queue, then all the adjacent nodes of node 5 except the visited nodes will be added in the Queue. The adjacent nodes of node 5 are 1 and 2. Since both the nodes have already been visited; therefore, there is no vertex to be inserted in a Queue.

The next node is 6. So, 6 would be deleted from the Queue. It gets printed and marked as visited as shown below:



**Result : 0, 1, 3, 2, 5, 6**

Once the node 6 gets removed from the Queue, then all the adjacent nodes of node 6 except the visited nodes will be added in the Queue. The adjacent nodes of node 6 are 1 and 4. Since the node 1 has already been visited and node 4 is already added in the Queue; therefore, there is not vertex to be inserted in the Queue.

The next element in the Queue is 4. So, 4 would be deleted from the Queue. It gets printed and marked as visited.

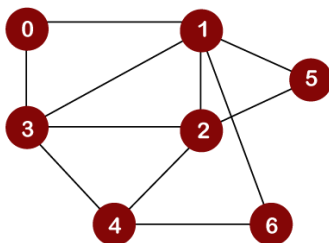
Once the node 4 gets removed from the Queue, then all the adjacent nodes of node 4 except the visited nodes will be added in the Queue. The adjacent nodes of node 4 are 3, 2, and 6. Since all the adjacent nodes have already been visited; so, there is no vertex to be inserted in the Queue.

## What is DFS?

DFS stands for Depth First Search. In DFS traversal, the stack data structure is used, which works on the LIFO (Last In First Out) principle. In DFS, traversing can be started from any node, or we can say that any node can be considered as a root node until the root node is not mentioned in the problem.

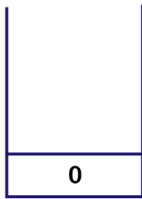
In the case of BFS, the element which is deleted from the Queue, the adjacent nodes of the deleted node are added to the Queue. In contrast, in DFS, the element which is removed from the stack, then only one adjacent node of a deleted node is added in the stack.

**Let's consider the below graph for the Depth First Search traversal.**

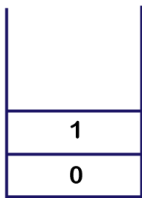


Consider node 0 as a root node.

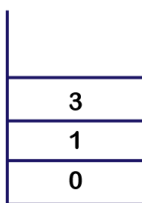
First, we insert the element 0 in the stack as shown below:



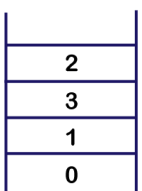
The node 0 has two adjacent nodes, i.e., 1 and 3. Now we can take only one adjacent node, either 1 or 3, for traversing. Suppose we consider node 1; therefore, 1 is inserted in a stack and gets printed as shown below:



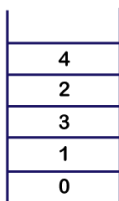
Now we will look at the adjacent vertices of node 1. The unvisited adjacent vertices of node 1 are 3, 2, 5 and 6. We can consider any of these four vertices. Suppose we take node 3 and insert it in the stack as shown below:



Consider the unvisited adjacent vertices of node 3. The unvisited adjacent vertices of node 3 are 2 and 4. We can take either of the vertices, i.e., 2 or 4. Suppose we take vertex 2 and insert it in the stack as shown below:



The unvisited adjacent vertices of node 2 are 5 and 4. We can choose either of the vertices, i.e., 5 or 4. Suppose we take vertex 4 and insert in the stack as shown below:

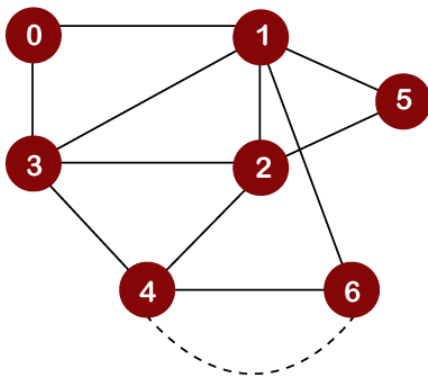


Now we will consider the unvisited adjacent vertices of node 4. The unvisited adjacent vertex of node 4 is node 6. Therefore, element 6 is inserted into the stack as shown below:

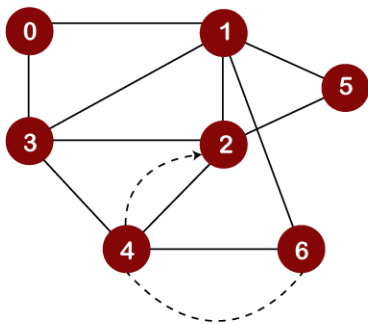
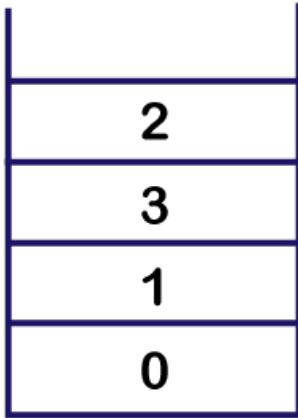
|   |
|---|
| 6 |
| 4 |
| 2 |
| 3 |
| 1 |
| 0 |

After inserting element 6 in the stack, we will look at the unvisited adjacent vertices of node 6. As there is no unvisited adjacent vertices of node 6, so we cannot move beyond node 6. In this case, we will perform **backtracking**. The topmost element, i.e., 6 would be popped out from the stack as shown below:

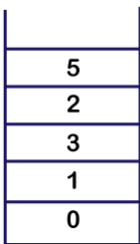
|   |
|---|
|   |
| 4 |
| 2 |
| 3 |
| 1 |
| 0 |



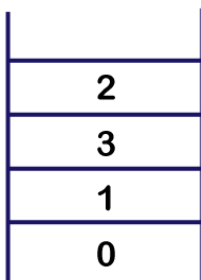
The topmost element in the stack is 4. Since there are no unvisited adjacent vertices left of node 4; therefore, node 4 is popped out from the stack as shown below:



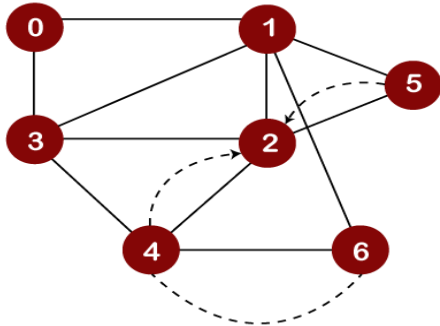
The next topmost element in the stack is 2. Now, we will look at the unvisited adjacent vertices of node 2. Since only one unvisited node, i.e., 5 is left, so node 5 would be pushed into the stack above 2 and gets printed as shown below:



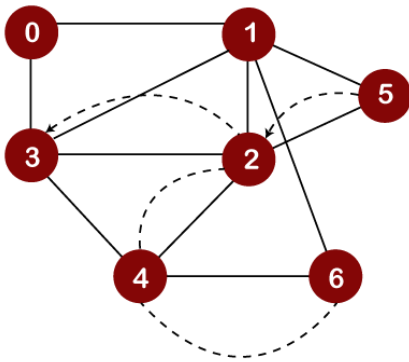
Now we will check the adjacent vertices of node 5, which are still unvisited. Since there is no vertex left to be visited, so we pop the element 5 from the stack as shown below:



We cannot move further 5, so we need to perform backtracking. In backtracking, the topmost element would be popped out from the stack. The topmost element is 5 that would be popped out from the stack, and we move back to node 2 as shown below:

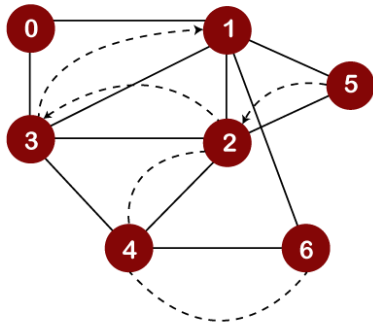


Now we will check the unvisited adjacent vertices of node 2. As there is no adjacent vertex left to be visited, so we perform backtracking. In backtracking, the topmost element, i.e., 2 would be popped out from the stack, and we move back to the node 3 as shown below:

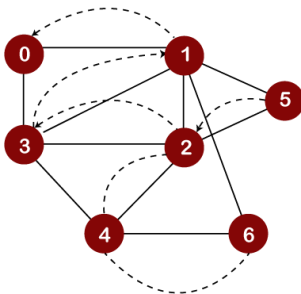


|   |
|---|
|   |
| 3 |
| 1 |
| 0 |

Now we will check the unvisited adjacent vertices of node 3. As there is no adjacent vertex left to be visited, so we perform backtracking. In backtracking, the topmost element, i.e., 3 would be popped out from the stack and we move back to node 1 as shown below:



|   |
|---|
|   |
| 0 |



We will check the adjacent vertices of node 0, which are still unvisited. As there is no adjacent vertex left to be visited, so we perform backtracking. In this, only one element, i.e., 0 left in the stack, would be popped out from the stack as shown below:





**Empty**

As we can observe in the above figure that the stack is empty. So, we have to stop the DFS traversal here, and the elements which are printed is the result of the DFS traversal.

### Differences between BFS and DFS

The following are the differences between the BFS and DFS:

|                        | <b>BFS</b>                                                                                                                    | <b>DFS</b>                                                                                                                                                                                |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Full form</b>       | BFS stands for Breadth First Search.                                                                                          | DFS stands for Depth First Search.                                                                                                                                                        |
| <b>Technique</b>       | It a vertex-based technique to find the shortest path in a graph.                                                             | It is an edge-based technique because the vertices along the edge are explored first from the starting to the end node.                                                                   |
| <b>Definition</b>      | BFS is a traversal technique in which all the nodes of the same level are explored first, and then we move to the next level. | DFS is also a traversal technique in which traversal is started from the root node and explore the nodes as far as possible until we reach the node that has no unvisited adjacent nodes. |
| <b>Data Structure</b>  | Queue data structure is used for the BFS traversal.                                                                           | Stack data structure is used for the BFS traversal.                                                                                                                                       |
| <b>Backtracking</b>    | BFS does not use the backtracking concept.                                                                                    | DFS uses backtracking to traverse all the unvisited nodes.                                                                                                                                |
| <b>Number of edges</b> | BFS finds the shortest path having a minimum number of edges to traverse from the source to the destination vertex.           | In DFS, a greater number of edges are required to traverse from the source vertex to the destination vertex.                                                                              |

|                                      |                                                                                                         |                                                                                                                                         |
|--------------------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Optimality</b>                    | BFS traversal is optimal for those vertices which are to be searched closer to the source vertex.       | DFS traversal is optimal for those graphs in which solutions are away from the source vertex.                                           |
| <b>Speed</b>                         | BFS is slower than DFS.                                                                                 | DFS is faster than BFS.                                                                                                                 |
| <b>Suitability for decision tree</b> | It is not suitable for the decision tree because it requires exploring all the neighboring nodes first. | It is suitable for the decision tree. Based on the decision, it explores all the paths. When the goal is found, it stops its traversal. |
| <b>Memory efficient</b>              | It is not memory efficient as it requires more memory than DFS.                                         | It is memory efficient as it requires less memory than BFS.                                                                             |

## Stack vs Heap

Before understanding the differences between the Stack and Heap data structure, we should know about the stack and heap data structure separately.

### What is Stack?

A [Stack is a data structure](#) that is used for organizing the data. Stack is similar to the stack a way organizing the objects in the real world. Some examples of stack of real world are stack of dinner plates, mathematical puzzle known as Tower of Hanoi containing three rods having multiple discs and stack of tennis balls. Stack is a collection with a property that an item or an object must be removed from one end known as the **top of the stack**.

It cannot be considered as the property rather it is considered as the constraint or restriction applied to the stack. In other words, we can say that only top of the stack is accessible and any item can be removed or inserted from the top of the stack. It follows LIFO (Last In First Out) principle in which recently added element must be removed from the stack first.

A stack is a list or a collection with a restriction that the insertion and deletion will take place from one end known as the top of the stack.

### Operation

**The following are the two operations available with a stack ADT:**

**Push:** The process of inserting an element in a stack is known as push operation. The push operation can be written as:

Push(x): It inserts an element x into the stack.

**Pop:** The process of deleting an element from the stack is known as a pop operation. The pop operation can be represented as:

**Pop():** It removes the most recent element from the stack.

### Representation of stack

Stack is shown below:

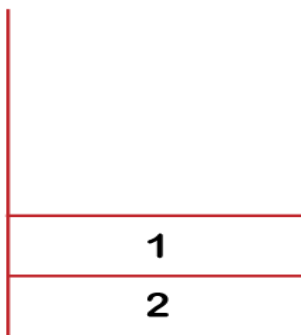


**Stack**

We can observe in the above figure that stack looks like a container which is opened from one side. It can be represented logically as a three-sided figure as a container open from one side. The above representation is an empty stack and let's assume that stack is 's'. It is a stack of integers. Now we will perform push and pop operations on the stack. Suppose we want to insert 2 element in the stack. After push operation, the stack would look like:



Since there is only one element in the stack, so 2 element would be at the top of the stack. If we want to insert 1 element in the stack, then the stack would like:



Since element 1 comes above the element 2, so element 1 would be considered the top of the stack. If we perform pop operation, then the topmost element, i.e., 1 would be removed from the stack as shown below:



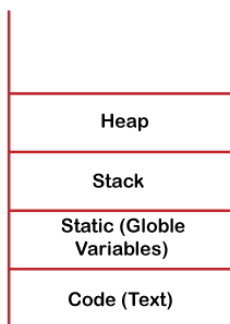
## What is Heap?

Heap is also a data structure or memory used to store the global variables. By default, all the global variables are stored in the heap memory. It allows dynamic memory allocation. The heap memory is not managed by CPU. Heap data structure can be implemented either using arrays or trees.

It is a complete binary tree that satisfies the condition of the heap property where complete binary tree is a tree in which all the levels are completely filled except the last level. In the last level, all the nodes are far as left as possible.

### Pointers and Dynamic memory

Here, we will see the architecture of memory. It is very crucial to know that how system manages the memory and accessible to us as programmers. The memory which is assigned to the program or application in a typical architecture is divided into four segments. One segment of the memory stores the instructions which are to be executed. Another segment of the memory stores the global or the static variables. The global variables are the variables which are declared outside the function and the lifetime is throughout the program. The third segment, i.e., stack is used to store all the function calls and the local variables. When any function is called then it occupies some space in the memory known as a stack memory.



Let's understand the stack memory through an example.

```
#include <stdio.h>
```

```
int square(int s)
```

```
{
```

```
    return s*s;
```

```
}
```

```
int sum(int x,int y)
```

```
{
```

```
    int z= x+y;
```

```

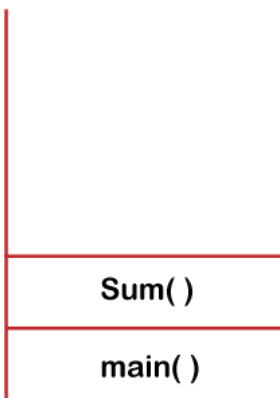
    int result = square(z);
    return result;
}
int main()
{
    int a=10;
    int b=20;
    int x = sum(a,b);
    printf("Output is : %d", x);
    return 0;
}

```

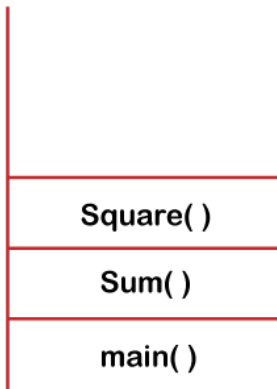
In the above code, execution starts from the main() method. So main() method would be given a memory in the stack as shown below:



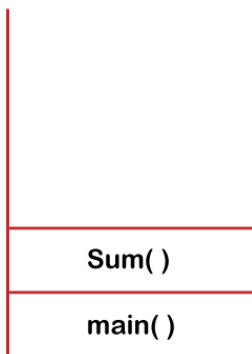
When the sum() method is called, the control moves to the **sum()** function.



The sum() method calls the square() method; therefore, the square method would be given a memory in the stack as shown below:



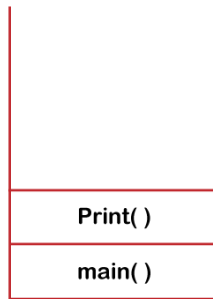
Once the `square()` method return statement is executed, the control moves back to the `sum()` method and the `square()` method gets removed from the stack as shown below:



When the return statement of the `sum()` method is executed, the control moves to the `main()` method and `sum()` method gets removed from the stack as shown below:



In the `main()` method, `printf()` function is called so it gets memory in the stack as shown below:



Once the execution of `printf()` statement is completed, the `printf()` and `main()` methods are removed from the stack memory as shown below:



There are some limitations of using a stack memory. Suppose operating system reserves 1MB stack memory for the program. If program keeps calling the functions again then stack memory would not be sufficient and it leads to the stack overflow condition. The stack overflow causes a program to crash. So, we can say that the stack memory does not grow runtime.

Another limitation of stack is that the scope of the variable cannot be manipulated. The allocation and deallocation of memory onto the stack are set by the rule, i.e., when the function is called then it is pushed onto the top of the stack and when `pop()` operation is called then the element is removed from the top of the stack.

The third limitation of the stack is that if we define the large data type such as array and the size of the array is not defined at the compile time. We want to define the size of the array based on some parameters, and defining the size of the array at the runtime is not possible with the stack. So, to allocate the large chunks of memory and keep the memory aside till the time we want, we can use a **heap** data structure. Unlike stack data structure, the size of the heap memory can vary and it is not fixed throughout the lifetime of the application. In heap memory, there is no set rule for the allocation and deallocation of memory. A programmer can itself manually handle the memory. The abstracted way for the programmer of looking at the heap as a large free of memory available to use and we can use it as per our needs.

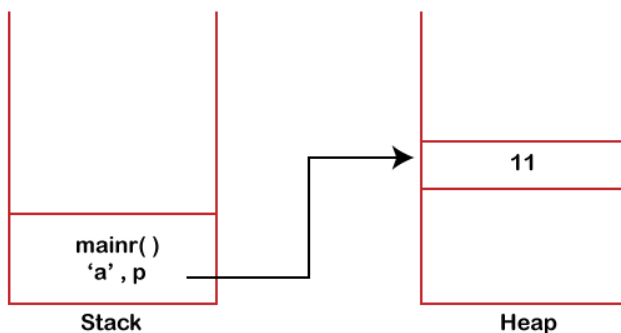
Heap is also known as a dynamic memory and using heap, can be considered as the dynamic memory allocation. To use dynamic memory, we need to use some functions. In [C language](#), we can use `malloc()` and `calloc()` to allocate the memory and `free()` function to deallocate the memory, whereas, in [C++ language](#), we use `new` operator for the allocation and `delete` operator for the deallocation.

**Let's understand the dynamic memory through an example.**

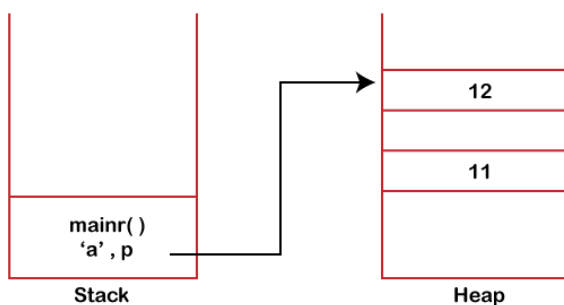
```
#include<stdio.h>
int main()
{
    int a; // goes onto stack.
    int *p;
    p = (int*)malloc(sizeof(int));
    *p = 11;
    free(p);
    p = (int*)malloc(sizeof(int));
    *p = 12;
}
```

### Explanation of the above code.

First, we declared the 'a' variable and it gets allocated within the stack frame of the **main()** method in the stack as shown below:



To allocate something in the **heap memory**, we need to use the `malloc()` function. We have used the `malloc()` function in the above code in which we pass the `sizeof(int)` defines that 4bytes of block is allocated in the heap memory. This function returns the void pointer that contains the starting address of the block. The 'p' is a pointer variable local to the function so it gets stored in the stack as shown below:



Again, we have allocated new block of memory pointed by the 'p' variable, so 'p' does not hold the address of the previous block. The block having value 11 is an unnecessary consumption of memory. In heap, the memory is not automatically deallocated, we have to release the memory manually. So, in the above code, we have used the `free(p)` function in which we pass the 'p' to deallocate the memory pointed by 'p'.

## Differences between Stack and Heap



| Stack                                                                                                                                                                                | Heap                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Stack provides static memory allocation, i.e., it is used to store the temporary variables.                                                                                          | Heap provides dynamic memory allocation. By default, all the global variables are stored in the heap.                             |
| It is a linear data structure means that elements are stored in the linear manner, i.e., one data after another.                                                                     | It is hierarchical data structure means that the elements are stored in the form of tree.                                         |
| It is used to access the local variables.                                                                                                                                            | It is used to access the global variables by default.                                                                             |
| The size of the stack memory is limited which is dependent on the OS.                                                                                                                | The size of the memory is not limited.                                                                                            |
| As it is a linear data structure, so data is stored in the contiguous blocks.                                                                                                        | As it is hierarchical data structure, so elements are stored in the random manner.                                                |
| In stack, the allocation and deallocation are automatically managed.                                                                                                                 | In heap, the memory is manually managed.                                                                                          |
| The implementation of stack can be done in three forms using array, linked list and dynamic memory.                                                                                  | The implementation of heap can be done in two forms using arrays and trees.                                                       |
| The main issue that occurs with a stack is the shortage of memory because the memory size cannot be changed at the runtime. The size of the stack is determined at the compile time. | The main issue that occurs with a heap is the memory fragmentation. Here, memory fragmentation means that the memory gets wasted. |
| It is of fixed size.                                                                                                                                                                 | It is flexible to use as the size of the heap can vary as per our needs.                                                          |
| The access time in stack is faster.                                                                                                                                                  | The access time in heap is slower.                                                                                                |
| The size of the stack memory is decided by the operating system.                                                                                                                     | The size of the heap memory is decided by the programmers.                                                                        |
| The scope of the variable cannot be changed.                                                                                                                                         | The scope of the variable can be changed.                                                                                         |

# Bubble sort vs. Selection sort

Here we will look at the differences between the selection sort and bubble sort. Before understanding the differences, we should know about the selection sort and bubble sort individually.

## What is selection sort?

Sorting means arranging the elements of an array in ascending order. Selection sort is one sorting technique used for sorting the array. In selection sort, an array is divided into two sub-arrays, i.e., one is an unsorted sub-array, and the other is sorted sub-array. Initially, we assume that the sorted subarray is empty. First, we will find the minimum element from the unsorted subarray, and we will swap the minimum element with an element which is at the beginning position of the array. This algorithm is named as selection sort because it is selecting the minimum element and then performs swapping.

**Let's understand the selection sort through an example.**

|   |   |    |   |   |   |
|---|---|----|---|---|---|
| 7 | 4 | 10 | 8 | 3 | 1 |
| 0 | 1 | 2  | 3 | 4 | 5 |

As we can observe in the above array that it contains 6 elements. The above array is an unsorted array whose indexing starts from 0 and ends at 5. The following are the steps used to sort the array:

**Step 1:** In the above array, the minimum element is 1; swap element 1 with an element 7.

Now, the sorted array contains only one element, i.e., 1, while the unsorted array contains 5 elements, i.e., 4, 10, 8, 3, 7.

|              |   |                |   |   |   |
|--------------|---|----------------|---|---|---|
| 0            | 1 | 2              | 3 | 4 | 5 |
| 1            | 4 | 10             | 8 | 3 | 7 |
| Sorted Array |   | Unsorted Array |   |   |   |

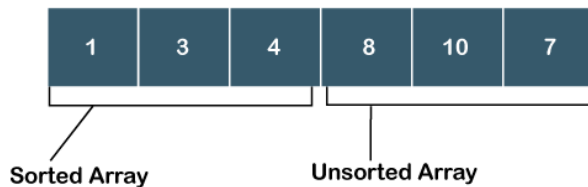
**Step 2:** In the unsorted sub-array, the minimum element is 3, so swap element 3 with an element 4, which is at the beginning of the unsorted sub-array.

Now the sorted array contains two elements, i.e., 1 and 3, while the unsorted array has four elements, i.e., 10, 8, 4, 7, as shown in the above figure.

|              |   |                |   |   |   |
|--------------|---|----------------|---|---|---|
| 1            | 3 | 10             | 8 | 4 | 7 |
| Sorted Array |   | Unsorted Array |   |   |   |

**Step 3:** Search the minimum element in the unsorted sub-array, and the minimum element is 4. Swap the element 4 with an element 10, which is at the beginning of the unsorted sub- array.

Now, the sorted array contains three elements, i.e., 1, 3, 4, while the unsorted array has three elements, i.e., 10, 8, 7



**Step 4:** Search the minimum element in the unsorted array, and the minimum element is 7. Swap element 7 with an element 10, which is at the beginning of the unsorted sub-array.



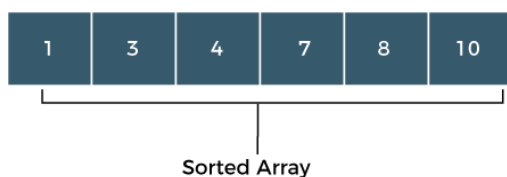
Now, the sorted array contains four elements, i.e., 1, 3, 4, 7, while the unsorted array has two elements, i.e., 10, 8.

**Step 5:** Search the minimum element in the unsorted array and the minimum element is 8. Swap the element 8 with an element 10 which is at the beginning of the unsorted sub-array.



Now, the sorted array contains the elements, i.e., 1, 3, 4, 7, 8.

**Step 6:** The last element is left in the unsorted sub-array. Move the last element to the sorted sub array shown as below:



## What is Bubble sort?

The bubble sort is also one of the sorting techniques used for sorting the elements of an array. The basic principle behind the bubble sort is that the two adjacent elements are to

be compared; if those elements are in correct order, then we move to the next iteration. Otherwise, we swap those two elements. Let's understand the bubble sort through an example.

**Consider the below array:**

|    |    |   |   |   |
|----|----|---|---|---|
| 15 | 16 | 6 | 8 | 5 |
| 0  | 1  | 2 | 3 | 4 |

The above array is an unsorted array. An array consists of 5 integers, i.e., 15, 16, 6, 8, 5.

**The following are the steps required used to sort the array:**

### **PASS 1**

**Step 1:** The  $a[0]$  element is compared with a  $a[1]$  element. The  $a[0]$  is less than  $a[1]$ , i.e.,  $15 < 16$ , so no swapping would be done as shown in the below figure:

|    |    |   |   |   |
|----|----|---|---|---|
| 15 | 16 | 6 | 8 | 5 |
| 0  | 1  | 2 | 3 | 4 |

**Step 2:** Now,  $a[1]$  would be compared with a  $a[2]$  element. Since  $a[1]$  is greater than the  $a[2]$  element, i.e.,  $16 > 6$ , so swap 16 and 6 as shown in the below figure:

|    |   |    |   |   |
|----|---|----|---|---|
| 15 | 6 | 16 | 8 | 5 |
| 0  | 1 | 2  | 3 | 4 |

**Step 3:** The  $a[2]$  would be compared with a  $a[3]$  element. Since  $a[2]$  is greater than the  $a[3]$  element, i.e.,  $16 > 8$ , so swap 16 and 8 elements as shown in the below figure:

|    |   |   |    |   |
|----|---|---|----|---|
| 15 | 6 | 8 | 16 | 5 |
| 0  | 1 | 2 | 3  | 4 |

**Step 4:** The  $a[3]$  would be compared with  $a[4]$  element. Since  $a[3]$  is greater than the  $a[4]$ , i.e.,  $16 > 5$ , so swap 16 and 5 elements as shown in the below figure:

|    |   |   |   |    |
|----|---|---|---|----|
| 15 | 6 | 8 | 5 | 16 |
| 0  | 1 | 2 | 3 | 4  |

As we can observe in the above array that the element which is the largest has been bubbled up to its correct position. In other words, we can say that the largest element has been placed at the last position of the array. The above steps are included in PASS 1 in which the largest element is at its correct position.

We will again start comparing the elements from the first position in PASS 2.

### **PASS 2:**

**Step 1:** First, we compare  $a[0]$  with  $a[1]$  element. Since  $a[0]$  element is greater than the  $a[1]$  element, i.e.,  $15 > 6$ , swap  $a[0]$  element with  $a[1]$  as shown in the below figure:

|   |    |   |   |    |
|---|----|---|---|----|
| 6 | 15 | 8 | 5 | 16 |
| 0 | 1  | 2 | 3 | 4  |

**Step 2:** The  $a[1]$  element would be compared with  $a[2]$  element. The  $a[1]$  is greater than the  $a[2]$ , i.e.,  $15 > 8$ , so swap  $a[1]$  with element  $a[2]$  as shown in the below figure:

|   |   |    |   |    |
|---|---|----|---|----|
| 6 | 8 | 15 | 5 | 16 |
| 0 | 1 | 2  | 3 | 4  |

**Step 3:** The  $a[2]$  element would be compared with a  $a[3]$  element. Since  $a[2]$  is greater than the  $a[3]$  element, i.e.,  $15 > 5$ , swap element 15 with element 5 as shown in the below figure:

|   |   |   |    |    |
|---|---|---|----|----|
| 6 | 8 | 5 | 15 | 16 |
| 0 | 1 | 2 | 3  | 4  |

**Step 4:** Now,  $a[3]$  is compared to  $a[4]$ . Since  $a[3]$  is less than  $a[4]$ , so no swapping would be done as shown in the below figure:

|   |   |   |    |    |
|---|---|---|----|----|
| 6 | 8 | 5 | 15 | 16 |
| 0 | 1 | 2 | 3  | 4  |

As we can observe above that the two elements are at the right position, largest (16) and the second largest element (15). In an array, three elements are unsorted, so again we will follow the same steps in PASS 3.

### PASS 3:

**Step 1:** First, we compare  $a[0]$  with  $a[1]$ . Since  $a[0]$  is less than  $a[1]$ , i.e.,  $6 < 8$ , so no swapping would be done as shown in the below figure:

|   |   |   |    |    |
|---|---|---|----|----|
| 6 | 8 | 5 | 15 | 16 |
| 0 | 1 | 2 | 3  | 4  |

**Step 2:** Now,  $a[1]$  would be compared with  $a[2]$ . As  $a[1]$  is greater than  $a[2]$ , so swap the element 8 with the element 5 as shown in the below figure:

|   |   |   |    |    |
|---|---|---|----|----|
| 6 | 5 | 8 | 15 | 16 |
| 0 | 1 | 2 | 3  | 4  |

**Step 3:** The  $a[2]$  would be compared with  $a[3]$ . Since  $a[2]$  is less than  $a[3]$ , i.e.,  $8 < 15$ , so no swapping would be done as shown in the below figure:

|   |   |   |    |    |
|---|---|---|----|----|
| 6 | 5 | 8 | 15 | 16 |
| 0 | 1 | 2 | 3  | 4  |

**Step 4:** The  $a[3]$  element would be compared with  $a[4]$ . Since  $a[3]$  is less than  $a[4]$ , i.e.,  $15 < 16$ , so no swapping would be done as shown in the below figure:

|   |   |   |    |    |
|---|---|---|----|----|
| 6 | 5 | 8 | 15 | 16 |
| 0 | 1 | 2 | 3  | 4  |

In PASS 3, three elements are at the right positions, largest, second largest and third largest. In an array, two elements are not sorted, so again we will follow the same steps in PASS 4.

**PASS 4:**

**Step 1:** First, we will compare  $a[0]$  and  $a[1]$ . Swap the  $a[0]$  with  $a[1]$  as  $a[0]$  is greater than  $a[1]$ .

|   |   |   |    |    |
|---|---|---|----|----|
| 5 | 6 | 8 | 15 | 16 |
| 0 | 1 | 2 | 3  | 4  |

The above array is a sorted array as all the elements are at the correct positions.

## Differences between Bubble sort and Selection sort

| Bubble sort                                                                                                                                | Selection sort                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| In bubble sort, two adjacent elements are compared. If the adjacent elements are not at the correct position, swapping would be performed. | In selection sort, the minimum element is selected from the array and swap with an element which is at the beginning of the unsorted sub array. |
| The time complexities in best case and worst case are $O(n)$ and $O(n^2)$ respectively.                                                    | The time complexity in both best and worst cases is $O(n^2)$ .                                                                                  |
| It is not an efficient sorting technique.                                                                                                  | It is an efficient sorting technique as compared to Bubble sort.                                                                                |
| It uses an exchanging method.                                                                                                              | It uses a selection method.                                                                                                                     |
| It is slower than the selection sort as a greater number of comparisons is required.                                                       | It is faster than the bubble sort as a lesser number of comparisons is required.                                                                |

## Stack vs Array

Data structure is a way of organizing the data and storing the data in a prescribed format so that the data can be accessed and modified in an efficient manner. Data structure basically provides the logical representation to store the data so that the various operations can be performed on the data. There are multiple ways of storing and retrieving the data, but the stack

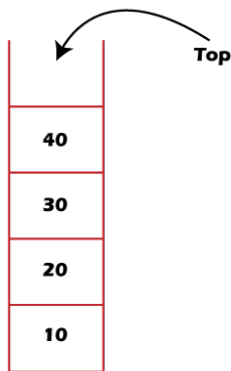
and array are the two most common ways of storing the data. Although the stack can be implemented with the help of array but there is a difference between these two. The major difference is access.

**Let's first understand both the data structures separately, then we will look at the differences between these two data structures.**

## What is Stack?

A stack is a linear list data structure having a sequential collection of elements in which elements are added on the top of the stack like piles of books, etc. The new item can be added, or existing item can be removed only from the top of the stack. Stack is considered as a dynamic data structure because the size of the stack can be changed at the run time. The two operations can be performed on the stack, i.e., push and pop. Here, push operation means that a new item is inserted into the stack, whereas the pop operation means that the existing item is removed from the stack. It strictly follows the **LIFO** (Last In First Out) principle in which the recently added item will be removed first, and the item which is added first will be removed last.

**The representation of the stack is given below:**



## What is Array?

An array is a linear data structure that has the collection of elements of a similar data type. The size of the array is pre-decided and the location at which the values are stored is known as the index of value. It is a static data structure because the memory is allocated at the compile-time, and it is fixed throughout the program. It is one of the efficient ways to store multiple elements belonging to the same data type. It is used to store multiple values of the same type, and we can access them through indices. It provides random access where all the elements are stored linearly and accessed directly through the index.

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|

**Total number of elements = 7**  
**First element with index a [ 0 ]**

## Differences between Stack and Array

**The following are the differences between the stack and array:**

### Definition

Stack is a linear data structure that can be defined as a collection of items arranged in the form

of piles of books. It is a sequential collection of elements and arranged in such a way that the elements can be added and removed only from one end, i.e., **top of the stack**. In contrast, an array is a random-access data structure used to store the multiple elements of similar data types to reduce the complexity of a program. In array, we can access any element directly through an index, and all the elements are stored one after another for efficient memory management.

### Data type

Stack is an abstract data type which is a sequential collection of objects that can store heterogeneous data. Here, heterogeneous data means that the data of various types can be stored in the stack. It is a limited-access data structure means that the element can be inserted or removed in a specific order. In other words, we can say that only top element of the stack can be accessed. In contrast, Array is a linear data structure that store homogeneous data. Homogeneous data means that data of a similar type can only be stored in the array. In array, access is not limited as we can access any element through indices.

### Working Principle

Stack is a linear data structure that follows **LIFO** (Last In First Out). The name LIFO itself suggests that the element which is added last will be removed first, or in other words, we can say that the element which is inserted first will be removed last. In contrast, in an array, any element can be accessed at any time irrespective of the order of elements.

### Size

The stack is a dynamic data structure means that size of the stack can grow or shrink at run time. In contrast, the size of the array is fixed, and it cannot be modified at run time.

**Let's look at the differences between the Stack and Array in a tabular form.**

| Array                                                                                                                                                 | Stack                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| It is a data structure that consists of a collection of elements that are identified by their indexes, where the first index is available at index 0. | It is an abstract data type that consists of a collection of elements that implements the LIFO (Last In First Out) principle.  |
| It is a collection of elements of the same data type.                                                                                                 | It is a collection of elements of different data types.                                                                        |
| It provides random-access, i.e., read and write operations would be performed to any element at any position through their indexes.                   | As it implements LIFO so it has limited-access. We can access the only top element of the stack using push and pop operations. |
| It is rich in methods or operations like sorting, traversing, reverse, push, pop, etc.                                                                | The limited operations can be performed on a stack like push, pop, peek, etc.                                                  |
| It is a data type. It is an abstract data type.                                                                                                       |                                                                                                                                |



It is used when we know all the data to be processed and require constant changes at any element.

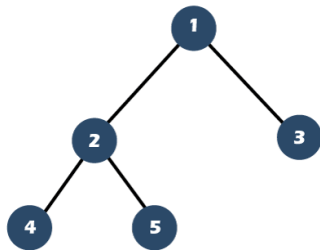
It is good to use when there are dynamic processes. It is useful when we do not know how much data would be required.

## Full Binary Tree vs. Complete Binary Tree

### What is a Full binary tree?

A full binary tree can be defined as a [binary tree](#) in which all the nodes have 0 or two children. In other words, the full binary tree can be defined as a binary tree in which all the nodes have two children except the leaf nodes.

The below tree is a full binary tree:



The above tree is a full binary tree as all the nodes except the leaf nodes have two children.

#### Full Binary tree theorem:

**Consider a Binary tree T to be a nonempty tree then:**

Let I be internal nodes in a tree and L to be a leaf node in a tree, then the number of leaf nodes would be equal to:

$$L = I + 1$$

If T has, I number of internal nodes and N to be the total number of nodes, then the total number of nodes would be equal to:

$$N = 2I + 1$$

If T contains 'N' total number of nodes and 'I' to be number of internal nodes, then the number of internal nodes would be equal to:

$$I = (N-1)/2$$

If 'T' has 'N' total number of nodes, and 'L' to be a number of leaf nodes, then the number of leaf nodes would be equal to:

$$L = (N+1)/2$$

If 'T' contains 'L' number of leaf nodes, then the total number of nodes would be equal to:

$$N = 2L - 1$$

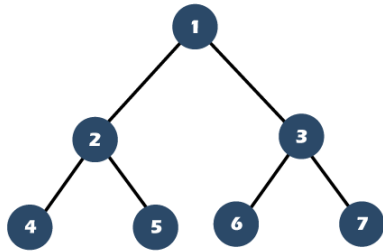
If 'T' has 'L' number of leaf nodes, and 'I' to be a number of internal nodes, then the number of internal nodes would be equal to:

$$I = L - 1$$

### What is a complete binary tree?

A binary tree is said to be a complete binary tree when all the levels are completely filled except the last level, which is filled from the left.

The below tree is a complete binary tree:



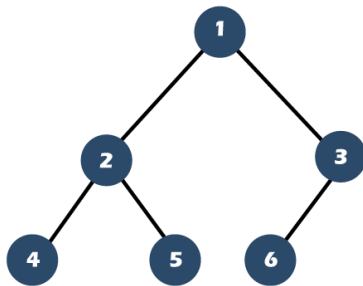
The complete binary tree is similar to the full binary tree except for the two differences which are given below:

The filling of the leaf node must start from the leftmost side.

It is not mandatory that the last leaf node must have the right sibling.

**Let's understand the above points through an example:**

**Consider the below tree:**



The above tree is a complete binary tree, but not a full binary tree as node 6 does not have its right sibling.

### **Creation of Complete Binary Tree**

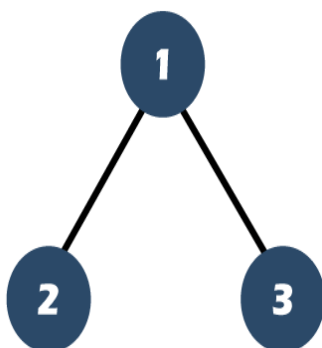
Suppose we have an array of 6 elements shown as below:



The above array contains 6 elements, i.e., 1, 2, 3, 4, 5, 6. The following are the steps to be used to create a complete binary tree:

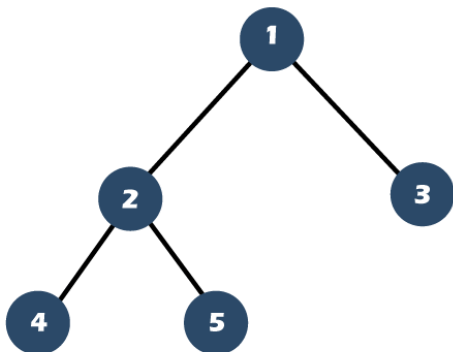
**Step 1:** First, we will select the first element of the array, i.e., 1, and make a root node of the tree. The number of elements available in the first level is 1.

**Step 2:** Now, we will select the second and third elements of the array. Keep the second element and third element of the array as the left and right child of the root node respectively shown as below:



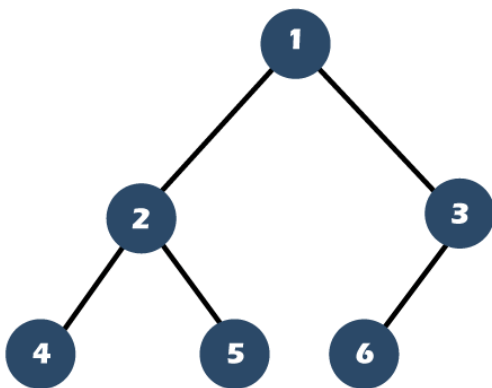
As we can observe above that the number of elements available in the second level is 2.

**Step 3:** Now, we will select the next two elements from the array, i.e., 4 and 5. Keep these two elements on the left and right of node 2 shown as below:



As we can observe above that nodes 4 and 5 are the left and right child of node 2 respectively.

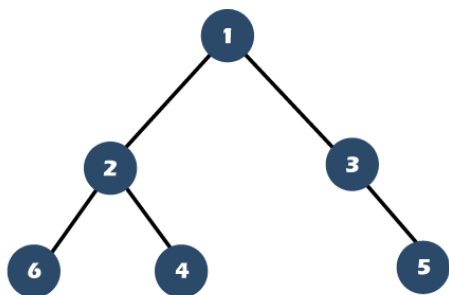
**Step 4:** Now, we will select the last element of the array, i.e., 6, and keep it as left child of the node 3 as we know that in a complete binary tree, the nodes are filled from the left side shown as below:



As we can observe that the second level contains 3 elements.

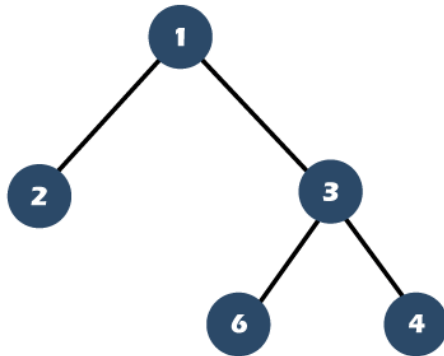
**Let's understand the differences between complete and full binary tree through the images.**

The binary tree which is shown below is neither a complete nor a full binary tree. It is not a full binary tree because node 3 has only one child. It is also not a complete binary tree as the nodes should be filled from the left side, but node 3 has a right child and does not have a left child.

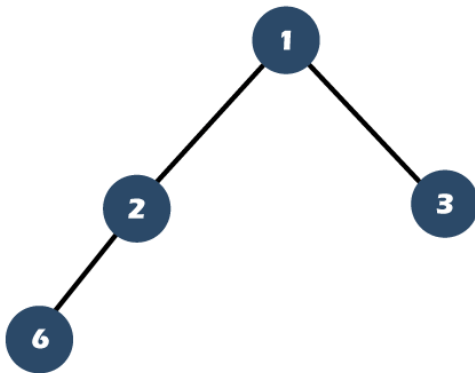


The binary tree, which is shown below, is a full binary tree but not a complete binary tree. It is a full binary tree because all the nodes have either 0 or 2 children. It is not a complete binary tree

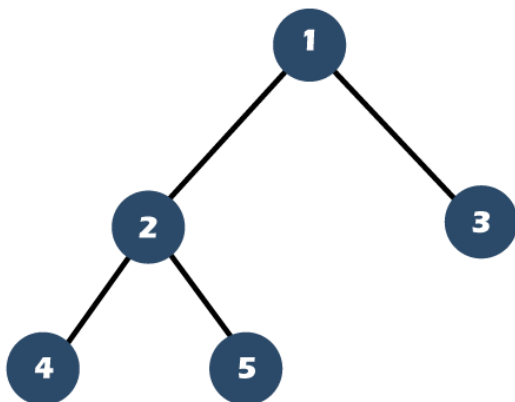
because node 3 does not have any children while node 2 has its children and we know that the nodes should be filled from the left side in a complete binary tree.



The binary tree which is shown below is a complete binary tree but not a full binary tree. It is a complete binary tree as all the nodes are left filled. It is not a full binary tree as node 2 has only one child.



The binary tree which is shown below is a complete as well as a full binary tree. It is a complete binary tree as all the nodes are left filled. It is a full binary tree as all the nodes have either 0 or 2 children.



## Binary Tree vs B Tree

Before understanding the differences between the binary tree and btree, we should know about **binary tree** and **btree** separately.

## What is Binary tree?

A [binary tree](#) is a tree that contains at most two child nodes. The binary tree has one limitation on the degree of the node as node in a binary tree cannot contain more than two child nodes. The topmost node in a binary tree is known as a root node and node mainly consists of two subtrees known as left subtree and right subtree. If the node in a binary tree does not contain any children, then it is known as a leaf node. Thus, node can have either 0, 1 or 2 children. The operations that can be performed on a binary tree are [insertion](#), deletion, and traversal.

### **Binary tree can be categorized into following types:**

Full binary tree: Each node in a binary tree can have zero or two child nodes.

Perfect binary tree: A perfect binary tree is a full binary tree except one condition that all the leaf nodes exist at the same level of depth.

Complete binary tree: A complete binary tree is a tree in which all the leaf nodes are left aligned as possible.

Balanced binary tree: A binary tree is said to be balanced if the height of the tree is as small as possible.

Binary search tree: A binary search tree is a tree in which all the keys are sorted to provide the faster search.

It can be used to implement the expression evaluation, parsers, data compression algorithms, storing router-tables, cryptographic applications, etc.

## What is BTree?

A [btree](#) is a self-balancing tree because its nodes are sorted in an inorder traversal. In contrast to binary tree, node in a btree can have more than two children. The height of btree is  $\log MN$  where M is the order of tree and N is the number of nodes. The height of a btree adjusts automatically, and the height in a btree is sorted in a specific order having the lowest value on the left and the highest value on the right.

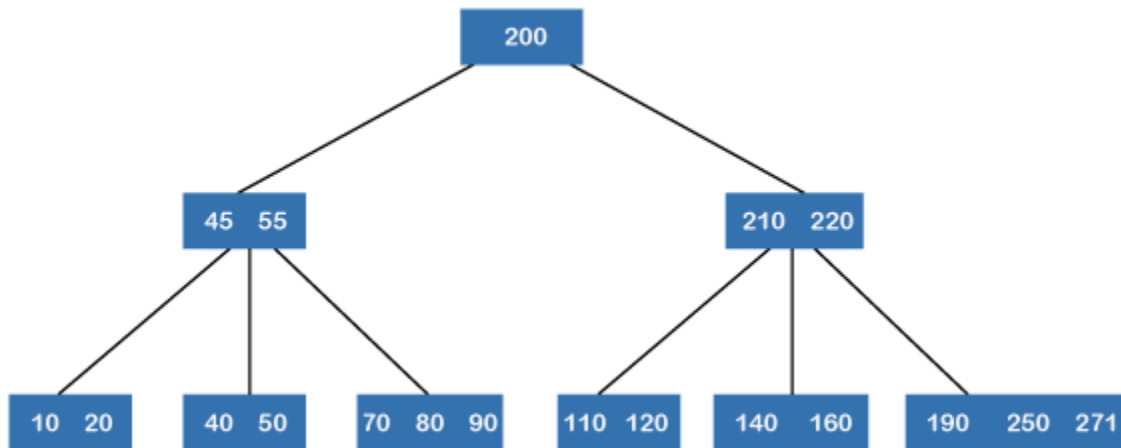
Btree is mainly used to store huge amount of data that cannot fit in the main memory. When the number of keys increases then the data is read from the disk in the form of blocks. As we know that disk access time is more than the memory access time. The main idea behind using the btree is to reduce the number of disk accesses. Most of the operations that are implemented on btree like search, delete, insert, max, min, etc have  $O(h)$  disk accesses where h is the height of the tree. Btree is a very wide tree. The idea behind constructing the btree by keeping the height of the tree as low as possible by attaching the maximum number of keys in a btree node. The size of the btree node is mainly made equal to the disk block size. Since the height of the tree is quite low so disk accesses are reduced significantly as compared to the balanced binary search tree like AVL tree, Red Black tree, etc.

### **Some important facts related to btree are given below:**

All the leaf nodes in a btree must be at the same level.

There should not be exists any empty subtree above the leaf nodes.

The height of the btree should be kept as low as possible.



In the above figure, we can observe that all the leaf nodes exist at the same level, and all the non-leaf nodes are non-empty sub trees having keys one less than the number of their children.

**Let's understand the differences between binary tree and btree in a tabular form.**

| B-tree                                                                                        | Binary tree                                                                                                 |
|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| A node in b tree can have maximum "M" number of child nodes where M is the order of the tree. | Unlike btree, binary tree can have maximum two children or two subtrees.                                    |
| A btree is a sorted tree because its nodes are sorted in an inorder traversal.                | A Binary tree is not a sorted tree A tree can be sorted either in inorder, preorder or postorder traversal. |
| The height of btree is $\log MN$ where M is the order of tree and N is the number of nodes.   | The height of binary tree is $\log_2 N$ where N is the number of nodes.                                     |
| Btree is implemented when the data is stored in the disk.                                     | Binary tree is implemented when the data is stored in RAM.                                                  |
| It is used in DBMS.                                                                           | It is used in Huffman coding and code optimization.                                                         |
| Insertion of data or a key in a btree is more complex than the binary tree.                   | Insertion of data in a binary tree is easier as compared to btree.                                          |

## Primitive vs non-primitive data structure

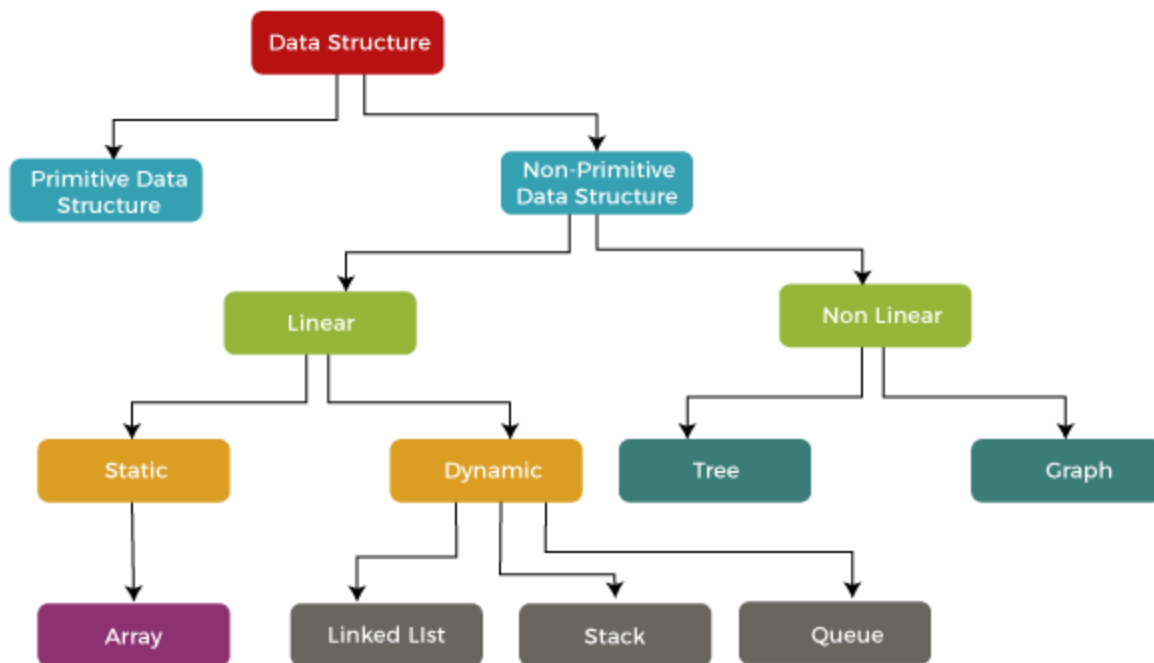
Data structure means organizing the data in the memory. The data can be organized in two ways either linear or non-linear way.

There are two types of data structure available for the programming purpose:

Primitive data structure

Non-primitive data structure

Primitive data structure is a fundamental type of data structure that stores the data of only one type whereas the non-primitive data structure is a type of data structure which is a user-defined that stores the data of different types in a single entity.



In the above image, we can observe the classification of the data structure. The [data structure](#) is classified into two types, i.e., primitive and non-primitive data structure. In the case of primitive data structure, it contains fundamental data types such as integer, float, character, pointer, and these fundamental data types can hold a single type of value. For example, integer variable can hold integer type of value, float variable can hold floating type of value, character variable can hold character type of value whereas the pointer variable can hold pointer type of value.

In the case of non-primitive data structure, it is categorized into two parts such as linear data structure and non-linear data structure. Linear data structure is a sequential type of data structure, and here sequential means that all the elements in the memory are stored in a sequential manner; for example, element stored after the second element would be the third element, the element stored after the third element would be the fourth element and so on. We have different linear data structures holding the sequential values such as [Array](#), [Linked list](#), [Stack](#), [Queue](#).

Non-linear data structure is a kind of random type of data structure. The non-linear data structures are Tree and Graph.

**Let's understand the differences between the primitive and non-primitive data structure.**

**Primitive data structure**

**Non-primitive data structure**

|                                                                                             |                                                                                                         |
|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Primitive data structure is a kind of data structure that stores the data of only one type. | Non-primitive data structure is a type of data structure that can store the data of more than one type. |
| Examples of primitive data structure are integer, character, float.                         | Examples of non-primitive data structure are Array, Linked list, stack.                                 |
| Primitive data structure will contain some value, i.e., it cannot be NULL.                  | Non-primitive data structure can consist of a NULL value.                                               |
| The size depends on the type of the data structure.                                         | In case of non-primitive data structure, size is not fixed.                                             |
| It starts with a lowercase character.                                                       | It starts with an uppercase character.                                                                  |
| Primitive data structure can be used to call the methods.                                   | Non-primitive data structure cannot be used to call the methods.                                        |

## Primitive data structure

Primitive data structure is a data structure that can hold a single value in a specific location whereas the non-linear data structure can hold multiple values either in a contiguous location or random locations

The examples of primitive data structure are float, character, integer and pointer. The value to the primitive data structure is provided by the programmer. The following are the four primitive data structures:

**Integer:** The integer data type contains the numeric values. It contains the whole numbers that can be either negative or positive. When the range of integer data type is not large enough then in that case, we can use long.

**Float:** The float is a data type that can hold decimal values. When the precision of decimal value increases then the Double data type is used.

**Boolean:** It is a data type that can hold either a True or a False value. It is mainly used for checking the condition.

**Character:** It is a data type that can hold a single character value both uppercase and lowercase such as 'A' or 'a'.

## Non-primitive data structure

The non-primitive data structure is a kind of data structure that can hold multiple values either in a contiguous or random location. The non-primitive data types are defined by the programmer. The non-primitive data structure is further classified into two categories, i.e., linear and non-linear data structure.

In case of linear data structure, the data is stored in a sequence, i.e., one data after another data. When we access the data from the linear data structure, we just need to start from one place and will find other data in a sequence.



**The following are the types of linear data structure:**

**Array:** An array is a data structure that can hold the elements of same type. It cannot contain the elements of different types like integer with character. The commonly used operation in an array is insertion, deletion, traversing, searching.

**For example:**

```
int a[6] = {1,2,3,4,5,6};
```

The above example is an array that contains the integer type elements stored in a contiguous manner.

**String:** String is defined as an array of characters. The difference between the character array and string is that the string data structure terminates with a 'NULL' character, and it is denoted as a '\0'.

**String data structure:**

```
char name[100] = "Hello javaTpoint";
```

In the above example, the length of the string is 17 as the last character is the NULL character which denotes the termination of the string.

**Char Representation:**

```
char name[100] = {'H', 'e', 'l', 'l', 'o', ' ', 'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't' }
```

In the above example, the length of the string is 16 as it does not have any NULL character as the last character to denote the termination.

**Stack:** Stack is a data structure that follows the principle **LIFO** (Last In First Out). All the operations on the stack are performed from the top of the stack such as PUSH and POP operation. The push operation is the process of inserting element into the stack while the pop operation is the process of removing element from the stack. The stack data structure can be implemented by using either array or linked list.

**Queue:** Queue is a data structure that can be implemented by using array. The difference between the stack and queue data structure is that the elements in the queue are inserted from the rear end while the elements in the queue are removed from the front end.

## Data types vs data structure

### What is a data type?

**The two things we must know about the data types are:**

It defines a certain domain of values.

It defines operations allowed on those values.

**For example:**

**If the data is of int (integer) type**

Then it takes only integer values

Operations that can be performed on the data type are addition, subtraction, multiplication, bitwise operations, etc.

**If the data is of float (floating) type**

Then it takes only floating type values.

Operations that can be performed on the data type are addition, subtraction, multiplication, division, etc. (bitwise and % operations are not allowed).

So, it is cleared from the above examples that data type does not only define a certain domain of values but also defines operations that can be performed on those values.

## User-defined data type

In contrast to primitive data types, there is a concept of user-defined data types. The values and the operations that can be performed on the primitive data types are not specified by the language itself, but it is specified by the user only. The examples of user-defined data types are structure, union, and enumeration. By using the structures, we can define our own type by combining other primitive data types. Let's understand through an example.

```
struct point
```

```
{  
    int x;  
    int y;  
}
```

In the above code, we have created a user-defined data type named '**point**' that is made by combining two primitive data types of integer type named 'x' and 'y'.

## Abstract data types

Abstract data types are like user-defined data types, which define the operations on the values using functions without specifying what is inside the function and how the operations are performed.

### For example:

**Stack ADT:** Here, the stack consists of elements of the same type arranged in sequential order.

The following are the operations that can be performed on the stack are:

initialize(): This method initializes the stack to be empty.

push(): It is a method used for inserting an element into the stack.

pop(): It is a method used for removing the element from the stack.

isEmpty(): This method is used to check whether the stack is empty or not.

isfull(): It checks whether the stack is full or not.

**Note: We can think of ADT as a black box that hides the user's inner structure and design of the data type.**

There are multiple ways to implement an ADT. For example, a stack ADT can be implemented using arrays or linked lists.

## Why ADT?

The program which uses data structure is known as a client program, and it has access to the ADT or interface. The program that implements the data structure is known as implementation.

## Advantage

If someone wants to use the stack in the program, he can directly use the push and pop operations in the program without knowing its implementation details.

## What is data structure?

A Data structure is a collection of different types of data on which a specific set of operations can be performed. It is a collection of different data types. It is a way of organizing the data in memory. The various operations that can be performed on a data structure are insertion, deletion, and traversal. For example, if we want to store the data of many students where each student has a student name, student id, and a mobile number. To store such big data requires complex data management, which requires a data structure comprising multiple primitive data types.

## Differences between the data type and the data structure

| Data type                                                                                                                                             | Data structure                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| A Data type is one of the forms of a variable to which the value can be assigned of a given type only. This value can be used throughout the program. | A Data structure is a collection of data of different data types. This collection of data can be represented using an object and can be used throughout the program. |
| The implementation of data type is known as an abstract implementation.                                                                               | The implementation of data structure is known as a concrete implementation.                                                                                          |
| It can hold value but not data. Therefore, we can say that it is data-less.                                                                           | It can hold multiple types of data within a single object.                                                                                                           |
| In case of data type, a value can be assigned directly to the variables.                                                                              | In the case of data structure, some operations are used to assign the data to the data structure object.                                                             |
| There is no problem in the time complexity.                                                                                                           | When we deal with a data structure object, time complexity plays an important role.                                                                                  |
| The examples of data type are int, float, char.                                                                                                       | The examples of data structure are stack, queue, tree, graph.                                                                                                        |